

MATH4267: Deep Learning and Artificial Intelligence

Lecture Notes

James Liley

January 21, 2026

Contents

Survival Guide	7
SG.1 Welcome!	7
SG.2 Learning objectives	7
SG.3 Overview of course	8
SG.3.1 Structure	8
SG.3.2 Rationale for course	8
SG.3.3 Lectures	8
SG.3.4 Tutorials	10
SG.3.5 Office hours	10
SG.4 Assessment	10
SG.4.1 Formative	10
SG.4.2 Summative	10
SG.5 Textbooks	11
SG.6 Mathematical reference	12
SG.6.1 Assumed knowledge	12
SG.6.2 Notation	12
SG.7 General advice	14
 1 Overview of statistical learning	 15
1.1 Introduction	15
1.2 Supervised and unsupervised learning	15
1.3 A simple example	15
1.4 Data matrices	17
1.5 Decision rules	17
1.6 Special case: the 0-1 loss function	17
1.7 Training and test error	19
1.7.1 Bias-variance tradeoff	20
1.8 Sample complexity	20
 2 Introduction to the neuron	 25
2.1 Introduction	25
2.2 Neurons	25
2.2.1 Biological neurons	25
2.2.2 The abstract neuron	26
2.2.3 Neurons as regressions	27
2.3 Activation functions	28
2.3.1 Discriminatory activation functions	29
2.3.2 Simple discriminatory activation functions	29
2.3.3 Why are we interested in this?	29
2.3.4 Sigmoid functions	30
2.3.5 ReLU activation functions	30
 3 Neural networks	 33
3.1 Introduction	33
3.2 Neural networks	33
3.2.1 What can a single neuron do?	33

3.2.2	Neural networks	33
3.2.3	What can a neural network do that a single neuron can not?	36
4	Representation theorems	41
4.1	Introduction	41
4.2	One dimensional inputs	41
4.2.1	Preliminaries	41
4.2.2	Activation function properties	41
4.2.3	Main result	41
4.2.4	Starting out	42
4.2.5	Proceeding to the proof	43
4.2.6	Proving 4.3	44
4.2.7	Proving 4.4	44
4.2.8	Remarks	46
5	Cybenko's universal approximation theorem	49
5.1	Introduction	49
5.2	Main theorem statement	49
5.2.1	Discriminatory activation functions: simple definition	49
5.2.2	Main theorem statement	49
5.2.3	Discriminatory activation functions: full definition	50
5.3	General preliminaries	50
5.3.1	The space $C(I_n)$	50
5.3.2	Functional analysis	51
5.3.3	Borel measure	51
5.3.4	Single hidden layer networks as universal approximators	51
5.4	Proceeding with the main proof	51
5.4.1	Lemmata	51
5.5	Final lemmas and proof	53
5.5.1	Proof of main result	53
5.5.2	Remarks	54
5.6	Other important results and generalisations	54
5.6.1	Continuous non-constant activation functions	54
5.6.2	Approximation with arbitrary functions	55
6	Notions of complexity	59
6.1	Introduction	59
6.2	The aim of neural networks and machine learning	59
6.2.1	Sizes of spaces and function spaces	59
6.2.2	Machine learning	60
6.3	Depth or width	60
6.3.1	Example 1: The sawtooth function	60
6.3.2	Example 2: The Cantor set	61
6.3.3	Discussion	62
6.4	Kolmogorov complexity	63
6.4.1	Examples	63
6.4.2	The Mandelbrot set	64
6.5	How simple are function classes?	64
6.5.1	Number of parameters paradox	65
6.5.2	The VC dimension	66
6.5.3	VC dimension of various classifiers	66
6.5.4	VC dimension in sample complexity	67
6.6	Discussion	68
7	Information as sigma algebras	73
7.1	Introduction	73
7.2	Preliminaries	73

7.2.1	Sigma-algebras	73
7.2.2	Sigma-algebra as information	73
7.2.3	Probability space	74
7.2.4	Formal definition of a random variable, and conception	74
7.3	Information in a neural network	75
7.3.1	Input and output information	75
7.3.2	Target information	76
7.3.3	Lost information	76
7.3.4	Example	76
7.4	Compressible layers	77
7.4.1	Terminology	77
7.4.2	Conditions for a layer to be uncompressed	77
8	Entropy	81
8.1	Introduction	81
8.2	Information-theoretic concepts	81
8.2.1	Information	81
8.2.2	Entropy	82
8.2.3	Example - autoencoders	85
9	Mutual information	89
9.1	Introduction	89
9.2	Mutual information	89
9.2.1	Conditional entropy	89
9.2.2	Mutual information	90
9.2.3	Properties	91
9.2.4	Transformation property	92
9.2.5	Data Processing Inequalities	92
9.3	Applications to neural networks	93
9.4	Summary of ideas from information theory	93
9.4.1	Intuition	95
9.4.2	Kolmogorov complexity	95
9.4.3	Entropy	95
9.4.4	Generated σ -algebras	95
9.4.5	Mutual information	95
10	Backpropagation	99
10.1	Introduction	99
10.1.1	Heuristics	99
10.2	Notation	99
10.3	Two-layer model	100
10.3.1	Updating weights	101
10.3.2	Propagating δ s	101
10.4	General formula	102
11	Gradient descent	107
11.1	Introduction	107
11.2	Properties of minima	107
11.3	Gradient descent	107
11.4	Making gradient descent work	109
11.4.1	Adaptive choice of t	109
11.4.2	Momentum	110
12	Practicalities of neural network training	115
12.1	Introduction	115
12.1.1	The basic backpropagation algorithm	115
12.2	Batch training	116

12.3	Weight initialisation	116
12.3.1	Xavier initialisation	117
12.4	Dropout	118
13	Convolutional neural networks	121
13.1	Introduction	121
13.2	Signals	121
13.3	Convolutional neural networks	123
13.4	Some notes	123
13.4.1	Pooling	123
13.4.2	Channels	124
13.4.3	Overall architecture	124
13.4.4	Transfer learning	124
14	Recurrent neural networks	129
14.1	Introduction	129
14.2	Dynamical systems	129
14.3	Recurrent neural networks	129
14.4	Training recurrent neural networks	131
14.4.1	Backpropagation through time	131
14.4.2	Problems	132
14.5	Long short-term memory networks (LSTMs)	132
15	Generative adversarial networks	135
15.1	Introduction	135
15.2	Setup	135
15.3	Density estimation	135
15.4	Adversarial models	135
15.5	Generative adversarial networks	136
15.5.1	Loss function	137
15.5.2	Optimal discriminator	138
15.5.3	Training	138
15.5.4	Mode collapse	139
16	Boltzmann machines	143
16.1	Introduction	143
16.2	Stochastic neurons and perceptrons	143
16.3	Boltzmann machines	143
16.3.1	Energy of a network	145
16.3.2	Exact learning	148
17	Restricted Boltzmann machines	153
17.1	Introduction	153
17.2	Training Boltzmann machines	153
17.3	Restricted Boltzmann Machines (RBMs)	154
17.3.1	Training RBMs	155
18	Variational autoencoders	159
18.1	Introduction	159
18.2	Autoencoder architecture	159
18.2.1	The problem with autoencoders	160
18.3	Variational autoencoders	160
18.3.1	The problem with (naive) variational autoencoders:	161
18.4	Training VAEs	161
18.4.1	Training the decoder	162
18.4.2	Training the encoder	162
19	Transformers	167

19.1	Introduction	167
19.2	Long short-term memory networks (LSTMs)	167
19.2.1	LSTM setup	167
19.3	Attention	168
19.3.1	Beginnings	169
19.3.2	Weighting states for attention	169
19.3.3	Self-attention	170
19.4	Transformers	171
19.4.1	Positional encoding	171
19.4.2	The transformer architecture	172
20	Diffusion models	177
20.1	Introduction	177
20.2	Overall idea	177
20.2.1	Generative networks	177
20.2.2	The normalising constant	178
20.2.3	The score	178
20.2.4	Diffusion model overall form	178
20.3	Details	178
20.3.1	How do we estimate the score?	179
20.3.2	How do we use the score to sample from f_{data} ?	179
20.3.3	Putting this together	179
21	Revision lecture	183
21.1	Introduction	183
21.2	General overall principles	183
21.3	Statistical learning	184
21.4	Introduction to neural networks	184
21.5	Representation	185
21.6	Information theory	185
21.6.1	Notions of complexity	185
21.6.2	Sigma algebras	185
21.6.3	Entropy, mutual information	186
21.6.4	What you do not need to know	186
21.7	Training	186
21.8	Standard network architectures	186
21.8.1	Convolutional neural networks	187
21.8.2	Recurrent neural networks	187
21.8.3	Generative adversarial networks	187
21.8.4	What you do not need to do	187
21.9	Energy-based networks	187
21.10	Modern neural networks	188

Survival guide

SG.1 Welcome!

Welcome to Deep Learning and Artificial Intelligence!

In this course we will be looking at the idea of deep neural networks and how they work. This is a slightly paradoxical topic, because although neural networks are very simple (and their training is similarly straightforward), their analysis is not.

The study of deep neural networks is an applied field, driven by real-world problems, so it tends towards usefulness rather than elegance. The field moves very fast, and there are constant developments in the best way to do things. There are a range of software packages across many languages which implement deep learning methods. You are likely to end up using methods from deep learning in your careers. A general philosophy of deep learning is of automating solutions to ever-more difficult problems, eventually beginning to approximate the class of problems solvable by humans.

Although we will be looking at some applied material, particularly later in the course, and in tutorials, we will mostly be focussing on earlier material, which is more fundamental and theoretical. The reason for this is that you, as mathematics students, have an edge in being able to understand neural networks ‘from scratch’, which is a great advantage when designing and troubleshooting. We will seek to make the most of that advantage, and seek to build an understanding of deep neural networks from the ground up, and focus on formal ways of describing things.

We will (fortunately or unfortunately) need a range of mathematical methods, including parts of analysis, probability, calculus, linear algebra, and geometry. This quick guide is a non-exhaustive starting point for course details, important things to know, and unsolicited advice. Please also bear in mind that good places to find things are:

1. The reading list (for specific questions)
2. Learn Ultra (for formal material for this course)
3. Your academic tutor (for academic discussion in general)
4. Your college (for non-academic matters)

This is still quite a new course, so please give any feedback you think you can. In particular, if the course is too easy, too hard, or too uninteresting, we can modify it a bit as we go.

I hope you enjoy the course!

SG.2 Learning objectives

This course has the following objectives. They are not necessarily covered with equal depth!

1. *A systematic and coherent understanding of the mathematical theory underlying deep neural networks and their training:* we will cover the general structure of neurons and neural networks; universal approximation theorems; information processing in neural networks; and how to train neural networks,
2. *An understanding of the relationship of this theory to other statistical techniques:* we will review general ideas in statistical learning; look at methods of gradient descent; and consider the neuron as a linear regression,
3. *The ability to make appropriate modelling and algorithmic choices for a given problem or application:* we will look at convolutional and recurrent neural networks; compare depth and width of networks; and look at methods of training,

4. *The ability to implement those choices using currently available software packages, and test their validity and performance*; in tutorials we will use contemporary deep learning software to attack problems with near-state-of-the-art performance; we will cover general methods of statistical learning; and we will look at practicalities when training neural networks,
5. *Sufficient understanding and expertise to be able to expand their knowledge of theory and practice to encompass newly developed techniques and software*; we will cover autoencoding networks; and generative networks; and in general focus on understanding neural networks from the ground up.

SG.3 Overview of course

SG.3.1 Structure

The course consists of:

- 21 lectures, grouped into a series of topics (one reserved for revision):
- 5 computer practicals

Assessment is via one two-hour exam and one assignment.

SG.3.2 Rationale for course

We will begin the course by looking at a brief review of statistical learning. Although this will not be new to many of you, it is of fundamental importance, particularly the ideas of generalisation error and training/testing sets. We will then go through what a neural network is, and some basic properties.

Probably the most important overall task we will try and learn is to efficiently designing a neural network to solve a particular problem. This is very bespoke, and hard-and-fast rules are rare. A useful start is to look at how neural networks can approximate functions, so one of the first things we will look at is ‘universal approximation theorems’: looking at the conditions under which neural networks can approximate arbitrary functions. We will also look later at the relative benefits of *deep* rather than *wide* networks.

The great paradox of neural networks is how they fit things well while being overparametrised. We won’t exactly answer this question but we’ll work towards having a language to address it, by looking at the way information flows through a network.

We will then (somewhat later than maybe expected) come to how to actually get a neural network to learn things. This largely concerns computing derivatives, but we will also look at practical ways to improve learning.

We will then begin to look at how we can adapt neural networks to particular tasks. We will look at convolutional networks, which are usually used to model visual data, and recurrent neural networks, which are used to model sequential data.

We will get to looking at some of the exciting things we can do with neural networks: autoencoders, generative networks, transformers, and diffusion models. We will also have a look at a theoretical type of network characterised by an ‘energy’ function.

SG.3.3 Lectures

We will have two one-hour lectures per week. Lectures will be divided into seven sub-topics, which will inter-relate to each other. In general, the last lecture in a sub-topic will be slightly shorter and easier, so please e-mail me if anything in a lecture is unclear, and we can revise it again.

Lecture difficulty map

Lectures and topics are detailed below. I have made a rough indication of the conceptual difficulty, mathematical difficulty, and length of each lecture (4 being hardest/longest). In general, in this course, the conceptual difficulty will be greater than the mathematical difficulty: once you have the ideas ‘translated’ to mathematics, they are not too hard to work with.

The hardest modules are on representation theorems and information theory, although some elements of both of these are non-examinable.

Number	Topic	Conc.	Math.	Length
General ideas				
1	Overview of statistical learning	2	2	2
2	The abstract neuron	1	1	2
3	Neural network construction	2	2	3
Representation theorems				
4	A closer look at activation functions	2	3	2
5	General representation theorems*	3	4	4
Information processing in neural networks				
6	Notions of complexity	4	3	3
7	Information as σ -algebras	3	4	2
8	Information and entropy	3	2	2
9	Mutual information	2	2	1
Training neural networks				
10	Gradient descent*	2	2	4
11	Backpropagation	2	1	3
12	Practicalities of training	1	2	2
Network architectures				
13	Recurrent neural networks	3	1	2
14	Convolutional neural networks	2	1	1
15	Generative networks	2	3	2
Energy-based networks				
16	Boltzmann machines	2	3	2
17	Contrastive divergence	2	2	1
Network applications and modern networks				
18	Autoencoders	2	3	3
19	Transformers*	4	1	3
20	Diffusion models*	4	2	3

Table 1: Conc.: conceptual difficulty (1 easiest, 4 hardest). Math.: mathematical difficulty (1 easiest, 4 hardest). The symbol * indicates that material in the lecture is **partly** non-examinable.

Style of lectures/tutorials

I will generally lecture in a chalk-and-talk style, although will sometimes use code or slides on-screen. I will release lecture notes on ultra the week before lectures. Lecture notes may be updated throughout the term. Lectures will have some practice questions at the end.

Although I will sometimes use slides, these **will not cover material exhaustively** and should not be relied on for revision.

For computer tutorials, we will use GitHub codespaces, which some of you will be familiar with. We will look at how to set this up at the first tutorial.

Lecture attendance

It is best to try and attend all lectures! If you need to miss one or two, then please try and do the following to catch up:

1. Read through the online lecture notes carefully

2. Try and do the problems at the end of the lecture

If you still feel like you are struggling, then:

1. Read through the suggested reading
2. Google any ideas you do not understand
3. Come to office hours if you are stuck

SG.3.4 Tutorials

Tutorials will roughly be on:

1. Building neural networks from scratch
2. Training and statistical learning
3. Convolutional networks
4. Generative networks

Use of R in tutorials

We will use the programming language R with the `keras` package in tutorials. While it is possible to install this on your own machine, I do not recommend it. Instead, we will use GitHub codespaces (which most of you will have used in DSSC). Instructions for how to set up codespaces are available on ultra. I will go through these in more detail before the first tutorial.

In machine learning, it is more common to use Python for computational tasks. For all summative assessment, there will be an option to use Python instead if you prefer, although we will not cover the use of Python in this course.

SG.3.5 Office hours

I will hold office hours on Friday at 14:00-15:00. (MCS2086). Please e-mail me prior to coming to office hours so I have an idea of numbers. It will also help if you can let me know any questions or concerns in advance.

SG.4 Assessment

SG.4.1 Formative

Formative assessment will consist of three question sets. These will be released regularly throughout the term, and solutions will be made available. The assignments and feedback will be released via GradeScape. Practice problems in lecture notes will not be marked, but I suggest that these are worked through in private.

SG.4.2 Summative

Exam

This course will be partly assessed by exam.

- The exam will be two hours long, except for those in the MSc, who will sit a three-hour exam in combination with the module on high-dimensional statistics.
- The exam will cover only material from lecture notes, but will assume knowledge of general mathematical techniques. When lecture notes refer to an external source (e.g. ‘see [Calin, 2020, chapter 2]’), the material in the external source is non-examinable.
- Homework assignments and end-of-lecture problems will generally contain similar questions to those encountered in the exam

- When lecture notes cover non-examinable material, this will be marked. If not marked, assume that it is examinable. Sometimes, I will expect you to know and be able to use the result of a theorem, but not reproduce the proof.
- The exam will be slightly weighted towards earlier lectures and more fundamental material. Questions will typically be of the ‘prove/show/find’ type rather than the ‘describe/define/discuss’ type.

Assignment

The assignment format will be specified later in term. It will consist of one, both, or either of:

1. A detailed analysis of a recent paper in machine learning (out of a list I will supply)
2. A programming task, for which you will submit an answer and a document giving details of your approach.

SG.5 Textbooks

We will predominantly work from the following textbook

- *Calin, Ovidiu. Deep learning architectures. New York City: Springer International Publishing, 2020.*

This textbook covers most of the material in this course to an appropriate mathematical level. It also covers some further areas which may be of interest (particularly topological deep learning). It contains a range of appendices which outline necessary mathematical details, and contains a range of good exercises and answers. We will use similar notation to this textbook.

On the downside, this textbook does not do anything computational, and there is no implementation of results. It contains occasional errors.

If you have trouble getting hold of this textbook, please get in touch with me.

We will also use the following textbooks:

- Zhang, Aston; Lipton, Zachary C; Li, Mu; Smola, Alexander. Dive into deep learning. Arxiv, 2021.

This is a very good textbook containing both code and theory. It is available directly online at <https://d2l.ai/index.html>.

- Anthony, Martin; Bartlett, Peter L. Neural network learning: Theoretical foundations. Cambridge University Press, 2022

This is a somewhat classic textbook looking at the foundations of statistical learning theory as applied to neural networks. We will occasionally hint at various ideas to do with how many samples are needed to train a neural network to do a particular task, and how we can sensibly bound these numbers; the formal versions of these hints can be found in this textbook.

- Hastie, Trevor; Tibshirani, Robert; Friedman, Jerome. The elements of statistical learning: data mining, inference, and prediction. New York: Springer, 2009

This is an excellent (and widely-quoted) book on general statistical learning, covering material both informally and in mathematical depth, with particular focus on links between methods. It is available directly online at <https://hastie.su.domains/Papers/ESLII.pdf>.

For tutorials, we will work with the following textbook:

- Chollet, Francis; Kalinowski, Tomasz; Allaire, Joseph. Deep learning with R. Manning, 2018

This textbook runs through a range of deep learning applications using the R package Keras (which was developed by the authors). It also contains some theory, although with limited mathematical backup.

We will also use a range of primary resources (e.g. academic papers) which I will cite in the course notes.

SG.6 Mathematical reference

Students who take DLAI4 come from a range of backgrounds, and (aside from some background in statistics) we do not necessarily have a lot of common knowledge. This section gives a rough idea of what I will expect you to know or what I will not expect you to know, and of some of the notation we will be using.

SG.6.1 Assumed knowledge

I will assume some familiarity with the following ideas.

- Calculus: integration, differentiation, partial derivatives.
- Basic algebra: sums, products, etc
- Construct a logical proof, including by contradiction, induction, or contrapositive.
- Basic ideas about random variables: distribution, expectation, discrete and continuous random variables
- Basic rules of probability: conditional probability and Bayes rule, Bayes rule, law of total probability, marginals
- Set theory; the idea of of an object being in a set, one set being a subset of another, a function mapping one set to another, etc.
- Analysis: continuity, differentiability, limits, open and closed sets, countability
- The Greek alphabet, though I will avoid exotic letters (ξ , ζ , ψ , ν , η , v etc.) where possible.

We will cover the following ideas as we need them, so do not worry if you have not come across some of these ideas. However, some background and revision will help:

- Statistical learning theory: generalisation error, sample complexity, loss functions
- Neural networks: backpropagation, activation functions
- Probability and analysis: sigma-algebras, filtrations, function spaces
- Computer science: notions of asymptotic complexity,

Although the following are important (and sometimes very important) for understanding neural network theory, we will not cover them in detail in this course, or at least the elements we will cover are non-examinable.

- Analysis: Hilbert and Banach spaces, theory of operators
- Probability: stochastic processes, stochastic differential equations, Fokker-Planck equation, measure theory
- Computer science: understanding of GPUs, computability, multi-threaded processing, Python.

We will sometimes use examples from the natural sciences or other applications, but I will not assume any knowledge of these.

SG.6.2 Notation

This is a *rough* guide to notation we will use in the course: some will change! **If a definition in lecture notes is in conflict with a definition here, then please trust the lecture notes** (and let me know so I can fix it!)

Sets and set theory

- $\mathbb{R}$ denotes the real numbers
- $\{\cdot\}$ denotes a set: e.g. $\{1, 2, 3\}$ is the set containing the numbers 1,2,3
- $\{x : f(x)\}$ denotes the set of all values x such that condition(s) $f(x)$ hold; e.g. $\{x : 1 \leq x < 2\}$ is the set containing all real numbers greater than or equal to 1 and less than 2.
- $[\cdot]$ denotes a *closed* interval (includes its endpoints). So $[0, 1]$ means the set of real numbers x satisfying $0 \leq x \leq 1$.
- $(\cdot)$ denotes an *open* interval (does not include its endpoints). So $(0, 1)$ means the set of real numbers x satisfying $0 < x < 1$.
- $A \cup B$ indicates the union of sets A and B . So, if $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, then $A \cup B = \{1, 2, 3, 4, 5\}$.
- $A \cap B$ or indicates the intersection of sets A and B . So, if $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, then $A \cap B = \{3\}$.
- $A \setminus B$ indicates the difference of sets A and B . So, if $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, then $A \setminus B = \{1, 2\}$.
- $\emptyset$ denotes the empty set

Probability

- Unless otherwise stated, integrals are taken to be definite integrals from $-\infty$ to $+\infty$, so $\int f(x)dx = \int_{x=-\infty}^{x=\infty} f(x)dx$.
- Generally, capital letters will correspond to *events* or *random variables*, and lowercase letters to fixed values, but this is not always the case.
- X, Y indicates the event ‘event X AND event Y ’.
- $P(X)$ denotes the probability of the event X .
- $f_X(x)$ denotes the density of continuous random variable X evaluated at the value x
- $\mathbb{E}(X)$ denotes the expectation of random variable X .
- $\mathbb{E}_X(f(X, Y))$ denotes the expectation of quantity $f(X, Y)$, treating Y as constant.
- $\mathbb{E}_{X|Y=y}(f(X, Y))$ is the expectation of random variable $f(X, Y)$ where X is taken to have the distribution $X|Y = y$, and Y is fixed to value y .
- $\mathbb{E}_{X|Y}(f(X, Y))$ is the expectation of random variable $f(X, Y)$ where Y is treated as constant and X is taken to have the distribution $X|Y$.
- $N(\mu, \sigma^2)$ denotes the Gaussian or Normal distribution with mean μ and variance σ^2

Neural networks

- The subscript i will enumerate *inputs* to a neuron
- The subscript j will enumerate *neurons in a layer*
- The superscript (ℓ) will enumerate *layers*
- The letter w will normally denote a weight, so $w_{ij}^{(\ell)}$ is ‘the i th input to the j th neuron in the ℓ th layer’.
- We denote by $d^{(\ell)}$ the number of neurons in layer (ℓ) .

Information theory

- $\mathfrak{G}(X)$ denotes the σ -algebra generated by X
- $H(X)$, $H(X, Y)$, and $H(X|Y)$ denote the entropy, joint entropy and conditional entropy of random variables X and Y respectively. We will not differentiate entropy from differential entropy unless specified.
- $I(X, Y)$ denotes the mutual information between X and Y .

SG.7 General advice

Finally, a few things I think it is worth keeping in mind through the course:

1. Don't be intimidated by big formulas or complex-looking notation. There are lots of them in machine learning, and the underlying idea is often very simple.
2. It often takes a long time to write the setup for a simple question. Do not assume that a question is difficult just because a lot of things need to be specified before asking it.
3. This is a 'functional' area of mathematics, driven by what 'works' and what we 'need' rather than elegance or completeness. We will sometimes introduce an idea but not really use it.
4. If you feel like you don't understand something, a useful way to check whether you have it down is to try and implement it in R.

Overview of statistical learning

Reading list and references

[Hastie et al. \[2009\]](#), chapters 1-3.

[Bates et al. \[2023\]](#): a recent paper on cross-validation; not examinable.

1.1 Introduction

Welcome to DLAI4.

In this lecture, we will revise some concepts from the theory of statistical learning. From the point of view of practical machine learning and neural networks, this is probably the most important material we will cover!

We'll start by revising the idea of supervised and unsupervised machine learning. We will introduce the basic idea of a design matrix. We will look at the idea of a decision boundary, and what the optimal decision boundary is. We will then move on to the idea of generalisation error, and how we can approximate it.

1.2 Supervised and unsupervised learning

In the most general sense, we typically use machine learning to make predictions about things we can't observe. We usually have something we can observe - which, without defining further for the moment, we'll call X - and something we want to know about which we'll call Y . It's usually helpful to model X and Y as random variables, with some joint distribution X, Y .

Machine learning is typically subdivided into 'supervised' and 'unsupervised' learning (there's also 'reinforcement' learning, which is sort-of a combination of both). In supervised learning, we have some observations from the joint distribution of (X, Y) , from which we can try and draw some inferences about $P(Y|X)$. In unsupervised learning, we can only see X , and we are usually trying to find a simpler representation of X (which we might call X') which gives us a similar amount of information about Y ; that is, for which $P(Y|X) \approx P(Y|X')$. In practice, in both cases, we are trying to find a 'simpler' representation of X which preserves its useful properties.

In this course we will mostly be looking at supervised learning. But the boundary is blurrier than it immediately seems: the idea of 'simplifying' our object X , or distilling it into a useful form, is central to everything we will do, and we will look explicitly at information later on.

1.3 A simple example

We'll start by simulating a simple example. Let's say we're given two pieces of data (percent sugar and percent butter) about a bunch of cakes, and each cake is either chocolate or lemon, but we don't know which. Our job is to try and predict whether the cake is chocolate or lemon from the percentage of sugar and butter. We will

Let's imagine that in truth all of our cakes come from one of five recipes, each of which have a set proportion of butter and sugar:

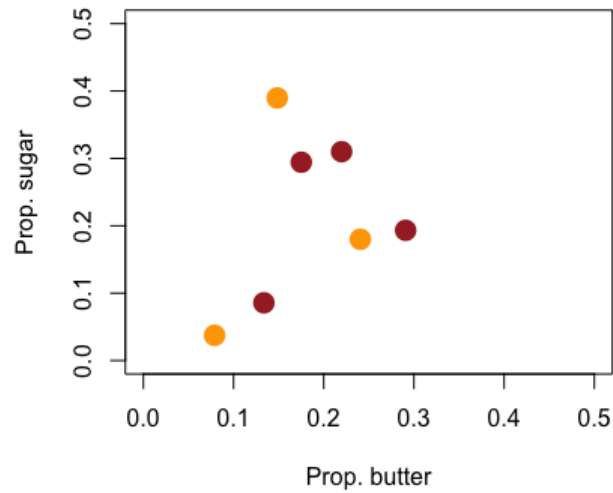


Figure 1.1: Means of butter and sugar proportions in each recipe

so, amongst all cakes made to a particular recipe, the mean amount of butter and sugar will be the corresponding dot. For a cake in a given recipe, let's say the proportions have a Gaussian distribution with covariance matrix $\frac{1}{10}I_2$ around the mean. Let's say we have a number of cakes which happen to be in equal proportion between the recipes. So our actual data look a bit like this

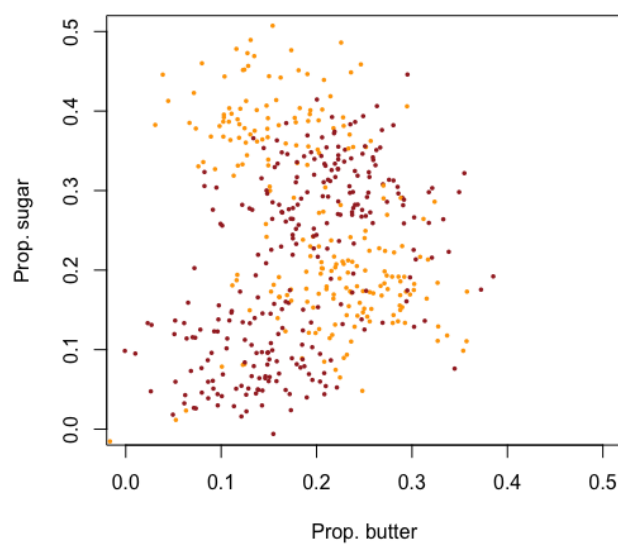


Figure 1.2: Actual sugar proportions in each recipe

1.4 Data matrices

We will typically assume that we begin with a *design matrix* comprising our observations of (X, Y) . The rows of the data matrix are *samples*, and the columns are *variables*.

Sample	Prop. sugar	Prop. butter	Type
1	0.21	0.35	Chocolate
2	0.33	0.31	Chocolate
3	0.24	0.18	Lemon
4	0.28	0.38	Lemon
4	0.28	0.38	Chocolate
...	...	...	...

We will generally assume that the rows are independent and identically distributed observations of a random variable (X, Y) . If X_1 is the random variable representing the proportion of sugar, and X_2 the random variable representing the proportion of butter, then we denote the PDF of $X_1, X_2 | Y = \text{chocolate}$ at the value (x, y) by

$$f_{X_1, X_2 | Y = \text{choc}}(x_1, x_2) \quad (1.1)$$

We can actually state exactly what this function is in this case (left as an exercise) but we will not worry too much about that for the moment. We will also want to keep track of the PDF of $X_1, X_2 | Y = \text{lemon}$:

$$f_{X_1, X_2 | Y = \text{lem}}(x_1, x_2) \quad (1.2)$$

Note that $f_{X_1, X_2 | Y = \text{choc}}(x_1, x_2)$ is the density of the brown points above, and $f_{X_1, X_2 | Y = \text{lem}}(x_1, x_2)$ the density of the orange.

1.5 Decision rules

Let's say that we can observe the above design matrix. We want to go from observations X_1, X_2 to 'predicted' outcomes $\hat{Y} = f(X_1, X_2)$, by means of some function f , which we get from our data. We note that, since we learn f from the samples in the data matrix, which we are modelling as random variables, we may also consider f itself a random variable. Most of this course will be concerned with how we go from the design matrix to this function f , in a general sense.

We measure how 'good' our function is by means of some similarity function $C(Y, \hat{Y})$, which tells us how 'similar' Y (the right answer) is to $\hat{Y}$ (the predicted answer). For some class of functions F , a sensible thing to try and minimise is the *generalisation error*:

$$GE(f) = \mathbb{E}_{X, Y} \{C(Y, f(X))\} \quad (1.3)$$

also called by various other names. We term the infimum $\inf_f GE(f)$ the 'Bayes loss' and any f achieving this a 'Bayes decision'.

1.6 Special case: the 0-1 loss function

Let us firstly look at a simple form of C when we have discrete Y : the 0-1 loss function. We will take our prediction $\hat{Y}$ as also being discrete; in fact, we will look at the very simple case when Y can take only two values, which we will call 0 and 1. We consider the 0-1 loss:

$$C(y, \hat{y}) = \begin{cases} 0 & \text{if } y = \hat{y} \\ 1 & \text{if } y \neq \hat{y} \end{cases} \quad (1.4)$$

Since we are making an estimate $\hat{Y} = f(x)$ for each point x , we are effectively partitioning the space of possible x s into two parts: those which we assign 0, and those which we assign 1.

In this case, we have the following theorem:

Theorem 1. Suppose that we have $Y \in \{0, 1\}$ and $X \in \mathbb{R}$, and both $X|Y = 1$ and $X|Y = 0$ admit probability density functions (denoted $f_{X|Y=1}$ and $f_{X|Y=0}$ respectively). Denote $\alpha = P(Y = 1)$. For 0-1 loss, the generalisation error is minimised when (almost surely):

$$f(x) = \mathbf{1}(\alpha f_{X|Y=1}(x) \leq (1 - \alpha)f_{X|Y=0}(x))$$

Proof. We have

$$\begin{aligned} GE(f) &= \mathbb{E}_{X,Y} \{C(Y, f(X))\} \\ &= \mathbb{E}_Y \{ \mathbb{E}_{X|Y} \{C(Y, f(X))\} \} \\ &= \mathbb{E}_Y \{ \mathbb{E}_{X|Y} \{Y f(X) + (1 - Y)(1 - f(X))\} \} \\ &= P(Y = 1) \mathbb{E}_{X|Y=1} \{f(X)\} + P(Y = 0) \mathbb{E}_{X|Y=0} \{1 - f(X)\} \\ &= \alpha \mathbb{E}_{X|Y=1} \{f(X)\} + (1 - \alpha) \mathbb{E}_{X|Y=0} \{1 - f(X)\} \\ &= \int (\alpha f(x) f_{X|Y=1}(x) + (1 - \alpha)(1 - f(x)) f_{X|Y=0}(x)) dx \end{aligned}$$

Considering $f(x)$ must be 0 or 1, this is minimised when for almost all x the function $f(x)$ takes whichever of 0, 1 minimises this expression. That is, the generalisation error is minimised when:

$$\begin{aligned} f(x) &= \begin{cases} 1 & \text{if } \alpha f_{X|Y=1}(x) \leq (1 - \alpha)f_{X|Y=0}(x) \\ 0 & \text{if } \alpha f_{X|Y=1}(x) > (1 - \alpha)f_{X|Y=0}(x) \end{cases} \\ &= \mathbf{1}(\alpha f_{X|Y=1}(x) \leq (1 - \alpha)f_{X|Y=0}(x)) \end{aligned}$$

□

The theorem informally states that:

Under a 0-1 loss, the optimal decision boundary is a contour of the ratio of PDFs $f_{X|Y=0}(x)/f_{X|Y=1}(x)$

Let's apply this to our lemon/chocolate example. The optimal decision boundary is given by a contour of the ratio of the PDFs above; that is, given a proportion of sugar x_1 and a proportion of butter x_2 we will minimise the 0-1 error of our guess if we guess 'chocolate' when:

$$\alpha f_{X_1, X_2|Y=\text{choc}}(x_1, x_2) \geq (1 - \alpha) f_{X_1, X_2|Y=\text{lem}}(x_1, x_2)$$

where $\alpha = P(Y = \text{choc})$. Our decision boundary is the set of points for which:

$$\frac{f_{X_1, X_2|Y=\text{choc}}(x_1, x_2)}{f_{X_1, X_2|Y=\text{lem}}(x_1, x_2)} = \frac{1 - \alpha}{\alpha}$$

that is, a contour of $g(x_1, x_2)$, where

$$g(x_1, x_2) = \frac{f_{X_1, X_2|Y=\text{choc}}(x_1, x_2)}{f_{X_1, X_2|Y=\text{lem}}(x_1, x_2)}$$

On our plot, this looks like:

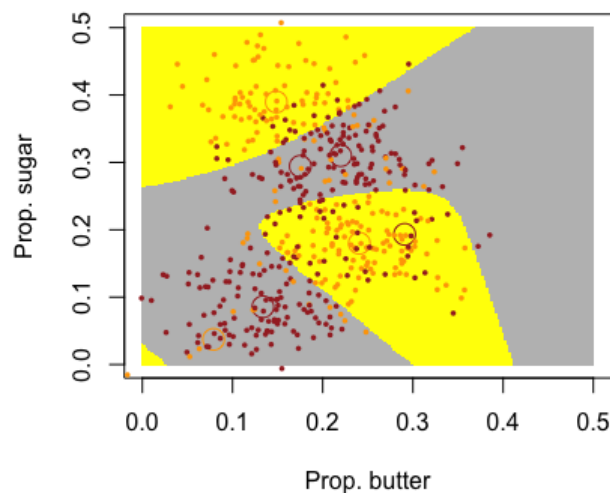


Figure 1.3: Optimal decision boundary

1.7 Training and test error

This is all very well, but we don't generally know the PDFs $f_{X_1, X_2|Y}$ in advance. The PDF $f_{X,Y}(x,y)$ in fact tells us everything about how X and Y are distributed: it is our object of inference, and if we knew it we would be done, but to even calculate the generalisation error, we need to know it!

If we have got some data, such as in our example above, and use it to choose some function $f(x)$, which hopefully has a low $C(Y, f(x))$, how do we go about assessing its generalisation error?

It is tempting, given data X_i, Y_i ($i \in 1..n$), where $(X_i, Y_i) \sim (X, Y)$, to say that:

$$GE(f) = \int C(X, f(X)) f_{X,Y}(x,y) dx dy \approx \frac{1}{n} \sum_{i=1}^n C(y_i, f(x_i)) \quad (1.5)$$

which is called the 'training error'. In fact, this looks unbiased, since we would expect that

$$\mathbb{E} \left\{ \frac{1}{n} \sum_{i=1}^n C(y_i, f(x_i)) \right\} = \frac{1}{n} \sum_{i=1}^n \mathbb{E} \{ C(y_i, f(x_i)) \} = \frac{1}{n} \sum_{i=1}^n \mathbb{E} \{ C(Y, f(X)) \} = \frac{1}{n} n GE(f) = GE(f) \quad (1.6)$$

but alas, this is wrong, since f depends on X (see exercises). Trying to minimise the training error leads to the familiar problem of *overfitting*.

In order to make the above argument work, a common approach is to divide our training sample into *training* and *testing* datasets, and only use the training dataset to decide our function, which we will call f_{train} . Now f_{train} is independent of the testing dataset, so if our test dataset is those of (x_i, y_i) for which i is in some set S , we have:

$$\mathbb{E} \left\{ \frac{1}{|S|} \sum_{i \in S} C(y_i, f_{train}(x_i)) \right\} = \frac{1}{|S|} \sum_{i \in S} \mathbb{E} \{ C(Y, f_{train}(X)) \} = \frac{1}{|S|} |S| GE(f_{train}) = GE(f_{train}) \quad (1.7)$$

so we have an important (but simple) theorem:

Theorem 2. Given samples X_i, Y_i , i in $1..n$, which are independent and identically distributed as $(X_i, Y_i) \sim (X, Y)$, if we designate a 'train set' $T \subset \{1, 2, \dots, n\}$ and a 'test set' $S = \{1, 2, \dots, n\} \setminus T$, and fit a function f_{train} only to

the training set, then the test error, defined as:

$$SE(f_{train}) = \sum_{i \in S}^{|S|} C(y_i, f_{train}(x_i)) \quad (1.8)$$

is an unbiased estimate of the generalisation error of f_{train} , called $GE(f_{train})$. The training error, defined as

$$TE(f_{train}) = \sum_{i \in T}^{|T|} C(y_i, f_{train}(x_i)) \quad (1.9)$$

is generally not an unbiased estimator of $GE(f_{train})$.

Proof. As above □

However, we usually want to use all our data to decide our function f . If f_{train} is the f we got using *only* the training data, then we then are making another approximation: we are saying that $GE(f_{train})$ (which we can estimate in an unbiased way with $SE(f_{train})$) is the same as $GE(f)$, which it generally is not.

How big should S be?

1.7.1 Bias-variance tradeoff

This brings us to the famed bias-variance tradeoff (which I will not look at very formally). If S is big, then we have a lot of samples on which to evaluate the generalisation error of f_{train} , so its variance will be small. This means that $SE(f_{train})$ is close to $GE(f_{train})$.

If T is big, then the function f_{train} will generally be more like the actual function f , since we're using 'almost' the whole dataset to train it. So $GE(f_{train})$ will be close to $GE(f)$; that is, its bias as an estimator of $GE(f)$ will be small.

We really want to know $GE(f)$. But all we can actually calculate is $SE(f_{train})$. If we make S bigger, we get $SE(f_{train})$ 'closer' to $GE(f_{train})$ in a sense, but we also move $GE(f_{train})$ further from $GE(f)$. This is the bias-variance tradeoff.

1.8 Sample complexity

We are now going to take a brief foray into the large and complex field of statistical learning theory. We are going to briefly describe an idea called 'sample complexity', governing (essentially) how difficult it is to learn a particular thing.

We will revisit this in the context of information theory later on.

Definition 1 (Learnability and sample complexity). Suppose we have:

- Some set $\mathcal{X}$ from which data can come, and 'labels' $\mathcal{Y}$ which can be either 0 or 1.
- A set of 'hypothesis' functions $\mathcal{F}$ which map $\mathcal{X}$ to $\{0, 1\}$, from which we will choose a function f .
- For a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, and a function $f \in \mathcal{F}$, we consider the generalisation error using 0-1 loss:

$$GE(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}} \{C(Y, f(X))\} = \mathbb{E}_{(X,Y) \sim \mathcal{D}} \{1\{f(X) \neq Y\}\} = P_{\mathcal{D}}(f(X) \neq Y)$$

- We have a 'learning algorithm' $\mathcal{A}$ which takes some number of samples from $(\mathcal{X} \times \mathcal{Y})$ and returns a function $f \in \mathcal{F}$.

We say that $\mathcal{A}$ is 'consistent', and $\mathcal{F}$ is 'learnable', if, given $\epsilon, \delta > 0$, we can find a large enough n such that if we have at least n training samples $D^n \sim \mathcal{D}^n$, then

$$P_{D^n \sim \mathcal{D}^n} \left(GE(\mathcal{A}(D^n)) - \inf_{f' \in \mathcal{F}} GE(f') > \epsilon \right) < \delta$$

The smallest $N = N(\mathcal{D}, \epsilon, \delta)$, considered as a function of $\mathcal{D}, \epsilon, \delta$, for which this holds, is called the *sample complexity* of $(\mathcal{A}, \mathcal{F})$.

We are concerned with the ‘agnostic’ learnability; that is, the worst possible sample complexity over all possible distributions $\mathcal{D}$ over $(\mathcal{X} \times \mathcal{Y})$:

$$N(\epsilon, \delta) = \max_{\mathcal{D}} N(\mathcal{D}, \epsilon, \delta)$$

We then define the agnostic learnability as follows:

Definition 2 (Agnostic learnability). A set of functions $\mathcal{F}$ which map $\mathcal{X}$ to $\mathcal{Y}$ is *agnostically learnable* if there exists a learning algorithm $\mathcal{A}$ such that $N(\epsilon, \delta)$ is finite for all $\epsilon, \delta \in (0, 1)^2$.

A few notes:

- We cannot necessarily make the generalisation error arbitrarily low. It may be that for some value x , we have $P_{\mathcal{D}}(Y|X = x) = 1/3$, but since our fitted function $f = \mathcal{A}(D^n)$ must assign 0 or 1 to the value x , we will be wrong at least $1/3$ of the time.
- We usually will use ‘empirical risk minimisation’ for our algorithm $\mathcal{A}$; that is, given data $D^n = \{(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)\}$ we set:

$$\mathcal{A}(D^n) = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n C(f(X_i), Y_i)$$

This will just about always suffice, so we will consider sample complexity to essentially depend only on $\mathcal{F}$.

Exercises

1. Suppose that there are n_{lem} lemon cake recipes, the i th recipe is used with frequency f_i , and the i th recipe has an average proportion of sugar s_i and an average proportion of butter b_i . Whenever someone makes a cake, they deviate from the average proportions of sugar and butter in that recipe so that the proportion has standard deviation σ . Give an expression for the sugar and butter proportions of a randomly-chosen lemon cake. (hint: this is a mixture Gaussian)

2. If $C(y, \hat{y}) = (y - \hat{y})^2$, and we have

$$\begin{aligned} X &\sim N(0, 1) \\ (Y|X = x) &\sim N(ax, 1) \end{aligned} \tag{1.10}$$

where $N(\mu, \sigma^2)$ is the normal distribution with mean μ and variance σ^2 , then:

- (a) Write the joint unconditional PDF $f_{X,Y}(x, y)$ of X, Y
- (b) For a predictor $f(x) = bx$, find an expression for the generalisation error of f
- (c) Show that this generalisation error is minimised when $b = a$.

The general result that, if $B^2 - 4AC < 0$

$$\int_{\mathbb{R}^2} \frac{1}{2\pi} \exp\left(\frac{1}{2}(Ax^2 - Bxy + Cy^2)\right) (Py - Qx)^2 dx dy = \frac{8\sqrt{-(B^2 - 4AC)}(AP^2 - BPQ + CQ^2)}{(B^2 - 4AC)^2} \tag{1.11}$$

will be of help.

3. Where does the argument in derivation 1.6 go wrong?
4. Suppose we have a fixed data set (X_i, Y_i) , $i \in 1..n$, with training dataset $T \subseteq \{1, 2, \dots, n\}$. We have to choose an f from a set of possible functions $\{f_t\}$, indexed by $t \in \Theta$ (effectively, Θ represents the space of possible parameters of f). Let us say that we restrict our choice of f to some subset $\theta \subset \Theta$, and choose our f_t to minimise the training error, to get f_θ ; that is:

$$f_\theta = \arg \min_{\{f_t: t \in \theta\}} TE(f_t) \tag{1.12}$$

and we will presume this minimum exists. Suppose we have $\theta_1 \subset \theta_2 \subset \Theta$. Show that $TE(f_{\theta_2}) \leq TE(f_{\theta_1})$. Hint: this is far simpler than it looks.

5. For a fixed distribution $\mathcal{D}$, show that the sample complexity $N(\mathcal{D}, \epsilon, \delta)$ is non-increasing in both ϵ and δ .

Solutions

1. Let us model the proportions of sugar and butter in a randomly-chosen lemon cake by the random variable (S, B) . Let's call the recipe associated with a random choice of lemon cake a random variable R (which has a multinomial distribution with frequencies $(f_1, f_2, \dots, f_{n_{\text{lem}}})$). We may not actually observe R .

If we happen to know that a sample uses the r th recipe, so $R = r$, then (S, B) follow a bivariate Gaussian, in distribution with mean (s_i, b_i) and variance $\begin{pmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{pmatrix}$. So the PDF of the random variable $(S, B)|R = r$ at values (s, b) is given by:

$$f_{S,B|R=r}(s, b) = \frac{1}{2\pi} \exp \left(-\frac{1}{2} \left(\left(\frac{s - s_i}{\sigma} \right)^2 + \left(\frac{b - b_i}{\sigma} \right)^2 \right) \right)$$

Now the unconditional density of (S, B) at (s, b) is given by:

$$\begin{aligned} f_{S,B}(s, b) &= \sum_{i=1}^{n_{\text{lem}}} f_{S,B,R=i}(s, b) \\ &= \sum_{i=1}^{n_{\text{lem}}} f_{S,B|R=i}(s, b) P(R = i) \\ &= \sum_{i=1}^{n_{\text{lem}}} f_{S,B|R=i}(s, b) f_i \\ &= \sum_{i=1}^{n_{\text{lem}}} \frac{1}{2\pi} \exp \left(\left(\frac{s - s_i}{\sigma} \right)^2 + \left(\frac{b - b_i}{\sigma} \right)^2 \right) f_i \end{aligned}$$

2. (a) We have

$$\begin{aligned} f_{X,Y}(x, y) &= f_{Y|X}(x, y) f_X(x) \\ &= \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2} (y - ax)^2 \right) \times \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2} x^2 \right) \\ &= \frac{1}{2\pi} \exp \left(-\frac{1}{2} ((y - ax)^2 + x^2) \right) \end{aligned}$$

- (b) The generalisation error is:

$$\begin{aligned} GE(f) &= \mathbb{E}_{X,Y} [C(Y, f(X))] \\ &= \int_{\mathbb{R}^2} (y - f(x))^2 f_{X,Y}(x, y) dx dy \\ &= \int_{\mathbb{R}^2} (y - bx)^2 f_{X,Y}(x, y) dx dy \\ &= \int_{\mathbb{R}^2} (y - bx)^2 \frac{1}{2\pi} \exp \left(-\frac{1}{2} ((y - ax)^2 + x^2) \right) dx dy \\ &= 1 + (a - b)^2 \end{aligned}$$

using the formula above, with $A = 1 + a^2$, $B = 2a$, $C = 1$, $P = 1$, $Q = b$, so $B^2 - 4AC = -4 < 0$.

- (c) The quantity $1 + (a - b)^2$ is clearly minimised for $b = a$

3. The step:

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E} \{C(y_i, f(x_i))\} = \frac{1}{n} \sum_{i=1}^n \mathbb{E} \{C(Y, f(X))\}$$

is wrong. Although we can expand the expectation over the sum (as in the first step), the expectation is not over only x_i, y_i , since f depends on all the other (x_j, y_j) s too. It is more obvious if we write it out more explicitly with what the expectation is over:

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{(x_1, y_1), (x_2, y_2) \dots (x_n, y_n)} \{C(y_i, f(x_i))\}$$

and note that it depends on all of these, not just x_i, y_i . It is generally worth explicitly saying what an expectation is over.

4. Suppose that, across $t \in \theta_1$, the value $TE(f_t)$ is minimised at $t = t_1$. Since $\theta_1 \subset \theta_2$, we have $t_1 \in \theta_2$, and hence

$$f_{\theta_2} = \arg \min_{\{f_t: t \in \theta_2\}} TE(f_t) \leq f_{t_1} = \arg \min_{\{f_t: t \in \theta_1\}} = f_{\theta_1}$$

as required.

5. Suppose that we have $N(\mathcal{D}, \epsilon_1, \delta_1)$ training samples, and $\epsilon_2 \geq \epsilon_1$, $\delta_2 \geq \delta_1$. Since $\epsilon_1 \leq \epsilon_2$, we have:

$$P_{D^n \sim \mathcal{D}^n} \left(GE(\mathcal{A}(D^n)) - \inf_{f' \in \mathcal{F}} GE(f') > \epsilon_2 \right) \leq P_{D^n \sim \mathcal{D}^n} \left(GE(\mathcal{A}(D^n)) - \inf_{f' \in \mathcal{F}} GE(f') > \epsilon_1 \right) < \delta_1 \leq \delta_2$$

so with this many samples, we have:

$$P_{D^n \sim \mathcal{D}^n} \left(GE(\mathcal{A}(D^n)) - \inf_{f' \in \mathcal{F}} GE(f') > \epsilon_2 \right) < \delta_2$$

and hence the minimum number of samples $N(\mathcal{D}, \epsilon_2, \delta_2)$ that we need for for this to happen is smaller than $N(\mathcal{D}, \epsilon_1, \delta_1)$

Introduction to the neuron

Reading list and references

[Calin, 2020, Chapters 1,5]

2.1 Introduction

In this lecture, we will go over the idea of a ‘neuron’ as an information processor, and a couple of basic properties. We will start with a cursory look at biological neuron, and the relevant elements. We will then revise the idea of a neuron as representing a generalised linear model (GLM), in which the ‘link’ function is re-cast as the ‘activation’ function and the constant term as a ‘bias’ input.

This lecture starts with only simple material. That is not just to ease you in to this: neural networks are, at their core, simple constructs from which complex behaviours can emerge. In lectures 4-5 we will begin to look at the class of functions we can approximate with neural networks.

2.2 Neurons

2.2.1 Biological neurons

A neuron is a cell which contains several *dendrites* and a single *axon*. The axon of one cell can connect to one or more dendrites of other cells (the connection is called a *synapse*).

Neurons process electrical signals. A signal passes from axon to dendrite in a stronger or weaker sense (which we may describe by a *weight*). The signals entering the dendrites of a neuron are collated by the neuron and *added together*. If this sum reaches a given threshold, the neuron will send an electrical signal through its axon (figure 2.1). The threshold for firing varies across neurons.

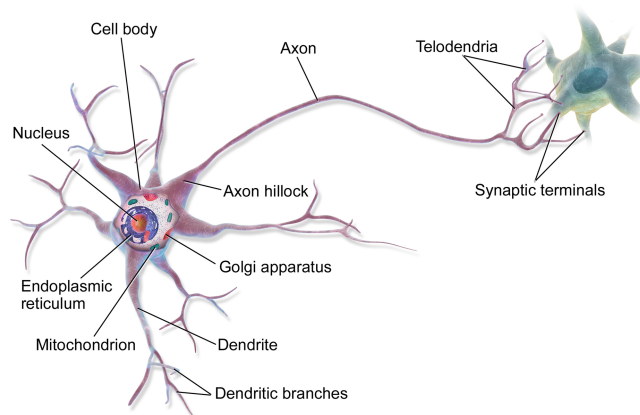


Figure 2.1: A simple schematic of a neuron. Reproduced from [Wikipedia](#)

We will disregard timing considerations, and denote the input to the i th dendrite as x_i . Suppose that the synapse at that dendrite results in applying a weight w_i , so that an electrical pulse of strength $w_i x_i$ travels down the dendrite (it is a bit more complex than this: ‘strength’ is partly voltage and partly frequency of ‘action potentials’, but this does not matter for our purposes)

Let’s denote by $H(x)$ (H here stands for ‘Heaviside step function’) the function

$$H(x) \triangleq \mathbf{1}_{x \geq t} \quad (2.1)$$

for some threshold t . Then we can write the output of the neuron (that is, the strength of the pulse coming down the axon) as

$$H\left(\sum_i w_i x_i\right)$$

where the sum is over all dendrites.

2.2.2 The abstract neuron

The abstract object above (taken as a function of the values x_i) is called a *perceptron*.

We may also wish to include a constant term w_0 in the sum above. When considered as a neuron, we usually call this the ‘bias’, which can be conceptualised as a dendrite with weight w_0 attached to an axon which is ‘always on’.

We arrive at our abstract model of a neuron (lifted from [Calin \[2020\]](#)), illustrated in figure 2.2:

Definition 3. An abstract neuron is a tuple $(\mathbf{x}, \mathbf{w}, \phi, y, b)$, where $\mathbf{x}^T = (x_1, \dots, x_n)$ is the input vector, $\mathbf{w}^T = (w_1, \dots, w_n)$ is the weights vector and ϕ is an activation function that defines the outcome function according to

$$y = \phi(\mathbf{x}^T \mathbf{w} - b) = \phi\left(\sum_{i=1}^n w_i x_i - b\right) \quad (2.2)$$

Sometimes we instead consider a neuron as a quadruple $(\mathbf{x}, \mathbf{w}, \phi, y)$ with $\mathbf{x}^T = (x_0, x_1, \dots, x_n)$ the input vector, $\mathbf{w}^T = (w_0, w_1, \dots, w_n)$ the weight vector, with $x_0 = -1$ and $w_0 = b$ (the bias), and

$$y = \phi(\mathbf{x}^T \mathbf{w}) = \phi\left(\sum_{i=1}^n w_i x_i - b\right) = \phi\left(\sum_{i=0}^n w_i x_i\right) \quad (2.3)$$

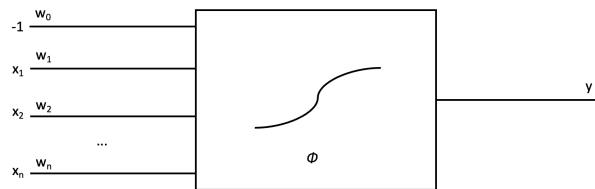


Figure 2.2: A simple schematic of an abstract neuron with n inputs with weights $w_1, w_2, \dots, w_n$ and activation function ϕ

Those who have done some control theory or courses on computer architecture may recognise that this looks similar to a circuitry symbol for a logic gate (figure 2.3)

Logic Gates

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR																																																																																																
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A + B$	$\overline{A + B}$	$A \oplus B$	$\overline{A \oplus B}$																																																																																																
Symbol																																																																																																							
Truth Table	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	0	1	0	0	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	1
A	X																																																																																																						
0	1																																																																																																						
1	0																																																																																																						
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					

Figure 2.3: Diagrams and specifications for logic gates (American National Standards Institute symbols)

This reflects a philosophical similarity, in which we build a complex function out of simple elements. We can quickly see that we can model logic gates using pseudo-biological ‘neurons’ with a step function activation; for instance, if we have a neuron with two dendrites for which $w_1 = w_2 = 1$, and which uses a threshold function (expression 2.1) with $t = 0$ and a bias $b = 1/2$, then the output of our neuron is:

$$\begin{aligned}
 H\left(\sum_i w_i x_i - b\right) &= H\left(x_1 + x_2 - \frac{1}{2}\right) \\
 &= \mathbf{1}_{x_1 + x_2 \geq \frac{1}{2}} \\
 &= (x_1 \text{ OR } x_2)
 \end{aligned}$$

As opposed to logic gates, we allow the inputs to a neuron to be real numbers (for instance, pixel intensities in an image). However, they will often come from a restricted set (e.g, integers, or $\{0, 1\}$). An interesting consideration is - suppose we have n dendrites, and we allow that the input can take β possible values. For a binary input, $\beta = 2$ (0 or 1); for a single-digit numerical input, $\beta = 10$ (0 to 9); for a real in $[0, 1]$ to q decimal digits, $\beta = 10^q$. We typically have many neurons and we want to code them as efficiently as possible. How do we get the most ‘bang for our buck’?

We want to be able to represent as many possible different inputs with our n dendrites with β states. We can represent β^n different inputs, which we want to be big. We can presume the computational cost of storing the neuron is

$$\text{Cost} = cn\beta \quad (2.4)$$

that is, cost scales linearly with the number of dendrites and the number of states a dendrite can be in. For a fixed cost, the number of states we can represent is:

$$\beta^n = \beta^{\frac{\text{Cost}}{c\beta}} = \left(\beta^{\frac{1}{\beta}}\right)^{\frac{\text{Cost}}{c}} \quad (2.5)$$

which is monotonic to $\beta^{1/\beta}$, which is maximised at $\beta = e = \exp(1)$. More importantly, $\beta^{1/\beta}$ for integral β is highest for $\beta = 3$, so using three-state inputs is most efficient.

2.2.3 Neurons as regressions

Let us replace H with an arbitrary function ϕ called the *activation function*.

$$\phi\left(\sum_i w_i x_i - b\right) \quad (2.6)$$

This immediately suggests a parallel with *generalised linear models* (GLMs). To revise, a GLM describes a set of dependent variables $\mathbf{Y}$ depending on a set of independent variables $\mathbf{X} = (x_1, x_2, \dots)$ via a coefficient vector $\mathbf{W} = (w_1, w_2, \dots)$ and an invertible function g according to:

$$\begin{aligned}\mathbb{E}(\mathbf{Y}|\mathbf{X}) &= g^{-1}(\mathbf{X} \cdot \mathbf{W}) \\ &= g^{-1}\left(\sum_i w_i x_i\right)\end{aligned}\tag{2.7}$$

whence the class of functions of x_i described by expression 2.6 is exactly that class of functions modelled by a GLM with link function $g = \phi^{-1}$. If we consider the sigmoid link function:

$$\phi(x) = g^{-1}(x) = \frac{1}{1 + e^{-x}} \triangleq \sigma(x)\tag{2.8}$$

we arrive at the familiar mean function used for logistic regression. When we are fitting a linear model or generalised linear model, we usually want to allow a calibrating ‘intercept’ term, which corresponds to the ‘bias’ in a neuron.

We consider the partial derivative of the output. This will be important when we look later at back-propagation.

Remark 1. The differential dy (a small change in y) can be expressed as

$$\begin{aligned}dy &= \sum_{i=1}^n \frac{\partial y}{\partial w_i} dw_i + \frac{\partial y}{\partial b} db \\ &= \sum_{i=1}^n \phi'(\mathbf{w}^T \mathbf{x} - b) x_i dw_i + \phi'(\mathbf{w}^T \mathbf{x} - b) db \\ &= \phi'(\mathbf{w}^T \mathbf{x} - b) \left(\sum_{i=1}^n x_i dw_i - db \right)\end{aligned}$$

hence

$$\begin{aligned}\nabla y &= \left(\frac{\partial y}{\partial w_1}, \frac{\partial y}{\partial w_2}, \dots, \frac{\partial y}{\partial b} \right) \\ &= \phi'(\mathbf{w}^T \mathbf{x} - b) \left(\sum_{i=1}^n x_i \frac{dw_i}{dw_1} - \frac{db}{dw_1}, \sum_{i=1}^n x_i \frac{dw_i}{dw_2} - \frac{db}{dw_2}, \dots, \sum_{i=1}^n x_i \frac{dw_i}{db} - \frac{db}{db} \right) \\ &= \phi'(\mathbf{w}^T \mathbf{x} - b) (x_1, x_2, \dots, -1)\end{aligned}$$

2.3 Activation functions

We will take a brief look at some common activation functions in use in realistic neural networks.

1. The *Heaviside* activation function:

$$H(x) = 1_{x>0}$$

2. The *Rectified Linear Unit (ReLU)* activation function:

$$ReLU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

3. The *Parametric Rectified Linear Unit (ReLU)* activation function:

$$PReLU(x) = \begin{cases} \alpha x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

for some $\alpha > 0$

4. The *logistic* activation function

$$\sigma(x) = \frac{1}{1 + e^{-cx}}$$

for some $c > 0$

5. The *SoftPlus* activation function:

$$sp(x) = \ln(1 + e^x)$$

2.3.1 Discriminatory activation functions

In order to get to some of our later results on universal approximation, we need to define a *discriminatory activation function* and prove a lemma regarding such functions.

We will describe a simple (but incomplete) definition of discriminatory activation functions. We will leave the (complex) complete description out of this course, since it involves measure theory, but it is fundamentally similar.

2.3.2 Simple discriminatory activation functions

Definition 4. Let $F = C[0, 1]^n$; the set of continuous functions $f : [0, 1]^n \rightarrow \mathbb{R}$. An activation function ϕ is n -discriminatory only if the only $f \in F$ for which:

$$\int_{I_n} \phi(w^t x + \theta) f(x) dx = 0$$

for all $w \in \mathbb{R}^n$ and $\theta \in \mathbb{R}$, is $f(x) = 0$.

Note we have said ‘only if’ and not ‘if and only if’; this is because we are using a simplified description (see [Calin \[2020\]](#) for the full definition). Alternatively: ϕ is *not* discriminatory if there exists $f \in F$ which is not $f(x) = 0$ such that for all $w \in \mathbb{R}^n$ and $\theta \in \mathbb{R}$ we have

$$\int_{I_n} \phi(w^t x + \theta) f(x) dx = 0$$

As an example: the function $\phi(x) = ax + b$ for $a, b \in \mathbb{R}$ is not discriminatory; the function $f(x) = 6x^2 - 6x + 1$ satisfies

$$\int_0^1 f(x) dx = 0 \text{ and } \int_0^1 x f(x) dx = 0$$

so

$$\int_0^1 \phi(w^t x + \theta) f(x) dx = \int_0^1 (a(wx + \theta) + b) f(x) dx = aw \int_0^1 x f(x) dx + (a\theta + b) \int_0^1 f(x) dx = 0$$

hence ϕ is not 1-discriminatory, and not discriminatory.

2.3.3 Why are we interested in this?

The first ‘universal approximation theorems’ for neural networks were proved for neurons with discriminatory activation functions. But why should this curious and unintuitive looking property matter for representation?

To give a rough reason as to why it matters, we recall an idea from the Calculus of Variations (in fact, a result sometimes called the ‘fundamental lemma of the Calculus of Variations’).

Theorem 3 (Fundamental Lemma of the Calculus of Variations). Let G be the set of smooth functions on $(0, 1)^1$. Then for a continuous function f , if:

$$\int_0^1 f(x) g(x) dx = 0 \text{ for all } g \in G, \text{ then } f(x) = 0$$

In the calculus of variations, we are wanting to find a *function* optimising something, so we want to consider a *large* space of functions representing essentially ‘every reasonable function’. The set of functions G in the lemma is the space of functions we are searching.

Consider an equivalent version of the definition of discriminatory:

¹Also must have compact support, but this is unimportant

Definition 5 ((Simple) discriminatory activation function). Let Φ be the set of functions $\phi(wx + \theta)$ for $w, \theta \in \mathbb{R}$, and ϕ discriminatory. Then for a continuous function f , if:

$$\int_0^1 f(x)p(x)dx = 0 \text{ for all } p \in \Phi, \text{ then } f(x) = 0$$

This can be seen to be almost analogous. In a sense, by discriminatory, we mean that the space Φ *behaves* like the space G , for the purposes of the lemma; that is, *as w and θ range over the real numbers, the function $\phi(wx + \theta)$ represents a large space of functions*. Since a universal approximation theorem looks at whether we can represent an arbitrarily complex function using compositions of ϕ , it makes sense that we would want a property like this to hold.

2.3.4 Sigmoid functions

Despite our counterexamples, it seems intuitively clear that most commonly used activation functions such as those above are discriminatory, but it is not obvious how to prove this. We will prove a useful theorem to show that certain activation functions are discriminatory.

Theorem 4. A sigmoid link function $\phi(x)$ is a function which satisfies:

$$\lim_{x \rightarrow -\infty} \phi(x) = 0 \quad \text{and} \quad \lim_{x \rightarrow \infty} \phi(x) = 1 \quad (2.9)$$

Any continuous sigmoid activation function is discriminatory

Proof. See [Calin, 2020, chapter 2]

□

2.3.5 ReLU activation functions

We finally note the following:

Theorem 5. The *ReLU* function is 1-discriminatory.

The proof is left as an exercise.

Exercises

1. Specify weights and bias for a single neuron to implement the AND function.
2. Suppose that, rather than equation 2.4 cost is respecified as

$$\text{Cost} = c(n + c_1)(\beta + c_2) \quad (2.10)$$

for some c, c_1, c_2 . If $c = 1, c_1 = 2, c_2 = 2, k = 100$, how many states should dendrites have in order to maximise the number of representable inputs for a fixed cost?

3. Plot the activation functions listed in section 2.3. Note whether they are bounded, bounded below, continuous, differentiable, or smooth.
4. Show that the counterexample above gives a $\phi(x)$ which is *not* discriminatory and not 0 everywhere. Hint: let $f(x) = (6x^2 - 6x + 1)$.
5. Find (a_0, b_0, a_1, b_1) such that

$$f(x) = \text{ReLU}(a_0x + b_0) - \text{ReLU}(a_1x + b_1)$$

is sigmoidal.

6. Prove theorem 5. Use the following steps (from Guilhoto [2018], but please have a go at this before looking up the answer). You may presume that ‘only if’ is ‘if and only if’ in the definition of discriminatory activation functions (the argument is virtually identical).

- (a) Assume that we have a function $\mu(x)$ such that for all $y, \theta \in \mathbb{R}$

$$\int \text{ReLU}(yx + \theta)\mu(x)dx = 0 \quad (2.11)$$

- (b) Define the function $f(x)$ as:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } 0 \leq x < 1 \\ 1 & \text{if } x > 1 \end{cases} \quad (2.12)$$

and show that

$$g(x) = f(yx + \theta) = \text{ReLU}(yx + \theta_1) - \text{ReLU}(yx + \theta_2)$$

for some values θ_1, θ_2 .

- (c) Using assumption 2.14, show that

$$\int f(yx + \theta)\mu(x)dx = 0$$

for any $y, \theta \in \mathbb{R}$. Since you know f is discriminatory, make a conclusion about ReLU .

Solutions

1. A neuron with weights $w_1 = w_2 = 1$, bias $b = 3/2$ and Heaviside activation function implements the function

$$H(w_1x_1 + w_2x_2 - b) = H(x_1 + x_2 - 3/2) = 1_{x_1+x_2>3/2} = 1_{x_1=x_2=1} = AND(x_1, x_2)$$

2. Suppose we fix $\text{Cost} = K$. Again the number of states we can represent is β^n , which now evaluates to:

$$\beta^n = \beta^{\frac{\text{Cost}}{c(\beta+c_2)} - c_1} = \beta^{\frac{K}{c(\beta+c_2)} - c_1} \quad (2.13)$$

Across integers B , with the given values of c, c_1, c_2, K , this is maximised at $B = 4$.

3. Bounded: Heaviside, logistic. Bounded below: all. Continuous: all except Heaviside. Differentiable: all differentiable other than at 0; logistic and softplus differentiable everywhere. Smooth: logistic and softplus.
4. The activation function $\phi(x) = x$ is not 1-discriminatory. If $f(x) = 6x^2 - 6x + 1$ then

$$\begin{aligned} \int_0^1 xf(x)dx &= \int_0^1 (6x^3 - 6x^2 + x)dx = 0 \\ \int_0^1 f(x)dx &= \int_0^1 (6x^2 - 6x + 1)dx = 0 \end{aligned}$$

so

$$\int_0^1 \phi(yx + \theta)f(x)dx = \int_0^1 (yx + \theta)f(x)dx = y \int_0^1 xf(x)dx + \theta \int_0^1 f(x)dx = 0 + 0 = 0$$

and since f is not everywhere 0, the activation function $\phi(x) = x$ is not discriminatory.

5. The function $ReLU(x+1) - ReLU(x-1)$ is sigmoidal.
6. Assume that we have a function $\mu(x)$ such that for all $y, \theta \in \mathbb{R}$

$$\int_0^1 ReLU(yx + \theta)\mu(x)dx = 0 \quad (2.14)$$

Define the function $f(x)$ as:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } 0 \leq x < 1 \\ 1 & \text{if } x > 1 \end{cases} \quad (2.15)$$

Now we have

$$f(yx + \theta) = ReLU\left(yx - \frac{\theta}{y}\right) - ReLU\left(yx + \frac{1-\theta}{y}\right) = ReLU(yx + \theta_1) - ReLU(yx + \theta_2)$$

where $\theta_1 = -\frac{\theta}{y}$, $\theta_2 = \frac{1-\theta}{y}$, when $y \neq 0$. If $y = 0$, then we can write

$$f(yx + \theta) = f(\theta) = ReLU\left(\theta + \frac{1}{2}\right) - ReLU\left(\theta - \frac{1}{2}\right)$$

so we can always write $f(yx + \theta)$ as a difference of two ReLU functions:

$$f(yx + \theta) = ReLU(y_1x + \theta_1) - ReLU(y_2x + \theta_2)$$

Now we have

$$\int_0^1 f(yx + \theta)\mu(x)dx = \int_0^1 ReLU(y_1x + \theta_1)\mu(x)dx - \int_0^1 ReLU(y_2x + \theta_2)\mu(x)dx = 0 - 0 = 0$$

where we have used the assumption above twice to say that $\int_0^1 ReLU(yx + \theta)\mu(x)dx = 0$. But this holds for any y, θ . The function f is sigmoidal and continuous, so is discriminatory, so this must mean $\mu(x) = 0$ everywhere by the earlier theorem. This means $ReLU$ is 1-discriminatory.

Neural networks

Reading list and references

[Calin, 2020, Chapters 5,6]

3.1 Introduction

In this lecture we will at how we arrange neurons into a network.

We are not going to look at *training* neural networks yet; we are first going to focus on *representativeness*. We will do this formally in later lectures, but will lay some of the conceptual groundwork in this lecture by looking at *why* neural networks can approximate well-behaved functions well.

3.2 Neural networks

We saw in the previous lecture how to define an abstract neuron, and that biological neural networks are constructed by joining the outputs (axons) of neurons to the inputs (dendrites) of another.

An abstract neural network is defined similarly. We may essentially characterise a network as a directed graph in which vertices are neurons and edges mark connections between neurons, directed to be ‘coming from’ the neuron for which the edge is an output. A *feedforward neural network* is a special case of a network in which this graph is acyclic. We will mostly be looking at feedforward networks.

3.2.1 What can a single neuron do?

Recall that a neuron implements a function parametrised by $\mathbf{w}$ and b :

$$f(\mathbf{x}) = \phi(\mathbf{x}\mathbf{w} - b)$$

which depends on $\mathbf{x}$ only through the linear function $\mathbf{x}\mathbf{w} - b$. Thus on any hyperplane for which $\mathbf{x}\mathbf{w}$ is constant, the neuron returns the same value. Put another way, the neuron can only differentiate points according to their projection on the vector w .

In two dimensions, this is readily visualised (see figure 3.1). The output of a perceptron at point (x_1, x_2) corresponds to the colour at the point (blue for 1, white for 0, varying between). The vector w is shown in red. Points with the same projection on w will have the same output from the neuron.

Suppose that we have a set of points, some of which are coloured black and some of which are coloured red, and we want to find a *perceptron* (recalling a perceptron uses the activation function $\phi(x) = \mathbf{1}_{x>0}$) that ‘separates’ them: that is, returns 1 for all the black points, and 0 for all the red. In the example below in figure 3.2, we can see that this is possible for the arrangement in the leftmost panel (as shown in the rightmost panel), but not in the middlemost panel.

3.2.2 Neural networks

The insight of *neural networks* is the idea of chaining several neurons together. This allows us to separate more complicated partitions of points (and as we will see in later lectures, in fact *any* partition of a finite number of points). Let us firstly consider a simple arrangement of three neurons (figure 3.3). We will use the convention of $w_{ij}^{(\ell)}$ for the weight of the i th input of the j th neuron in the ℓ th layer. There are $d^{(\ell)}$ neurons in the ℓ th layer, with

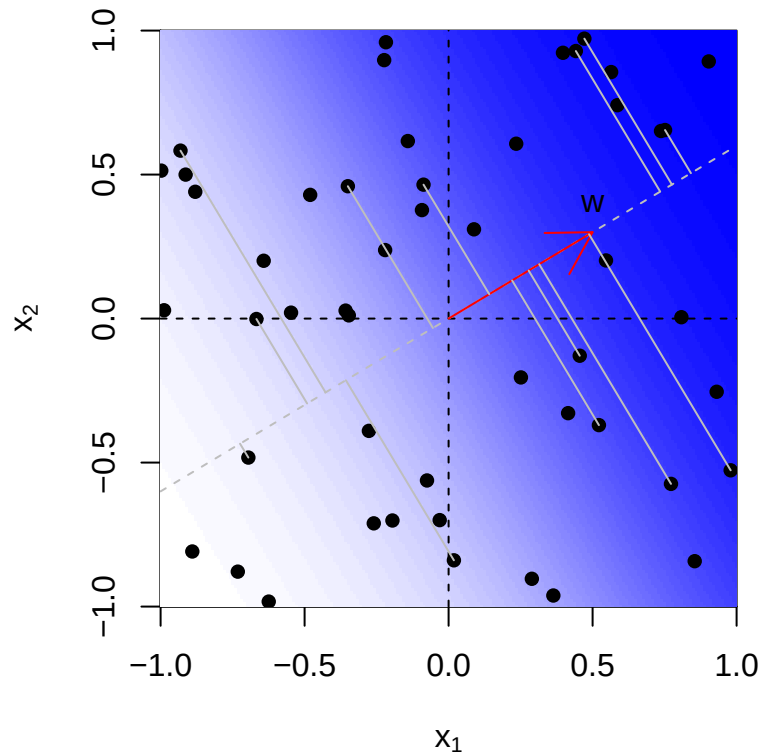


Figure 3.1: Output of a neuron with two inputs and parameters $w = (0.5, 0.3)$ and $b = 0$, using the activation function $\phi(x) = (1 + e^{-5x})^{-1}$

the convention that $d^{(0)}$ is the dimension of the input. The bias can be referred to as either the input $i = 0$, with corresponding weight $w_{0j}^{(\ell)}$, or as a separate variable $b_j^{(\ell)}$. The inputs of layer ℓ are the values $x_i^{(\ell-1)}$, $i \in 1 \dots d^{(\ell-1)}$, and the output is the values $x_i^{(\ell)}$, $i \in 1 \dots d^{(\ell)}$. The input is values $x_1, x_2, \dots, x_{d^{(0)}} = x_1^{(0)}, x_2^{(0)}, \dots, x_{d^{(0)}}^{(0)}$ and the output is the value $y_1, y_2, \dots, x_{d^{(\ell)}} = x_1^{(\ell)}, x_2^{(\ell)}, \dots, x_{d^{(\ell)}}^{(\ell)}$.

We firstly recognise that if we use a *linear* activation function $\phi(x) = ax + b$, then an arrangement of three neurons is equivalent to a single neuron.

Theorem 6. The output of an arrangement of three neurons as in figure 3.3 with common linear activation function $\phi(x) = ax + b$ is equivalent to that of a single neuron with input weights:

$$\mathbf{w} = \left(w_{11}^{(2)} w_{11}^{(1)} + w_{21}^{(2)} w_{12}^{(1)}, w_{11}^{(2)} w_{21}^{(1)} + w_{21}^{(2)} w_{22}^{(1)} \right) \quad (3.1)$$

and linear activation function $\phi(x) = a^2 x + (2ab + b)$

Proof. Denote $y_i^{(\ell)}$ as the output of neuron i in layer (ℓ) . The output $y_1^{(1)}$ of the first neuron in the first layer is

$$\begin{aligned} y_1^{(1)} &= \phi \left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 \right) \\ &= a \left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 \right) + b \end{aligned}$$

Similarly, we have

$$y_2^{(1)} = a \left(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 \right) + b$$

Now, for the second layer, we have

$$y_1^{(2)} = \phi \left(w_{11}^{(2)} y_1^{(1)} + w_{21}^{(2)} y_2^{(1)} \right)$$

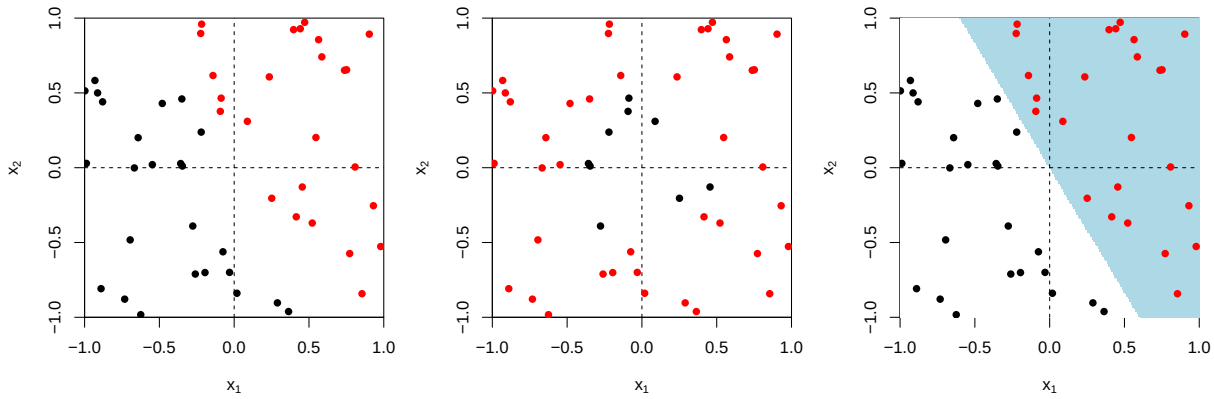


Figure 3.2: Sets of points which are linearly separable or not. The final panel shows the output of a perceptron with $b = 0$, $\phi(x) = \mathbf{1}_{x>0}$, $w = (0.5, 0.3)$

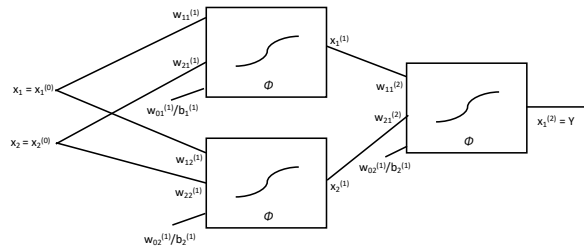


Figure 3.3: Arrangement of three neurons, each with two inputs and a bias. We will leave the activation functions unspecified for the moment.

$$\begin{aligned}
 &= \phi \left(w_{11}^{(2)} a \left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 \right) + b + w_{21}^{(2)} a \left(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 \right) + b \right) \\
 &= a \left(w_{11}^{(2)} a \left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 \right) + b + w_{21}^{(2)} a \left(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 \right) + b \right) + b \\
 &= a^2 \left(w_{11}^{(2)} \left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 \right) + w_{21}^{(2)} \left(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 \right) \right) + 2ab + b \\
 &= a^2 \left(\left(w_{11}^{(2)} w_{11}^{(1)} + w_{21}^{(2)} w_{12}^{(1)} \right) x_1 + \left(w_{11}^{(2)} w_{21}^{(1)} + w_{21}^{(2)} w_{22}^{(1)} \right) x_2 \right) + (2ab + b) \\
 &= \phi'(w'_1 x_1 + w'_2 x_2)
 \end{aligned}$$

which is the form of a single neuron with two inputs with weights $(w'_1, w'_2) = \left(w_{11}^{(2)} w_{11}^{(1)} + w_{21}^{(2)} w_{12}^{(1)}, w_{11}^{(2)} w_{21}^{(1)} + w_{21}^{(2)} w_{22}^{(1)} \right)$ and activation function $\phi'(x) = a^2 x + (2ab + b)$, as required. $\square$

The activation function ϕ' and weights (w'_1, w'_2) in the equivalent single neuron in theorem 6 are not unique. This extends to the more general result that:

Theorem 7. A feedforward network of a finite number of layers each of a finite number of neurons with (possibly different) linear activation functions is equivalent to a single neuron with a linear activation function.

An obvious corollary is that if we have a network of neurons with linear activation functions and replace the activation function of the final neuron with a step function, we will be equivalent to a single neuron whose activation function is a step function.

So using a linear activation function will not help us differentiate more partitions of points. We need to use nonlinear activation functions for neurons which are not in the final layer.

3.2.3 What can a neural network do that a single neuron can not?

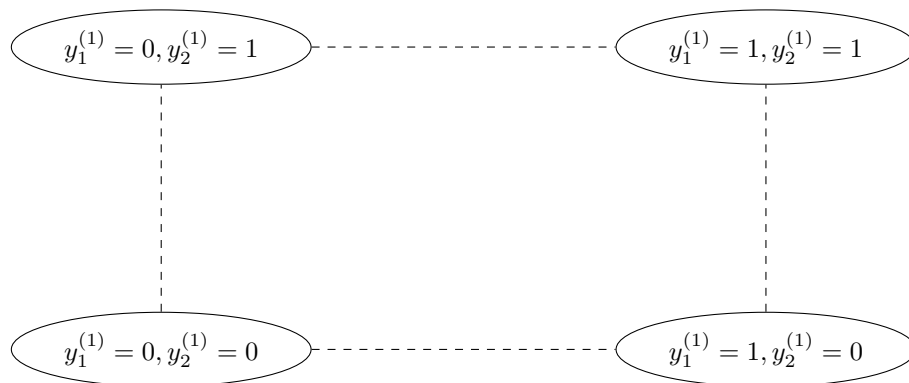
Let us consider what colourings of a set of points are separable by a three-neuron network of the type in figure 3.3 using step function activations. We will make a slight modification to figure 3.3 by allowing a bias in each neuron.

Denoting $\phi(x) = \mathbf{1}_{x>0}$. As above, we will denote $y_i^{(\ell)}$ as the output of neuron i in layer (ℓ) , and use the shorthand y for $y_1^{(2)}$, the final output. Now

$$\begin{aligned} y = 1 &\Leftrightarrow \phi\left(w_{11}^{(2)}y_1^{(1)} + w_{21}^{(2)}y_2^{(1)}\right) = 1 \\ &\Leftrightarrow -w_{01}^{(2)} + w_{11}^{(2)}y_1^{(1)} + w_{21}^{(2)}y_2^{(1)} \geq 0 \\ &\Leftrightarrow w_{11}^{(2)}y_1^{(1)} + w_{21}^{(2)}y_2^{(1)} \geq -w_{01}^{(2)} \end{aligned}$$

If we consider a point (x_1, x_2) in $\mathbb{R}^2$, recall that $y_1^{(1)}$ is 1 only for (x_1, x_2) in some closed half-plane (specifically the half-plane for which $w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 \geq w_{01}^{(1)}$). Together with the half-plane in which $y_2^{(1)} = 1$, these split the plane into up to four subsets.

Bear in mind that $y_1^{(1)}, y_2^{(1)} \in \{0, 1\}$, and that the final neuron also is 1 only in some half-plane. Let us consider the square



Since the final neuron can separate a half-plane, it can either separate one node from the other three, or two *adjacent* nodes from the other two. So an arrangement as above can separate *the intersection of two half-planes from its complement*, as well as emulate any single neuron. That is, it can separate the white regions below from the black in figure 3.4:

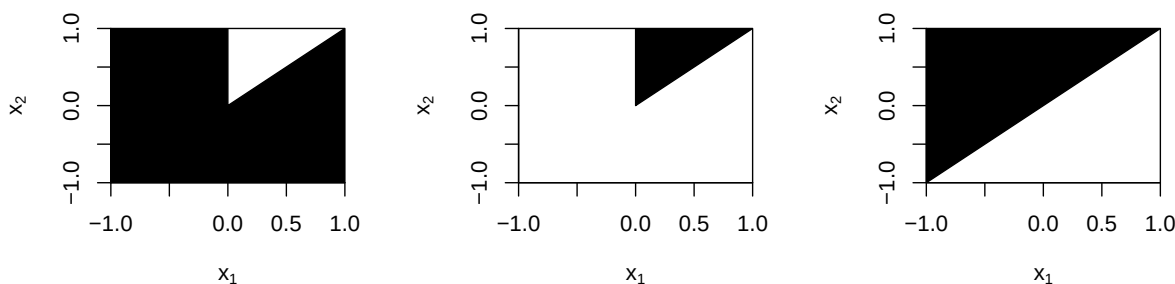


Figure 3.4: Sets of points which are separable using a three-neuron arrangement as above. In the third panel, the neural network is emulating a single neuron

Suppose we extend figure 3.3 to have an arbitrary (finite) number n of neurons in the leftmost layer. Each such neuron returns 1 if and only if the input (x_1, x_2) is in a particular half-plane. Let us draw all the boundaries of these half-planes together (figure 3.5)

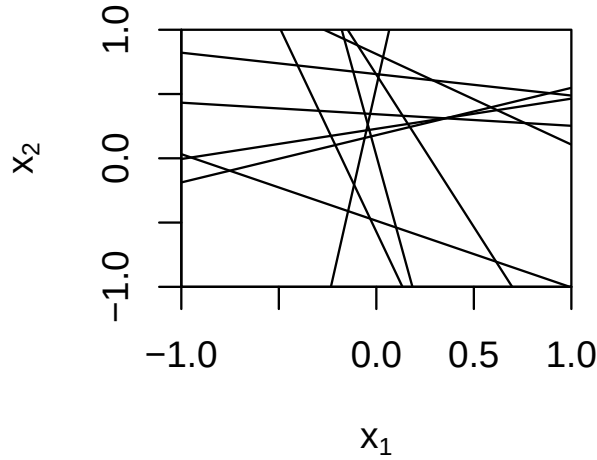


Figure 3.5: Boundaries of a lot of half-planes

This divides the plane into subregions. Suppose we want a function which returns 1 if and only if we are in one of those subregions. How will we implement this? Well, a single one of the subregions is defined by a specific inclusion/exclusion relation for each half-plane. Suppose we want the region *included* in the half-planes defined by neurons indexed by $S \subseteq [n]$ and *excluded* in the half-planes defined by neurons indexed by $([n] \setminus S) \subseteq [n]$. Then if we define

$$w_{i1}^{(2)} = \begin{cases} 1 & \text{if } i \in S \\ -1 & \text{if } i \notin S \end{cases} \quad (3.2)$$

and $w_{01}^{(2)} = n - \frac{1}{2}$, then (denoting H_i as the half-plane on which neuron i will return 1 if and only if (x_1, x_2) is in that half-plane) we have

$$\begin{aligned} y &= \phi \left(-w_{01}^{(2)} \sum_{i=1}^n w_{i1}^{(2)} y_i^{(1)} \right) \\ &= \mathbf{1}_{-w_{01}^{(2)} \sum_{i=1}^n w_{i1}^{(2)} y_i^{(1)} \geq 0} \\ &= \mathbf{1}_{\sum_{i=1}^n w_{i1}^{(2)} y_i^{(1)} \geq w_{01}^{(2)}} \\ &= \mathbf{1}_{\sum_{i \in S} y_i^{(1)} - \sum_{i \in ([n] \setminus S)} y_i^{(1)} \geq n - \frac{1}{2}} \\ &= \mathbf{1}_{\sum_{i \in S} ((x_1, x_2) \in H_i) + \sum_{i \in ([n] \setminus S)} ((x_1, x_2) \notin H_i) \geq n - \frac{1}{2}} \\ &= (\cap_{i \in S} ((x_1, x_2) \in H_i)) \cap (\cap_{i \in [n] \setminus S} ((x_1, x_2) \notin H_i)) \end{aligned}$$

which implements the function we need.

Now we have enough to show

Theorem 8. For any convex bounded subregion R of $\mathbb{R}^2$ with a continuous border, we may find a sequence of two-layer neural networks with step-function activations implementing a sequence of functions f_i such that

$$\lambda^* \left(\{ (x_1, x_2) \in \mathbb{R}^2 : f_i(x_1, x_2) \neq \mathbf{1}_{(x_1, x_2) \in R} \} \right) \rightarrow_{i \rightarrow \infty} 0 \quad (3.3)$$

where λ^* is the Lebesgue outer measure (effectively the difference in area).

Exercises

1. Prove theorem 7. To do this without horrible notation, use a matrix to represent the weights in a layer, and try and avoid explicit calculations.
2. Prove theorem 8. You may assume that for any region R of the type in the theorem we may find a sequence of convex polygons P_i containing R such that the Lebesgue outer measure of the difference between P_i and R converges to 0.
3. Sketch some more regions which can be separated by a three-neuron arrangement as in figure 3.3 with added biases. Specify the weights and biases to separate the regions in figure 3.4 (do not worry about the region boundaries).
4. Is it possible for a three-neuron arrangement as in figure 3.3 with added biases with step-function activations to separate the region comprising the $(+, +)$ and $(-, -)$ quadrants from its complement?
5. What does the bias in the two leftmost neurons in figure 3.3 add? In other words, characterise the regions which can be separated by a three-neuron arrangement as in figure 3.3 *without* biases in the two leftmost neurons.
6. What does the bias in the rightmost neuron in figure 3.3 add? Characterise the regions which can be separated by a three-neuron arrangement as in figure 3.3 *without* biases in the rightmost neuron.

Solutions

- Using our notation in the lecture for the weights, assemble the weights in layer ℓ into a matrix $W^{(\ell)}$, where the i, j th element is $w_{ij}^{(\ell)}$. We will give each neuron a bias and assemble these into a vector $b^{(\ell)}$. We will call the input to the ℓ th layer $X^{(\ell-1)}$ and the output $X^{(\ell)}$. Suppose the activation function is $\phi(x) = Ax + B$. Then

$$X^{(\ell)} = \phi(WX^{(\ell-1)} + b^{(\ell)}) = A(WX^{(\ell-1)} + b^{(\ell)}) + b = AWX^{(\ell-1)} + Ab^{(\ell)} + b \quad (3.4)$$

Suppose by the induction hypothesis that $X^{(\ell-1)}$ can be expressed as $M^t X^{(0)} + C$, for some matrix M and vector C . Then

$$\begin{aligned} X^{(\ell)} &= (AW)^t X^{(\ell-1)} + Ab^{(\ell)} + b \\ &= AW^t(M^t X^{(0)} + C) + Ab^{(\ell)} + b \\ &= (AW^t M^t) X^{(0)} + AW^t C + Ab^{(\ell)} + b \\ &= M_1^t X^{(0)} + C_1' \end{aligned}$$

where $M_1 = A(MW)$ and $C_1 = AW^t C + Ab^{(\ell)} + b$. Clearly M_1 and C_1 are still a matrix and vector, so by induction, Y must also be of the form $M_L^t X^{(0)} + C_L$ for some matrix M_L and vector C_L . We can clearly implement this with a one-layer network with weight matrix M , bias matrix C , and activation function $\phi(x) = x$

- Let us choose a sequence of convex polygons P_i , $i = 1, 2, \dots$ such that:

$$\lim_{i \rightarrow \infty} \lambda^* (\{(x_1, x_2) \in \mathbb{R}^2 : 1_{(x_1, x_2) \in P_i} \neq 1_{(x_1, x_2) \in R}\}) = 0 \quad (3.5)$$

that is, that the difference between P_i and R converges to 0 in area. Then P_i can be represented as the intersection of a finite number $n(i)$ of half-planes. For any half-plane H , a single neuron with step-function activation can implement the function $1_{(x_1, x_2) \in H}$. Let us assemble $n(i)$ neurons in the first layer, which implement the indicator functions for the $n(i)$ half-planes making up P_i . Then we may implement the ‘and’ function in the second-layer neuron by using weights 1 for each input neuron and a bias of $-n(i) - 1/2$, so the second-layer neuron will output 1 if and only if all the neurons in the first layer output 1. The neural network will thus implement P_i . By generating such a neural network f_i for each P_i , we can find a sequence of neural networks as in the theorem statement, and hence approximate R arbitrarily closely.

- Any intersection of two half-planes will do.
- No, it is not. As per the ‘square’ figure above, the final neuron would have to be non-monotonic.
- The bias means that the half-planes separated by the input neuron can be moved so that they do not intersect the origin.
- Without a bias in the rightmost neuron, the neuron can only implement functions of the type $OR(\pm x_1^{(1)}, \pm x_2^{(1)}, \dots)$, since it will output whenever the sum

$$\sum_{i=1}^{d^{(1)}} w_{1i}^{(1)} x_i^{(1)} \quad (3.6)$$

is positive.

Representation theorems

Reading list and references

[Calin, 2020, Chapters 7-9]

4.1 Introduction

In this and the next lecture, we will formally extend our simple approximation theorems in the previous lecture to fully describe the power of two-layer neural networks.

We will warm-up by showing a universal approximation result when we have a single input in $[0, 1]$, then develop this to more general results.

Please see Calin [2020] for an alternative exposition of these ideas.

4.2 One dimensional inputs

We will begin with a simple case in which we have a two-layer neural network with one input in $[0, 1]$, and we are attempting to learn a continuous function from $[0, 1]$ to $\mathbb{R}$.

Our neural network will have somewhat arbitrary activation functions in the hidden layer and a *linear* activation function in the final layer. A rough diagram is shown in figure 9.1. Since this is a simple setting we will use simpler notation than in the previous lecture: the weight of the single input in the i th neuron in the first layer will be w_i and the weight of the output of the i th neuron in the second layer will be c_i . We will denote the bias of the i th neuron in the first layer by θ_i .

4.2.1 Preliminaries

We define (as usual) $C[0, 1]$ as the set of continuous functions from $[0, 1]$ to $\mathbb{R}$, and recall a special case of the Heine-Borel theorem (proof)

Theorem 9 (Heine-Borel theorem). Any continuous real-valued function g defined on a closed interval $[a, b]$ is uniformly continuous: that is, for all $\epsilon > 0$, there exists $\delta > 0$ such that for all $x, x' \in [a, b]$ with $|x - x'| \leq \delta$ we have $|g(x) - g(x')| \leq \epsilon$.

4.2.2 Activation function properties

We will define a *permissible* activation function as follows:

Definition 6. A permissible activation function is a function $\phi(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ satisfying

1. ϕ is increasing
2. $(\lim_{x \rightarrow \infty} \phi(x)) - (\lim_{x \rightarrow -\infty} \phi(x)) = 1$

4.2.3 Main result

Our main result is the following:

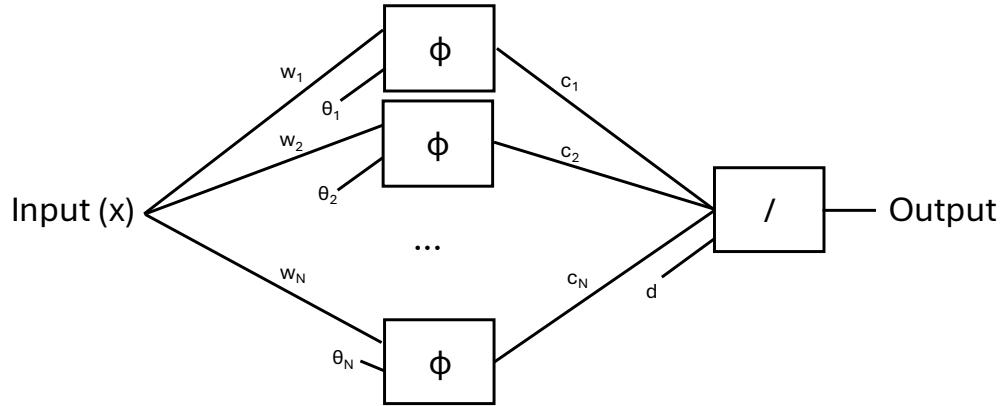


Figure 4.1: A neuron taking a single input, which we will show is a universal approximator.

Theorem 10. Suppose ϕ is a permissible activation function and g is a function $C[0, 1]$. Then for any $\epsilon > 0$ there exists a finite N and $\{c_i, w_i, \theta_i\}_{i \in \{1, 2, \dots, N\}} \in \mathbb{R}$ such that:

$$\sup_{x \in [0, 1]} \left| g(x) - \left(\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d \right) \right| < \epsilon$$

Note that the value

$$\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d$$

is the output of the neural network depicted in figure 9.1.

4.2.4 Starting out

We firstly notice a couple of things about the definition of a ‘permissible’ activation function. Firstly, since

$$\left(\lim_{x \rightarrow \infty} \phi(x) \right) - \left(\lim_{x \rightarrow -\infty} \phi(x) \right) = 1$$

both limits must exist. If we say $(\lim_{x \rightarrow -\infty} \phi(x)) = A$, then $(\lim_{x \rightarrow \infty} \phi(x)) = A + 1$. We can now write:

$$\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d = \sum_{i=1}^N c_i (\phi(w_i x + \theta_i) - A) + A \sum_{i=1}^N c_i + d = \sum_{i=1}^N c_i \phi'(w_i x + \theta_i) - A + d'$$

where $\phi'(x) = \phi(x) - A$ and $d' = A \sum_{i=1}^N c_i + d$. So the theorem is true for a function ϕ if and only if it is true for $\phi'(x) = \phi(x) - A$, so we can assume $A = 0$ and that:

$$\left(\lim_{x \rightarrow -\infty} \phi(x) \right) = 0 \tag{4.1}$$

$$\left(\lim_{x \rightarrow \infty} \phi(x) \right) = 1 \tag{4.2}$$

We next note that $\phi(x)$ has to stay in $[0, 1]$ for all x . If $\phi(x) = \gamma < 0$ for some x , then since ϕ is increasing, we would have $\phi(y) \leq \gamma$ for all $y < x$, which contradicts limit 4.1. Similarly, we cannot have $\phi(x) > 1$.

Thinking back to the definition of a limit, we can say $\epsilon_1 > 0$ we can find a δ^- and δ^+ such that:

$$0 \leq \phi(x) \leq \epsilon_1 \text{ for } x \leq -\delta^-$$

$$1 - \epsilon_1 \leq \phi(x) \leq 1 \text{ for } x \geq \delta^+$$

Calling $\delta = \max(\delta^-, \delta^+)$, using the fact that ϕ is increasing again, we can say that

$$\begin{aligned} 0 &\leq \phi(x) \leq \epsilon_1 \text{ for } x \leq -\delta \\ 1 - \epsilon_1 &\leq \phi(x) \leq 1 \text{ for } x \geq \delta \end{aligned}$$

Suppose we have two small positive values $\epsilon_1, \epsilon_2 > 0$. Then, choosing δ corresponding to ϵ_1 as above, and setting $s = \frac{\delta}{\epsilon_2}$, we have $x \geq \epsilon_2 \Leftrightarrow sx \geq \delta$, and

$$\begin{aligned} 0 &\leq \phi(sx) \leq \epsilon_1 \text{ for } x \leq -\epsilon_2 \\ 1 - \epsilon_1 &\leq \phi(sx) \leq 1 \text{ for } x \geq \epsilon_2 \end{aligned}$$

That is, by zooming out far enough, we can make sure that our function ϕ ‘looks’ like the Heaviside function, differing by at most ϵ_1 in the vertical direction except for an interval of width $2\epsilon_2$ about the origin.

4.2.5 Proceeding to the proof

Now let’s consider our function g , and our value ϵ . Using the Heine-Borel theorem, since g is continuous, we will choose a value δ such that for all $x, x' \in [a, b]$ with $|x - x'| \leq \delta$ we have $|g(x) - g(x')| \leq \frac{\epsilon}{2}$. Note the use of $\frac{\epsilon}{4}$: we will need this.

We will (for convenience) assume that $\frac{1}{\delta}$ is not an integer; if it is, we can choose another smaller δ in its place. We will denote by $N_\delta = \lceil \frac{1}{\delta} \rceil$ the smallest integer exceeding $\frac{1}{\delta}$, and we choose $\epsilon_3 < \frac{1}{\delta} - N_\delta$. We will then divide the interval $[0, 1]$ up into N_δ sub-intervals of length δ

$$\left[-\frac{\epsilon_3}{2}, \delta - \frac{\epsilon_3}{2}\right], \left[\delta - \frac{\epsilon_3}{2}, 2\delta - \frac{\epsilon_3}{2}\right], \dots, \left[(N_\delta - 1)\delta - \frac{\epsilon_3}{2}, N_\delta\delta - \frac{\epsilon_3}{2}\right] := [\delta_0, \delta_1], [\delta_1, \delta_2], \dots, [\delta_{N_\delta-1}, \delta_{N_\delta}]$$

noting that:

- These intervals overlap at the endpoints,
- The values 0 and 1 are interior points of the first and last interval

We will also slightly extend g to take the value $g(0)$ on $[\delta_0, 0]$ and $g(1)$ on $[1, \delta_{N_\delta}]$. We then consider the step function:

$$g_0(x) = \begin{cases} g(\delta_0) & \text{if } 0 \leq x < \delta_1 \\ g(\delta_1) & \text{if } \delta_1 \leq x < \delta_2 \\ \dots & \\ g(\delta_i) & \text{if } \delta_i \leq x < \delta_{i+1} \\ \dots & \\ g(\delta_{N_\delta-1}) & \text{if } \delta_{N_\delta-1} \leq x \leq \delta_{N_\delta} \end{cases}$$

We are now going to show two things: firstly, that:

$$\sup_{x \in [0, 1]} |g(x) - g_0(x)| \leq \frac{\epsilon}{4} \quad (4.3)$$

and secondly that we can find values w_i, θ_i, c_i, d such that:

$$\sup_{x \in [0, 1]} \left| \left(\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d \right) - g_0(x) \right| \leq 3 \frac{\epsilon}{4} \quad (4.4)$$

Using these values of w_i, θ_i, c_i, d , we will then have:

$$\begin{aligned} \sup_{x \in [0, 1]} \left| \left(\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d \right) - g(x) \right| &= \sup_{x \in [0, 1]} \left| \left(\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d \right) - g_0(x) + g_0(x) - g(x) \right| \\ &\leq \sup_{x \in [0, 1]} \left[\left| \left(\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d \right) - g_0(x) \right| + |g_0(x) - g(x)| \right] \end{aligned}$$

$$\begin{aligned}
&\leq \sup_{x \in [0,1]} \left[\left| \left(\sum_{i=1}^N c_i \phi(w_i x + \theta_i) + d \right) - g_0(x) \right| + \sup_{x \in [0,1]} |g_0(x) - g(x)| \right] \\
&\leq \frac{\epsilon}{4} + 3 \frac{\epsilon}{4} \\
&= \epsilon
\end{aligned}$$

which will establish the result.

4.2.6 Proving 4.3

To prove the first part, we simply need to note that for any x , the nearest ‘break point’ in the step function going backwards (that is, the first δ_i we get to going backwards) is at a distance at most δ from x . So, for any $x \in [0, 1]$, we have

$$|x - \delta_i| \leq \delta$$

and hence, by the construction of δ and g_0 :

$$|g(x) - g_0(x)| = |g(x) - g(\delta_i)| \leq \frac{\epsilon}{4}$$

which is what we needed.

4.2.7 Proving 4.4

To prove the second part, we construct a function as follows. Choose $\epsilon_1 \leq \frac{1}{N_\delta - 1} = \frac{1}{N}$, $\epsilon_2 < \frac{\epsilon_3}{3}$, and choose s as in the previous part. We now consider the function:

$$g_1(x) = g(\delta_0) + \sum_{i=1}^{N_\delta - 1} [g(\delta_i) - g(\delta_{i-1})] \phi(sx - s\delta_i)$$

This is of the form representable by the neural network: we are using $N = N_\delta - 1$, $c_i = g(\delta_i) - g(\delta_{i-1})$, $w_i = s$, $\theta_i = s\delta_i$, and $d = g(\delta_0)$.

What does this function look like? If we consider the simple case where $\phi(x) = H(x)$, then for $x \in (\delta_0, \delta_1)$, it is $g(\delta_0) = g(0)$. For $x \in (\delta_i, \delta_{i+1})$ it is:

$$[g(\delta_i) - g(\delta_{i-1})] + [g(\delta_{i-1}) - g(\delta_{i-2})] + \dots + [g(\delta_1) - g(\delta_0)] = g(\delta_i)$$

so it is a step function agreeing with $g(x)$ at the values δ_i ; exactly g_0 . Since our function $\phi(sx)$ is ‘similar’ to $H(x)$ for large x , our general functions will be similar too.

Firstly, we consider a value $x \in [\delta_j + \epsilon_2, \delta_{j+1} - \epsilon_2]$ for some $j \in \{0, 1, \dots, N_{\text{delta}} - 1\}$. For x in this interval, and $i \leq j$ we have

$$\begin{aligned}
-\frac{\epsilon}{4} &\leq g(\delta_i) - g(\delta_{i-1}) \leq \frac{\epsilon}{4} \\
0 &\leq 1 - \phi(sx - s\delta_i) \leq \epsilon_1
\end{aligned}$$

the first because of our choice of δ in the Heine-Borel theorem; the second because of our choice of s . So

$$-\frac{\epsilon\epsilon_1}{4} \leq (g(\delta_i) - g(\delta_{i-1})) (1 - \phi(sx - s\delta_i)) \leq \frac{\epsilon\epsilon_1}{4}$$

and hence

$$g(\delta_i) - g(\delta_{i-1}) - \frac{\epsilon\epsilon_1}{2} \leq (g(\delta_i) - g(\delta_{i-1})) \phi(sx - s\delta_i) \leq g(\delta_i) - g(\delta_{i-1}) + \frac{\epsilon\epsilon_1}{4}$$

For $i > j$, we have, since $0 \leq \phi(sx - s\delta_i) \leq \epsilon_1$:

$$-\frac{\epsilon\epsilon_1}{4} \leq (g(\delta_i) - g(\delta_{i-1})) \phi(sx - s\delta_i) \leq \frac{\epsilon\epsilon_1}{4}$$

Thus we have (allowing slightly different derivations for $j = 0$ or $j = N_\delta - 1$)

$$g_1(x) = g(\delta_0) + \sum_{i=1}^{N_\delta - 1} [g(\delta_i) - g(\delta_{i-1})] \phi(sx - s\delta_i)$$

$$\begin{aligned}
&\geq g(\delta_0) + \sum_{i=0}^j (g(\delta_i) - g(\delta_{i-1})) - N \frac{\epsilon \epsilon_1}{4} \\
&= g(\delta_j) - N \frac{\epsilon \epsilon_1}{4} \\
&= g_0(x) - N \frac{\epsilon \epsilon_1}{4}
\end{aligned}$$

and similarly

$$g_1(x) \leq g_0(x) + N \frac{\epsilon \epsilon_1}{4}$$

so, bearing in mind our choice of ϵ_1 :

$$|g_1(x) - g_0(x)| \leq N \frac{\epsilon \epsilon_1}{4} \leq \frac{\epsilon}{4}$$

Now let's consider a value in $[\delta_j - \epsilon_2, \delta_j + \epsilon_2]$, for $j \in \{1, 2, \dots, N_\delta - 1\}$. Since $\epsilon_2 \leq \frac{\epsilon_3}{2}$, we have now considered all points in $[0, 1]$.

We mostly have the same bounds as above; for $i < j$:

$$g(\delta_i) - g(\delta_{i-1}) - \frac{\epsilon \epsilon_1}{4} \leq (g(\delta_i) - g(\delta_{i-1})) \phi(sx - s\delta_i) \leq g(\delta_i) - g(\delta_{i-1}) + \frac{\epsilon \epsilon_1}{4}$$

and for $i > j$:

$$-\frac{\epsilon \epsilon_1}{4} \leq (g(\delta_i) - g(\delta_{i-1})) \phi(sx - s\delta_i) \leq \frac{\epsilon \epsilon_1}{4}$$

The difference is, for j itself, we only know:

$$0 \leq \phi(sx - s\delta_j) \leq 1$$

so, again using $|g(\delta_j) - g(\delta_{j-1})| \leq \frac{\epsilon}{4}$:

$$-\frac{\epsilon}{4} \leq (g(\delta_j) - g(\delta_{j-1})) \phi(sx - s\delta_j) \leq \frac{\epsilon}{4}$$

and we have:

$$\begin{aligned}
g_1(x) &= g(\delta_0) + \sum_{i=1}^{N_\delta-1} [g(\delta_i) - g(\delta_{i-1})] \phi(sx - s\delta_i) \\
&\geq g(\delta_0) + \sum_{i=0}^{j-1} (g(\delta_i) - g(\delta_{i-1})) - N \frac{\epsilon \epsilon_1}{4} - \frac{\epsilon}{4} \\
&= g(\delta_{j-1}) - \frac{\epsilon}{4} - N \frac{\epsilon \epsilon_1}{4} \\
&\geq g(\delta_j) - \frac{\epsilon}{4} - \frac{\epsilon}{4} - N \frac{\epsilon \epsilon_1}{4} \\
&= g(\delta_j) - \frac{\epsilon}{2} - N \frac{\epsilon \epsilon_1}{4}
\end{aligned}$$

and similarly

$$\begin{aligned}
g_1(x) &\leq g(\delta_{j-1}) + \frac{\epsilon}{4} + N \frac{\epsilon \epsilon_1}{4} \\
&\leq g(\delta_j) + \frac{\epsilon}{2} + N \frac{\epsilon \epsilon_1}{4}
\end{aligned}$$

We hit a small snag: since $x \in [\delta_j - \epsilon_2, \delta_j + \epsilon_2]$, we could have either $g_0(x) = \delta_{j-1}$ (if $x \in [\delta_j - \epsilon_2, \delta_j]$) or $g_0(x) = \delta_j$ (if $x \in [\delta_j, \delta_j + \epsilon_2]$). But either way, we have, bearing in mind our choice of ϵ_1 :

$$|g_1(x) - g_0(x)| \leq \frac{\epsilon}{2} + N \frac{\epsilon \epsilon_1}{4} \leq \frac{\epsilon}{2} + \frac{\epsilon}{4} = 3 \frac{\epsilon}{4}$$

which is what we needed, and we are done.

4.2.8 Remarks

This somewhat laborious proof belies the very simple idea of

1. Starting with a well-behaved function; or in particular, one for which knowledge of value at a point gives us some indication of its behaviour in a neighbourhood of that point
2. Approximating this function with a step function
3. Noting that we can approximate a step function by using a sum of permissible activation functions, appropriately scaled
4. Concluding that we can closely approximate our original function with a sum of permissible activation functions.

The idea of approximation of an arbitrary function in terms of a simple function (in this case, a sum) of simpler functions is a very well-worn tenet of applied mathematics. The useful thing about this particular representation is it shows that we can do it with a neural network which is relatively easy to quickly *train*.

Exercises

1. Verify that the logistic activation function is permissible
2. What functions g are ‘harder’ to approximate in this way, in that they need a larger N ?
3. Give an example of a function which is continuous on the *open* interval $(0, 1)$ which is not approximable by a finite sum of step functions (in the sense of theorem [11](#))

Solutions

1. The logistic activation function is $\phi(x) = (1 + e^{-x})^{-1}$. We have

(a) $\frac{d\phi(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} > 0$

(b) $\lim_{x \rightarrow -\infty} \phi(x) = \lim_{x \rightarrow -\infty} \frac{e^x}{e^x + 1} = \lim_{x \rightarrow -\infty} \frac{0}{0+1} = 0$ and $\lim_{x \rightarrow \infty} \phi(x) = \lim_{x \rightarrow \infty} (1 + 0)^{-1} = 1$.

(c) $\frac{d\phi(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} < \frac{1+e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} = \phi(x) < 1$, and we have already showed that $\frac{d\phi(x)}{dx} > 0$.

so $\phi(x)$ is admissible.

2. More quickly-changing functions are harder to approximate.
3. For instance, $g(x) = \sin\left(\frac{1}{x}\right)$, which we cannot approximate near $x = 0$.

Cybenko's universal approximation theorem

Reading list and references

[Calin, 2020, Chapters 7-9]
Cybenko [1989]: a paper on universal representation by 'discriminatory' activation functions
Leshno et al. [1993]: a paper on very general universal representation by non-polynomial activation functions.
Hornik et al. [1989]: a paper on universal approximation by arbitrary continuous non-constant functions.

5.1 Introduction

In this lecture, we will extend the ideas from the previous lecture to show the main representation theorems concerning two-layer neural networks.

As in the previous lecture, the analytic formalities bely a fairly simple underlying idea: that if we can guarantee that a function doesn't vary too much in some sense, then we can approximate the whole by dividing it into parts in which it does not vary much, and approximating it in each part.

In this lecture, we will go over some preliminaries we will need to prove our main result, which was shown by Cybenko. **You should know the result of this theorem, and the 'simple' definition of a discriminatory activation function, but the 'full' definition of a discriminatory activation function and the proof of the theorem are non-examinable.**

Again, we will go through results essentially as presented in Calin [2020], with some additions and modifications.

5.2 Main theorem statement

Define I_n as the unit hypercube; that is, $[0, 1]^n$.

5.2.1 Discriminatory activation functions: simple definition

Let $F = C[0, 1]^n$; the set of continuous functions $f : [0, 1]^n \rightarrow \mathbb{R}$. An activation function ϕ is n -discriminatory only if the only $f \in F$ for which:

$$\int_{I_n} \phi(w^t x + \theta) f(x) dx = 0$$

for all $w \in \mathbb{R}^n$ and $\theta \in \mathbb{R}$, is $f(x) = 0$.

Alternatively: ϕ is *not* discriminatory if there exists $f \in F$ which is not $f(x) = 0$ such that for all $w \in \mathbb{R}^n$ and $\theta \in \mathbb{R}$ we have

$$\int_{I_n} \phi(w^t x + \theta) f(x) dx = 0$$

5.2.2 Main theorem statement

Theorem 11. If σ is a continuous discriminatory activation function, then the set of functions G of the form:

$$\sum_{j=1}^N \alpha_j \sigma(w_j^T x + \theta_j)$$

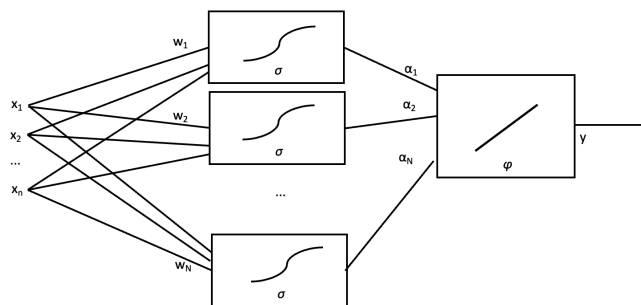


Figure 5.1: A neuron taking a set of real-valued inputs in $[0, 1]$, which we will show is a universal approximator for several function classes

with $w_j \in \mathbb{R}^n, \alpha_j, \theta_j \in \mathbb{R}$, and finite N , are dense in $C(I_n)$; that is, for any $\epsilon > 0$ and $f \in C(I_n)$, there exists a function $g \in G$ such that:

$$|f(x) - g(x)| < \epsilon$$

for all $x \in I_n$

5.2.3 Discriminatory activation functions: full definition

Lecture not examinable after here.

We denote by $M(I_n)$ the set of finite, signed Borel measures on I_n . Recall that a Borel measure is a measure defined on the Borel sets, which are members the σ -algebra generated by the products of open intervals of $[0, 1]$

Definition 7. Let μ be in $M(I_n)$. An n -discriminatory activation function is a function for which if

$$\int_{I_n} \sigma(w^t x + \theta) d\mu(x) = 0 \tag{5.1}$$

for all vectors $w \in \mathbb{R}^n, \theta \in \mathbb{R}$, then $\mu = 0$.

A discriminatory activation function is n -discriminatory for all $n > 0$.

5.3 General preliminaries

In this section, we will be concerned with neural networks of the type shown in figure 5.1. We will presume that neurons in the first layer have an activation function σ , which will be of a type we will define called ‘discriminatory activation functions’. The final layer will again have a linear activation function, this time with weights $\alpha_1, \alpha_2, \dots$

We will denote by x our input variable, and presume we are trying to approximate a function $f(x)$. We will presume f comes from a space of functions $\mathcal{S}$ endowed with a metric d . We will aim to approximate functions in $\mathcal{S}$ with functions g implementable by neural networks. We denote by $\mathcal{U}$ the space of such functions, and generally presume it is a subspace of $\mathcal{S}$ (although it need not necessarily be; recall from the previous lecture that we may approximate continuous functions arbitrarily closely with non-continuous step functions, for appropriate d). We call $\mathcal{S}$ the *target space* and $\mathcal{U}$ the *approximation space*.

We define what we mean by a universal approximator:

Definition 8. A neural network design is a *universal approximator* for the metric space $(\mathcal{S}, d)$ if the space $\mathcal{U}$ of functions it implements is d -dense in $\mathcal{S}$; that is, for all $f \in \mathcal{S}$, for any $\epsilon > 0$, there exists some $g \in \mathcal{U}$ such that $d(f, g) < \epsilon$

5.3.1 The space $C(I_n)$

We will mostly be concerned with the space $C(K)$: the class of continuous functions on a compact set K (in particular, on products of closed intervals). We considered a subclass of this in the previous lecture. As before, we

use the distance metric:

$$d(f, g) = \sup_{x \in K} |f(x) - g(x)| = \max_{x \in K} |f(x) - g(x)| \quad (5.2)$$

where we can replace $\sup(\cdot)$ by $\max(\cdot)$ because $|f - g|$ is continuous and we are working on a compact set. The norm corresponding to this metric is $\|f\| = \max_{x \in K} |f(x)|$.

We will usually use the space $K = I_n$, where I_n is the *open* unit hypercube in $\mathbb{R}^n$ (that is, the product of n copies of the open interval $[0, 1]$).

We could also consider $L^1(K)$: the space of real-valued integrable functions on a compact set K . The usual distance metric in this case is:

$$d(f, g) = \int_K |f(x) - g(x)| dx \quad (5.3)$$

The corresponding norm is the L^1 norm $\|f\| = \int_K |f(x)| dx$. Another important space is $L^2(K)$: the space of real-valued square-integrable functions on a compact set K . The distance function is

$$d(f, g) = \left(\int_K |f(x) - g(x)|^2 dx \right)^{\frac{1}{2}} \quad (5.4)$$

with corresponding norm $\|f\| = \left(\int_K |f(x)|^2 dx \right)^{\frac{1}{2}}$. We will not explicitly look at approximation in these spaces, but corresponding results to those we will derive can be found in [Calin, 2020, chapter 9].

5.3.2 Functional analysis

We will use the following concepts from the analysis of linear operators:

1. A *normed linear space* will be taken to be a space endowed with a notion of addition, denoted '+', and multiplication with real scalars, denoted '.', and a norm, usually denoted $\|\cdot\|$, which satisfies (i) $\|x\| \geq 0$ for all x , (ii) $\|x\| = 0$ if and only if $x = 0$, and (iii) $\|x + y\| \leq \|x\| + \|y\|$ for all x, y
2. A *linear subspace* is a subspace of a normed vector space S which is closed under the addition and scalar multiplication operations of S .
3. A *linear functional* is a function L from a linear space X to $\mathbb{R}$ such that for any $a_i \in \mathbb{R}$ and $x_i \in X$ we have $L(a_1 x_1 + a_2 x_2) = a_1 L(x_1) + a_2 L(x_2)$
4. A linear functional $L : X \rightarrow \mathbb{R}$ is *bounded* if there exists a constant $M > 0$ such that $|L(x)| \leq M \|x\|$ for all x , where $\|\cdot\|$ is the norm of X . We denote $M = \|L\|$.

5.3.3 Borel measure

We denote by $M(I_n)$ the set of finite, signed Borel measures on I_n . Recall that a Borel measure is a measure defined on the Borel sets, which are members the σ -algebra generated by the products of open intervals of $[0, 1]$. A *finite* measure μ has the property that $\mu([0, 1])$ is finite, and a *signed* measure means that the measure can take negative values.

5.3.4 Single hidden layer networks as universal approximators

We will show that a neural network with one 'hidden' layer (that is, two layers total) is a universal approximator for the spaces $C(I_n)$.

This will in practice mean that if we allow a wide enough network, we can approximate any function in this spaces arbitrarily closely, where 'close' is with respect to the relevant metric.

5.4 Proceeding with the main proof

5.4.1 Lemmata

Statements and proofs not examinable

We begin with a lemma concerning non-dense subspaces.

Lemma 1. Let U be a linear subspace of a normed linear space $(X, \|\cdot\|)$ and suppose that there exists some $x_0 \in X$ such that for some $\delta > 0$ we have

$$d(x_0, u) = \|x_0 - u\| > \delta \quad (5.5)$$

for all $u \in U$. Then there exists a bounded linear functional L on X satisfying

1. $\|L\| \leq 1$
2. $L(u) = 0$ for all $u \in U$
3. $L(x_0) = \delta$

Proof. We will construct the functional explicitly on U and use the Hahn-Banach theorem to extend it to X . Denote

$$T = \{t : t = u + \lambda x_0, u \in U, \lambda \in \mathbb{R}\} \quad (5.6)$$

and define $L : T \rightarrow \mathbb{R}$ as $L(t) = L(u + \lambda x_0) = \lambda \delta$. We firstly recognise that this functional is well-defined, in that we can only express a member of X as $u + \lambda x_0$ in at most one way. Suppose that for some $x \in X$ we had

$$x = u_1 + \lambda_1 x_0 = u_2 + \lambda_2 x_0$$

with $\lambda_1 \neq \lambda_2$. Then:

$$x_0 = \frac{u_1 - u_2}{\lambda_2 - \lambda_1}$$

But this means we are expressing x_0 as a sum of scalars multiplied by elements of U - so, since U is closed under addition and scalar multiplication (being a linear subspace) we would have to have $x_0 \in U$, which it cannot be by definition. So we can express any $x \in X$ as $u + \lambda x_0$ in at most one way.

Now we must show that L is a linear functional. We have

$$L(a_1 t_1 + a_2 t_2) = L(a_1(u_1 + \lambda_1 x_0) + a_2(u_2 + \lambda_2 x_0)) = a_1 \lambda_1 \delta + a_2 \lambda_2 \delta = a_1 L(t_1) + a_2 L(t_2) \quad (5.7)$$

as required. Now we show that $L(t) \leq \|t\|$ for all $t \in T$ so $\|L\| \leq 1$. If $u \in U$, then $(-\frac{1}{\lambda})u \in U$ so

$$\begin{aligned} \|x_0 + \frac{u}{\lambda}\| &= \|x_0 - \left(-\frac{1}{\lambda}\right)u\| \geq \delta \\ \Leftrightarrow |\lambda|\delta &\leq \|u + \lambda x_0\| \end{aligned}$$

so

$$L(t) = L(u + \lambda x_0) = \lambda \delta \leq |\lambda| \delta \leq \|u + \lambda x_0\| = \|t\|$$

as required. We can apply the [Hahn-Banach theorem](#) to extend L to a linear functional on X such that $L(x) \leq \|x\|$ for all $x \in X$. Finally, we have

$$\begin{aligned} L(u) &= L(u + 0x_0) = 0\delta = 0 \\ L(x_0) &= L(0 + 1x_0) = 1\delta = \delta \end{aligned}$$

as required. □

As an example, let:

1. our normed linear space be $C([0, 1])$ with norm $\|f(x)\| = \sup_{x \in [0, 1]} (|f(x)|) = \max_{x \in [0, 1]} (|f(x)|)$, recalling that a continuous function on a compact set attains its supremum,
2. U be $\{f(x) = ax; a \in \mathbb{R}\}$, the set of linear functions with intercept 0, and
3. x_0 be the function $f(x) = x^2$

Then we may set $\delta = 1/100$, $T = \{ax + bx^2, a \in \mathbb{R}, b \in \mathbb{R}\}$ and $L(ax + bx^2) = a/100$. It's a little difficult to visualise how to extend this to $C([0, 1])$, but one way is to define the equivalence class $\simeq$ as

$$(f \simeq g) \Leftrightarrow (f - g)(x) = ax + bx^2 \quad (5.8)$$

and for each equivalence class F , define $L(f_0) = 0$ for $f_0 = \arg \min_{f \in F} \{||f||\}$ ¹ and

$$L\{f(x)\} = L\{f_0(x) + ax + bx^2\} = b\delta = b/100 \quad (5.9)$$

otherwise. Now $L(f)$ is a linear functional satisfying the conditions of lemma 1 (this is a bit tedious to show but we will do it in lectures if we have time). We note that in contrast to neural networks, *we cannot approximate arbitrary functions in $C([0, 1])$ by sums of linear functions.*

We note that we can restate lemma 1 in a form which looks more useful:

Lemma 2. Let U be a linear *non-dense* subspace of a normed linear space X . Then there is a bounded linear functional on X such that $L \neq 0$ on X and $L(u) = 0$ for all $u \in U$.

5.5 Final lemmas and proof

We will need a result (sometimes called the Riesz–Markov–Kakutani representation theorem, or just the Riesz representation theorem) which we will state without proof in the somewhat specific form that we need:

Theorem 12 (Riesz–Markov–Kakutani). Let F be a bounded linear functional on $C(I_n)$. Then exists a unique finite signed Borel measure μ on I_n such that for all $f \in C(I_n)$ we have:

$$F(f) = \int_{I_n} f(x) d\mu(x) \quad (5.10)$$

and $||F|| = |\mu|(I_n)$

We now show that with a non-dense subspace of functions, we can find a measure which vanishes on that subspace, but is not zero:

Lemma 3. Let U be a linear, non-dense subspace of $C(I_n)$. Then there is a measure $\mu \in M(I_n)$ such that for all $h \in U$ we have

$$L(h) = \int_{I_n} h d\mu = 0 \quad (5.11)$$

but for which there exists $f \in C(I_n)$ such that $L(f) \neq 0$.

Proof. Choose a linear functional (as per lemma 2) for which $L(f) \neq 0$ for some $f \in C(I_n)$ and for which $L(f) = 0$ for all $h \in U$.

By theorem 12 we can now choose a measure $\mu \in M(I_n)$ such that

$$L(f) = \int_{I_n} f(x) d\mu(x) \quad (5.12)$$

This gives us exactly what we need: we have $L(h) = 0$ for all $h \in U$, but $L(f) \neq 0$ for some $f \in C(I_n)$. □

5.5.1 Proof of main result

It's now relatively straightforward to prove our main result.

Since our (discriminatory) activation function is continuous, we have that:

$$U = \left\{ G : G(x) = \sum_{j=1}^N \alpha_j \sigma(w_j^T x + \theta_j); N < \infty; w_j \in \mathbb{R}^n; \alpha_j \in \mathbb{R} \right\} \quad (5.13)$$

is a linear subspace of $C(I_n)$.

If U is *not* dense in $C(I_n)$ then by lemma 3 we may choose $\mu \in M(I_n)$ such that

$$\int_{I_n} g d\mu = 0 \quad (5.14)$$

¹To see why the minimum exists, we note that for any $f \in F$ the function $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ given by $g(a, b) = ||f(x) + ax + bx^2||$ diverges in all directions, so approaches its infimum inside some (compact) closed ball.

for all $g \in U$. But this means for any $w_j \in \mathbb{R}^n; \alpha_j \in \mathbb{R}$ we have:

$$\begin{aligned} 0 &= \int_{I_n} \left(\sum_{j=1}^N \alpha_j \sigma(w_j^T x + \theta_j) \right) d\mu \\ &= \sum_{j=1}^N \alpha_j \int_{I_n} \sigma(w_j^T x + \theta_j) d\mu \end{aligned}$$

If we choose $\alpha_1 = 1, \alpha_2 = \alpha_3 = \dots = 0$, we attain

$$\int_{I_n} \sigma(w_1^T x + \theta_1) d\mu = 0 \quad (5.15)$$

for any $w_1 \in \mathbb{R}^n; \alpha_1 \in \mathbb{R}$. But this contradicts the assumption that σ is discriminatory.

5.5.2 Remarks

We firstly note that the curious property of ‘discrimination’ allows us to use the Riesz-Markov-Kakutani theorem to prove this quickly. We are also aided by $C(I_n)$ being a well-behaved space. We recall that any *sigmoidal* function is discriminatory, so this is a helpful result.

We note that the space U is a proper subspace of $C(I_n)$: there are certainly functions in $C(I_n)$ which are not in U . For a simple example, set $n = 1$, choose a differentiable activation function so that all $u \in U$ are differentiable, and set f as a non-differentiable function on $[0, 1]$ (e.g. $f(x) = |x - \frac{1}{2}|$). Then $f \notin U$.

To restate in a more useful form, theorem 11 states that for any $f \in C(I_n)$ and $\epsilon > 0$, we can find a neural network encoding a function $G(x)$ such that:

$$\max_{x \in I_n} |G(x) - f(x)| \leq \epsilon \quad (5.16)$$

5.6 Other important results and generalisations

We will briefly look at a couple of other important results in the theory of approximation by neural networks. We will not go through the proofs of these results, but this is largely due to their being analytically laborious rather than technically difficult.

5.6.1 Continuous non-constant activation functions

The intuition behind sigmoidal functions being universal approximators is that for inputs of sufficiently large absolute value, they are close to 0 or 1. Thus, by ‘squashing’ them horizontally, we can use a sigmoidal activation function like an indicator function on a limited region, and guarantee that it will not have much influence anywhere else. By composing lots of these together, we can essentially approximate anything bit by bit.

ReLU is similar, in that we can roughly ‘put together’ two ReLUs to get a sigmoidal activation function (see exercises). Polynomials are not (also see exercises). What can we say about neural networks in which the first layer has polynomial activation functions?

We can sort-of manage if we allow a different type of neural network. If we allow that a neuron can take the *product* of its inputs, rather than the sum (we do not need to consider separate weights for each input), then a three-layer network can be a universal approximator. The following theorem is due to Hornik, Stinchcombe and White (1989) [Hornik et al., 1989]

Theorem 13 (Hornik, Stinchcombe, White, 1989). Let $\phi : \mathbb{R} \rightarrow \mathbb{R}$ be any continuous nonconstant activation function. Then the finite sums of finite products of the form

$$G(x) = \sum_{k=1}^M \beta_k \prod_{j=1}^{N_k} \phi(w_{jk}^T x + \theta_{jk}) \quad (5.17)$$

with $w_{jk} \in \mathbb{R}^n, \beta_j, \theta_{jk} \in \mathbb{R}, x \in I_n, M, N_k \in \mathbb{N}$, are dense in $C(I_n)$.

For the proof, see [Calin, 2020, Theorem 9.3.8].

5.6.2 Approximation with arbitrary functions

We know polynomial activation functions cannot be discriminatory in a two-layer neural network. It turns out this is essentially the only case in which functions are not. The following remarkable theorem from Leshno, Lin, Pinkus and Schocken (1993) establishes that *any* non-polynomial activation function can allow a neural network to be a universal approximator. We state it in a slightly less general form:

Theorem 14 (Leshno, Lin, Pinkus and Schocken (1993)). Let σ be a bounded function from I_n to $\mathbb{R}$ such that

1. The closure of the set of points of discontinuity of σ has zero Lebesgue measure
2. For any polynomial $p(x)$, the set $\{x \in I_n : \sigma(x) \neq p(x)\}$ has Lebesgue measure greater than 0. That is, σ is not almost-everywhere a polynomial.

Then the set of functions:

$$U = \left\{ G : G(x) = \sum_{j=1}^N \alpha_j \sigma(w_j^T x + \theta_j); N < \infty; w_j \in \mathbb{R}^n; \alpha_j \in \mathbb{R} \right\} \quad (5.18)$$

is dense in $C(I_n)$ with respect to the supremum norm.

For a proof, see [Leshno et al. \[1993\]](#) (the paper is well-written and straightforward to follow).

Exercises

1. Consider the function $k(x) = 1_{x \in [0,1]}$. Consider the class of functions defined by

$$\mathcal{S} = \left\{ \sum_{i=1}^N c_i k(w_i x - \theta_i) \right\} \quad (5.19)$$

for finite N and $c_1, c_2, \dots, c_N, w_1, w_2, \dots, w_N, \theta_1, \theta_2, \dots, \theta_N \in \mathbb{R}$. Prove that this is a universal approximator for the space $\mathcal{S}$ of continuous functions on the closed interval $[0, 1]$ with respect to the norm $\|f\| = \sup_{x \in [0,1]} f(x)$. Hint: show firstly that $g \in \mathcal{S}$ can be approximated to a supremum error ϵ by a step function, then show that step functions are in the above class.

2. Is the above class still a universal approximator for $\mathcal{S}$ if all the weights $w_1, w_2, \dots, w_N$ are equal to 1?
3. Verify that $C([0, 1])$, $L^1([0, 1])$, $L^2(K)$ are normed linear spaces with the norms above.
4. Sketch the form of a sum-product neural network implementing the function in theorem 13.
5. Suppose we have a two-layer neural network, with N_1 neurons in the first layer, N_2 in the second, and one in the third.
 - (a) Write a formula for the output of this network
 - (b) Show that the outputs of all possible networks of this type form a linear space of functions.

Solutions

1. As per the previous lecture, continuous functions on $[0, 1]$ are uniformly continuous, so given $\epsilon > 0$, we may choose δ such that whenever $|x - x'| < \delta$ we have $|f(x) - f(x')| < \epsilon$. Let us partition $[0, 1]$ into N equal intervals $[x_0 = 0, x_1), [x_1, x_2), \dots, [x_{N-1}, x_N = 1]$ with $N > 1/\delta$, so that each interval has width $< \delta$. For each interval, we choose a u_i with $x_{i-1} \leq u_i < x_i$ (with the obvious allowances for $i = N$). Let

$$U(x) = \begin{cases} u_i & \text{if } x_i \leq x < x_{i+1} \\ u_N & \text{if } x = 1 \end{cases}$$

For $x \in [x_i, x_{i+1})$ we have $|u_i - x| = \frac{1}{N} < \delta$ so $|f(x) - U(x)| = |f(x) - f(u_i)| < \epsilon$, so

$$\sup_{x \in [0, 1]} |f(x) - U(x)| < \epsilon$$

Now we note that we can write

$$U(x) = \sum_{i=1}^N u_i k(Nx - x_k)$$

which is of the required form, so we can approximate $U(x)$ exactly by a function in $\mathcal{S}$, and hence approximate f to a supremum error of at most ϵ .

2. Yes - we can write

$$\begin{aligned} k(w_i x - \theta_i) &= 1_{w_i x - \theta_i \in [0, 1]} \\ &= 1_{(w_i x - \theta_i \geq 0) \text{ AND } (w_i x - \theta_i \leq 1)} \\ &= 1_{(w_i x - \theta_i \geq 0) \text{ AND NOT } (w_i x - \theta_i \geq 1)} \\ &= 1_{w_i x - \theta_i \geq 0} - 1_{w_i x - \theta_i \geq 1} \\ &= 1_{x - \frac{\theta_i}{w_i} \geq 0} - 1_{x - \frac{\theta_i + 1}{w_i} \geq 0} \\ &= H\left(x - \frac{\theta_i}{w_i}\right) - H\left(x - \frac{\theta_i + 1}{w_i}\right) \\ &= k(x - u_i) - k(x - v_i) \end{aligned}$$

noting that on $[0, 1]$, we have $H(x - a) = k(x - a)$, and setting $u_i = \frac{\theta_i}{w_i}$, $v_i = \frac{\theta_i + 1}{w_i}$. So we can write any function in $\mathcal{S}$ in which $w_i \neq 1$ as a function in which all w_i s are 1, but for which N may need to be larger.

3. The space $C[0, 1]$ under the supremum norm, where $x = f(z) \in C[0, 1]$, $y = g(z) \in C[0, 1]$ satisfies:

- (a) $\|x\| = \sup_{z \in (0, 1)} |f(z)| \geq 0$
- (b) $\|x\| = \sup_{z \in (0, 1)} |f(z)| = 0$ implies $f(z) = 0$ for all $z \in [0, 1]$
- (c) $\|x + y\| = \sup_{z \in (0, 1)} |f(z) + g(z)| \leq \sup_{z \in (0, 1)} |f(z)| + \sup_{z \in (0, 1)} |g(z)| = \|x\| + \|y\|$

The space $L^1[0, 1]$ under the L^1 norm, with $x = f(z) \in L^1[0, 1]$, $y = g(z) \in L^1[0, 1]$, satisfies

- (a) $\|x\| = \int_0^1 |f(z)| dz \geq 0$
- (b) $\|x\| = \int_0^1 |f(z)| dz = 0$ implies $f(z) = 0$ for all $z \in [0, 1]$
- (c) $\|x + y\| = \int_0^1 |f(z) + g(z)| dz \leq \int_0^1 (|f(z)| + |g(z)|) dz = \int_0^1 |f(z)| dz + \int_0^1 |g(z)| dz = \|x\| + \|y\|$

The space $L^2[0, 1]$ under the L^2 norm with $x = f(z) \in L^2[0, 1]$, $y = g(z) \in L^2[0, 1]$, satisfies

- (a) $\|x\| = \sqrt{\int_0^1 f(z)^2 dz} \geq 0$
- (b) $\|x\| = \sqrt{\int_0^1 f(z)^2 dz} = 0$ implies $f(z) = 0$ for all $z \in [0, 1]$

(c) $\|x + y\| = \sqrt{\int_0^1 (f(z) + g(z))^2 dz} \leq \sqrt{\int_0^1 f(z)^2 dz} + \sqrt{\int_0^1 g(z)^2 dz} = \|x\| + \|y\|$ by Minkowski's inequality (or just the triangle inequality)

4. Let λ_j be the weights and b_j the biases in the first layer, w_{ji} be the weights and β_i the biases in the second layer, and α_i be the weights and c the bias in the final layer. Denote θ as all of these parameters. The output is:

$$f(x; \theta) = c + \sum_{i=1}^{N_2} \alpha_i \sigma \left(\sum_{j=1}^{N_1} w_{ji} \sigma(\lambda_j x + b_j) + \beta_i \right) \quad (5.20)$$

5. We can write $a_1 f(x; \theta_1) + a_2 f(x; \theta_2) = f(x; \theta_3)$ for some θ_3 , which establishes closure under linear operators. So the space of functions $f(x, \theta)$ parameterised by θ is a linear space.

Notions of complexity

Reading list and references

[Raj \[2018\]](#)
[Telgarsky \[2016\]](#)
[Kolmogorov \[1965\]](#)
[Bartlett and Mendelson \[2002\]](#)
[\[Anthony and Bartlett, 2002, Chapter 3\]](#)
[\[Calin, 2020, Chapter 13\]](#)

6.1 Introduction

In this lecture, we will look at the idea of complexity, and why it matters.

A fundamental question in the history of neural networks is why deep neural networks work well, as opposed to the ‘wide’ neural networks encountered in the universal approximation theorem. We will look at this question and try to quantify the answer, relating it back to the general notion of trying to compress information in a useful way.

We will look at an example of a function which can be learned efficiently with a deep neural network but not with a shallow one. We will use this to discuss a very general idea of ‘complexity’, proposed by Kolmogorov, which seems at first too general to be useful. We will then start to look at the other side of the coin; complexity of classes of functions. We will introduce an idea called the Vapnik-Chervonenkis dimension, which describes how ‘complex’ a class of functions is.

6.2 The aim of neural networks and machine learning

Recall that our general aim in neural networks is to approximate some function $f : X \rightarrow Y$, for some sets X and Y , which comes from some class of functions $f \in F$. We do this by means of a neural network which implements functions in some class G . That is, we want to find a $g \in G$ such that g is ‘close’ to f . For example, if f is a continuous function from $[0, 1]$ to $[0, 1]$, we may want to find a function $g \in G$ such that:

$$\sup_{x \in [0, 1]} |f(x) - g(x)| < \epsilon$$

for a suitably small ϵ .

We have talked about the kind of function classes G that we can so implement. With a single neuron, and a Heaviside activation function, we can implement the class of functions like $g(x) = 1_{x > r}$, for some r . With more layers, we can implement more complicated classes of functions, and if we allow lots of neurons in a single layer, we can implement arbitrary functions arbitrarily well (by the universal approximation theorem).

6.2.1 Sizes of spaces and function spaces

When *training* a neural network, we have to ‘search’ through G to find a good g . For this reason, we want a fairly ‘small’ G , or a G that is easy to search. In order to specify a ‘small’ G , it is helpful to have an idea of how ‘big’ or ‘small’ the class F is: that is, we want to have the smallest G possible that allows us to represent the class F . What do we mean by ‘small’ though? This turns out to be a difficult question.

6.2.2 Machine learning

We discussed, early in the course, the notion that both supervised learning and unsupervised learning tend to concern information *compression* - that is, the efficient representation of an object X (which tends to be considered as a random variable):

$$X \rightarrow X' \leftrightarrow Y$$

Put (slightly) more formally: if our data is considered as an instance of a random variable X , we want to find some function h and $X' = h(X)$ such that $(Y|X) \sim (Y|X')$; that is, X' gives us the same information about Y that X does, but X' is *simpler* or *smaller*.

We generally don't mind if h is not invertible; all we care about is Y , and the information we lose going from X to X' is fine if it doesn't tell us anything useful about Y .

This is obviously an analogous question to the idea of 'smallness' of F and G above, and this is no coincidence!

6.3 Depth or width

Since Cybenko's approximation theorem, and other similar celebrated results (which we covered in the last two weeks), researchers in machine learning were aware that depth was not technically needed to approximate arbitrarily complex functions.

We will look at a construction by [Telgarsky \[2016\]](#) of a function which can be represented by a deep network easily, but a shallow network only with difficulty.

6.3.1 Example 1: The sawtooth function

Let $\sigma(x) = \max(x, 0)$ (the ReLU function). Now let

$$h(x) = \sigma \left(2\sigma(x) - 4\sigma \left(x - \frac{1}{2} \right) \right)$$

We are interested in the composition of h with itself. What does the plot of $h(x)$ look like? It's a pain to figure it out, so let's just plot it, along with $h^2(x) = h(h(x))$ and $h^n(x) = \underbrace{h(h(\dots h(x) \dots))}_{n \text{ times}}$ (figure 6.1):

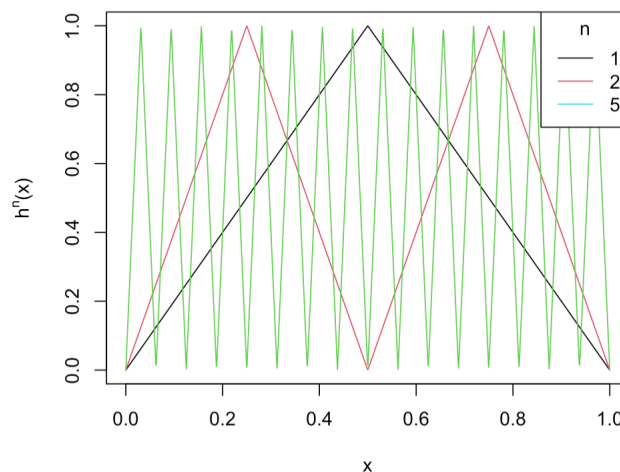
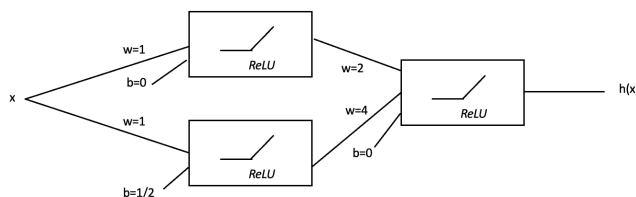


Figure 6.1: Plots of $h(x)$, $h^2(x)$ and $h^5(x)$

It is clear that $h^n(x)$, for $x \in (0, 1)$, zig-zags between zero and one 2^n times. The function $h^n(x)$ is very easy to implement with a neural network with ReLU activations. To implement $h(x)$ we just need the following network (figure 6.2):

Figure 6.2: Neural network implementing $h(x)$

Implementing $h^n(x)$ is straightforward by chaining these together, using $2n$ layers with $3n$ neurons in total. How many neurons does it take in a two-layer network, with ReLU activation functions, to implement $h^n(x)$? Considering the class of functions that can be implemented by a function of the form:

$$f_m(x) = \sigma \left(\sum_{i=0}^m \alpha_i \sigma(w_i x + b_i) \right) \quad (6.1)$$

it's pretty clear that adding another neuron to the hidden layer (i.e. increasing m by 1) adds at most one discontinuity to f_m . Since our function $h^n(x)$ has $2^n - 1$ discontinuities, we will need $m = 2^n$ neurons to implement this function.

So we need only a *linear* number of neurons to implement this with a deep network, but an *exponential* number of neurons to implement it with a shallow network.

6.3.2 Example 2: The Cantor set

The [Cantor set](#), which many of you will have heard of, is an object with many unintuitive properties which is often used to construct counterexamples in analysis. It is defined as follows.

1. We define the 0-Cantor set as the interval $[0, 1]$
2. We remove the middle third of the interval $[0, 1]$ to get the 1-Cantor set
3. Given the n -Cantor set, we remove the middle third of all intervals in the set to get the $n + 1$ -Cantor set

The Cantor set is the limit of the n -Cantor set as $n \rightarrow \infty$. The first six steps give the following (from Wikipedia):



We are interested in the *indicator function* for the n -Cantor set; that is, the function $f(x)$ where:

$$f_n(x) = \begin{cases} 1 & \text{if } x \in n\text{-Cantor set} \\ 0 & \text{otherwise} \end{cases}$$

How many neurons do we need to implement this with a deep or wide network?

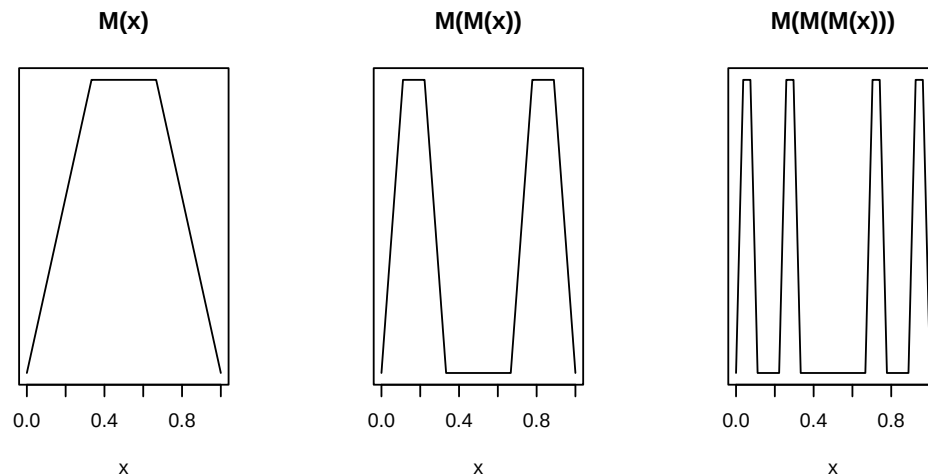
We will allow ourselves the luxury of a special function in the final layer which is the indicator function for the complement of the set $\{0, 1\}$; that is, we define the activation function for the final layer as σ_{final} where:

$$\sigma_{\text{final}}(x) = \begin{cases} 1 & \text{if } x \notin \{0, 1\} \\ 0 & \text{otherwise} \end{cases}$$

With this in mind, we now define the ‘M-function’:

$$M(x) = \text{ReLU}(3x) + \text{ReLU}(3x - 3) - \text{ReLU}(3x - 1) - \text{ReLU}(3x - 2)$$

If we plot $M(x)$, $M(M(x))$, and $M(M(M(x)))$:



we begin to see a clear pattern: we can say that

$$f_n(x) = \sigma_{\text{final}} \left(\underbrace{M(M(\dots M(x)))}_{n \text{ times}} \right)$$

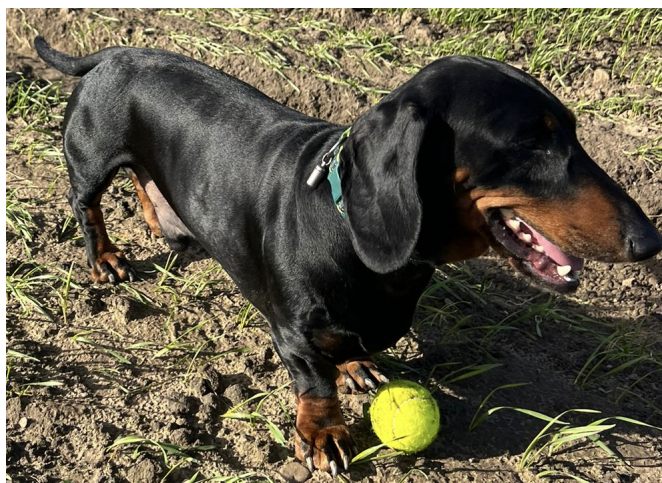
As for the sawtooth function, we can implement this with $O(n)$ neurons when we can use multiple layers, but (given the number of discontinuities) we need $O(e^n)$ neurons in a single layer.

6.3.3 Discussion

What is special about the sawtooth function and the indicator function for the Cantor set?

Intuitively, they both have ‘repeated structure’: the same motif appears, possibly rescaled to different sizes, repeatedly in the graph of the function. We use depth to ‘refer’ to such patterns more than once, and thereby implement the function more efficiently. However, **if the function is arbitrary and does not have substructure**, we cannot make it simpler by using depth: we may as well use a very wide neural network.

This idea is maybe the single most important idea in the DLAI4 course. The reason deep neural networks tend to be useful is that *most real world data which we are interested in has some kind of structure, and deep neural networks can model a wide range of structures*. For example, we will discuss a type of network called a convolutional neural network for image processing later in the course. When processing an image, the first layer of neurons may pick up very basic features like *edges* or *block colours*. The next layer may combine the information from the first layer into more complex features like *shapes* or *boundaries*. The next layer may combine these shapes into *faces* or *paws*. The next layer may combine these elements into the sort of elements humans associate with pictures - people, animals, houses.



Quantifying exactly what we mean by ‘structure’ is difficult and there is no perfect way to do it (in fact, it comes down to some very deep questions in computer science). But we will try.

6.4 Kolmogorov complexity

We can marry the idea of

- X' being ‘simpler’ than X
- A ‘sawtooth’ being ‘simpler’ than a non-sawtooth

in a very fundamental way. [Kolmogorov \[1965\]](#) suggested quantifying the complexity of a set based on *how difficult it is to describe*. Formally,

Definition 9. Given a binary sequence x , the Kolmogorov complexity of x is the minimum length of a program in a standard language which outputs x

Kolmogorov complexity is defined with reference to a ‘standard language’, which we can think of as a programming language, although English also works. It is also defined in terms of a sequence, rather than (e.g.) a picture, but that is largely for convenience; we can convert a picture to a binary sequence and back straightforwardly.

We are not really interested in the *exact* complexity of a sequence so much as how the Kolmogorov complexity of a particular type of sequence changes with the length of that sequence. Really, we are interested in the question:

Given a series A_n of sets of binary sequences, let $\bar{K}(n)$ be the mean Kolmogorov complexity for a sequence in A_n . How does $\bar{K}(n)$ change with n ?

To do this we will quickly revise asymptotic notation, which we use to describe the growth ‘rate’ of messy functions for which more specific inequalities are not practical:

- If $\bar{K}(n) = O(n)$ then we are saying ‘ $\bar{K}(n)$ grows at most as fast as a linear function of n ’, or formally: there exists $r, N > 0$ such that for all $n > N$ we have $\bar{K}(n) \leq rn$.
- If $\bar{K}(n) = \Omega(n)$ then we are saying ‘ $\bar{K}(n)$ grows at least as fast as a linear function of n ’, or: there exists $r, N > 0$ such that for all $n > N$ we have $\bar{K}(n) \geq rn$.
- If $\bar{K}(n) = \Theta(n)$ we are saying ‘ $\bar{K}(n)$ grows linearly in n ’, or: there exists r_1, r_2 with $0 < r_1 \leq r_2$ and $N > 0$ such that for all $n > N$ we have $r_1 n \leq \bar{K}(n) \leq r_2 n$

6.4.1 Examples

Let us use the programming language R as our standard language, and consider some examples.

- If A_n is the set of *all* binary sequences of length n , then we generally cannot do any better than writing the whole sequence out in the program. For instance, for the sequence 01001000110001100101, we can really only write

```
print("01001000110001100101")
```

So as n increases, we generally need to write a program of length cn (for some c) to output a member of A_n . So $\bar{K}(n) = \Theta(n)$.

- If A_n is the set of binary sequences of the form 000...0011...1100...000; for instance, 0000001111111111000, then we can write a program like:

```
print(paste0(
  rep("0",6),
  rep("1",11),
  rep("0",3)
))
```

This is much more efficient! We do need to consider that we still have to specify the lengths of the 0/1 sequences in the program (in this example, 6,11,3), and for large n this will be the biggest part of the program. Since we need about $\log_{10}(m)$ digits to specify m , and the largest of the three numbers in the program will have a similar number of digits to n , the shortest program will be of length $c \ln(n)$ for some constant c , so $\bar{K}(n) = \Theta(\ln(n))$

- If A_n is the set of binary sequences containing only 1's, then likewise $\bar{K}(A_n) = \Theta(\ln(n))$ (since we need to specify the length of the sequence in the program).

It is clear that we cannot do any better (asymptotically) than $\bar{K}(n) = \Theta(\ln(n))$ (since we need to specify the length of the sequence) and we cannot do any worse (asymptotically) than $\bar{k}(n) = \Theta(n)$ (since this allows us to directly specify the sequence in the program).

6.4.2 The Mandelbrot set

To illustrate the usefulness of Kolmogorov complexity, suppose we consider the Mandelbrot set. If, for complex z , we define:

$$f_c(z) = z^2 + c$$

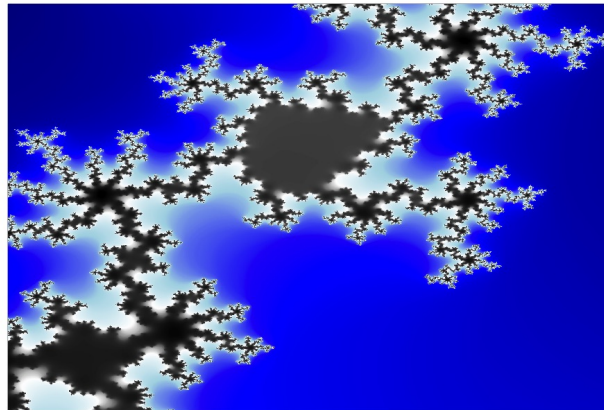
and

$$h^1(c) = 0, \quad h^n(c) = f_c(h^{n-1}(c)) \text{ for } n > 1$$

then the Mandelbrot set is defined as:

$$\{c : c \in \mathbb{C}; \lim_{n \rightarrow \infty} |h^n(c)| \text{ diverges}\}$$

The [Mandelbrot set](#) is fantastically complex:



but from a Kolmogorov complexity perspective, it is actually very simple (since we can define it very quickly). If we consider a line segment in the complex plane, consider n equally spaced points along the line, and define A_n as binary sequence which is 1 according to whether each of these points is in the Mandelbrot set¹, then the Kolmogorov complexity turns out to be $\bar{K}(n) = \Theta(\ln(n))$.

6.5 How simple are function classes?

We have looked at how to quantify how complex *the function we are trying to approximate* is, and then looking at how to approximate it. Another way of looking at this is to ask: given a particular class of approximators (for instance, a particular neural network) how complex is the class of functions which we are able to approximate?

We have already looked at this a bit (or rather a lot): we looked at simple neurons, which could implement half-planes; and simple neural networks which could implement indicator functions for convex shapes. We will now formalise this a bit.

¹Actually we can only do this for a finite approximation of the Mandelbrot set; we cannot determine in finite time whether any given point escapes or not. See exercises.

6.5.1 Number of parameters paradox

Another place you have probably seen this is in the number of *degrees of freedom*, or rather the number of parameters, of a statistical model. This is a very useful idea but it can be misleading.

Suppose we want to fit a linear model to some data, which consists of pairs $x_i, y_i, \in 1 \dots n$. We have the option of either using a simple linear model:

$$Y = b_0 + b_1 X + \epsilon, \quad \epsilon \sim N(0, \sigma^2) \quad (6.2)$$

or we could use some higher-order terms for X , say 10:

$$Y = b_0 + b_1 X + b_2 X^2 + \dots b_{10} X^{10} + \epsilon, \quad \epsilon \sim N(0, \sigma^2) \quad (6.3)$$

The second model has more parameters (12, rather than 3), so it looks like a more complicated model which is able to fit more complex functions, and indeed it is.

But simply counting the number of parameters can lead us astray. We will consider an extreme example:

Theorem 15. There exists a bijective map $f: \mathbb{R} \rightarrow \mathbb{R}^n$ for any finite n .

Proof. We will begin by showing a bijective map $f'(\cdot)$ from $(0, 1)$ to $(0, 1)^n$.

Consider the usual decimal representation of real numbers; that is, equating $0.999\dots$ to 1.

Given a point $X = (x_1, x_2, \dots, x_n)$ in $\mathbb{R}^n$, write the decimal representations of x_i as

$$\begin{aligned} x_1 &= 0.a_1 b_1 c_1 \dots \\ x_2 &= 0.a_2 b_2 c_2 \dots \\ &\dots \\ x_3 &= 0.a_n b_n c_n \dots \end{aligned}$$

We associate this point X with the decimal representation:

$$f'(X) = 0.a_1 a_2 \dots a_n b_1 b_2 \dots b_n \dots$$

that is, splicing the digits of the x_i . It is obvious that this map is bijective: we can reconstruct X perfectly from $f'(X)$.

To extend this to a map from $\mathbb{R}$ to $\mathbb{R}^n$, we can simply chain f' with a bijective function from $(0, 1)$ to $\mathbb{R}$; for instance if $g(x) = \tan\left(\pi\left(x - \frac{1}{2}\right)\right)$; $g^{-1}(x) = \frac{1}{2} + \frac{1}{\pi} \tan^{-1}(x)$, and

$$f(x) = g^{-1}(f'(g(x)))$$

□

Why is this annoying? Let's say we denote a model fitted according to equation 6.2 as:

$$\gamma(\beta_0, \beta_1, \sigma)$$

and a model fitted according to equation 6.3 as

$$\zeta(\beta_0, \beta_1, \dots, \beta_{10}, \sigma)$$

But let's denote the bijective function mapping $\mathbb{R}^{12}$ to $\mathbb{R}^1$ as g . Then we can define:

$$\xi(\alpha) = \xi(g(\beta_0, \beta_1, \dots, \beta_{10}, \sigma)) = \zeta(\beta_0, \beta_1, \dots, \beta_{10}, \sigma)$$

that is, we can write the model fitted to 6.3 equally well with just one parameter, making it look even simpler than the model fitted to 6.2! We have done this bijectively, so we have not lost any information about the model either.

To actually resolve this question, we need to be quite analytically careful and say something about the topology of the model space and the parameters space. We will leave that to the pure mathematicians, who enjoy that sort of thing, but we will just remember for the moment to be careful with 'parameter count' as an index of complexity.

6.5.2 The VC dimension

We will look at an idea called the Vapnik-Chervonenkis dimension [Anthony and Bartlett, 2002, Chapter 3] (which we will refer to as the VC dimension) which gives a somewhat more useful quantification of the complexity of a general class of functions. We will define the VC dimension in slightly less than full generality, which will suffice for our purposes. To define the VC dimension, we first define ‘shattering’ of a set of data points as follows.

Definition 10. Given a class F of functions from $\mathbb{R}^d \rightarrow \{0, 1\}$, and a set $S = \{x_1, x_2, \dots, x_r\}$ with $x_i \in \mathbb{R}^d$, we say that F ‘shatters’ S if for any subset $S' \subseteq S$, there exists an $f \in F$ such that

$$f(x_i) = 1 \text{ for all } x_i \in S'$$

and

$$f(x_i) = 0 \text{ for all } x_i \in S \setminus S'.$$

That is, for any subset S' of the points of S , we can find an $f \in F$ whose decision boundary separates S' from the remainder of the points in S .

The VC dimension is defined as follows:

Definition 11. Given a class of functions F from $\mathbb{R}^d \rightarrow \{0, 1\}$, the VC dimension of F is the size r of the largest set of points $S = \{x_1, x_2, \dots, x_r\}$ in general position in $\mathbb{R}^d$ such that F shatters S .

By ‘general position’, we mean points that are:

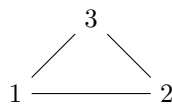
- Not coincident
- No three points lie on a line, no four points lie on a plane, ...

We will only really be concerned with $d = 2$, so ‘general position’ means that no three points lie on a line and no two points are coincident.

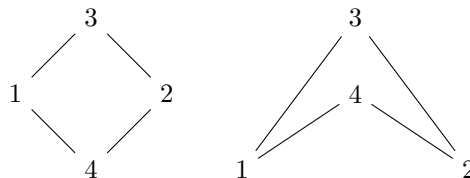
6.5.3 VC dimension of various classifiers

Let us consider a few classes F of functions from $\mathbb{R}^2$ to $\{0, 1\}$ (that is, binary classifiers of points in the plane).

- Consider the class $F = \{f(x) = 0, f(x) = 1\}$. That is, F contains only the trivial functions which are 1 everywhere or 0 everywhere. Then we cannot shatter any set of two points: if S is a set of two points, and S' is one of them, then no function in F can be 1 for the point in S' and 0 for the other. So the VC dimension of F is 1.
- Consider the class F of functions which can be implemented by a single neuron with a Heaviside activation function. We know that this is the set of indicator functions for half-planes. If we have three points in the plane:

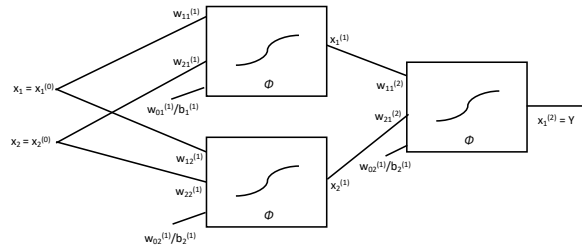


then we can clearly separate any two from the other with a half-plane, so the VC dimension of F is at least 3. If we have four points (labelled 1,2,3,4) then consider the triangle formed by points 1,2,3. Either the fourth point is outside this triangle (e.g. the quadrilateral formed by the points is convex) or it is not (the quadrilateral is nonconvex):

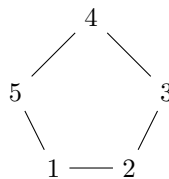


If the fourth point is outside the triangle (left figure) then we cannot separate $S' = \{1, 2\}$ from $S \setminus S' = \{3, 4\}$ with a half-plane. If point 4 is inside the triangle (right figure) then we cannot $S' = \{4\}$ from $S \setminus S' = \{1, 2, 3\}$ with a hyperplane. So the VC dimension is 3.

- Consider the set F of functions implemented by a neural network with one hidden layer containing two neurons:



As we discussed in the first module, this network can implement indicator functions of *intersections of two half-planes*. Consider five points forming a convex pentagon:



With a half-plane, we can separate any one point off, or divide the figure into a set of two points and a set of three points. With an additional half-plane, we can shatter the set of two points or the set of three points. In general, we can separate any subset of points from its complement using the intersection of two half-planes. So the VC dimension of this classifier is *at least 5*.

6.5.4 VC dimension in sample complexity

The VC dimension shows up in a lot of important results [Blumer et al., 1989] about how effectively an unknown function can be learned (which, recalling the first lecture in this course, is one conception of the core aim of machine learning).

We will go over one important theorem in this area, concerning the sample complexity (or in general, PAC learning). This is barely brushing the surface of a very large body of work, but is sometimes called the ‘Fundamental Theorem of Statistical learning’ [Shalev-Shwartz and Ben-David, 2014, Theorem 6.7], with good reason: it essentially says that a finite sample complexity is equivalent to a finite VC dimension, or that we can consistently learn a function only if it comes from a reasonably ‘simple’ class. We use a couple of simplifying assumptions:

- We are trying to learn a *binary* function; that is, one returning 0 or 1. For example, we might be trying to classify pictures as being of a cat or a dog, or trying to predict whether a person will have a heart attack in the coming year or not.
- We use a 0-1 loss function; that is, we want to minimise the *probability of making an error*.

We recall the definition of sample complexity from an earlier lecture:

Definition 12 (Learnability and sample complexity). Suppose we have:

- Some set $\mathcal{X}$ from which data can come, and ‘labels’ $\mathcal{Y}$ which can be either 0 or 1.
- A set of ‘hypothesis’ functions $\mathcal{F}$ which map $\mathcal{X}$ to $\{0, 1\}$, from which we will choose a function f .

- For a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, and a function $f \in \mathcal{F}$, we consider the generalisation error using 0-1 loss:

$$GE(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}} \{C(Y, f(X))\} = \mathbb{E}_{(X,Y) \sim \mathcal{D}} \{1\{f(X) \neq Y\}\} = P_{\mathcal{D}}(f(X) \neq Y)$$

- We have a ‘learning algorithm’ $\mathcal{A}$ which takes some number of samples from $(\mathcal{X} \times \mathcal{Y})$ and returns a function $f \in \mathcal{F}$.

We say that $\mathcal{A}$ is ‘consistent’, and $\mathcal{F}$ is ‘learnable’, if, given $\epsilon, \delta > 0$, we can find a large enough n such that if we have at least n training samples $D^n \sim \mathcal{D}^n$, then

$$P_{D^n \sim \mathcal{D}^n} \left(GE(\mathcal{A}(D^n)) - \inf_{f' \in \mathcal{F}} GE(f') > \epsilon \right) < \delta$$

The smallest $N = N(\mathcal{D}, \epsilon, \delta)$, considered as a function of $\mathcal{D}, \epsilon, \delta$, for which this holds, is called the *sample complexity* of $(\mathcal{A}, \mathcal{F})$.

We are concerned with the ‘agnostic’ learnability; that is, the worst possible sample complexity over all possible distributions $\mathcal{D}$ over $(\mathcal{X} \times \mathcal{Y})$:

$$N(\epsilon, \delta) = \max_{\mathcal{D}} N(\mathcal{D}, \epsilon, \delta)$$

We then define the agnostic learnability as follows:

Definition 13 (Agnostic learnability). A set of functions $\mathcal{F}$ which map $\mathcal{X}$ to $\mathcal{Y}$ is *agnostically learnable* if there exists a learning algorithm $\mathcal{A}$ such that $N(\epsilon, \delta)$ is finite for all $\epsilon, \delta \in (0, 1)^2$.

We now have:

Theorem 16. The set of functions $\mathcal{F}$ is agnostically learnable if and only if it has finite VC dimension.

Indeed, we can even say *what the sample complexity is* in terms of the VC dimension:

Theorem 17. Suppose $\mathcal{F}$ has VC dimension d . Then there exist constants c, C such that:

$$c \frac{d + \log\left(\frac{1}{\delta}\right)}{\epsilon^2} \leq N(\epsilon, \delta) \leq C \frac{d + \log\left(\frac{1}{\delta}\right)}{\epsilon^2}$$

For proofs (non-examinable), see [Shalev-Shwartz and Ben-David, 2014, Theorem 6.7]. A few notes:

- This theorem essentially says that the function can only have limited complexity.
- Not every set of functions has a finite VC dimension: if we consider e.g. any continuous function, then it’s unsurprising that we can’t necessarily learn this no matter how many samples we have.
- It’s relatively obvious that we can learn a function if $\mathcal{H}$ is *finite*: we only need to check finitely many cases. Learnability is less obvious if $\mathcal{H}$ is infinite: it turns out to be its VC dimension that matters.
- There are other ways of assessing complexity.

6.6 Discussion

We have looked at notions of how ‘complex’ a dataset (taken a binary string) or set of functions is, and looked at why this matters in terms of representability by *deep* neural networks. In the next couple of lectures we will look at sigma algebras, entropy and mutual information.

Exercises

1. Suppose we have a function $f : \mathbb{R} \rightarrow \mathbb{R}$ which satisfies:

- f is bounded and differentiable and has bounded derivative,
- f is periodic with period 1
- f is symmetric (that is, $f(x) = f(-x)$)

We will try and approximate f on an interval $[0, n]$, where n is an integer greater than 1, using a neural network in which all neurons have ReLU activations, except for the neuron in the output layer which has an identity activation. Show the following:

- Show that, for any $\epsilon > 0$, we can approximate $f(x)$ on $[0, 1]$ by a function $g(x)$ where $|f(x) - g(x)| < \epsilon$ and $g(x)$ is exactly implemented by a network as above with one hidden layer.
 - Show that if $s_n(x)$ is the ‘sawtooth’ function with 2^n peaks, then $f(x) = f(s_n(2^n x))$.
 - Using the previous two parts, show that for any fixed ϵ we can design a network as above with $\Theta(\ln(n))$ neurons which implements a function h satisfying $|f(x) - h(x)| < \epsilon$ for all $x \in [0, n]$.
2. Can you find a two-dimensional analogue of the sawtooth function? That is, a function $f(x, y)$ which implements a ‘grid’ of $(2^n)^2$ pyramids, which can be implemented using $\Theta(n)$ ReLU neurons?
3. Find the asymptotic average Kolmogorov complexity (in terms of n and k) for the following sets of 0-1 sequences:
- The set A_n of strings of length n beginning with k 1s
 - The set A_n of strings of length n consisting of a sequence of ‘runs’ of 0’s or 1’s of length k , padded with 0s at the end if k does not divide n (for instance, with $n = 17, k = 5$, a sequence like 000001111111111100)
4. Write a function `km` in R which takes the real and imaginary parts (x, y) of a complex number z and returns 1 if z is in the k -Mandelbrot set (for some fixed value of k), defined as: z is in the Mandelbrot set if $|h^k(z)| < 2$ (where $h^k(\cdot)$ is defined as per the section above on the Mandelbrot set). Use this to write a script which outputs a sequence $A_n = \{km(z_1), km(z_2), \dots, km(z_n)\}$, where $z_1, z_2, \dots, z_n$ are a sequence of equally-spaced points between $z_1 = -1 + 0i$, $z_n = 0 + 1i$.
5. Extend theorem 15 to the case where n is countably infinite. Hint: write a list of the decimal representations of the x_i as a grid and think of a way to traverse this grid.
6. Suppose we have a neural network with all-Heaviside activations with two inputs, one hidden layer with *three* neurons, and one neuron in the output layer. Let G_3 be the class of functions expressible by this network. Show that the VC dimension of G_3 is at least six.
7. Suppose we consider the network discussed earlier in the notes, which is the same as the network in the previous question but with two neurons in the hidden layer instead of three. Let G_2 be the class of functions expressible by this network. Show that the VC dimension of G_2 is less than 17. Hint: look at the [happy ending problem](#).

Solutions

1. (a) Suppose that $|f'(x)| < c$. Since f has bounded derivative and is bounded, we can divide $[0, 1]$ uniformly into $k(\epsilon) := \frac{c}{\epsilon}$ equal blocks, such that $f(x)$ varies by less than ϵ in any block. We can then consider a continuous piecewise linear function $g(x)$ which agrees with $f(x)$ at the boundaries of the blocks (that is, the points $x = 0, \frac{\epsilon}{c}, 2\frac{\epsilon}{c}, \dots$) and this piecewise linear function can be exactly output by a network with one hidden layer, $k(\epsilon)$ neurons in the hidden layer, ReLU activation functions in the hidden layer, and an identity activation function in the final layer.
- (b) The function $s_n(2^n x)$ increases linearly from 0 to 1 on any interval $[2k, 2k + 1]$, so

$$s_n(2^n x) = \begin{cases} \{x\} & \text{if } \lfloor x \rfloor \text{ is even} \\ 1 - \{x\} & \text{if } \lfloor x \rfloor \text{ is odd} \end{cases}$$

where we denote $\{x\} = x \bmod 1$ as the fractional part of x , and $\lfloor x \rfloor$ as the largest integer not exceeding x .

Since f is periodic, we have $f(x) = f(\{x\})$, and hence $f(1 - x) = f(-x)$. Since f is symmetric, we have $f(-x) = x$, so

$$f(1 - \{x\}) = f(-\{x\}) = f(\{x\}) = f(x).$$

Hence we have

$$f(s_n(2^n x)) = f\left(\begin{cases} \{x\} & \text{if } \lfloor x \rfloor \text{ is even} \\ 1 - \{x\} & \text{if } \lfloor x \rfloor \text{ is odd} \end{cases}\right) = \begin{cases} f(\{x\}) & \text{if } \lfloor x \rfloor \text{ is even} \\ f(1 - \{x\}) & \text{if } \lfloor x \rfloor \text{ is odd} \end{cases} = f(x)$$

- (c) Let $k = \lceil \log_2(n) \rceil$ (where $\lceil x \rceil$ is the smallest integer at least as large as x). We can implement the function $s_k(2^k x)$ using a ReLU network with $O(k)$ neurons. If we take the output of this network and push it through the network we described in part a, then we get

$$h(x) = g(s_k(2^k x)).$$

Since $s_k(2^k x) \in [0, 1]$ and $|f(x) - g(x)| < \epsilon$ for $x \in [0, 1]$ we have

$$|f(s_k(2^k x)) - g(s_k(2^k x))| < \epsilon$$

for $x \in [0, 2^k]$, so

$$|f(s_k(2^k x)) - g(s_k(2^k x))| = |f(x) - h(x)| < \epsilon$$

for $x \in [0, 2^k]$ also (using the second part). The number of neurons in part a depends only on ϵ , not n , and the number of neurons to implement the sawtooth function is $\Theta(\log(n))$. So overall we have approximated f on $[0, n]$ to accuracy ϵ using $\Theta(\ln(n))$ neurons.

2. There are a few ways to do this. One is as follows. Define:

$$R(x) = \text{ReLU}(2x) - 2\text{ReLU}(2x - 1) + \text{ReLU}(2x - 2),$$

and

$$R_n(x) = \underbrace{R(R(\dots R(x)))}_{n \text{ times}}$$

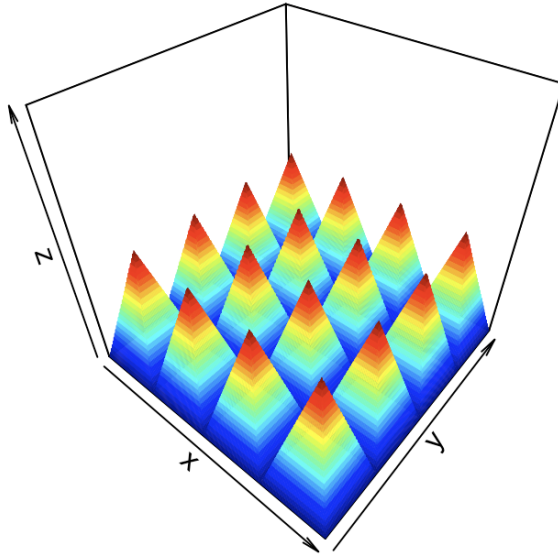
. Now if we define

$$p(x, y) = \text{ReLU}\left(R\left(\frac{x+y}{2}\right) + R\left(\frac{1+x-y}{2} - 1\right)\right),$$

then

$$p(R_n(x), R_n(y))$$

gives the requisite function (shown below for $n = 3$):



3. Something like the following:

```
k=50
km=function(x,y) {
  hx=0; hy=0
  for (i in 2:k) {
    hxb = hx^2 - hy^2 + x
    hyb = 2*hx*hy + y
    hx=hxb
    hy=hyb
    r=sqrt(hx^2 + hy^2)
    if (r>2) return(0) # This needs to be here to avoid overflow
  }
  return(1)
}

# Set length of sequence
n=100

# Generate sequences of Re(zi) and Im(zi)
xm=seq(-1,0,length=n)
ym=seq(0,1,length=n)

# Generate sequence
An=rep(NA,n) # initialise
for (i in 1:n) An[i]=km(xm[i],ym[i])

# Output sequence An
```

The only line which depends on the number n is 'n=100', so this program has length $\Theta(\ln(n))$.

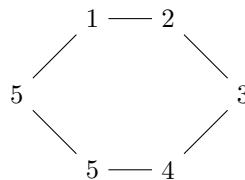
4. We want to find a one-to-one map function from a countably infinite list of real numbers $Z = \{z_1, z_2, \dots\}$ to real numbers. We will presume all numbers are in $(0, 1)$ as in the previous proof, with the usual extension to the whole real line. Suppose $z_1 = 0.d_1^1 d_2^1 \dots$, $z_2 = 0.d_1^2 d_2^2 \dots$ where $0.d_1^i d_2^i \dots$ is the decimal representation of

the i th number in Z . Write these in a table as:

$$\begin{array}{cccc} d_1^1 & d_2^1 & d_3^1 & \dots \\ d_1^2 & d_2^2 & d_3^2 & \dots \\ d_1^3 & d_2^3 & d_3^3 & \dots \\ \vdots & \vdots & \vdots & \vdots \end{array} \quad (6.4)$$

and traverse these in a zig-zag fashion to get the sequence $d_1^1, d_2^1, d_1^2, d_3^2, d_2^3, d_1^3, d_4^3, \dots$. Concatenating these numbers into one number gives the map we need.

5. The set G_3 includes the set of *indicator functions for triangles* and their complements, as well as indicator functions for unions and intersections of half-planes. Given a regular hexagon, we can either draw a triangle around any subset of points, or draw a ‘strip’ round two points and a half plane around the third. So we can shatter the points in a regular hexagon using G_3 , and the VC dimension is at least six.
6. As per the happy ending theorem, any set of 17 points in general position contains a convex hexagon. Given a convex hexagon with the points labelled 1-6 going around:



there is no way to separate points 1,3, and 5 from points 2,4, and 6 with two half-planes. Clearly we cannot separate them with two half-planes: the first half-plane either

- separates the set into subsets of two and four points, in which case another half-plane cannot shatter the set of four points; or
- separates the set into two sets of three points, but this would mean that the set of points 1,3,5 would include points on opposite sides of the first half-plane.

So we cannot shatter a convex hexagon with G_2 , and hence the VC dimension of G_2 is at most 30.

Information as sigma algebras

Reading list and references

[Calin, 2020, Chapters 12]
Ash [2012]

7.1 Introduction

In this and the next lecture we will look formally at neural networks as *information processors*. We will consider the concepts of entropy and mutual information, and look at how we can describe the information lost and retained as we proceed through the layers of a neural network.

Why does this matter? Fundamentally, a neural network (and machine learning more generally) is about data compression. If we want to take a series of photos and determine what they are, we don't care about the colour of the top-left pixel; we want to assign photos to a (relatively short) list of categories. Taken more mathematically, we want to take the space in which photos live - roughly $[256]^{3 \times m \times n}$ - and spit out a much smaller categorisation.

In the next couple of lectures we'll try and formally define this compression. The aim is to give us a mathematical language to talk about this (mostly derived from information theory) and to formally describe various ideas like 'lossless compression'.

We will define input, output, target information, and information loss and retention, mostly using the language of probability. Our aim in this lecture will be to establish when a layer is compressing information or not, and describe conditions on the neurons in the layer under which the input to the neuron is reconstructible from its output.

7.2 Preliminaries

7.2.1 Sigma-algebras

We will be working with sigma-algebras a lot and we will revise what they mean.

Given a set X , we denote the set of subsets of X as 2^X . A set S is a σ -algebra S on a set X if:

1. $S \subseteq 2^X$
2. If $A \in S$ then $(X \setminus A) \in S$ (complementation)
3. If $A_1, A_2, \dots \in S$ then $A_1 \cup A_2 \cup \dots \in S$ (and $A_1 \cap A_2 \cap \dots \in S$)

The σ -algebra *generated* by a set $K \subseteq X$ is the smallest valid σ -algebra S such that $K \subseteq S$.

7.2.2 Sigma-algebra as information

A rough intuition for the definitions of σ -algebras for our purposes is 'the set of things to which we should be able to assign probabilities'. If we consider a set X to be a set of potential outcomes, and $A_1, A_2, \dots$ various outcomes that can occur, then

- It makes sense that if we can say ' A occurred' then we can say ' A did not occur' - hence $X \setminus A$ should be in S
- If we can give a probability to each of A_1, A_2 , occurring, then we should be able to give a probability to A_1 or A_2 occurring

- It doesn't necessarily make sense that we should be able to say what the probability is for an *uncountable* number of sets. In particular, suppose we drop a coin onto a line on the floor, and summarise its position with a number $x \in (0, 1)$. Suppose that the coin has an equal chance of landing anywhere on the line. Then the probability of it landing at any one spot is 0. It makes intuitive sense to have the axiom that if we have a set of events S with probability 0 then the probability of *any* event in S happening is also 0. But this goes out the window if we allow S to be uncountable.

In a sense a σ -algebra describes the precision at which we can observe the sample space. If we are training a neural network to recognise written numbers, then the input information is observed at very fine detail - we see the whole photo. But the output information is just a number 0-9 - so dividing the sample space into ten parts.

7.2.3 Probability space

We define a probability space as a triple (Ω, F, P) , where Ω is a set, F is a σ -algebra on Ω , and P is a function mapping F to $[0, 1]$. Having used a σ -algebra to list the sets of values *for which we should be able to calculate probabilities*, the function P is what we use to actually assign these probabilities. The properties P must have come as little surprise:

1. $P(\emptyset) = 0$ (the probability of nothing happening is nothing)
2. $P(x) \geq 0$ for all $x \in F$ (probabilities are non-negative)
3. If we have a *countable* sequence $X_1, X_2, \dots \in F$ with $(X_i \cap X_j) = \emptyset$, then $P(X_1 \cup X_2 \cup \dots) = P(X_1) + P(X_2) + \dots$ (probabilities are additive for mutually exclusive events)
4. $P(\Omega) = 1$ (the probability of *any* event happening is 1. Why is Ω necessarily in F ? (see exercises))

7.2.4 Formal definition of a random variable, and conception

A *random variable* is a probability space (Ω, F, P) together with a space (Y, E) (where E is a σ -algebra on Y) and a measurable function $f : \Omega \rightarrow Y$. Recall that a measurable function means that for any $\mathcal{E} \in E$ we have

$$f^{-1}(\mathcal{E}) \triangleq \{x \in X : f(x) \in \mathcal{E}\} \in F \quad (7.1)$$

The space Y can be thought of as 'what we can observe', and for a set of observations $\mathcal{E} \in E$, its pre-image in X is the set of things which could have given rise to the set of observations. This naturally shows why we need f to be a measurable function: for any reasonable set of observations, we need to be able to describe probabilistically the set of events which could have led to them.

Why do we think of a random variable as a function, not just a probability space? A probability space seems to be all we need: a random variable, for most purposes, is just something which can take different sets of values with different probabilities. Use of a function for a random variable is largely for convenience - firstly, it makes it easy to define several random variables on the same probability space. Secondly, it tends to be a more natural definition. We observe a random variable, but it often will not tell us *everything* about the underlying random process, or it might tell us *more*. Suppose we drop a coin on a line as before, but this time we colour half the line black and half red and record only the colour. We could just associate the colour on which the coin falls with a probability space, but it is more natural to associate it with the larger probability space showing exactly where the coin fell. [Some examples](#).

An important theorem

We note the following important definition and theorem, which is a lot of help when dealing with σ -algebras generated by random variables:

Definition 14. Suppose X is a random variable with the probability space (Ω, F, P) and measurable space (Y, E) . The σ -algebra generated by X , which we will denote $\mathfrak{S}(X)$, is the σ -algebra generated by:

$$\{X^{-1}(\mathcal{E}), \mathcal{E} \in E\} \quad (7.2)$$

Essentially, F describes a σ -algebra over Ω and is a list of things to which we *should* be able to assign probabilities.

A random variable is sort of ‘what we can observe’: we observe the occurrence of events in E . But this doesn’t necessarily mean we can use X to assign probabilities to everything in F . Roughly, the σ -algebra generated by X is the set of things to which we *can* assign probabilities. As a very simple example, suppose roll a twenty-sided dice, and Ω consists of $1 \dots 20$, and F is all subsets of $\{1, 2, \dots, 20\}$. We can define a random variable X which is 1 if the die comes up even and 0 if it is odd. Then $Y = \{0, 1\}$ and E is all subsets of $\{0, 1\}$, but just knowing X doesn’t enable us to give probabilities to everything in F .

We now have the following intuitive but important theorem:

Theorem 18. Suppose X is a random variable representing a measurable function from probability space (Ω, F, P) to (Y, E) . Let $f : Y \rightarrow Z$ be a deterministic function (technically, let Z be a space Z and D a σ -algebra on Z , and f a function measurable with respect to D). Then for the random variable $f(X)$, we have

$$\mathfrak{S}(f(X)) \subseteq \mathfrak{S}(X)$$

In fact, the converse is also true: if $\mathfrak{S}(Y) \subseteq \mathfrak{S}(X)$ then Y is ‘determined’ by X (that is, $Y = f(X)$ for some deterministic function f).

Proof. **Not examinable.** See Cinlar [2011], proposition 4.4 □

7.3 Information in a neural network

We will consider the input and output of a neural network to be random variables. We will call the set of possible inputs of the neural network Ω and define H as a σ -algebra on Ω to which we can assign probabilities. The random variable X representing the input is a function $X : \Omega \rightarrow \mathbb{R}^n$.

We will use **slightly different notation** to Calin [2020] in this section so as to distinguish the σ -algebra over the range of a random variable from the σ -algebra induced by a random variable. In other words, for a random variable X from the probability space to (Ω, F, P) to the measurable space (E, M) , what we call $X^{-1}(M)$ will be just called M by Calin [2020], with the actual σ -algebra over E generally left unspecified.

7.3.1 Input and output information

We can now define the input information:

Definition 15. The *input information*, which we call I , is a σ -algebra on $\mathbb{R}^n$ (the range of the input random variable X) such that $X^{-1}(i) \in H$ for $i \in \mathcal{I}$. We assume that the input information is as large as possible: that is, for any set $s \in 2^{\mathbb{R}^n}$ such that $X^{-1}(s) \in H$, we have $s \in I$ (note that this is not a necessary assumption for a random variable).

The neural network implements a function g , whose domain is X . We suppose the set of possible outputs of the neural network as $\mathbb{R}$ and denote the output random variable as $Y : \Omega \rightarrow \mathbb{R}$. The output information can now be defined:

Definition 16. The *output information*, which we call E , is a σ -algebra on $\mathbb{R}$, such that for $e \in E$ we have $Y^{-1}(e) \in H$. As for the input information, we assume that the output information is as large as possible.

If the neural network implements a function g , then we have that $Y = g(X)$, or for $w \in \Omega$, we have $Y(w) = g(X(w)) = (g \circ X)(w)$.

Given a function f and a σ -algebra S we define the pre-image of S as the set of pre-images of the elements of S , e.g.

$$f^{-1}(S) = \{x : x = f^{-1}(s), s \in S\} \quad (7.3)$$

We define the σ -algebra *generated by* X as the pre-image $X^{-1}(H)$. We now have the following simple theorem:

Theorem 19. With (Ω, H, P) as a probability space and X, Y, H, E defined as above, the σ -algebra generated by Y is a subset of the σ -algebra generated by X ; that is, we have $Y^{-1}(E) \subseteq X^{-1}(H)$.

Proof. Suppose on the contrary that there existed $e \in \mathcal{E}$ such that $Y^{-1}(e) \notin X^{-1}(H)$. Denote $G = g^{-1}(e)$. We have

$$Y^{-1}(e) = (g \circ X)^{-1}(e) = X^{-1}(g^{-1}(e)) = X^{-1}(G)$$

By assumption, $Y^{-1}(e) \in H$. But $G = X(Y^{-1}(e))$. From the assumption that I is as large as possible, we must have $G \in I$. But this means that $Y^{-1}(e) \in X^{-1}(I)$, contradicting the assumption. □

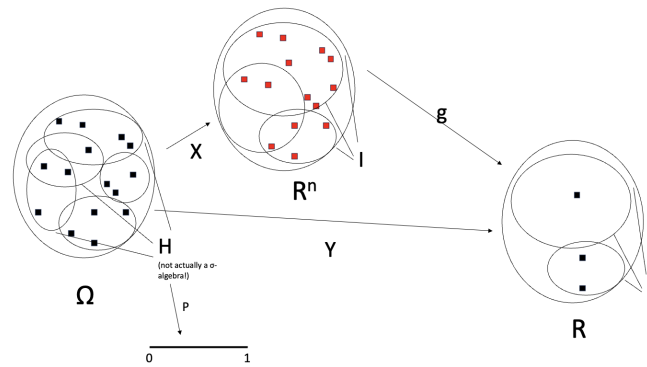


Figure 7.1: Schematic of probability space (Ω, H, P) , and random variables X and Y .

A schematic of the input and output information is shown in figure 7.1.

7.3.2 Target information

We define the *target information* as a σ -algebra Z over Ω which represents what we are trying to predict. If $Z \subseteq X^{-1}(I)$ then we say that Z is *learnable* from X .

7.3.3 Lost information

The *lost information* $\mathfrak{L}$ is (unsurprisingly) the information *lost* when we go from X to Y ; that is, the σ -algebra generated by the difference between $X^{-1}(I)$ and $Y^{-1}(E)$:

$$\mathfrak{L} = \mathfrak{G}(X^{-1}(I) \setminus Y^{-1}(E)) \quad (7.4)$$

7.3.4 Example

Suppose our learning task is to take a photograph of a handwritten number and return the number that is written (we will do this task in workshops). We take the photo (for simplicity) as a 256×256 pixel image, in which each pixel intensity is a real number. We return a natural number 0-9.

In this case, we have:

- Ω represents the space of *actual handwritten drawings*, which is covered by a much larger space than $\mathbb{R}^{256 \times 256}$.
- H represents a large σ -algebra over Ω , in which each element of H is a subset of Ω for which, given an observation $\omega \in \Omega$, we could decide whether ω is in that subset. In particular, H includes the subsets $\{\omega \in \Omega : \omega \text{ is a drawing of a } 7\}$ etc.
- The probability measure over H is known; that is, we know the proportion of pictures we are likely to encounter which are of a 7.
- The random variable X represents a function mapping the actual image to a 256×256 photo of the image, which is in the space $\mathbb{R}^{256 \times 256}$.
- For each ‘reasonable subset’ h in H , there is a corresponding set in I consisting of the set of photos of the elements of h . The input information I is the set of such sets of photos.
- The function g maps $\mathbb{R}^{256 \times 256}$ to the set $[9] = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. The output information is the σ -algebra $2^{[9]}$.
- The target information Z is the σ -algebra generated by *the set of pictures of each number*; e.g., the set of pictures which are intended to depict a 7.

- The target information is learnable if $X^{-1}(H)$ actually contains Z ; that is, the set of photos of pictures intended to be a 7 can be distinguished from the others.
- The lost information is all ‘identifiable sets of photos’ (that is, subsets of $\mathbb{R}^{256 \times 256}$ in the σ -algebra I) other than those exactly corresponding to the sets of pictures intended to depict give numbers.

7.4 Compressible layers

Since we generally lose information going from inputs to outputs, we would expect that this happens in a sequential way: that is, we lose information as we go through layers.

7.4.1 Terminology

We will consider a deep feedforward neural network (in contrast to the two-layer neural networks we have considered in the lectures on universal representation). We will presume the network has L layers ($L-1$ hidden layers) with $d^{(\ell)}$ neurons in the ℓ th layer. The network input is an n -dimensional random variable $X = X^{(0)} = (x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)})^T$, and the output (activation) from the ℓ th layer ($\ell > 0$) is a $d^{(\ell)}$ -dimensional random variable which we denote $X^{(\ell)} = (x_1^{(\ell)}, x_2^{(\ell)}, \dots, x_{d^{(\ell)}}^{(\ell)})^T$. The output of the neural network is given by $Y = X^{(L)} = (x_1^{(L)}, x_2^{(L)}, \dots, x_{d^{(L)}}^{(L)})^T$.

We denote the σ -algebra of the output from the ℓ th layer as $I^{(\ell)}$, and we give the induced σ -algebra on Ω the explicit name $\Omega^{(\ell)}$, so $I^{(L)} = E$ and $\Omega^{(L)} = Y^{-1}(E)$.

We have

$$X^{-1}(I) = X^{-1}(I^{(0)}) = \Omega^{(0)} \supseteq \Omega^{(1)} \supseteq \dots \supseteq \Omega^{(L)} = Y^{-1}(I^{(L)}) = Y^{-1}(E) \quad (7.5)$$

We denote by $L^{(\ell)}$ the information lost at layer ℓ ; namely

$$L^{(\ell)} = \mathfrak{S} \left(\Omega^{(\ell-1)} \setminus \Omega^{(\ell)} \right) \quad (7.6)$$

We now define *compressed* layers as follows

Definition 17. We call the ℓ th layer of the network:

1. Uncompressed if $L^{(\ell)}$ is trivial; that is, $L^{(\ell)} = \{\emptyset, \Omega\}$
2. Totally compressed if $L^{(\ell)} = \Omega^{(\ell-1)}$; that is, all information is lost
3. Partially compressed if $\{\emptyset, \Omega\} \subset L^{(\ell)} \subset \Omega^{(\ell-1)}$.

7.4.2 Conditions for a layer to be uncompressed

Intuitively, a layer will be uncompressed if we can completely recover the values of $X^{(\ell)}$ from the values $X^{(\ell-1)}$. This suggests that the function implemented by the layer must be invertible. Indeed, we have the following theorem:

Theorem 20. Denote the weights of the j th neuron in layer ℓ as the j th column in a matrix $W^{(\ell)}$ of dimensions $d^{(\ell-1)} \times d^{(\ell)}$, and the biases in a vector $B^{(\ell)}$ of dimension $d^{(\ell)}$. Suppose all neurons in the layer have a common activation function ϕ . Then layer ℓ is uncompressed if:

1. The function ϕ is invertible
2. The number of ingoing and outgoing variables is the same: that is, $d^{(\ell-1)} = d^{(\ell)}$
3. $\det(W^{(\ell)}) \neq 0$

Proof. We have:

$$X^{(\ell)} = \phi \left(W^{(\ell)T} X^{(\ell-1)} - B^{(\ell)} \right) \quad (7.7)$$

applying ϕ to each element of the vector. Since ϕ is invertible we have

$$W^{(\ell)T} X^{(\ell-1)} = \phi^{-1} \left(X^{(\ell)} \right) + B^{(\ell)}$$

and since $W^{(\ell)}$ is invertible ($\det(W^{(\ell)}) \neq 0$) we have

$$X^{(\ell-1)} = \left(W^{(\ell)T}\right)^{-1} \left(\phi^{-1}(X^{(\ell)} + B^{(\ell)})\right) \quad (7.8)$$

which is a measurable function, so $\Omega^{(\ell-1)} \supseteq \Omega^{(\ell)}$. Since we already know $\Omega^{(\ell-1)} \subseteq \Omega^{(\ell)}$, we conclude $\Omega^{(\ell-1)} = \Omega^{(\ell)}$, so the layer is uncompressed. $\square$

Note that the invertibility of ϕ is satisfied by many activation functions, but not ReLU.

Exercises

1. Suppose we have a set X and a subset $A \subseteq X$. What is the σ -algebra generated by A ?
2. Show that the notion of a σ -algebra *generated* by a set of subsets $K \subseteq 2^X$ is well-defined if X is finite. Hint: show that if A and B are σ -algebras containing K , then $A \cap B$ is a σ -algebra containing K .
3. If the set Ω is finite, then what properties must a σ -algebra on Ω have (that is, how can they be simplified)?
4. Show that the axioms of σ -algebras mean that $\emptyset \in S$ and $\Omega \in S$ (σ -algebras must be non-empty)
5. Give examples of neural networks for which the input information I and the output information E satisfy i) $I = E$; ii) $E \subset I$.

Solutions

1. The set $\{\emptyset, X, A, X \setminus A\}$
2. Suppose A and B are σ -algebras on X containing all sets in K . Obviously $k \in (A \cup B)$ for all $k \in K$. We need to show that $A \cup B$ is a σ -algebra. Going through the necessary properties:
 - (a) Since $A \subseteq 2^X$ and $B \subseteq 2^X$, we have $A \cap B \subseteq 2^X$
 - (b) Suppose $x \in A \cap B$. Then $x \in A$ and $x \in B$, and from the complementation property, $(X \setminus x) \in A$ and $(X \setminus x) \in B$. Thus $(X \setminus x) \in (A \cap B)$ and the second property holds.
 - (c) If $x_1, x_2, \dots \in A \cap B$ then $x_1, x_2, \dots \in A$ and $x_1, x_2, \dots \in B$. So $x_1 \cup x_2 \cup \dots \in A$ and $x_1 \cup x_2 \cup \dots \in B$, and hence $x_1 \cup x_2 \cup \dots \in A \cap B$, so the third property holds

So if A and B are σ -algebras containing K , then so is $A \cap B$. If there were no smallest σ -algebra containing K , then there would have to be two σ -algebras of minimum size, neither of which were subsets of the other. But if we call these subsets A and B , then we can make a smaller σ -algebra $A \cap B$, which is a subset of both A and B , giving a contradiction. Note that this argument does not (unfortunately) work for infinite X , though the result does hold.

3. The first two properties are the same. For the third, for *any* set $Q \subset S$, we must have $\cup_{q \in Q} Q \in S$: that is, we no longer need to restrict ourselves to the countability requirement, since 2^Ω is finite.
4. Let A be a set in S . Then by property 2, $\Omega \setminus A \in S$. By property 3, $A \cup (\Omega \setminus A) \in S$, and $A \cup (\Omega \setminus A) = \Omega$, so $\Omega \in S$. By property 2, $\Omega \setminus \Omega \in S$, and $\Omega \setminus \Omega = \emptyset$.
5. (a) For $I = E$: a one-layer neural network with two inputs X_1, X_2 , two neurons, and two outputs, for which the first neuron has input weights $(1, 0)$, and the second has input weights $(0, 1)$, and both have invertible activation functions ϕ and zero bias. Now

$$Y = \begin{pmatrix} Y_1 \\ Y_2 \end{pmatrix} = \begin{pmatrix} \phi(1X_1 + 0X_2) \\ \phi(0X_1 + 1X_2) \end{pmatrix} = \phi \begin{pmatrix} X_1 \\ X_2 \end{pmatrix}$$

which is invertible.

- (b) Again with two inputs, a single neuron with input weights $1, 1$ and any activation function (say, for instance, $\phi(x) = x$). Then the one-dimensional output is

$$Y = \phi(1X_1 + 1X_2) = X_1 + X_2$$

If we say that X_1 and X_2 can both take values $0, 1, 2, 3$, and $\mathfrak{S}(X) = 2^{\{1,2,3\} \times \{1,2,3\}}$, then $\{(X_1, X_2) = (1, 2)\} \in \mathfrak{S}(X)$, but it is not in $\mathfrak{S}(Y)$, since Y returns the same value for $(X_1, X_2) = (1, 2)$ as it does for $(X_1, X_2) = (1, 2)$, so any set in $\mathfrak{S}(Y)$ containing $\{(X_1, X_2) = (1, 2)\}$ must also contain $\{(X_1, X_2) = (2, 1)\}$.

Entropy

Reading list and references

[Calin, 2020, Chapters 13]
Ash [2012]

8.1 Introduction

In this lecture, we will go through some ideas from information theory: in particular, the ideas of entropy and mutual information, and some of their properties in relation to information flow through neural networks.

Why do we care about these? It's helpful to have a language to describe information movement. Information can be thought of as an invariant in a similar way to physical quantities such as energy or momentum, in that we cannot gain information beyond what we are initially given. Thinking about how we decide which information to lose, and which to keep, is the central idea of a lot of machine learning.

8.2 Information-theoretic concepts

8.2.1 Information

We denote the 'information' contained in an *observation* of a discrete random variable as the negative log of the probability of making that observation. Why do we do this?

Surprisingly, it arises from a few simple axioms we want 'information' to satisfy (other than the choice of a natural log)

1. The information we gain from observing an event is a function of the probability of observing it;
2. The information is continuous with respect to the probability;
3. For two independent events x, y , information is additive; that is, the information from x and y is the sum of the information from x and the information from y .

and we employ the following theorem:

Theorem 21. If $f(x)$ is continuous and $f(xy) = f(x) + f(y)$, then $f(x) = c \ln(x)$

Proof. We will prove this in the case that f is differentiable.

Let $y = 1 + \epsilon$. Now from the assumptions

$$f(xy) = f(x + x\epsilon) = f(x) + f(1 + \epsilon) \quad (8.1)$$

Using the continuity of f we have

$$\lim_{\epsilon \rightarrow 0} f(x + x\epsilon) = f(x) + \lim_{\epsilon \rightarrow 0} f(1 + \epsilon)$$

so $f(1) = 0$. Now

$$\lim_{\epsilon \rightarrow 0} \frac{f(1 + \epsilon)}{\epsilon} = \lim_{\epsilon \rightarrow 0} \frac{f(1 + \epsilon) - f(1)}{\epsilon} = f'(1)$$

Dividing equation 8.1 through by $x\epsilon$ gives

$$\frac{f(x + x\epsilon) - f(x)}{x\epsilon} = \frac{f(1 + \epsilon)}{x\epsilon}$$

and taking the limit with $\epsilon \rightarrow 0$ gives

$$f'(x) = \frac{1}{x} f'(1) \quad (8.2)$$

Letting $c = f'(1)$ and integrating both sides yields the result. $\square$

Given the first axiom, we can write the information in observing an event as a function $f(p)$ of the probability p of observing the event. Since the probability of two independent events p, q is pq , the third axiom tells us that $f(p \text{ and } q) = f(pq) = f(p) + f(q)$, which along with the second axiom (continuity of f) and theorem 21, we have

$$f(p) = c \ln(p) = -\frac{\ln(p)}{\ln(e^{-1/c})} = -\log_b(p)$$

for some c , where $b = e^{1/c}$. The b essentially specifies ‘units’ of information:

- If $b = 2$, the resulting units of information are called ‘Shannons’ (after Claude Shannon, a pioneer of information theory).
- If $b = e$, so $f(p) = \ln(p)$, the resulting units of information are called ‘natural units’.
- If $b = 10$, the resulting units of information are called ‘bans’ (coined by Turing) or ‘Hartleys’.

We note a few properties of information:

- Observing an almost certain event (e.g., with probability 1) confers no information
- One Shannon is the information in a fair coin flip (e.g., $p = 1/2$)
- One nat is the information in an observation with probability $1/e$. What has probability $1/e$? One example is the limiting probability that a given sample is in a bootstrap sample.

8.2.2 Entropy

We now come on to the slippery idea of ‘Entropy’. Recalling that the information we defined previously is a function of an event (technically defined on the σ -algebra associated with a probability space), entropy is a property of a random variable.

Information-theoretically, entropy is essentially the *expected* information we get from observing a random variable. For a discrete random variable X taking values x_i with probability p_i , the entropy (termed H , analogously to the thermodynamic quantity) is defined as:

$$H = \mathbb{E}\{-\log_b P_X(X)\} = \sum_i (-p_i \log_b(p_i)) \quad (8.3)$$

taking $p \log(p)$ to be 0 if $p = 0$.

For a continuous random variable X admitting a density function $f_X(\cdot)$, the *differential entropy* (which is **not quite the same**) is defined as:

$$H = \mathbb{E}\{-\log_b f_X(X)\} = \int (-\log_b(f_X(x))) f_X(x) dx \quad (8.4)$$

We begin with a helpful lemma:

Lemma 4. Let X be a discrete random variable taking values in $\mathcal{X}$ and f a function from $\mathcal{X}$ to a space $\mathcal{Y}$. Then $H[f(X)] \leq H(X)$, with equality if and only if $f(x_i)$ is distinct for all $x_i \in \mathcal{X}$ such that $P(X = x_i) > 0$.

Proof. We may consider only x_i with $P(X = x_i > 0)$, since all other x_i contribute nothing to the entropy. For $x_i \in \mathcal{X}$ denote $y_i = f(x_i)$. If the y_i are all distinct, then since the entropy depends only on the probabilities of events, not the value of the events themselves, we have $H[f(X)] = H(X)$.

If the y_i are not distinct, then some x_i 's are 'merged' by f . We may consider f as a composition of a sequence of functions $f^1, f^2, \dots$, possibly countably infinite; say

$$X = X^0 \rightarrow f^1(X) := X^1 \quad X^1 \rightarrow f^2(X^1) := X^2 \quad \dots \quad (8.5)$$

with $f(X) = \lim_{r \rightarrow \infty} X^r$, and where each function f^r 'merges' the value of two elements x_i^{r-1}, x_j^{r-1} of X^{r-1} into a single value x_i^r of X^r . It is easiest to consider the composition of functions as follows: We will show that $H(X) > H(X_1) > H(X_2) \dots$, which will suffice to show that $H(X) > H(f(X))$. Suppose f_r merges elements x_i, x_j . Then, abbreviating $p_k^r = P(X^r = x_k^r)$:

$$\begin{aligned} H(X^{r-1}) &= H[f(X^{r-1})] \\ &= - \sum_k p_k^{r-1} \log[p_k^{r-1}] \\ &= - \sum_{k \neq i, j} p_k^{r-1} \log[p_k^{r-1}] - p_i^{r-1} \log[p_i^{r-1}] - p_j^{r-1} \log[p_j^{r-1}] \\ &> - \sum_{k \neq i, j} p_k^{r-1} \log[p_k^{r-1}] - (p_i^{r-1} + p_j^{r-1}) \log[p_i^{r-1} + p_j^{r-1}] \\ &= - \sum_{r \neq i} p_k^r \log[p_k^r] - p_i^r \log[p_i^r] \\ &= - \sum_r p_k^r \log[p_k^r] \\ &= H(X^r) \end{aligned}$$

using the inequality that $p \log p + q \log q < (p + q) \log(p + q)$ for $0 < p, q \leq 1$. This gives the result. $\square$

The Source Coding Theorem

Why does entropy matter for neural networks? This comes back to the idea of *compression*: the efficient representation of an input. It turns out that entropy is a very useful way of describing this.

We talked about the Kolmogorov complexity of a binary string, which describes the *shortest computer program which outputs it*. This is not necessarily the whole story with neural networks, though: we don't generally need to be perfectly correct, so the *probability* of seeing various inputs also matters.

Suppose that you want to send a (very expensive) message to your friend on the moon, saying who won the Premier League this year (equivalently, you want to build a neural network to predict the outcome, with as small an output as possible). Let's say at this stage in the season there are four possibilities:

- Newcastle
- Manchester City
- Brentford
- Tottenham Hotspur

In order to send the message to your friend, or have your neural network be able to output all possibilities, you need two bits of information to code this: 00 for Newcastle, 01 for Manchester City, 10 for Brentford and 11 for Tottenham.

But let's say that the probabilities of winning are as follows:

Newcastle	0.5	with apologies to Tottenham supporters. Now we still need two bits to fully
Manchester City	0.499	
Brentford	0.001	
Tottenham Hotspur	0	

convey the message, but we will never be conveying 11, and it is very unlikely we will have to convey 10. So we could do it with 1 bit, and we will usually be fine.

Let's generalise this a bit, proceeding roughly. Suppose (for simplicity) that only two teams are in the running, team A and team B (for fairness sake) with probabilities of winning $p, 1 - p$ for some $0 < p < 1$; that is, the winning team can be considered a random variable X with $P(X = A) = p$. We want to send the results of the last n years. How efficiently can we do this? Well, as n gets large, the proportion of times team A won gets close to p , so we are sending a message with close-to np 'A's. Any particular message with np A's has a probability of occurring given by:

$$q = p^{np}(1 - p)^{n - np} = 2^{n(p \log_2(p) + (1 - p) \log_2(1 - p))}$$

So *almost any* message will have about this log probability of occurring. There are going to be roughly $1/q$ of such messages, and

$$\frac{1}{q} = 2^{-n(p \log_2(p) + (1 - p) \log_2(1 - p))} = 2^{nH(X)}$$

We can encode k objects with a binary string of length $\log_2 k$. So we can encode almost-all possible messages using a binary string of length

$$\log_2 \left(\frac{1}{q} \right) = nH(x)$$

This argument generalises into the *Shannon Source Coding Theorem*, which can be informally stated as:

A message consisting of n IID samples from a random variable X can usually be compressed to a message of approximately $nH(X)$ bits, in such a way that the original message is uniquely recoverable.

More formally:

Theorem 22. Suppose Z is an ordered sequence of n independent and identically distributed discrete random variables X . Then, for any $\epsilon > 0$, there exists an N such that we can generate an encoder function that, for $n > N$, maps Z to a binary string of length $n(H(Z) + \epsilon)$, such that Z is uniquely recoverable from the binary string with probability at least $1 - \epsilon$

The [proof](#) is not too difficult (and uses the same intuition as above), though not examinable. The source coding theorem is another useful quantification of how 'complex' an object is; namely when the object is a random variable. How much we can compress the object roughly corresponds to how efficiently we can 'compress' it with a neural network.

Application to neural networks

Now to look at how information and entropy relate to neural networks.

In the last lecture, we considered the transit of information through neural networks in terms of a sequence of σ -algebras. We are going to approach the idea this time using entropy, looking at layer activations as a sequence of random variables. We will treat them as *discrete* random variables, to avoid difficulties with differential entropy.

Recall that we will call the inputs to the neural network $X^{(0)}$, and we will refer to the activations (outputs) from layer $\ell \in \{1, 2, \dots, L\}$ as $X^{(\ell)}$. Each $X^{(\ell)}$ is a function of $X^{(\ell-1)}$. We consider layer (ℓ) and layer $(\ell - 1)$, with entropies $H(X^{(\ell)})$ and $H(X^{(\ell-1)})$. We call the sequence of entropies $H(X^{(0)}), H(X^{(1)}), \dots, H(X^{(L)})$ the *entropy flow* of the network. We define the *entropy leak* as:

$$\Lambda(\ell - 1, \ell) = H(X^{(\ell)}) - H(X^{(\ell-1)}) \quad (8.6)$$

From the earlier lemma, we can see that:

Lemma 5. The entropy leak of layer ℓ is positive, and zero if and only if the layer maps unique values of $X^{(\ell-1)}$ to unique values of $X^{(\ell)}$. In particular, the entropy leak is zero if layer ℓ is invertible.

What happens when we go through a neural network? We typically start with a large messy dataset; for instance, a picture. The random variable associated with the variety of pictures we can get will have a very high entropy. But we're trying to get a simple piece of information out of the picture; for instance, is it of a cat, or of a dog? So our final output variable $X^{(L)}$ will have very low entropy: as low as 1 Shannon.

Note - this is *not* true for the differential entropy, which we must use if we consider the random variable $X^{(0)}$ to be continuous.

8.2.3 Example - autoencoders

An autoencoder, which we will come back to in a later lecture, is a neural network which takes a high-dimensional input (such as a picture), and compresses it to a smaller (sometimes lower-dimensional) space, then tries to recreate the original picture from the lower-dimensional information only, thereby ‘automatically encoding’ the original space of pictures onto a lower-dimensional space. In particular, let’s suppose we are encoding a 256×256 image of black-white pixel values onto a 16×16 image (for simplicity).

The network setup looks a bit like this:

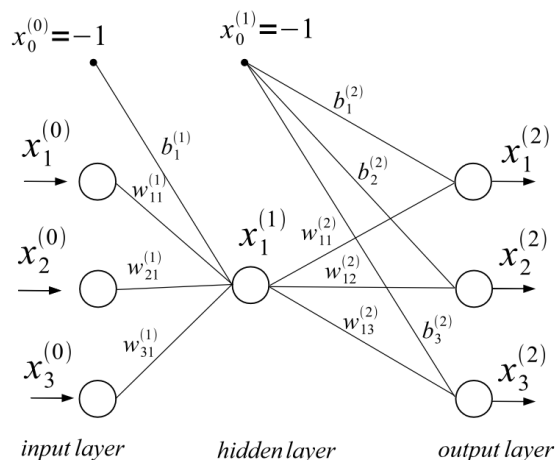


Figure 8.1: Schematic of an autoencoder (taken directly from [Calin \[2020\]](#), figure 12.1. There are three inputs: $x_1^{(0)}$, $x_2^{(0)}$, $x_3^{(0)}$, and three outputs $x_1^{(2)}$, $x_2^{(2)}$, $x_3^{(2)}$. The three inputs are ‘compressed’ down to a single value $x_1^{(1)}$.

It is worth noting that if we were considering pictures as coming from a *continuous* space, we would need to deal with the ‘problem with degrees of freedom’: the amount of information we can encode with three real numbers as an input (or indeed, a thousand-dimension input picture) is the same as what we can encode with a single channel. We can circumvent this difficulty (which we would usually manage by considering topologies) by considering the range of possible pictures to be *large but finite* so the distribution of input information is *discrete*.

The first layer of the network ‘encodes’ the input into the value $x_1^{(1)}$, and the second layer ‘decodes’ $x_1^{(1)}$ into the vector of values $x^{(2)}$. Let’s suppose the entire network implements a function f (mapping $x_1^{(0)}$, $x_2^{(0)}$, $x_3^{(0)}$ to $x_1^{(2)}$, $x_2^{(2)}$, $x_3^{(2)}$). We want to faithfully reconstruct the original picture, so we want $x^{(2)} \approx x^{(0)}$ - and hence $H(x^{(2)}) \approx H(x^{(0)})$.

But how are we going to manage this? Technically we could encounter *every possible picture* for $x^{(1)}$ - but there are far more 256×256 pictures than there are 16×16 (constituting $x^{(1)}$), so we are going to need to ‘merge’ many pictures together, as per the earlier lemma.

The key is that we want to map $x^{(1)}$ to a lower-dimensional variable $x^{(1)}$ while preserving the entropy. We recognise that *some pictures are much more likely to be encountered than other pictures*: with a uniform distribution over possible pictures, most pictures appear to be messy grayscale randomness, but most training sets will include pictures of various objects. Analogously to sending a signal, and reserving short signals for more common events, we want to keep the values of $x^{(1)}$ distinct for *probable* values of $x^{(0)}$, and merge rare events.

By doing this, we can ensure that $H(x^{(2)}) \approx H(x^{(0)})$, even though the entropy $H(x^{(2)})$ is necessarily lower.

Exercises

1. Extend the proof of theorem 21 to the case where f is continuous. To do this, first show that if a function g satisfies $g(x+y) = g(x)+g(y)$ then for some c we have $g(x) = cx$ for all rational x . Conclude that the only continuous g satisfying this equation are of the form $g(x) = cx$ for all x . Consider the function $g(x) = f(e^x)$ to complete the argument.
2. Prove that the entropy of a discrete random variable is positive.
3. Suppose that X is a random variable given by the outcome of a coin flip of a biased coin with probability p of heads. What is the entropy of X , and what is the limiting entropy as $p \rightarrow 0$? For what value of p is the entropy of X maximised?
4. Give a continuous random variable with negative differential entropy (hint: consider the entropy of a Gaussian)
5. Let X be a continuous random variable defined over $\mathbb{R}$, and $Y = aX + b$, with $a > 0$ and b constant. Show that $H(Y) = H(X) + \ln(a)$.

Solutions

1. The equation $g(x+y) = g(x) + g(y)$ is [Cauchy's functional equation](#) which for f continuous is satisfied only for functions $g(x) = cx$. If we set $g(x) = f(e^x)$ then $g(x+y) = f(e^{x+y}) = f(e^x e^y) = f(e^x) + f(e^y) = g(x) + g(y)$, and g is continuous, so $g(x) = cx$. Hence $f(e^x) = cx$, so setting $y = e^x$ we have $f(y) = c \ln(x)$.
2. The entropy of a discrete random variable X taking values x_i with probability p_i is

$$H(X) = - \sum p_i \ln(p_i)$$

The function $f(x) = -x \ln(x)$ is convex when $x > 0$, since $f''(x) = -\frac{1}{x} < 0$. Hence by [Jensen's inequality](#), we have:

$$H(x) = - \sum p_i \ln(p_i) = \sum_i f(p_i) \geq f(\sum_i p_i) = f(1) = -1 \ln 1 = 0 \quad (8.7)$$

3. The entropy is

$$H(X) = - \sum p_i \ln(p_i) = -(p \ln(p) + (1-p) \ln(1-p))$$

and

$$\lim_{p \rightarrow 0} H(X) = - \lim_{p \rightarrow 0} (p \ln(p) + (1-p) \ln(1-p)) = - \lim_{p \rightarrow 0} (p \ln(p)) = - \lim_{p \rightarrow 0} p (1 + p + O(p^2)) = 0$$

The value of $H(X)$ is maximised when $p = 1/2$

4. For a Gaussian distribution $X \sim N(0, \sigma^2)$, the entropy is

$$\begin{aligned} H(X) &= - \int_{\mathbb{R}} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}x^2} \ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}x^2} \right) dx \\ &= - \frac{1}{\sqrt{2\pi\sigma^2}} \ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) \int_{\mathbb{R}} e^{-\frac{1}{2\sigma^2}x^2} dx + \frac{1}{\sqrt{2\pi\sigma^2}} \int_{\mathbb{R}} \frac{1}{2\sigma^2} x^2 e^{-\frac{1}{2\sigma^2}x^2} dx \\ &= \frac{1}{2} \ln(2\pi e \sigma^2) \end{aligned}$$

which is negative if $\sigma < \sqrt{\frac{1}{2\pi e}}$

5. Denoting $Y = aX + b = g(X)$, we derive the PDF of Y from the usual transformation formula for PDFs:

$$\begin{aligned} f_Y(y) &= f_X(g^{-1}(y)) \left| \frac{d}{dy} g^{-1}(y) \right| \\ &= \frac{1}{a} f_X \left(\frac{1}{a}(y-b) \right) \end{aligned}$$

so, noting $dy/dx = a$ and making the substitution $x = \frac{1}{a}(y-b)$

$$\begin{aligned} H(Y) &= - \int f_Y(y) \ln(f_Y(y)) dy \\ &= - \int \frac{1}{a} f_X \left(\frac{1}{a}(y-b) \right) \ln \left(\frac{1}{a} f_X \left(\frac{1}{a}(y-b) \right) \right) dy \\ &= - \int \frac{1}{a} f_X \left(\frac{1}{a}(y-b) \right) \ln \left(\frac{1}{a} \right) dy - \int \frac{1}{a} f_X \left(\frac{1}{a}(y-b) \right) \ln \left(f_X \left(\frac{1}{a}(y-b) \right) \right) dy \\ &= - \frac{1}{a} \ln \left(\frac{1}{a} \right) \int f_X \left(\frac{1}{a}(y-b) \right) dy - \frac{1}{a} \int f_X \left(\frac{1}{a}(y-b) \right) \ln \left(f_X \left(\frac{1}{a}(y-b) \right) \right) dy \\ &= - \frac{1}{a} \ln \left(\frac{1}{a} \right) \int f_X(x) \frac{dy}{dx} dx - \frac{1}{a} \int f_X(x) \ln(f_X(x)) \frac{dy}{dx} dx \\ &= - \frac{1}{a} \ln \left(\frac{1}{a} \right) \int f_X(x) a dx - \frac{1}{a} \int f_X(x) \ln(f_X(x)) a dx \end{aligned}$$

$$\begin{aligned} &= \ln(a) - \int f_X(x) \ln(f_X(x)) dx \\ &= H(x) + \ln(a) \end{aligned}$$

Mutual information

Reading list and references

[Calin, 2020, Chapters 13]
Ash [2012]

9.1 Introduction

This is our final lecture on the ideas of information theory and neural networks. So far, we have looked at assessing the information in a network using the σ -algebras corresponding to network activations, and using information and entropy. In this lecture we'll continue with our look at entropy and look at the idea of 'mutual information'.

In this lecture, we will slightly inappropriately conflate the terms 'entropy' and 'differential entropy'.

9.2 Mutual information

Having described 'information' of an observation of a random variable, and 'entropy' of a random variable in the previous lecture, we will now go on to describe a concept called the 'mutual information'.

9.2.1 Conditional entropy

We must first define the conditional entropy, and will give some basic properties.

The *conditional entropy* between random variables X and Y is denoted $H(Y|X)$ and is the *expected amount of information we attain about Y if we know X* , in the sense of 'information' we used in the previous lecture.

We will motivate the definition for continuous X and Y by considering firstly the case for discrete X and Y . By directly calculating the entropy of the random variable $Y|X = x_i$, we get:

$$H(Y|X = x_i) = - \sum_j P(Y = y_j | X = x_i) \ln(P(Y = y_j | X = x_i)) \quad (9.1)$$

The conditional entropy is now the weighted mean of this, over various x_i :

$$H(Y|X) = \sum_i P(X = x_i) H(Y|X = x_i) \quad (9.2)$$

We now appeal to the observation that $P(X = x_i)P(Y = y_j | X = x_i) = P(X = x_i, Y = y_j)$, so

$$H(Y|X) = \sum_i \sum_j P(X = x_i, Y = y_j) \ln(P(Y = y_j | X = x_i)) \quad (9.3)$$

If (X, Y) have joint distribution $\mathcal{Z}$, we can write this as

$$H(Y|X) = \mathbb{E}_{(X,Y) \sim \mathcal{Z}} \{\ln(P(Y|X))\} \quad (9.4)$$

with the obvious continuous analogue: for continuous X, Y with joint PDF f_{XY} and domains $\mathbb{R}$, we have

$$H(Y|X) = - \int_{\mathbb{R}} \int_{\mathbb{R}} f_{XY}(x, y) \ln(f_{Y|X}(x, y)) \quad (9.5)$$

where $f_{Y|X} = \frac{f_{XY}(x, y)}{f_X(x)}$.

We now note a basic property (Calin [2020], proposition 12.6.1):

Theorem 23. Let X and Y be random variables such that $Y = f(X)$, where f is deterministic. Then $H(Y|X) = 0$

Proof. Left as exercise. $\square$

We now note that conditional entropy decreases as additional ‘conditions’ are added. Intuitively, conditioning on a further variable reduces the ‘surprise’.

Theorem 24. For three random variables X, Y, Z , we have $H(X|Y, Z) \leq H(X|Z)$

Proof. We will show this for continuous random variables. The discrete case is similar. We will use the shorthand $f_{A|B}(a, b) = \frac{f_{AB}(a, b)}{f_B(b)}$. We have

$$\begin{aligned}
 H(X|Z) - H(X|Y, Z) &= - \int f_{XZ}(x, z) \ln(f_{X|Z}(x, z)) dx dz \\
 &\quad + \int f_{XYZ}(x, y, z) \ln(f_{X|Y, Z}(x, y, z)) dx dy dz \\
 &= - \int f_{XYZ} f_{XZ}(x, y, z) \ln(f_{X|Z}(x, z)) dx dy dz \\
 &\quad + \int f_{XYZ}(x, y, z) \ln(f_{X|Y, Z}(x, y, z)) dx dy dz \\
 &= \int f_{XYZ}(x, y, z) \ln\left(\frac{f_{X|YZ}(x, y, z)}{f_{X|Z}(x, z)}\right) dx dy dz \\
 &= \int f_{XYZ}(x, y, z) \ln\left(\frac{\frac{f_{XYZ}(x, y, z)}{f_Y(y, z)}}{\frac{f_{XZ}(x, z)}{f_Z(z)}}\right) dx dy dz \\
 &= \int f_{XYZ}(x, y, z) \ln\left(\frac{f_{XYZ}(x, y, z)}{f_{X|Z}(x, z) f_{YZ}(y, z)}\right) dx dy dz \\
 &= - \int f_{XYZ}(x, y, z) \ln\left(\frac{f_{X|Z}(x, z) f_{YZ}(y, z)}{f_{XYZ}(x, y, z)}\right) dx dy dz \\
 &\geq \int f_{XYZ}(x, y, z) \left(1 - \frac{f_{X|Z}(x, z) f_{YZ}(y, z)}{f_{XYZ}(x, y, z)}\right) dx dy dz \\
 &= \int f_{XYZ}(x, y, z) dx dy dz - \int f_{X|Z}(x, z) f_{YZ}(y, z) dx dy dz \\
 &= 1 - 1 \\
 &= 0
 \end{aligned} \tag{9.6}$$

by the same argument

9.2.2 Mutual information

If we have two random variables X and Y which are independent, then we expect that conditioning Y on X will not decrease its information content; that is $H(Y) = H(Y|X)$.

What if X and Y are dependent? Then we have (for continuous X, Y):

$$\begin{aligned}
 H(Y) - H(Y|X) &= - \int f_Y(y) \ln(f_Y(y)) dy + \int \int f_{XY}(x, y) \ln\left(\frac{f_{XY}(x, y)}{f_Y(y)}\right) dx dy \\
 &= \int \int f_{XY}(x, y) \ln\left(\frac{f_{XY}(x, y)}{f_X(x) f_Y(y)}\right) dx dy \\
 &= - \int \int f_{XY}(x, y) \ln\left(\frac{f_X(x) f_Y(y)}{f_{XY}(x, y)}\right) dx dy \\
 &\geq \int \int f_{XY}(x, y) \left(1 - \frac{f_X(x) f_Y(y)}{f_{XY}(x, y)}\right) dx dy \\
 &= \int \int f_{XY}(x, y) dx dy - \int \int f_X(x) f_Y(y) dx dy
 \end{aligned}$$

$$= 1 - 1 = 0$$

so if X and Y are dependent (that is, $f_{XY}(x, y) \neq f_X(x)f_Y(y)$ holds f_{XY} -almost everywhere), then conditioning on X reduces the information in Y . This matches our intuition.

The difference is the *mutual information* denoted $I(Y, X)$. From the form in the derivation above we note that

$$I(Y, X) = H(Y) - H(Y|X) = D_{KL}(f_{XY}(x, y) || f_X(x)f_Y(y)) \quad (9.7)$$

where $D_{KL}(\cdot || \cdot)$ is the *Kullback-Leibler* divergence of the two distributions.

Definition 18 (Kullback-Leibler divergence, discrete). Suppose we have two different discrete distributions $p_1(\cdot)$, $p_2(\cdot)$ defined over the same set S . Then the Kullback-Leibler divergence (K-L divergence) between p_1 and p_2 , denoted $D_{KL}(p_1 || p_2)$, is given by:

$$D_{KL}(p_1 || p_2) = \sum_{s \in S} p_1(s) \log \left(\frac{p_1(s)}{p_2(s)} \right)$$

For continuous distributions:

Definition 19 (Kullback-Leibler divergence, continuous). Suppose we have two different continuous distributions with densities $f_1(\cdot)$, $f_2(\cdot)$ supported on the same set S . Then the Kullback-Leibler divergence (K-L divergence) between f_1 and f_2 , denoted $D_{KL}(f_1 || f_2)$, is given by:

$$D_{KL}(f_1 || f_2) = \int_S f_1(s) \log \left(\frac{f_1(s)}{f_2(s)} \right) ds$$

Note that the K-L divergence is in general *not* symmetrical: that is, $D_{KL}(p_1 || p_2) \neq D_{KL}(p_2 || p_1)$ and $D_{KL}(f_1 || f_2) \neq D_{KL}(f_2 || f_1)$ in general. So we have:

Definition 20 (Mutual information, discrete). Suppose that two random variables X, Y have a joint discrete distribution $P(X = x, Y = y)$, with (X, Y) taking values in S . Denote the functions $p_1(x, y) = P(X = x, Y = y)$ and $p_2(x, y) = P(X = x)P(Y = y)$. Then the mutual information of X and Y is given by

$$I(X, Y) = D_{KL}(p_1 || p_2) = \sum_{s \in S} p_1(s) \log \left(\frac{p_1(s)}{p_2(s)} \right) = \sum_{x, y \in S} P(X = x, Y = y) \log \left(\frac{P(X = x, Y = y)}{P(X = x)P(Y = y)} \right)$$

For continuous distributions:

Definition 21 (Kullback-Leibler divergence, continuous). Suppose that two random variables X, Y have a joint continuous distribution $f_{X,Y}(x, y)$, with support in a set S . Denote the functions $f_1(x, y) = f_{X,Y}(x, y)$ and $f_2(x, y) = f_X(x)f_Y(y)$. Then the mutual information of X and Y is given by

$$I(X, Y) = D_{KL}(f_1 || f_2) = \sum_{s \in S} f_1(s) \log \left(\frac{f_1(s)}{f_2(s)} \right) = \sum_{x, y \in S} f_{X,Y}(x, y) \log \left(\frac{f_{X,Y}(x, y)}{f_X(x)f_Y(y)} \right)$$

9.2.3 Properties

We notice a few properties:

Theorem 25. The mutual information $I(X, Y)$ of two random variables X, Y satisfies:

1. Non-degeneracy: $I(X, Y) = 0$ if and only if X and Y are independent.
2. Non-negativity: $I(X, Y) \geq 0$
3. Symmetry: $I(Y, X) = I(X, Y)$

Proof. We have already shown (1) and (2). (3) follows from equation 9.7, noting that the expression in terms of D_{KL} is symmetric. $\square$

Given symmetry, the following is unsurprising:

Theorem 26. We have

$$I(Y, X) = I(X, Y) = H(X) + H(Y) - H(X, Y)$$

Proof. Left as exercise □

Information can, in a sense, be considered analogously to areas, with mutual information as ‘shared area’, in that

$$I(X, Y) = H(X) + H(Y) - H(X, Y) \quad (9.8)$$

and, for two regions X, Y

$$\text{area}(X \cup Y) = \text{area}(X) + \text{area}(Y) - \text{area}(X \cap Y) \quad (9.9)$$

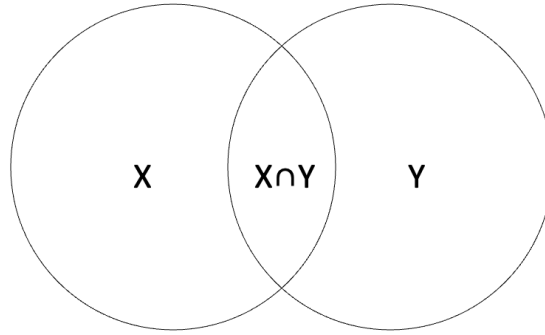


Figure 9.1: Areas; in a sense, $I(Y, X)$ behaves like the area of $X \cup Y$, in t.

9.2.4 Transformation property

We now come to an important property of mutual information: that for well behaved functions, the mutual information $I(X, Y)$ of two random variables X, Y is left unchanged by transformation through those functions (Calin [2020], proposition 12.7.5)

Theorem 27. Suppose that X, Y are random variables taking values in $\mathbb{R}^n$. Suppose ϕ and ψ are invertible and differentiable functions $\phi, \psi : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Then

$$I(X, Y) = I(\phi(X), \psi(Y)) \quad (9.10)$$

Proof. Not examinable: see Calin [2020], proposition 12.7.5 □

9.2.5 Data Processing Inequalities

We briefly show a theorem applying to information flow in Markov chains. □

Theorem 28. Suppose $X \rightarrow Y \rightarrow Z$ is a Markov chain, so $P(Z|X, Y) = P(Z|Y)$. Then $I(X, Y) \geq I(X, Z)$ and $I(Y, Z) \geq I(X, Z)$.

Proof. We have

$$\begin{aligned} I(X, Y) - I(X, Z) &= (H(X) - H(X|Y)) - (H(X) - H(X|Z)) \\ &= H(X|Z) - H(X|Y) \\ &= H(X|Z) - H(X|Y, Z) \geq 0 \end{aligned}$$

using the Markov property and theorem 24. □

This tells us that mutual information behaves in the intuitive way in a Markov chain: that is, the mutual information decreases the further along the chain we are.

9.3 Applications to neural networks

We will now turn our attention to the application of these ideas to neural networks. For our purposes here, we will treat the random variables associated with the input and activations as *discrete*. This is mostly to avoid dealing with difficult distributions with negative entropy. In practice, this does not make much difference; in practice, real inputs are discrete, but potentially have a very large number of states. We note that activations $X^{(0)}, X^{(1)}, \dots, X^{(L)}$ form a Markov chain, and the data processing inequalities apply.

We begin by revisiting a theorem from the previous lecture, looking at the entropy flow of a network under this discrete assumption (Calin [2020], proposition 12.8.1), with an easier proof involving mutual information:

Theorem 29. Suppose that the activations $X^{(0)}$ (input), $X^{(1)}, X^{(2)}, \dots, X^{(L)}$ (output) are discrete random variables. Then the entropy flow of the network is decreasing; that is

$$H(X^{(0)}) \geq H(X^{(1)}) \geq \dots \geq H(X^{(L)}) \quad (9.11)$$

Proof. Since the activation of layer $\ell > 0$ depends deterministically on the activation of layer $\ell - 1$, we have $H(X^{(\ell)}|X^{(\ell-1)}) = 0$ (theorem 23). Now

$$\begin{aligned} I(X^{(\ell)}, X^{(\ell-1)}) &= H(X^{(\ell)}) - H(X^{(\ell)}|X^{(\ell-1)}) = H(X^{(\ell)}) \\ I(X^{(\ell)}, X^{(\ell-1)}) &= H(X^{(\ell-1)}) - H(X^{(\ell-1)}|X^{(\ell)}) \leq H(X^{(\ell-1)}) \end{aligned}$$

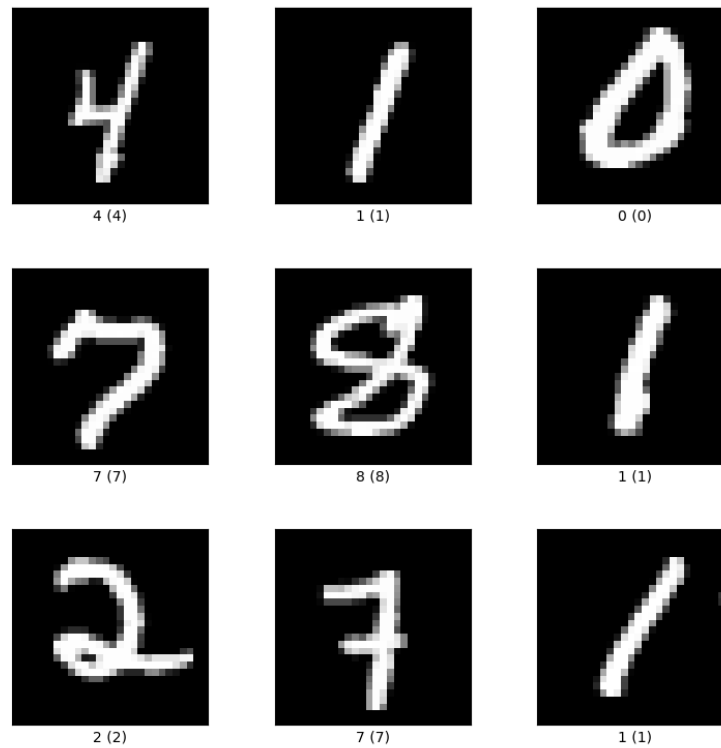
since $H(X^{(\ell-1)}|X^{(\ell)}) \geq 0$, as the variables are discrete. Thus $H(X^{(\ell)}) \leq H(X^{(\ell-1)})$ as required.

□

9.4 Summary of ideas from information theory

We will now summarise some ideas from information theory that we can use to describe a typical machine learning task. In fact, we will look at *the* typical machine learning task - the classification of the MNIST dataset.

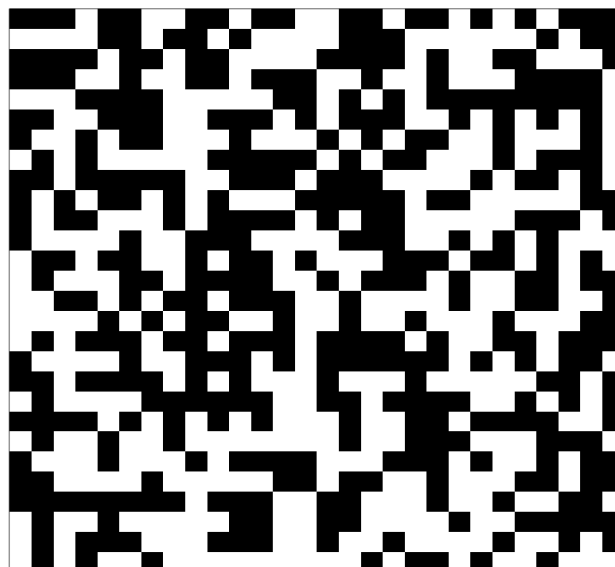
The MNIST dataset is a collection of 10,000 handwritten digits (0-9) from high school students in the USA, reduced to 28×28 pixel images:



The goal is to ‘read’ the digit. In other words, given an image described as X (with a particular distribution), we want to find an X' which is our best guess of the digit depicted. To illustrate the ideas of mutual information, we will assume that Y has some randomness and it is not possible to guess the correct number with certainty. According to our now-familiar schematic:

$$X \rightarrow X' : Y$$

we will assume X' is our ‘best guess’ of Y . As a comparison, we will consider all possible 28×28 pixel black-and-white images, along with a (random) assignment of such images to the numbers 0-9:



9.4.1 Intuition

Intuitively, for the random image problem, we will struggle to design a small neural network to efficiently classify these images, since the images with a common number assigned to them have nothing in common. We can do no better than going through the 2^{28^2} images, seeing if our candidate image matches, and returning 1 if it does. We can do this with a single-layer network with 2^{28^2} ($= 101\,745\,825\,697\,019\,260\,773\,923\,519\,755\,878\,567\,461\,315\,282\,017\,759\,829\,107\,608\,914\,364\,075\,275\,235\,254\,395\,622\,580\,447\,400\,994\,175\,578\,963\,163\,918\,967\,182\,013\,639\,660\,669\,771\,108\,475\,957\,692\,810\,857\,098\,847\,138\,903\,161\,308\,502\,419\,410\,142\,185\,759\,152\,435\,680\,068\,435\,915\,159\,402\,496\,058\,513\,611\,411\,689\,167\,650\,816$) neurons in the hidden layer, each of which checks whether the input matches one of the images in the set. A computer program is no better; we need to write a huge long programme to check every image in the set.

The MNIST database seems easier. There is not a clear common structure to each image; what is a picture of a number exactly? But images tend to be similar and have common elements. We could write a more complex computer program which would find numbers, without necessarily needing to scan for every possible image of a particular number, like we did for the arbitrary task. When we look at convolutional neural networks, we will see a way to do this more efficiently with neural networks. Indeed, we would expect that we could probably learn to quickly classify numbers with a neural network, since our own brains are themselves neural networks and can do this very well!

9.4.2 Kolmogorov complexity

This is reflected by the mean Kolmogorov complexity of typical inputs in the MNIST data (or in a general hand-written digit) as compared to the complexity of an arbitrary image. The Kolmogorov complexity of an arbitrary image is a small amount larger than 28^2 , since we can write a program which outputs the image. But we could generally write a program to output an image of a number more efficiently: we could, for example, use ‘run-length encoding’ where we encode a length of identical pixels as e.g. $10b$ (ten black pixels). This would substantially reduce the length of a program to output a number, but would not help (on average) for an arbitrary image.

9.4.3 Entropy

Suppose we consider a random variable X_a which is equally likely to take any black-and-white 28×28 pixel image. We also consider a random variable X_m which represents images of scanned handwritten digits. Although technically X_m can take the value of any black-and-white 28×28 pixel image, most images are very unlikely: and images with high probability will closely resemble various digits.

Since X_a has a *discrete uniform* distribution, it has the maximum entropy over all distributions over possible 28×28 pixel images. The random variable X_m has a markedly non-uniform distribution, so has a much lower entropy. In other words, we can ‘summarise’ it with a neural network quite efficiently, to a fairly small ‘barcode’, which is easier to process than the non-compressible X_a .

The interim random variable X' , to which X is mapped, has a much lower entropy still: it has only 10 possible values, so has entropy at most 10.

9.4.4 Generated σ -algebras

Let us consider X_a to be a function from a set of size 2^{28^2} to the space of 28×28 pixel images, and X' to be a function from the space of 28×28 pixel images to the numbers $0 - 9$. The σ -algebra generated by X_a is relatively easy to characterise; it is every set of values of X_a to which we can assign a probability. This is every set of possible 28×28 pixel images, so the σ -algebra is of size $2^{2^{28^2}}$ (needless to say, a large number).

The σ -algebra associated with X' is much smaller, though, reflecting the fact that X' is a very compressed version of X , maintaining only the useful information. The pre-image of, say, $X' = 4$ is all the values of X_a which mapped to ‘4’.

9.4.5 Mutual information

Let us presume that we have encoded all our information as well as possible, using the function f , where $X' = f(X_a)$. That is, for any image x_a , and any value y of Y , we presume

$$P(Y = y|X = x_a) = P(Y = y|X' = f(x_a)).$$

Now what are the mutual information values $I(X, Y)$ and $I(X', Y)$? We have:

$$\begin{aligned}
 I(X, Y) &= \sum_{x_a, y} P(X_a = x_a, Y = y) \ln \left(\frac{P(X_a = x_a, Y = y)}{P(X_a = x_a)P(Y = y)} \right) \\
 &= \sum_{x_a, y} P(X_a = x_a, Y = y) \ln \left(\frac{1}{P(Y = y)} P(Y = y | X_a = x_a) \right) \\
 &= \sum_{x_a, y} P(X_a = x_a, Y = y) \ln \left(\frac{1}{P(Y = y)} P(Y = y | X' = f(x_a)) \right) \\
 &= \sum_{x_a, y} P(X_a = x_a, Y = y) \ln \left(\frac{P(X' = f(x_a), Y = y)}{P(X' = f(x_a))P(Y = y)} \right) \\
 &= \sum_{i=1}^9 \sum_{x_a: f(x_a)=i} P(X_a = x_a, Y = y) \ln \left(\frac{P(X' = f(x_a), Y = y)}{P(X' = f(x_a))P(Y = y)} \right) \\
 &= \sum_{i=1}^9 \sum_{x_a: f(x_a)=i} P(X_a = x_a, Y = y) \ln \left(\frac{P(X' = i, Y = y)}{P(X' = i)P(Y = y)} \right) \\
 &= \sum_{i=1}^9 \ln \left(\frac{P(X' = i, Y = y)}{P(X' = i)P(Y = y)} \right) \sum_{x_a: f(x_a)=i} P(X_a = x_a, Y = y) \\
 &= \sum_{i=1}^9 \ln \left(\frac{P(X' = i, Y = y)}{P(X' = i)P(Y = y)} \right) P(X' = i, Y = y) \\
 &= I(X', Y)
 \end{aligned}$$

so, although X' is much simpler than X_a (having a smaller σ -algebra, a lower entropy, and a lower Kolmogorov complexity) we maintain the mutual information with Y : we have extracted all the ‘useful’ information out of it.

This is not entirely realistic, of course: in some cases the digit is much more obvious than others, so $P(Y = y | X_a = x_a)$ will sometimes be higher than $P(Y = y | f(X_a) = f(x_a))$ and sometimes lower, so the mutual information $I(X', Y)$ will be less than the mutual information $I(X, Y)$. The interpretation of this is, by making a specific ‘guess’ for Y for each image, we are losing some information: namely, we are giving up on the information on how ‘certain’ we are, retaining only information on the most likely outcome.

Exercises

1. Prove theorem 23. Note that the entropy of a random variable taking on a constant value c with probability 1 will be 0.
2. Is $I(Y, X)$ a metric on the space of random variables?
3. Prove theorem 26.
4. Show that the total compression ρ is given by $\rho = \rho_1 \rho_2 \dots \rho_L$.

Solutions

1. The entropy of a constant c satisfies $H(c) = -\sum p_i \ln(p_i) = -1 \ln 1 = 0$. The entropy of Y conditioned on $X = x_i$ is

$$H(Y|X = x_i) = H(f(X)|X = x_i) = H(f(x_i)) = 0 \quad (9.12)$$

so

$$H(Y|X) = \sum_i p(x_i) H(Y|X = x_i) = 0 \quad (9.13)$$

2. No. Since $I(X, X) = H(X)$, which is often positive, it cannot be a metric.
3. We will show this for differential entropy; the discrete case is similar. We have that $\ln(f_{Y|X}(x, y)) = \ln\left(\frac{f_{X,Y}(x, y)}{f_X(x)}\right) = \ln(f_{X,Y}(x, y)) - \ln(f_X(x))$. Now

$$\begin{aligned} I(X, Y) &= H(Y) - H(Y|X) \\ &= H(Y) + \int f_{X,Y}(x, y) \ln(f_{Y|X}(x, y)) dx dy \\ &= H(Y) + \int f_{X,Y}(x, y) \ln(f_{X,Y}(x, y)) dx dy - \int f_{X,Y}(x, y) \ln(f_X(x)) dx dy \\ &= H(Y) - H(X, Y) - \int f_X(x) \ln(f_X(x)) dx \\ &= H(Y) + H(X) - H(X, Y) \end{aligned}$$

as required.

4. We have:

$$\rho_1 \rho_2 \cdots \rho_L = \frac{I(X^{(0)}, X^{(1)})}{I(X^{(0)}, X^{(0)})} \frac{I(X^{(0)}, X^{(2)})}{I(X^{(0)}, X^{(1)})} \cdots \frac{I(X^{(0)}, X^{(L)})}{I(X^{(0)}, X^{(L-1)})} = \frac{I(X^{(0)}, X^{(L)})}{I(X^{(0)}, X^{(0)})} = \frac{I(X^{(0)}, X^{(L)})}{H(X^{(0)})} = \rho_0$$

Backpropagation

Reading list and references

[[Calin, 2020](#), Chapters 4,6]

10.1 Introduction

In this lecture, we will now turn our attention to the practicalities of neural networks, and look at how to train them, using the idea of backpropagation.

10.1.1 Heuristics

Backpropagation is a slightly frustrating area to study in that the idea of it is extremely simple, but it is somewhat tedious to deal with mathematically.

Roughly, given some input and some output, each ‘connection’ between neurons contributes to that output. The connection is associated with a weight. Increasing the weight will either increase or decrease the value of the output.

The rough idea of backpropagation is to look at the output we *get*, and look at what the output *should be*. Then:

- if the output is *too small*, then we strengthen the connections (that is, increase the weights) which increase the value of the output, and we weaken the connections (that is, decrease the weights) which decrease it; and
- if the output is *too large*, then we weaken the connections which increase the value of the output and strengthen the connections which decrease it; and
- we do this change more drastically for bigger differences between what the outcome is and what it should be

This is, unsurprisingly, also expressible through the medium of gradient descent. The useful thing about neural networks is that the *gradient can be simplified* in allowing us to update one layer at a time.

10.2 Notation

We will use our usual notation for neural networks:

1. The bracketed index (ℓ) refers to layer, with $\ell \in 0 \dots L$; quantities with $\ell = 0$ (for instance, $X^{(0)}$) refer to the input and quantities with $\ell = L$ (e.g. $X^{(L)}$) refer to the output.
2. The vector $X^{(\ell)}$ is the set of *inputs* to layer $\ell + 1$.
3. We will identify the *inputs* for the ℓ th layer and the *outputs* of layer $\ell - 1$ as $X^{(\ell-1)}$ for $\ell > 1$
4. We denote the weight matrix for layer ℓ as $W^{(\ell)}$, with entries $w_{ij}^{(\ell)}$. Index ij indicates the i th input to the j th neuron.
5. We will denote the bias for neuron j in layer ℓ as $b_j^{(\ell)}$. We will also consider these as weights $w_{1j}^{(\ell)} = b_j^{(\ell)}$ with constant inputs $x_{0j}^{(\ell-1)} = -1$

6. We denote the number of neurons in the ℓ th layer by $d^{(\ell)}$.

We will presume that all neurons have a common differentiable discriminatory activation function ϕ . We will also use the shorthand

$$s^{(\ell)} = \begin{pmatrix} s_1^{(1)} \\ s_2^{(\ell)} \\ \vdots \\ s_{d^{(\ell)}}^{(\ell)} \end{pmatrix} = W^{(\ell)} X^{(\ell-1)} - b^{(\ell)} = \begin{pmatrix} w_{11}^{(\ell)} & w_{21}^{(\ell)} & \cdots & w_{d^{(\ell-1)}1}^{(\ell)} \\ w_{12}^{(\ell)} & w_{22}^{(\ell)} & \cdots & w_{d^{(\ell-1)}2}^{(\ell)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1d^{(\ell)}}^{(\ell)} & w_{2d^{(\ell)}}^{(\ell)} & \cdots & w_{d^{(\ell-1)}d^{(\ell)}}^{(\ell)} \end{pmatrix} \begin{pmatrix} X_1^{(\ell-1)} \\ X_2^{(\ell-1)} \\ \vdots \\ X_{d^{(\ell-1)}}^{(\ell-1)} \end{pmatrix} - \begin{pmatrix} b_1^{(\ell)} \\ b_2^{(\ell)} \\ \vdots \\ b_{d^{(\ell)}}^{(\ell)} \end{pmatrix} \quad (10.1)$$

so that

$$X^{(\ell)} = \begin{pmatrix} X_1^{(1)} \\ X_2^{(\ell)} \\ \vdots \\ X_{d^{(\ell)}}^{(\ell)} \end{pmatrix} = \phi \left(W^{(\ell)} X^{(\ell-1)} - b^{(\ell)} \right) = \phi(s^{(\ell)}) \quad (10.2)$$

Notation is illustrated in figure 10.1 (taken from Calin [2020], figure 6.1).

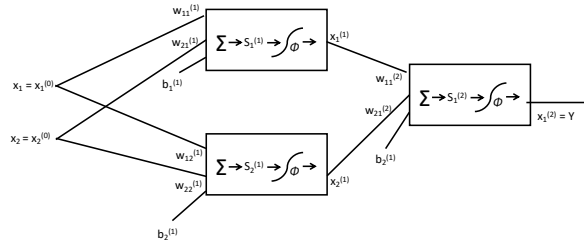


Figure 10.1: Notational setup for a standard feedforward neural network. In our case this has two neurons.

10.3 Two-layer model

We will begin by establishing how to perform gradient descent for the two-layer three-neuron model in figure 10.1. We will then extend this in a natural way to the general feedforward case.

Suppose our neural network implements a function $F_{W,B}(X^{(0)})$, where $W = \{W^{(1)}, W^{(2)}\}$ is the weights of both layers, and $B = \{b^{(1)}, b^{(2)}\}$ is the set of biases (which we will consider as separate to the weights, for the moment).

We have a *cost function* $C(W, B)$ which tells us how far away the function thus implemented is from the function we want to implement. For instance, if we have some *training data* X^{train} in the same space as $X^{(0)}$ with labels Y^{train} , then our cost function may be the mean-squared error:

$$C(W, B) = MSE(Y^{train}, F_{W,B}(X^{train})) = \frac{1}{|Y^{train}|} \sum_i (Y^{train} - F_{W,B}(X^{train}))^2 \quad (10.3)$$

We will presume that our cost function is differentiable. In order to *minimise* the cost function, we need to compute its gradient.

Neural networks have lots of parameters. In our simple two-neuron case, we have nine parameters (six weights, three biases). In a fully-connected network with n inputs, L hidden layers of m neurons, and an output layer of one neuron, we have $\Theta(Lm^2 + mn)$ parameters. The neural network is costly to differentiate directly (see exercises). Backpropagation allows us to speed it up.

10.3.1 Updating weights

We will call our cost function C . Our first (important) definition is of δs :

$$\delta_i^{(\ell)} = \frac{\partial C}{\partial s_i^{(\ell)}} \quad (10.4)$$

We will now first consider the weights in the second layer. Note that C depends on $X^{(L)} = \phi(s^{(2)})$, and hence depends on weights $w_{ij}^{(2)}$ only through $s^{(2)}$. We have

$$\begin{aligned} \frac{\partial C}{\partial w_{11}^{(2)}} &= \frac{\partial C}{\partial s_1^{(2)}} \frac{\partial s_1^{(2)}}{\partial w_{11}^{(2)}} = \delta_1^{(2)} X_1^{(1)} \\ \frac{\partial C}{\partial w_{21}^{(2)}} &= \frac{\partial C}{\partial s_1^{(2)}} \frac{\partial s_1^{(2)}}{\partial w_{21}^{(2)}} = \delta_1^{(2)} X_2^{(1)} \end{aligned}$$

For the bias in layer 2, we have

$$\frac{\partial C}{\partial b_1^{(2)}} = \frac{\partial C}{\partial s_1^{(2)}} \frac{\partial s_1^{(2)}}{\partial b_1^{(2)}} = -\delta_1^{(2)}$$

Now let's look at the first layer ($\ell = 1$). We have $X^{(\ell)} = \phi\left(W^{(2)T} \phi(s^{(1)}) - b^{(2)}\right)$, so $X^{(\ell)}$ depends on weights $w_{ij}^{(1)}$ in the first layer only through $s_j^{(1)}$. Now

$$\frac{\partial C}{\partial w_{11}^{(1)}} = \frac{\partial C}{\partial s_1^{(1)}} \frac{\partial s_1^{(1)}}{\partial w_{11}^{(1)}} = \delta_1^{(1)} X_1^{(0)}$$

and

$$\frac{\partial C}{\partial b_1^{(1)}} = \frac{\partial C}{\partial s_1^{(1)}} \frac{\partial s_1^{(1)}}{\partial b_1^{(1)}} = -\delta_1^{(1)}$$

and generally, we have

$$\begin{aligned} \frac{\partial C}{\partial w_{ij}^{(1)}} &= \frac{\partial C}{\partial s_j^{(1)}} \frac{\partial s_j^{(1)}}{\partial w_{ij}^{(1)}} = \delta_j^{(1)} X_i^{(0)} \\ \frac{\partial C}{\partial b_j^{(1)}} &= \frac{\partial C}{\partial s_j^{(1)}} \frac{\partial s_j^{(1)}}{\partial b_j^{(1)}} = -\delta_j^{(1)} \end{aligned} \quad (10.5)$$

10.3.2 Propagating δs

Since our aim is to calculate $\nabla f(X^{(0)})$, we must thus be able to calculate the values $X^{(\ell)}$ and the values $\delta_j^{(\ell)}$.

The values $X^{(\ell)}$ are easy to quickly calculate: we simply propagate the values $X^{(0)}$ through the network, using the current values of W and B . What about the δs ?

The values $\delta_j^{(2)}$ are easy to calculate. Since we may write $C = C(X^{(2)}) = C(\phi(s^{(2)}))$, we have

$$\delta_j^{(2)} = \frac{\partial C}{\partial s_j^{(2)}} = \frac{\partial C}{\partial X^{(2)}} \frac{\partial X^{(2)}}{\partial s_j^{(2)}} = \frac{\partial C}{\partial X^{(2)}} \phi'(s_j^{(2)}) \quad (10.6)$$

It turns out we can use the chain rule to calculate the values $\delta_j^{(1)}$ very efficiently. We note the ‘dependency tree’ shown in figure 10.2 (Calin [2020], figure 6.4), in which every quantity affects C only through the quantity above it.

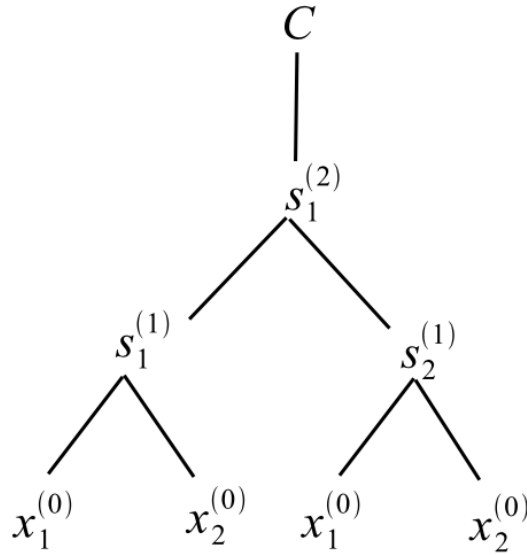


Figure 10.2: Dependency tree for two-layer three-neuron neural network in figure 10.1. Each quantity affects C only through the quantities on the path from it to C .

With this dependency structure in mind, we have

$$\delta_1^{(1)} = \frac{\partial C}{\partial s_1^{(1)}} = \frac{\partial C}{\partial s_1^{(2)}} \frac{\partial s_1^{(2)}}{\partial s_1^{(1)}} = \delta_1^{(2)} \frac{\partial s_1^{(2)}}{\partial s_1^{(1)}}$$

Now we have

$$s_1^{(2)} = -b_1^{(2)} + w_{11}^{(2)} \phi(s_1^{(1)}) + w_{21}^{(2)} \phi(s_2^{(1)})$$

so

$$\frac{\partial s_1^{(2)}}{\partial s_1^{(1)}} = w_{11}^{(2)} \phi'(s_1^{(1)})$$

and

$$\delta_1^{(1)} = \delta_1^{(2)} w_{11}^{(2)} \phi'(s_1^{(1)}) \quad (10.7)$$

All of these terms are easy to calculate! We can quickly calculate $\delta_1^{(2)}$ as above. We have $s_1^{(1)}$ from a forward pass through the network. And we already know $w_{11}^{(2)}$.

We are essentially ‘partitioning the error’ across neurons in the layer, and then adjusting the weights of each neuron separately in order to appropriately account for the portion of error attributable to it.

10.4 General formula

The general formula for backpropagation to calculate the gradient of a neural network is similar.

We will denote:

1. The network to have L layers in total
2. The common activation function as ϕ
3. Layer ℓ to have $d^{(\ell)}$ neurons
4. The inputs to the ℓ th layer as $x_i^{(\ell-1)}$, $i \in 1, 2, \dots, d^{(\ell-1)}$, so the overall input is $x_i^{(0)}$, $i \in 1, 2, \dots, d^{(0)}$; and the overall output is $x_i^{(L)}$, $i \in 1, 2, \dots, d^{(L)}$.

5. The weights in the j th neuron in the (ℓ) th layer to be $w_{ij}^{(\ell)}$, with $i \in 1, 2, \dots, d^{(\ell-1)}$, $j \in 1, 2, \dots, d^{(\ell)}$, and $\ell \in 1, 2, \dots, L$.
6. The bias for the j th neuron in the (ℓ) th layer to be $b_j^{(\ell)}$, with $j \in 1, 2, \dots, d^{(\ell)}$, and $\ell \in 1, 2, \dots, L$.
7. The overall cost function to be C

$$s_j^{(\ell)} = \sum_{i=1}^{d^{(\ell-1)}} w_{ij}^{(\ell)} x_i^{(\ell-1)} - b_j^{(\ell)}, \quad 1 \leq j \leq d^{(\ell)} \quad (\text{Forward propagation})$$

$$x_j^{(\ell)} = \phi(s_j^{(\ell)}), \quad 1 \leq j \leq d^{(\ell)} \quad (\text{Forward propagation})$$

$$\delta_j^{(L)} = \frac{\partial C}{\partial x_j^{(L)}} \phi'(s_j^{(L)}), \quad 1 \leq j \leq d^{(L)} \quad (\text{Deltas in final layer})$$

$$\delta_i^{(\ell-1)} = \phi'(s_i^{(\ell-1)}) \sum_{j=1}^{d^{(\ell)}} \delta_j^{(\ell)} w_{ij}^{(\ell)}, \quad 1 \leq i \leq d^{(\ell-1)}, \quad \ell = L, L-1, \dots, 2 \quad (\text{Compute deltas})$$

$$\frac{\partial C}{\partial w_{ij}^{(\ell)}} = \delta_j^{(\ell)} x_i^{(\ell-1)}, \quad 1 \leq i \leq d^{(\ell-1)}, 1 \leq j \leq d^{(\ell)}, \quad \ell = L, L-1, \dots, 1 \quad (\text{Derivatives w.r.t. weights})$$

$$\frac{\partial C}{\partial b_i^{(\ell)}} = -\delta_i^{(\ell)}, \quad 1 \leq i \leq d^{(\ell)}, \quad \ell = L, L-1, \dots, 1 \quad (\text{Derivatives w.r.t. biases})$$

Exercises

1. In a fully-connected network with n inputs, L hidden layers of m neurons, and an output layer of one neuron, where each neuron has a bias, exactly how many parameters are there, and how many of these are biases?
2. Adapt the general backpropagation algorithm for the case where there are different activation functions in each layer (say that layer ℓ has activation function $\phi^{(\ell)}$)
3. Suppose we have a *repeated* weight in some layer; that is, we want to fit a network while ensuring that weights $w_{ij}^{(\ell)}$ and $w_{pq}^{(r)}$ are equal. How do we adapt backpropagation to this setting? This will be important when we look at convolutional neural networks. Hint: recall or show that if partial derivatives are continuous, then if:

$$d_1(x, y) = \frac{\partial F(x, y)}{\partial x}$$

$$d_2(x, y) = \frac{\partial F(x, y)}{\partial y}$$

then

$$\frac{\partial F(z, z)}{\partial z} = d_1(z, z) + d_2(z, z) \quad (10.8)$$

4. Show that if $\phi(x) = (1 + \exp(-x))^{-1}$ and $|w_{ij}^{(\ell)}| < c_1$, $|x_j^{(0)}| < c_2$, and $\left| \frac{\partial C}{\partial x_j^{(L)}} \right| < c_3$ for all i, j, ℓ , then $\frac{\partial C}{\partial w_{ij}^{(\ell)}} = O(4^{-(L-\ell)})$ and $\frac{\partial C}{\partial b_i^{(\ell)}} = O(4^{-(L-\ell)})$.

Solutions

- Each of the m neuron in the first layer has n inputs and hence n weights and a bias, so these neurons have $m(n+1)$ parameters and n biases. In the remaining $L-1$ hidden layers, each of the m neurons has m inputs and weights and a bias, so these layers collectively have $(L-1)m(m+1)$ parameters and $(L-1)m$ biases. The final layer has m weights and a bias. So in total there are $m(n+1) + (L-1)m(m+1) + m+1$ weights of which $mL+1$ are biases.
- We note that $\frac{\partial C}{\partial w_{ij}^{(\ell)}}$ depends on the activation functions $\phi^{(\ell)}$ only through the δ s. We may also propagate δ s normally; in that $\delta^{(\ell)}$ depends on previous activation functions only through previous $\delta^{(\ell-1)}, \delta^{(\ell-2)}, \dots$. So we need to define:

$$\delta_j^{(L)} = \frac{\partial C}{\partial x_j^{(L)}} (\phi^{(L)})'(s_j^{(L)}), \quad 1 \leq j \leq d^{(L)}$$

and

$$\delta_i^{(\ell-1)} = (\phi^{(\ell)})'(s_i^{(\ell-1)}) \sum_{j=1}^{d^{(\ell)}} \delta_j^{(\ell)} w_{ij}^{(\ell)}, \quad 1 \leq i \leq d^{(\ell-1)}, \quad \ell = L, L-1, \dots, 2$$

and leave the rest of the equations unchanged.

- Let us write θ as all parameters of the neural network other than $w_{ij}^{(\ell)}$ and $w_{pq}^{(r)}$, and write the overall loss function as:

$$C(w_{ij}^{(\ell)}, w_{pq}^{(r)}, \theta) = C_\theta(w_{ij}^{(\ell)}, w_{pq}^{(r)})$$

Let us first perform normal backpropagation as though we were not enforcing that $w_{ij}^{(\ell)} = w_{pq}^{(r)}$ and calculate the values:

$$\begin{aligned} d_1(w_{ij}^{(\ell)}, w_{pq}^{(r)}) &= \frac{\partial C}{\partial w_{ij}^{(\ell)}} \\ d_2(w_{ij}^{(\ell)}, w_{pq}^{(r)}) &= \frac{\partial C}{\partial w_{pq}^{(r)}} \end{aligned}$$

Now, supposing we fix $w_{ij}^{(\ell)} = w_{pq}^{(r)} = w$, we have:

$$\frac{\partial C}{\partial w} = \frac{\partial}{\partial w} C(w, w, \theta) = \frac{\partial}{\partial w} C_\theta(w, w) = d_1(w, w) + d_2(w, w)$$

using the rule in the hint. This extends by induction in the obvious way (e.g., if we want lots of weights to be forced to be equal, we compute the derivative of the network with respect to the common weight as the sum of the partial derivatives with respect to the individual weights). To show that the rule holds, we simply note that:

$$\begin{aligned} \frac{\partial}{\partial z} F(z, z) &= \lim_{h \rightarrow 0} \frac{F(z+h, z+h) - F(z, z)}{h} \\ &= \lim_{h \rightarrow 0} \frac{F(z+h, z+h) - F(z+h, z) + F(z+h, z) - F(z, z)}{h} \\ &= \lim_{h \rightarrow 0} \left(\frac{F(z+h, z+h) - F(z+h, z)}{h} + \frac{F(z+h, z) - F(z, z)}{h} \right) \\ &= \lim_{h \rightarrow 0} \left(\frac{\partial F(x, y)}{\partial y} \Big|_{(x, y)=(z+h, z)} + \frac{\partial F(x, y)}{\partial x} \Big|_{(x, y)=(z, z)} \right) \\ &= \lim_{h \rightarrow 0} (d_1(z+h, z) + d_2(z, z)) \\ &= d_1(z, z) + d_2(z, z) \end{aligned}$$

using the property that partial derivatives are continuous.

4. We have

$$\phi'(x) = \frac{d}{dx} \frac{1}{1 + e^{-x}} = \frac{1}{e^x + e^{-x} + 2} \leq \frac{1}{4}$$

Now

$$|\delta_j^{(L)}| = \left| \frac{\partial C}{\partial x_j^{(L)}} \phi'(s_j^{(L)}) \right| = \left| \frac{\partial C}{\partial x_j^{(L)}} \right| \left| \phi'(s_j^{(L)}) \right| < \frac{c_3}{4}$$

Suppose that for some i with $0 \leq i < L$ we have $\delta_j^{(L-i)} < r_i 4^{-(i+1)}$, where r_i depends only on values d^ℓ, c_1, c_3 .
Now

$$\begin{aligned} |\delta_i^{(L-i-1)}| &= \left| \phi'(s_i^{(L-i-1)}) \sum_{j=1}^{d^{(L-i)}} \delta_j^{(L-i)} w_{ij}^{(L-i)} \right| \\ &= \left| \phi'(s_i^{(L-i-1)}) \right| \left| \sum_{j=1}^{d^{(L-i)}} \delta_j^{(L-i)} w_{ij}^{(L-i)} \right| \\ &\leq \left| \phi'(s_i^{(L-i-1)}) \right| \sum_{j=1}^{d^{(L-i)}} \left| \delta_j^{(L-i)} \right| \left| w_{ij}^{(L-i)} \right| \\ &< \frac{1}{4} \sum_{j=1}^{d^{(L-i)}} c_1 r_i 4^{-(i+1)} \\ &= d^{(L-i)} c_1 r_i 4^{-(i+2)} \\ &= r_{i+1} 4^{-(i+2)} \end{aligned}$$

where $r_{i+1} = d^{(L-i)} c_1 r_i$ depends only on values d^ℓ, c_1, c_3 . So by induction we have $\delta_j^{(L-i)} < r_i 4^{-(i+1)}$ for all i with $0 \leq i < L$. So:

$$\begin{aligned} \left| \frac{\partial C}{\partial w_{ij}^{(\ell)}} \right| &= \left| \delta_j^{(\ell)} x_i^{(\ell-1)} \right| = \left| \delta_j^{(\ell)} \right| \left| x_i^{(\ell-1)} \right| < r_i c_2 4^{-(L-\ell)} \\ \left| \frac{\partial C}{\partial b_i^{(\ell)}} \right| &= \left| \delta_j^{(\ell)} \right| < r_i 4^{-(L-\ell)} \end{aligned}$$

as required.

Gradient descent

Reading list and references

[Calin, 2020, Chapter 4]

11.1 Introduction

In the next three lectures, we will turn our attention to the practicalities of neural networks, and look at how to train them, using the idea of backpropagation.

Our starting point will be a formal look at the gradient descent algorithm, and a couple of common extensions. We will demonstrate a couple of common properties which are intuitive in two dimensions, but possibly not in higher dimensions.

11.2 Properties of minima

We will begin by considering continuous functions f defined on compact sets D , which will usually be of the form:

$$D = [a_1, b_1] \times [a_2, b_2] \dots [a_n, b_n]$$

Since f is continuous, and D compact, f achieves a maximum and minimum value on D . Let us assume $f(x) = f((x_1, x_2, \dots, x_n)^t)$ is twice differentiable inside D , and suppose f attains its minimum value $c = f(c)$ at a point c in the interior of D . By Taylor's theorem we have

$$\begin{aligned} f(x) &= f(c) + \sum_{i=1}^n \frac{\partial f}{\partial x_i}(c)(x_i - c_i) + \frac{1}{2} \sum_{1 \leq i, j \leq n} \frac{\partial^2 f}{\partial x_i \partial x_j}(c)(x_i - c_i)(x_j - c_j) + o\left(\sum_{i=1}^n |x_i - c_i|^3\right) \\ &= f(c) + (x - c)^t \nabla f(c) + \frac{1}{2} (x - c)^t \nabla^2 H_f(c) (x - c) + o(\|x - c\|^2) \end{aligned}$$

where we call $[H_f(x)]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}(x)$ the *Hessian* of f . We can now state another hopefully familiar result ([Calin, 2020], proposition 4.1.1):

Theorem 30. Suppose that for some c on the interior of D we have $\nabla f(c) = 0$ and, for x in some neighbourhood of c the Hessian $H_f(x)$ is positive definite. Then c is a local minimum of f .

Proof. Exercise □

11.3 Gradient descent

Suppose we have a differentiable function $f : \mathbb{R}^p \rightarrow \mathbb{R}$, defined on some compact set $D \subseteq \mathbb{R}^p$. We wish to find an x^* for which $f(x^*) = \min_{x \in D} f(x)$, presuming this minimum exists.

The rough gradient descent algorithm is as follows: The termination criterion varies; we may use a maximum number of iterations (e.g. terminate when $k > k_{max}$, for some maximum number of iterations k_{max}), use a minimum gradient (e.g. terminate when $\|\nabla f(x_k)\| \leq \nabla_0$ for some ∇_0) or use a minimum change in x_n (e.g. terminate when $|x_n - x_{n-1}| \leq \epsilon$ for some ϵ). In any case, we are essentially ‘walking downhill’ at a speed given by t . Unsurprisingly, if there is a minimum point nearby, we eventually hit it, with some caveats.

Algorithm 1 Gradient descent**Require:** Function f , domain D **Require:** k_{max} , ∇_0 or ϵ (see below)

```

1: Choose some  $x_0 \in D$ 
2:  $k \leftarrow 0$ 
3: while Termination criteria (determined by  $k_{max}$ ,  $\nabla_0$  or  $\epsilon$ ) not reached do
4:   Choose some  $t > 0$ 
5:    $x_{k+1} \leftarrow x_k - t\nabla f(x_k)$ 
6:    $k \leftarrow k + 1$ 
7: end while
8: Return  $x^* = x_k$ 

```

We will informally denote the set of points for which the gradient descent algorithm ‘works’ (that is, the union of points x such that if the gradient descent algorithm is started at x , and run with a sufficiently small t and for a long enough time, eventually x_k will be within distance ϵ of x^* , for any $\epsilon > 0$) as the *basin of attraction* of c .

The following theorem formalises this (see [discussion](#)); in particular indicating that we need $t \leq \frac{1}{L}$ when f has Lipschitz norm L .

Theorem 31. Suppose that $f : \mathbb{R}^p \rightarrow R$ is convex and differentiable and that there exists $L > 0$ such that for any x, y :

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2 \quad (11.1)$$

that is, ∇f is L -Lipschitz. Suppose also that $f(x)$ takes a minimum value at $x = x^*$. Then if we run algorithm 1 for k steps, and $t \leq \frac{1}{L}$, we have

$$f(x_k) - f(x^*) \leq \frac{\|x_0 - x^*\|_2^2}{2tk} \quad (11.2)$$

Proof. The Lipschitz condition on ∇f with constant L gives, for all x, y :

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2 \quad (11.3)$$

Setting $y = x^+ = x - t\nabla f(x)$ (the ‘next’ x) gives:

$$f(x^+) \leq f(x) - \left(1 - \frac{Lt}{2}\right)t\|\nabla f(x)\|_2^2$$

If $0 < t \leq \frac{1}{L}$ we thus have:

$$f(x^+) \leq f(x) - \left(1 - \frac{Lt}{2}\right)t\|\nabla f(x)\|_2^2 \leq f(x) - \frac{t}{2}\|\nabla f(x)\|_2^2 \quad (11.4)$$

From the convexity of f we have

$$f(x) - f(x^*) \leq \nabla f(x)^T(x - x^*) \quad (11.5)$$

Putting 11.4 and 11.5 together, we have

$$\begin{aligned}
f(x^+) &\leq f(x^*) + \nabla f(x)^T(x - x^*) - \frac{t}{2}\|\nabla f(x)\|_2^2 \\
&= f(x^*) + \frac{1}{t}(x - x^+)^T(x - x^*) - \frac{t}{2}\frac{1}{t^2}\|x - x^+\|_2^2 \\
&= f(x^*) + \frac{1}{2t}(2(x - x^+)^T(x - x^*) - (x - x^+)^T(x - x^+)) \\
&= f(x^*) + \frac{1}{2t}((x - x^*)^T(x - x^*) - (x^+ - x^*)^T(x^+ - x^*)) \\
&= f(x^*) + \frac{1}{2t}(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2)
\end{aligned}$$

Now we have, noting $(x^{(i)})^+ = x^{(i+1)}$:

$$\sum_{i=1}^k \left(f(x^{(i)}) - f(x^*)\right) \leq \sum_{i=1}^k \frac{1}{2t} \left(\|x^{(i)} - x^*\|_2^2 - \|x^{(i+1)} - x^*\|_2^2\right)$$

$$\begin{aligned}
&= \frac{1}{2t} \left(\|x^{(0)} - x^*\|_2^2 - \|x^{(1)} - x^*\|_2^2 \right. \\
&\quad + \|x^{(1)} - x^*\|_2^2 - \|x^{(2)} - x^*\|_2^2 \\
&\quad + \dots \\
&\quad \left. + \|x^{(k)} - x^*\|_2^2 - \|x^{(k+1)} - x^*\|_2^2 \right) \\
&= \frac{1}{2t} \left(\|x^{(0)} - x^*\|_2^2 - \|x^{(k+1)} - x^*\|_2^2 \right) \\
&\leq \frac{1}{2t} \|x^{(0)} - x^*\|_2^2
\end{aligned} \tag{11.6}$$

and since $f(x^{(i)}) \geq f(x^{(i+1)})$, by design, we have

$$f(x^{(k)}) - f(x^*) \leq \frac{1}{k} \sum_{i=1}^k \left(f(x^{(i)}) - f(x^*) \right) \leq \frac{\|x^{(0)} - x^*\|_2^2}{2tk}$$

as required. $\square$

This also allows us to say something about the *rate* of convergence: namely, that it usually goes at about $1/t$.

11.4 Making gradient descent work

The version of gradient descent in algorithm 1 uses a fixed step size t . Theorem 31 suggests choosing $t \leq \frac{1}{L}$ to make the procedure ‘work’, but we don’t usually know L . If t is too small, then values $x^{(i)}$ will converge too slowly.

11.4.1 Adaptive choice of t

One option is to change t in each iteration. An algorithm called ‘backtracking line search’ is a standard way to achieve this (See [CMU lecture notes](#), [Calin, 2020])

Algorithm 2 Backtracking line search

Require: Values of α, β with $0 < \alpha \leq 1/2$ and $0 < \beta < 1$,

Require: For the k th step of algorithm 1:

- 1: $t_k \leftarrow 1$
 - 2: **while** $f(x_k - t_k \nabla f(x_k)) > f(x_k) - \alpha t_k \|\nabla f(x_k)\|_2^2$ **do**
 - 3: $t_k \leftarrow \beta t_k$
 - 4: **end while**
 - 5: Proceed with the k th step of algorithm 1 with $t = t_k$
-

It is not immediately obvious that this algorithm terminates; the proof of this for differentiable f is left as an exercise.

An alternative, simpler algorithm is *exact line search*:

Algorithm 3 Backtracking line search

Require: For the k th step of algorithm 1:

- 1: $t_k \leftarrow \min_t \{f(x_k - t \nabla f(x_k))\}$
 - 2: Proceed with the k th step of algorithm 1 with $t = t_k$
-

An illustration of the path of a line search method is shown in figure 11.1.

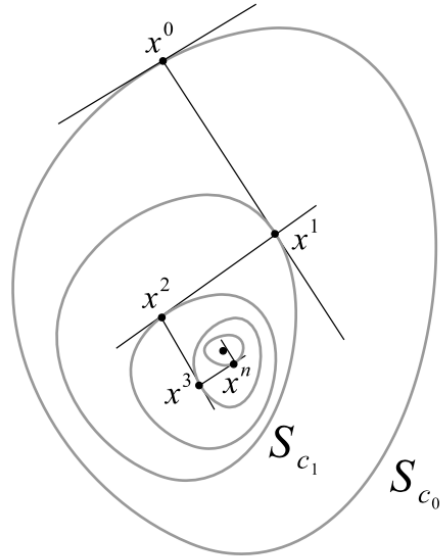


Figure 11.1: Illustration of solution path $\{x_t\}$ using gradient descent for exact line search. Figure from [Calin, 2020], fig. 4.10

Exact line search does not usually give any useful advantage over backtracking line search.

11.4.2 Momentum

Two frequent problems we encounter with gradient descent are:

1. Converging to a local minimum rather than a global
2. Getting ‘stuck’ on a flat part of $f(x)$ (e.g. where $\|\nabla f\|_2^2 \approx 0$)

A standard approach which can help in both of these cases is the ‘momentum’ method.

Let’s reformulate what we are doing kinematically. Suppose we have a ball with unit mass travelling along a path $x(t) \in f(D)$ with velocity $v(t) = \frac{\partial x}{\partial t}$ (we will consider a two-dimensional surface for clarity, but the arguments work in higher dimensions). Let us presume that $f(x_0) = 0$ at the minimum $x_0 \in D$.

The kinetic energy of the ball due to its movement is

$$E_k = \frac{1}{2} \|v\|^2 \quad (11.7)$$

and its potential energy due to height is simply $E_p = f(x)$. Its total energy E_{tot} is given by

$$E_{tot} = E_k + E_p = \frac{1}{2} \|v\|^2 + f(x) \quad (11.8)$$

Let us now suppose that there is a frictional force acting on the ball. This force is proportional to speed $\|v\|$ and in the opposite direction to velocity, with some damping coefficient ρ . Newton’s second law of motion for the ball gives the acceleration of the ball $a(t)$ as:

$$a(t) = \frac{\partial^2}{\partial t^2} x(t) = \underbrace{-\nabla f(x(t))}_{\text{Force acting due to gravity}} - \underbrace{\rho \frac{\partial}{\partial t} x(t)}_{\text{Force acting due to friction}} \quad (11.9)$$

How does E_{tot} change along the ball’s path? (you may remember a lot of this from elementary physics, but bear with me):

$$\frac{\partial}{\partial t} E_{tot}(t) = \frac{\partial}{\partial t} \left(\frac{1}{2} \|v\|^2 + f(x) \right)$$

$$\begin{aligned}
&= \frac{\partial}{\partial t} \left(\frac{1}{2} v v^T + f(x) \right) \\
&= \left(\frac{\partial x}{\partial t} \right)^T \frac{\partial^2 x}{\partial t^2} + \left(\frac{\partial x}{\partial t} \right)^T \nabla f(x) \\
&= \left(\frac{\partial x}{\partial t} \right)^T \left(\frac{\partial^2 x}{\partial t^2} + \nabla f(x) \right) \\
&= -\rho \left(\frac{\partial x}{\partial t} \right)^T \frac{\partial x}{\partial t} \\
&= -\rho \|v\|^2
\end{aligned}$$

We note a couple of things:

1. Since $\frac{\partial}{\partial t} E_{tot}(t) \leq 0$, the total energy decreases as the ball moves;
2. If $\rho = 0$ (a frictionless environment), the total energy of the ball remains constant

Writing equation 11.9 as a pair of first-order differential equations for the functions $x(t)$, $v(t)$:

$$\begin{aligned}
x'(t) &= v(t) \\
v'(t) &= -\rho v(t) - \nabla f(x(t))
\end{aligned}$$

and turning these into finite-difference equations over a time interval $\eta = t_{k+1} - t_k$:

$$\begin{aligned}
x_{k+1} - x_k &= v_{k+1} \eta \\
v_{k+1} - v_k &= -\eta \rho v_k - \nabla f(x_k) \eta
\end{aligned}$$

If we rescale $v(t)$ as $\tilde{v}(t) = \eta v(t)$, and set $\mu = \eta \rho$, we get

$$\begin{aligned}
x_{k+1} &= x_k + \tilde{v}_{k+1} \\
\tilde{v}_{k+1} &= \mu \tilde{v}_k - \eta \nabla f(x_k)
\end{aligned}$$

These equations comprise the classic *momentum* method for gradient descent [Polyak, 1964]. This can help us get over ‘bumps’ and past ‘plateaus’. It requires keeping track of the ‘velocity’ $\tilde{v}_k$ of the sequence x_k .

The ‘momentum’ method is something of a misnomer - it is more like ‘frictional’.

Other algorithms

Other slightly more complex optimisation algorithm include:

1. Nesterov Accelerated Gradient Descent [Nesterov, 1983], in which we essentially replace the $\nabla f(x_k)$ in the original formulation of momentum with $\nabla f(x_k + \mu \tilde{v}_k)$, the updated position of x_k .
2. Adaptive Gradient algorithm [Duchi et al., 2011] (ADAGrad), which uses a faster learning rate for certain parameters
3. Adaptive Moment Estimation (ADAM) [Adam et al., 2014], which again adapts learning rate for each parameter, but does so by scaling by an average of recent gradients for that weight.

Generally, the ADAM optimiser is preferentially used.

Exercises

1. Suppose f is a *harmonic* function defined on a region $K = [a_1, b_1] \times [a_2, b_2] \dots [a_n, b_n]$; that is, a function for which

$$\sum_{i=1}^n \frac{\partial^2 f}{\partial x_i^2} = 0 \quad (11.10)$$

for all $x \in K$. Show that f can attain a maximum or minimum only on the boundary of K (you may assume that at a minimum, the second derivative in any direction must be negative).

2. Find a function $f : \mathbb{R} \rightarrow \mathbb{R}$ which is everywhere differentiable, Lipschitz, non-constant, has a single minimum, and for which gradient descent can never find a minimum.

Solutions

1. In D , not all second derivatives can be negative, for then we would have

$$\sum_{i=1}^n \frac{\partial^2 f}{\partial x_i^2} < 0 \tag{11.11}$$

2. For instance,

$$f(x) = x^2 + |x| \left(1 + \cos \left(\frac{1}{x} \right) \right)$$

Practicalities of neural network training

Reading list and references

[Calin, 2020, Chapter 6]
[Zhang et al., 2021, Chapter 5]

12.1 Introduction

In this lecture, we will look at how to practically train neural networks. We have previously looked at the direct backpropagation algorithm and gradient descent, along with some minor modifications.

We will look at the justification for training in *batches* rather than sample-by-sample. We will also look at the idea of weight initialisation; effectively, where to start.

We will then turn our attention to the idea of ‘dropout’, in which we leave neurons out while training.

12.1.1 The basic backpropagation algorithm

Thus far, we have abstractly introduced an algorithm which roughly works as follows: We start with:

1. A neural network, parametrised by θ , implementing a function $f(x; \theta)$
2. Some training data pairs $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
3. A function $L(y, \hat{y})$ which tells us how good an approximation $\hat{y}$ is of a value y .

and we do something like:

1. Choose a learning rate η
2. Initialise θ to a value θ_0 somehow; for instance, choosing weights and biases uniformly randomly in some interval
3. Evaluate the derivative function of $f(x; \theta)$ and $L(y, f(x; \theta))$ using backpropagation:

$$\nabla_{\theta} L(y, f(x; \theta)) = \frac{\partial L(y, f(x; \theta))}{\partial \theta}$$

4. Loop through the training data, updating θ according to

$$\theta_{r+1} = \theta_r - \eta \nabla_{\theta} L(y_i, f(x_i; \theta))|_{\theta=\theta_r}$$

5. Keep looping through the training data until $|\theta_{r+1} - \theta_r| < \epsilon$, or some other stopping criterion is met.

Clearly, there are a lot of potential problems and a lot of scope for adaptation.

12.2 Batch training

A couple of problems immediately arise with this algorithm. It is clearly conceivable that the same training example could lead to updates in opposite directions. For instance, if we have a dataset of observations of the random variable X, Y for which

$$(Y|X) \sim \text{Bernoulli}(h(X)) \quad (12.1)$$

for some function h with $h(X) \in [0, 1]$, and we have a neural network $f(x, \theta)$, which we are evaluating using mean-squared error loss, then

$$\text{sign} \nabla_{\theta} L(1, f(x; \theta)) = -\text{sign} \nabla_{\theta} L(0, f(x; \theta)) \quad (12.2)$$

so if we encounter the samples $(x, 0)$ and $(x, 1)$ (which is perfectly possible) then we will ‘update’ in opposite directions.

An obvious solution is to compute the ‘directions’ for all training samples, and average them; that is, to update according to the rule:

$$\theta_{r+1} = \theta_r - \eta \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} L(y_i, f(x_i; \theta))|_{\theta=\theta_r} \quad (12.3)$$

but this tends to be computationally slow: our estimate of θ may be egregiously wrong, and we would know this after looking at only a few samples, but we must labor through the whole training set before readjusting θ .

For this reason, we usually update in ‘batches’; that is, we select a subset of the training data, and compute the average direction of update suggested by that subset. That is, we nominate an $N \ll n$, choose N samples $(x_{(1)}, y_{(1)}), (x_{(2)}, y_{(2)}), \dots, (x_{(N)}, y_{(N)})$ from $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, and update as:

$$\theta_{r+1} = \theta_r - \eta \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L(y_{(i)}, f(x_{(i)}; \theta))|_{\theta=\theta_r}$$

It is common to partition the set n into blocks of size N , and pass through these blocks, updating n/N times in total. One pass through the dataset in this way is called an *epoch*. This method is called *Stochastic gradient descent*.

12.3 Weight initialisation

We will now put some thought into how to initialise θ , or rather how to choose θ_0 . It is fairly obvious why this matters; the closer we manage to start to the optimal value of θ , the less we have to update. To note a couple of other trivial properties:

1. If we set θ_0 to very large values, then we may also find we have very large gradients, and be susceptible to overflows when we try and calculate the gradient of the neural network by backpropagation
2. If we initialise all weights to the *same* value, then we have no way to induce differences between neurons; that is, if all the neurons have the same weights initially, they always will.
3. If weights are small enough, then the variance of the input signal decreases as we go through the network. Suppose we use activation function ϕ , with $\phi'(x) \leq C$, and assume that the inputs of the ℓ th layer $X^{(\ell)}$ are independent random variables. Denote the weights in the ℓ th layer by $W_{ij}^{(\ell)}$ and the bias by $b_j^{(\ell)}$, so the output of the ℓ th layer X^{ℓ} is given by

$$X_j^{(\ell)} = \phi \left(\sum_i W_{ij}^{(\ell)} X_i^{(\ell-1)} - b_j^{(\ell)} \right)$$

Given that $\text{var}(f(X)) \approx f'(\mathbb{E}(X))^2 \text{var}(X)$ (see exercise), we have:

$$\begin{aligned} \text{var}(X_j^{(\ell)}) &\approx \phi' \left(\mathbb{E} \left(\sum_i W_{ij}^{(\ell)} X_i^{(\ell-1)} - b_j^{(\ell)} \right) \right)^2 \text{var} \left(\sum_i W_{ij}^{(\ell)} X_i^{(\ell-1)} - b_j^{(\ell)} \right) \\ &= \phi' \left(\sum_i W_{ij}^{(\ell)} \mathbb{E}(X_i^{(\ell-1)}) - b_j^{(\ell)} \right)^2 \left(\sum_i W_{ij}^{(\ell)} \text{var}(X_i^{(\ell-1)}) \right) \end{aligned}$$

$$\begin{aligned}
&\leq C^2 \left(\sum_i W_{ij}^{(\ell)} \text{var}(X_i^{(\ell-1)}) \right) \\
&\leq C^2 \left(\sum_i (W_{ij}^{(\ell)})^2 \right)^{\frac{1}{2}} \left(\sum_i \text{var}(X_i^{(\ell-1)})^2 \right)^{\frac{1}{2}}
\end{aligned}$$

using the Cauchy-Schwarz inequality in the last step. If we denote:

$$V^{(\ell)} = \sum_i \text{var}(X_i^{(\ell)})^2 \quad (12.4)$$

then summing and squaring the previous expression gives

$$V^{(\ell)} < C^4 \left(\sum_i (W_{ij}^{(\ell)})^2 \right) V^{(\ell-1)} \quad (12.5)$$

so for small weights, the value $V^{(\ell)}$ can shrink very rapidly with increasing ℓ . We can, of course, get the opposite problem if weights are too large.

It is clear that we should initialise weights to something that is variable, but neither too large nor too small.

Let us presume we begin by initialising the biases in layer (j) to 0, and the weights $W_{ij}^{(\ell)}$ to be independently chosen from a Gaussian distribution with mean 0 and variance σ^2 ; that is, $W_{ij}^{(\ell)} \sim N(0, \sigma^2)$. We will also choose $W_{ij}^{(\ell)}$ independently of $X^{(\ell)}$. What should we choose σ to be?

A sensible choice would be to approximately preserve the variance of each component; so $\text{var}(X_i^{(\ell)}) \approx \text{var}(X_i^{(\ell-1)})$. To do this, let's assume that $X_i^{(\ell)}$ are IID, and $X_i^{(\ell-1)}$ are also IID, and that there are $d^{(\ell)}$ neurons in layer ℓ . We'll start with the approximate formula $\text{var}(f(X)) \approx f'(\mathbb{E}(X))^2 \text{var}(X)$. Now we have $\mathbb{E}(W_{ij}^{(\ell)} X_i^{(\ell)}) = 0$ and $\text{var}(W_{ij}^{(\ell)} X_i^{(\ell)}) = 0$, so

$$\begin{aligned}
\text{var}(X_j^{(\ell)}) &\approx \phi' \left(\mathbb{E} \left(\sum_i^{d^{(\ell)}} W_{ij}^{(\ell)} X_i^{(\ell-1)} - b_j^{(\ell)} \right) \right)^2 \text{var} \left(\sum_i^{d^{(\ell)}} W_{ij}^{(\ell)} X_i^{(\ell-1)} - b_j^{(\ell)} \right) \\
&= \phi' \left(\sum_i^{d^{(\ell)}} \mathbb{E}(W_{ij}^{(\ell)} X_i^{(\ell-1)}) - b_j^{(\ell)} \right)^2 \left(\sum_i^{d^{(\ell)}} \text{var}(W_{ij}^{(\ell)} X_i^{(\ell-1)}) \right) \\
&= \phi' \left(-b_j^{(\ell)} \right)^2 \left(\sum_i^{d^{(\ell)}} \sigma^2 \text{var}(X_i^{(\ell-1)}) \right) \\
&= \sigma^2 d^{(\ell)} \phi'(0)^2 \text{var}(X_i^{(\ell-1)})
\end{aligned}$$

so we should choose σ such that $\sigma^2 d^{(\ell)} \phi'(0)^2 = 1$, or rather

$$\sigma^2 = \frac{1}{d^{(\ell)} \phi'(0)^2}$$

12.3.1 Xavier initialisation

We derived a rough formula for how to choose $\text{var}(W_{ij}^{(\ell)})$ by preserving the variance of $X_j^{(\ell)}$ across layers ℓ . For the purposes of backpropagation, we may be more interested in preserving the variance of the gradient of the loss with respect to the weights; that is, to ensure that

$$\frac{\partial L(y, f(x; \theta))}{\partial W_{ij}^{(\ell)}} \approx \frac{\partial L(y, f(x; \theta))}{\partial W_{ij}^{(\ell-1)}} \quad (12.6)$$

We will not go through the derivation of where to go from here, but this essentially leads to the equation:

$$\text{var}(W_{ij}^{(\ell)}) = \frac{2}{d^{(\ell)} + d^{(\ell-1)}} \quad (12.7)$$

or roughly the mean of the number of neurons in the ℓ and $(\ell - 1)$ th layers

12.4 Dropout

Let's recall a note from statistical learning. If we have data X_i, Y_i ($i \in 1..n$), where $(X_i, Y_i) \sim (X, Y)$, the generalisation error is

$$GE(f) = \int C(Y, f(X)) f_{X,Y}(x, y) dx dy$$

We want to minimise this, but can't directly estimate it from the data on which we train the neural network. We usually hold-out a set of 'test' data and evaluate the neural network on this to estimate the generalisation error. An 'overfitted' model will tend to be very non-robust to small perturbations in the data.

It would be nice if we could fit a neural network without overfitting too much. One way to combat overfitting is to add noise to the input of a network, and try and make the output of our network robust to that noise. This turns out to be roughly equivalent to 'Ridge' regression in linear models [Bishop, 1995].

A better way is to introduce noise *throughout* the network. This can be accomplished with 'dropout' [Srivastava et al., 2014]¹. Dropout works by literally 'dropping' neurons out from internal layers during training (Figure 12.1)

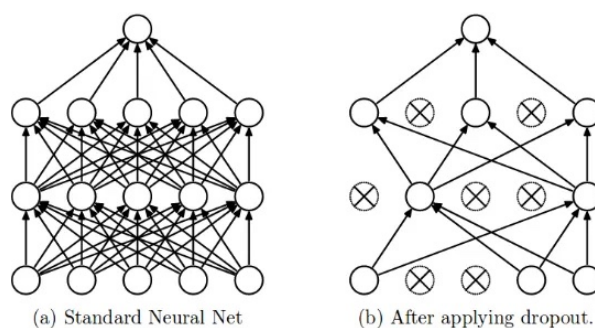


Figure 12.1: Illustration of dropout (adapted from [Srivastava et al., 2014])

Suppose that the input to layer ℓ is $X^{(\ell-1)}$. The usual output of the layer, $X^{(\ell)}$, is generated by:

$$X_i^{(\ell)} = \phi \left(\sum_j W_{ij}^{(\ell)} X_j^{(\ell-1)} + b_i \right) \quad (12.8)$$

To add dropout for layer $\ell - 1$, we first nominate a dropout probability p . We replace the calculation above with

$$\begin{aligned} r_j^{(\ell-1)} &\sim \text{Bernoulli}(p) \\ \tilde{X}^{(\ell-1)} &= r^{(\ell-1)} X^{(\ell-1)} \frac{1}{1-p} \\ X_i^{(\ell)} &= \phi \left(\sum_j W_{ij}^{(\ell)} \tilde{X}_j^{(\ell-1)} + b_i \right) \end{aligned}$$

that is, for each neuron in the layer, we either drop it out (with probability $1 - p$) or don't; we then change our input layer to be 0 for dropped-out neurons, and for non-dropped out neurons, we scale it. We then propagate forwards as usual.

This means the output of the neural network for a fixed input and fixed weights is now a random variable. To use dropout in practice, we treat this random variable as the output of the network, and backpropagate through the dropped-out network, to estimate the gradient for updating.

This ensures that *the output of the neural network cannot depend too much on one layer*.

¹With nearly 40,000 citations, this paper is in the top-20 most cited papers in machine learning research

Exercises

1. Show statement [12.2](#) holds; that is, if we encounter samples $(x, 0)$ with parameters θ we will update in the opposite direction to if we encounter samples $(x, 1)$ with parameters θ .
2. Demonstrate that (under reasonable conditions) the batch update is an unbiased estimator of the ‘correct’ direction of update, and that its variance is roughly n/N times that of the whole-data update in equation [12.3](#)
3. Demonstrate that for a continuous random variable X and differentiable function f we have $\text{var}(f(X)) \approx f'(\mathbb{E}(X))^2 \text{var}(X)$. Use a Taylor expansion of f about $\mathbb{E}(X)$.

Solutions

1. We have

$$\begin{aligned}
 \nabla_{\theta} L(1, f(x_i; \theta)) &= \frac{\partial L(y, f(x; \theta))}{\partial \theta} \\
 &= \frac{\partial}{\partial \theta} (f(x; \theta) - 1)^2 \\
 &= 2 \frac{\partial f(x; \theta)}{\partial \theta} (f(x; \theta) - 1) \\
 \nabla_{\theta} L(0, f(x_i; \theta)) &= 2 \frac{\partial f(x; \theta)}{\partial \theta} f(x; \theta)
 \end{aligned} \tag{12.9}$$

and $f(x; \theta)$ and $f(x; \theta) - 1$ have opposite sign.

2. Denote $Z_i = \nabla_{\theta} L(y_{(i)}, f(x_{(i)}; \theta)|_{\theta=\theta_r})$. Then the expected value of the change to θ is:

$$\mathbb{E}(\theta_{r+1} - \theta_r) = \mathbb{E} \left(\frac{1}{N} \sum_i Z_i \right) = \frac{1}{N} \sum_i \mathbb{E}(Z_i) = \frac{1}{N} N \mathbb{E}(Z_i) = \mathbb{E}(Z_i)$$

and $\mathbb{E}(Z_i) = \mathbb{E}(\nabla_{\theta} L(Y, f(X; \theta)|_{\theta=\theta_r}))$, which is what it ‘should’ be. The variance statement follows from the central limit theorem applied to $\frac{1}{N} \sum_i Z_i$.

3. Denote $m = \mathbb{E}(X)$. In a neighbourhood of m , we have, using a Taylor expansion about m :

$$f(x) = f(m) + f'(m)(x - m)^2 + O((x - m)^2) \tag{12.10}$$

so

$$\begin{aligned}
 \text{var}(f(x)) &\approx \text{var}(f(m)) + \text{var}(f'(m)(x - m)^2) \\
 &= 0 + f'(m)^2 \text{var}(X - m) \\
 &= f'(m)^2 \text{var}(X)
 \end{aligned}$$

as required.

Convolutional neural networks

Reading list and references

[Calin, 2020, Chapters 16]
[Zhang et al., 2021, Chapters 7-8]

13.1 Introduction

In this section, we will look at ‘convolutional’ neural networks (CNNs). CNNs are feed-forward networks which are classically used to classify images, or spatially dependent data.

Typically, CNNs are sparse (most weights are 0), with some consequences for their training. Usually, ReLU is used as an activation function.

13.2 Signals

In order to make some sense of the convolution operator which defines convolutional networks, we will look at two-dimensional ‘discrete signals’. These are mostly analogous to the one-dimensional case. The higher-dimensional analogues are pretty obvious.

Definition 22. A *two-dimensional discrete signal* is a set $[y_{ij}]$, with $i, j \in \mathbb{Z}$ and $y_{ij} \in \mathbb{R}$.

A discrete signal is:

- a *finite signal* if $\max|y_{ij}| < \infty$
- a *signal of compact support* if there exists N with $y_{ij} = 0$ for all $|i| > N$ or $|j| > N$

We define a *filter* as a discrete signal consisting of weights, e.g.

$$w = [\dots, w_{-2}, w_{-1}, w_0, w_1, w_2, \dots] \quad (13.1)$$

and the *convolved signal* $z = y * w$ as the signal $\{z_{k\ell}\}$ defined by

$$z_{k\ell} = \sum_{-\infty < i, j < \infty} y_{(k+i)(\ell+j)} w_{ij} \quad (13.2)$$

essentially, the dot-product of y and w after shifting y to put k at 0. We will denote the *convolution operator* applied to a signal y by $\mathcal{C}(y)$, leaving the kernel itself unspecified. We note some basic properties, with deeper interpretations later:

Theorem 32. The convolution operator is invariant to translations; that is, if $T_{a,b}$ is a ‘translation’ operator mapping discrete signals to discrete signals via:

$$(T_{a,b}y)_{ij} = y_{(i-a)(j-b)} \quad (13.3)$$

then we have

$$\mathcal{C}(T_{a,b}y) = T_{a,b}\mathcal{C}(y) \quad (13.4)$$

Proof. Simple; see exercises. $\square$

Suppose our input $X^{(0)}$ is an $m \times n$ array of pixels (or, initially, just a vector of length mn). After convolving with a kernel, we get another $m \times n$ array $X^{(1)}$. We can encode convolution using mn neurons: after all, each element of $X^{(1)}$ is simply a weighted sum of elements of $X^{(0)}$.

This immediately suggests the idea of *chaining* convolutions. If we can use the first convolutional layer to ‘see’ corners, or edges, then can we convolve again to find squares, polygons, or more complicated images?

We can, but there is a subtlety. A straightforward convolution can be encoded as a neuron *with a linear activation function*. Recalling results from an earlier lecture, a two-layer network of linear neurons is equivalent to a single-layer network. This leads to the not-immediately obvious result that *two convolutional layers with linear activations are equivalent to a single layer*.

It is not, however, immediate that this new single layer can be represented as a convolution itself. We will now show this.

Theorem 33. Suppose y is a discrete linear signal with support N , w^1 is a discrete linear kernel with support n_1 , and w^2 is discrete linear kernel with support n_2 . Then

$$(y * w^1) * w^2 = y * W \quad (13.5)$$

where $W = (w^2 * w^1)$ is a discrete kernel with support at most $n_1 + n_2$.

Proof. By direct calculation we have:

$$\begin{aligned} (y * w^1)_{k\ell} &= \sum_{-\infty < i, j < \infty} y_{(k+i)(\ell+j)} w^1_{ij} \\ &= \sum_{|i|, |j| < n_1} y_{(k+i)(\ell+j)} w^1_{ij} \end{aligned}$$

so

$$\begin{aligned} ((y * w^1) * w^2)_{pq} &= \sum_{-\infty < k, \ell < \infty} (y * w^1)_{(p+k)(q+\ell)} w^2_{k\ell} \\ &= \sum_{|k|, |\ell| < n_2} (y * w^1)_{(p+k)(q+\ell)} w^2_{k\ell} \\ &= \sum_{|k|, |\ell| < n_2} \left(\sum_{|i|, |j| < n_1} y_{(p+k+i)(q+\ell+j)} w^1_{ij} \right) w^2_{k\ell} \\ &= \sum_{|i|, |j| < n_1} \sum_{|k|, |\ell| < n_2} y_{(p+k+i)(q+\ell+j)} w^1_{ij} w^2_{k\ell} \\ &= \sum_{|i|, |j| < n_1} \sum_{|k|, |\ell| < n_2} y_{(p+k+i)(q+\ell+j)} w^1_{ij} w^2_{k\ell} \\ &= \sum_{-\infty < a, b < \infty} y_{(p+a)(q+b)} \left(\sum_{\substack{(i,j,k,\ell): \\ i+k=a, q+\ell=b}} w^1_{ij} w^2_{k\ell} \right) \\ &= \sum_{-\infty < a, b < \infty} y_{(p+a)(q+b)} W_{ab} \\ &= (y * W)_{pq} \end{aligned}$$

where

$$\begin{aligned} W_{ab} &= \sum_{\substack{(i,j,k,\ell): \\ i+k=a, q+\ell=b}} w^1_{ij} w^2_{k\ell} \\ &= \sum_{-\infty < i, j < \infty} \sum_{-\infty < k, \ell < \infty} w^1_{(-i)(-j)} w^2_{(a+i)(b+j)} \\ &= (w^2 * w^{-1})_{ab} \end{aligned} \quad (13.6)$$

where $w_{ij}^{-1} = w_{(-i)(-j)}^1$. To see that W has support $n_1 + n_2$, note that if $a > n_1 + n_2$ and $|i| < n_1$ then

$$n_1 + n_2 < a = a - i + i \leq |a - i| + |i| < |a - i| + n_1 \quad (13.7)$$

and $|a - i| > n_2$; hence either $|i| > n_1$ or $|a - i| > n_2$, and each term in the summation in equation 13.6 is 0. $\square$

Why is convolution helpful for learning? Let's think about learning as 'extracting the useful information', and suppose we are interested in finding the position of 2×2 squares in a picture (a reasonable sort of thing we might want to do).

A naive ML approach might be to just input all the pixels values (in $\{0, 1\}$) into a neural network, and try and train it to find the square locations. However, this is going to be complicated: the network has to 'figure out' which inputs correspond to squares which are next to each other. Eventually, the network will figure out that an answer of 'square at $(2, 3)$ ' means that a certain four pixels are activated, and will do the same for each other co-ordinate.

But this is inefficient! This neural network can learn any map from $[[n]^2]^4 \rightarrow [n]^2$ equally well. This is a vastly more complicated task than it needs to do - a much greater reduction in the information about the input is required.

A convolution allows us to *send positional information* to the network directly.

With Theorem 33 in mind, in our convolutional layer, it makes sense to use a non-linear activation function. The weights of the neuron (which are common to all neurons in the layer) can be learned or trained like other weights in the network.

13.3 Convolutional neural networks

In general, when we convolve a discrete signal with compact support with a kernel with compact support, we shall ignore the 'bits around the outside': that is, we shall treat the result of convolving an $m \times n$ matrix with an $m_1 \times n_1$ kernel as a matrix of dimension $(m - m_1 + 1) \times (n - n_1 + 1)$. To write this out explicitly, suppose we are convolving the 3×3 matrix:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

with the 2×2 kernel

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

we will get

$$\begin{aligned} \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} * \begin{pmatrix} a & b \\ c & d \end{pmatrix} &= \begin{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} * \begin{pmatrix} a & b \\ c & d \end{pmatrix} & \begin{pmatrix} 2 & 3 \\ 5 & 6 \end{pmatrix} * \begin{pmatrix} a & b \\ c & d \end{pmatrix} \\ \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} * \begin{pmatrix} a & b \\ c & d \end{pmatrix} & \begin{pmatrix} 5 & 6 \\ 8 & 9 \end{pmatrix} * \begin{pmatrix} a & b \\ c & d \end{pmatrix} \end{pmatrix} \\ &= \begin{pmatrix} 1a + 2b + 3c + 4d & 2a + 3b + 5c + 6d \\ 4a + 5b + 7c + 8d & 5a + 6b + 8c + 9d \end{pmatrix} \end{aligned}$$

13.4 Some notes

13.4.1 Pooling

Convolutional neural networks reduce the information in an image, but not altogether drastically: using a 4×4 convolution on a 512×512 image will reduce it to just a 508×508 image. We generally want to reduce it down to a small multinomial distribution (e.g. recognise digits) so clearly something a bit more drastic is needed.

That thing is *pooling*. Roughly, we care mostly about 'whether something is in the image' rather than 'where in the image is it'. If we can 'find' it by using a convolution to light it up (e.g. the following convolution will give a high activation if the pattern of pixels is matched exactly:)

$$\begin{array}{cccccccc} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \quad (13.8)$$

then we want to combine the outputs of all the activations to say whether it is there or not (e.g., is at least one activation high?)

Pooling is when we replace a set of activations with a summary statistic: usually max, min, or mean. In this case, we want the maximum activation, to answer whether this is implemented in the image.

This is a little difficult to back-propagate through, but works as expected: only the neuron which actually returned the maximum has its weights updated.

13.4.2 Channels

We may want to run several convolutional networks in parallel, and later combine the results. This is usually referred to as using *channels*. We use several convolutional kernels to process the image simultaneously, to generate a series of convolved images, and (generally) combine these convolved images later.

13.4.3 Overall architecture

The overall architecture of a convolutional neural network ends up looking something like this:

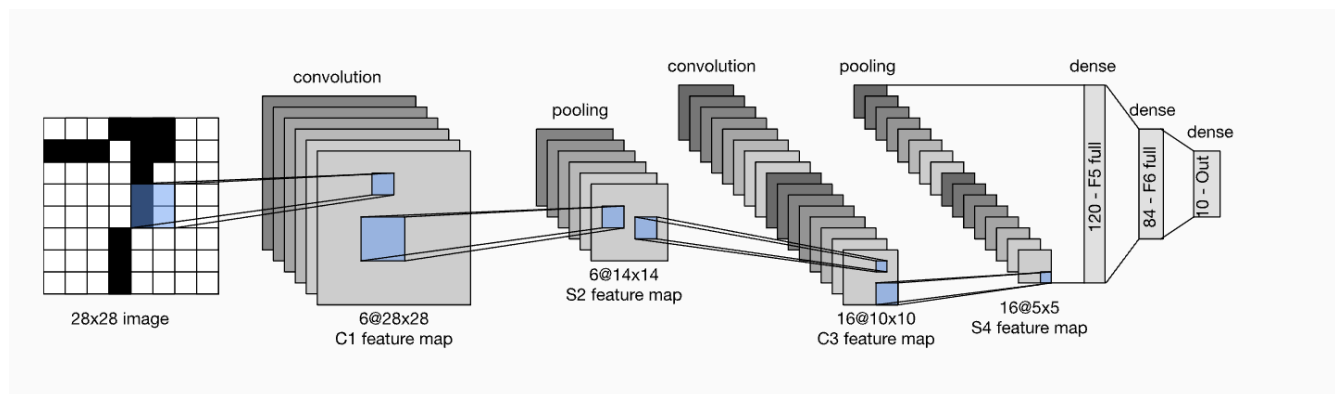


Figure 13.1: The LeNet, a typical convolutional neural network. From [Zhang et al. \[2021\]](#)

We begin with an image, and a series of convolutions each of which search for a particular visual ‘feature’. Each convolution gives us a new ‘channel’ (second column). We then partially pool the results (usually using maximum pooling) to reduce the size of the data. This architecture then uses another convolutional layer, expanding the number of channels (fourth column), then pools again (fifth column). Finally, we use a standard feedforward neural network (columns 6-8) to do what we want to do with the image (usually classify what the image is of).

13.4.4 Transfer learning

We will finish with a short note on the idea of ‘Transfer learning’. This is a deep idea in machine learning theory, but it has a particularly nice manifestation in convolutional neural networks.

Suppose we want to train a convolutional neural network to differentiate between X-ray images of two subtypes of a rare type of lung disease (e.g., small-cell lung cancer from adenocarcinoma). We have a few thousand images from a hospital from people with one of these two types.

X-ray images are large; usually several thousand pixels in each direction. Even with a very efficiently-designed convolutional neural network, there will be thousands of parameters to learn. This is generally going to be intractable with only our few thousand images. It seems like a useful idea to be able to use the information in larger, existing studies to make our own one easier. This idea is called ‘Transfer learning’.



Figure 13.2: A typical chest x-ray ([Wikipedia](#))

But there are millions of chest x-ray images available to train on which aren't from people with these rare types of lung cancer. Moreover, there are huge machine learning models already trained to them. When we want to use our machine learning model to tell two rare types of lung cancer apart, most of the work is common to any other machine learning model trying to find things in chest x-rays: find the lungs, identify the ribs, identify the heart, and so on (figure 13.2)

This 'grunt work' is just feature extraction - or what is done in the first few layers of the convolutional neural network. And indeed, a useful way to do transfer learning is to just take the first few layers of a trained model (in figure 18.1, maybe the first three or five columns) and retrain the last bit.

This is philosophically similar to 'dimension reduction' approaches like principal component analysis, but we are choosing the way to compress the information (and reduce the entropy of the dataset) via use of an already-trained model.

Exercises

1. Prove that a finite-energy signal with compact support is L^1 -finite.
2. For a finite energy signal y , specify the weight signal w such that $z = y * w$ is a moving average of order $2n + 1$ of y .
3. Prove theorem [32](#).

Solutions

1. Suppose $y_k \leq C$. We have:

$$\sum_{k=-\infty}^{\infty} |y_k| = \sum_{|k| \leq N} |y_k| + \sum_{|k| > N} |y_k| = \sum_{|k| \leq N} |y_k| \leq (2N+1) \max |y_k| \leq (2N+1)C < \infty$$

2. A moving average of order $2n+1$ of y is a sequence z_k with

$$z_k = \frac{1}{2n+1} \sum_{i=k-n}^{k+n} y_i$$

so is equal to the convolution $y * w$, where w_k is equal to $1/(2n+1)$ if $-n \leq k \leq n$ and 0 otherwise.

3. Suppose the kernel operator is a convolution with the kernel z_{ij} ; that is $\mathcal{C}(y) = y * z$. Now, evaluating at indices k, r :

$$\begin{aligned} \mathcal{C}(T_{a,b}(y))_{k,r} &= ((T_{a,b}y) * z)_{k,r} \\ &= \sum_i \sum_j (T_{a,b}(y))_{k+i, r+j} z_{i,j} \\ &= \sum_i \sum_j y_{k+i-a, r+j-b} z_{i,j} \\ &= \sum_i \sum_j y_{(k-a)+i, (r-b)+j} z_{i,j} \\ &= (y * z)_{k-a, r-b} \\ &= T_{a,b}((y * z)_{k,r}) \end{aligned} \tag{13.9}$$

as required.

Recurrent neural networks

Reading list and references

[Calin, 2020, Chapter 17]
[Zhang et al., 2021, Chapters 9,10]

14.1 Introduction

So far, we have only really looked at *feedforward* neural networks, characterised by an architecture in which we start with inputs, progress through layers, and get to an output.

In this lecture we will look at *recurrent* neural networks, and depart from this architecture a bit. Recurrent neural networks are neural networks specialised to model *sequential* data.

14.2 Dynamical systems

We shall start by considering a deterministic dynamical system.

Definition 23. A deterministic discrete dynamical system $\{h_t\}$, $t \in \{1, 2, \dots\}$, is a sequence of random variables given by

$$h_t = f(h_{t-1}; \theta) \quad (14.1)$$

for $t > 1$, where f is called the ‘transition function’ and θ is a set of parameters.

We assume states h_t are random variables, although all randomness comes from the first state h_0 . We can describe this in terms of sigma-algebras: recalling our notation of $\mathfrak{S}(x)$ as the σ -algebra generated by x , we denote by $\mathfrak{S}(h_t)$ the σ -algebra generated by h_t , and note that $\mathfrak{S}(h_t) \subseteq \mathfrak{S}(h_{t-1})$.

We now consider a more general dynamical system, in which we inject new randomness at each t , governed by a random process X_t (for our purposes, a random process is just a sequence of random variables, not necessarily independent, but defined on the same probability space).

Definition 24. A discrete dynamical system $\{h_t\}$, $t \in \mathbb{N}$, is a sequence of random variables given by

$$h_t = f(h_{t-1}, X_t; \theta) \quad (14.2)$$

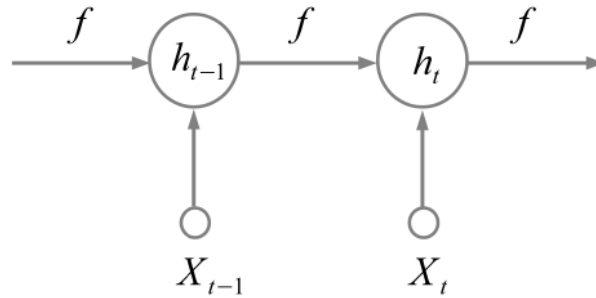
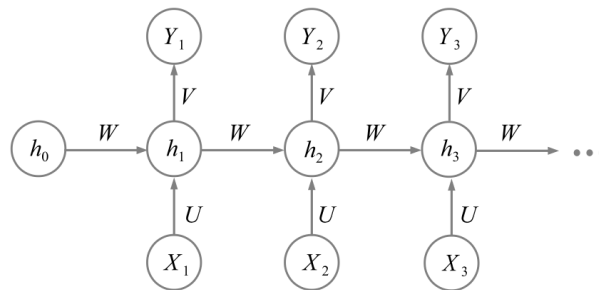
for $t > 1$, where f is a Borel-measurable function, X_t is a random process, and θ is a set of parameters.

Now our sequence of σ -algebras is a little more complicated. Denoting $I_t = \mathfrak{S}(X_t)$ we have

$$\mathfrak{S}(h_t) \subseteq \mathfrak{S}(\{h_{t-1}, X_t\}) = \mathfrak{S}(\mathcal{H}(t-1) \cup I_t)$$

14.3 Recurrent neural networks

Let’s suppose that we draw a discrete dynamical system as in figure 14.1, and add an ‘output’ from each h_t as an outcome Y_t as in figure 14.2.

Figure 14.1: Illustration of a dynamical system with random process X_t Figure 14.2: Illustration of a dynamical system with random process X_t and output Y_t

If we consider each block $X_t \rightarrow h_t \rightarrow Y_t$ separately, and model this as a neural network with output Y_t , hidden layer h_t and inputs X_t and h_{t-1} , then we get what is called a *recurrent* neural network.

Making this setup a bit more formal, let us say that the sample space of h_t is $\mathbb{R}^k$, and the sample space of X^t is $\mathbb{R}^p$. We set this up as:

$$\begin{aligned} h_t &= \phi(W h_{t-1} + U X_t + b) \\ Y_t &= V h_t + C \end{aligned}$$

with appropriately sized matrices W , U , and V . We immediately note a few points

1. Note that each h_t depends also on h_{t-1} : this is what distinguishes a recurrent neural network from a series of feedforward networks
2. The matrices W , U , and V do not depend on t . This corresponds to the function f being fixed with t .
3. The values h_t are vectors with all elements in the range of $\phi(\cdot)$.
4. The parameter θ is (W, U, b)
5. We can quickly see the resemblance to a hidden Markov model
6. Recurrent neural networks typically use the activation function $\phi(\cdot) = \tanh(\cdot)$.

The essence of the ‘sequence’ of data comes from the matrix W (or the function f in our dynamical system formulation).

14.4 Training recurrent neural networks

Recurrent neural networks encounter training examples X_t , Y_t sequentially, updating as they go. This does not lend itself directly to the backpropagation algorithm we have studied for feedforward networks.

Suppose our network has input X_t , output Y_t , and *target* output Z_t , over T total times. We have a series of loss estimates L_t comparing how similar each Y_t is to each Z_t , with total loss L as

$$L = \sum_{t=0}^T L_t$$

As usual, we have a range of choices for L .

14.4.1 Backpropagation through time

We will consider a backpropagation for $T = 2$ with activation function ϕ . We have:

$$\begin{aligned} h_i &= \phi(W h_{i-1} + U X_i + b) \\ Y_i &= V h_i + c \end{aligned}$$

for $i \in \{1, 2\}$. We will use the shorthand:

$$a_i = W h_{i-1} + U X_i + b \quad (14.3)$$

In order to perform gradient descent on the loss function, we need to compute five gradients:

$$\nabla L = \left(\frac{\partial L}{\partial W}, \frac{\partial L}{\partial V}, \frac{\partial L}{\partial U}, \frac{\partial L}{\partial b}, \frac{\partial L}{\partial c} \right) \quad (14.4)$$

We start with

$$\frac{\partial h_i}{\partial W} = \frac{\partial}{\partial W} \phi(a_i) = \phi'(a_i) \frac{\partial a_i}{\partial W} = \phi'(a_i) h_{i-1}$$

We similarly have

$$\frac{\partial h_i}{\partial b} = \phi'(a_i) \quad (14.5)$$

The value $L_i = \text{Loss}(Y_i, Z_i)$ depends on h_i only through Y_i . Let us write $\gamma_i = \frac{\partial L_i}{\partial Y_i}$. Now we have

$$\frac{\partial L_i}{\partial h_i} = \frac{\partial L_i}{\partial Y_i} \frac{\partial Y_i}{\partial h_i} = \gamma_i V$$

and

$$\frac{\partial h_i}{\partial h_{i-1}} = \frac{\partial}{\partial h_{i-1}} \phi(a_i) = \phi'(a_i) \frac{\partial a_i}{\partial h_{i-1}} = \phi'(a_i) W$$

and

$$\frac{\partial h_i}{\partial U} = \frac{\partial}{\partial U} \phi(a_i) = \phi'(a_i) \frac{\partial a_i}{\partial U} = \phi'(a_i) X_i$$

Now we can compute the gradients we need. Since L_i depends on V only through Y_i , we have

$$\begin{aligned} \frac{\partial L}{\partial V} &= \sum_i \frac{\partial L_i}{\partial V} \\ &= \sum_i \frac{\partial L_i}{\partial Y_i} \frac{\partial Y_i}{\partial V} \\ &= \sum_i \frac{\partial L_i}{\partial Y_i} \frac{\partial}{\partial V} (V h_i + c) \\ &= \sum_i \gamma_i h_i \end{aligned}$$

and

$$\frac{\partial L}{\partial c} = \sum_i \frac{\partial L_i}{\partial c}$$

$$\begin{aligned}
&= \sum_i \frac{\partial L_i}{\partial Y_i} \frac{\partial Y_i}{\partial c} \\
&= \sum_i \frac{\partial L_i}{\partial Y_i} \frac{\partial}{\partial c} (Vh_i + c) \\
&= \sum_t \frac{\partial L_i}{\partial Y_i}
\end{aligned}$$

Computing the gradient of L with respect to W gets difficult. The value L_1 depends on W only through h_1 , but L_2 depends on W through both h_1 and h_2 (see figure 14.2) and in general L_i depends on W through all the values $h_1, h_2, \dots, h_i$. To make it more difficult, h_i also depends on h_{i-1} . Let us start by considering just L_2 :

$$\begin{aligned}
\frac{\partial L_2}{\partial W} &= \frac{\partial L_2}{\partial h_2} \frac{dh_2}{dW} \\
&= \frac{\partial L_2}{\partial h_2} \frac{d}{dW} \phi(W h_1 + U X_2 + b) \\
&= \frac{\partial L_2}{\partial h_2} \left(\frac{\partial h_2}{\partial W} + \frac{\partial h_2}{\partial h_1} \frac{dh_1}{dW} \right) \\
&= \gamma_2 V (\phi'(a_2) h_1 + \phi'(a_2) W \phi'(a_1) h_0)
\end{aligned} \tag{14.6}$$

To calculate $\frac{\partial L_i}{\partial W}$, let us denote

$$d_i = \frac{dh_i}{dW} \tag{14.7}$$

so $d_1 = \phi'(a_1) h_0$, and $d_2 = \phi'(a_2) h_1 + \phi'(a_2) W \phi'(a_1) h_0 = \phi'(a_2) h_1 + \phi'(a_2) W d_1$. Now, working recursively:

$$d_i = \frac{dh_i}{dW} = \frac{d}{dW} \phi(W h_{i-1} + U X_i + b) = \frac{\partial h_i}{\partial W} + \frac{\partial h_i}{\partial h_{i-1}} \frac{dh_{i-1}}{dW} = \phi'(a_i) h_{i-1} + \phi'(a_i) W d_{i-1}$$

and, finally,

$$\begin{aligned}
\frac{\partial L}{\partial W} &= \sum_i \frac{\partial L_i}{\partial W} \\
&= \sum_i \frac{\partial L_i}{\partial h_i} \frac{dh_i}{dW} \\
&= \sum_i \gamma_i V d_i
\end{aligned}$$

Similar derivations hold for $\frac{\partial L}{\partial U}$ and $\frac{\partial L}{\partial b}$, which are left as exercises.

14.4.2 Problems

This does not look great - there are lots of V 's and W 's getting multiplied together - and indeed, recurrent neural networks are very susceptible to both exploding gradients and vanishing gradients. If the eigenvalues of W are all less than 1, we are very susceptible to vanishing gradients; if any of them exceed 1, we are susceptible to exploding gradients.

14.5 Long short-term memory networks (LSTMs)

A tidy way to (partly) overcome the vanishing gradient problem intrinsic to recurrent neural networks is the long short-term memory network (LSTM) [Hochreiter and Schmidhuber \[1997\]](#). We will look at these further (the material is non-examinable) in our second-to-last lecture on Transformers.

Exercises

1. For a deterministic discrete dynamical process $\{h_t\}$, show that $\mathcal{H}(t) = \mathcal{H}(t-1)$ for all random variables h_1 of appropriate dimension if and only if the transition function $f(x, \theta)$ is invertible in f .
2. Let $\alpha_i = \frac{dh_i}{dW}$, $\beta_i = \frac{\partial h_i}{\partial W}$, and $\gamma_i = \frac{\partial h_i}{\partial h_{i-1}}$. Find a recursion for α_i , and show that for $n > 1$

$$\alpha_n = \beta_n + \sum_{i=1}^{n-1} \left(\prod_{j=i+1}^n \gamma_j \right) \beta_i \quad (14.8)$$

3. Derive the backpropagation-through-time formula for $\frac{\partial L}{\partial U}$

Solutions

1. Since $h_t = f(h_{t-1}, \theta)$, we have $\mathcal{H}(t) \supseteq \mathcal{H}(t-1)$. If we have $\mathcal{H}(t) = \mathcal{H}(t-1)$ then we also have $\mathcal{H}(t) \subseteq \mathcal{H}(t-1)$, so we must have $h_{t-1} = g(h_t)$ for some deterministic function g . Thus $f(x, \theta)$ is invertible and its inverse is g . The converse is clear.
2. We have $h_i = \phi(W h_{i-1} + U X_i + b)$. The value of W affects h_i both directly through this equation, and through h_{i-1} . Thus we have for $i > 1$:

$$\begin{aligned}\alpha_i &= \frac{dh_i}{dW} = \frac{\partial h_i}{\partial W} + \frac{\partial h_i}{\partial h_{i-1}} \frac{dh_{i-1}}{dW} \\ &= \beta_i + \gamma_i \alpha_{i-1}\end{aligned}$$

If $n = 2$ then we have

$$\alpha_2 = \beta_2 + \gamma_2 \alpha_1 = \beta_2 + \sum_{i=1}^{2-1} \left(\prod_{j=1+1}^2 \gamma_j \right) \beta_i$$

Suppose by induction that the formula in the question holds for $n = 1, 2 \dots n-1$. Then we have

$$\begin{aligned}\alpha_n &= \beta_n + \gamma_n \alpha_{n-1} \\ &= \beta_n + \gamma_n \left(\beta_{n-1} + \sum_{i=1}^{n-2} \left(\prod_{j=i+1}^{n-1} \gamma_j \right) \beta_i \right) \\ &= \beta_n + \gamma_n \beta_{n-1} + \sum_{i=1}^{n-2} \gamma_n \left(\prod_{j=i+1}^{n-1} \gamma_j \right) \beta_i \\ &= \beta_n + \sum_{i=n-1}^{n-1} \left(\prod_{j=i+1}^n \gamma_j \right) \beta_i + \sum_{i=1}^{n-2} \left(\prod_{j=1+1}^n \gamma_j \right) \beta_i \\ &= \beta_n + \sum_{i=1}^{n-1} \left(\prod_{j=1+1}^n \gamma_j \right) \beta_i\end{aligned}$$

which gives the requisite result by induction.

3. We have

$$\begin{aligned}\frac{\partial L}{\partial U} &= \frac{\partial L_1}{\partial U} + \frac{\partial L_2}{\partial U} + \frac{\partial L_3}{\partial U} \dots \\ &= \frac{\partial L_1}{\partial Y_1} \frac{\partial Y_1}{\partial U} + \frac{\partial L_2}{\partial Y_2} \frac{\partial Y_2}{\partial U} + \frac{\partial L_3}{\partial Y_3} \frac{\partial Y_3}{\partial U} \dots \\ &= \frac{\partial L_1}{\partial Y_1} \frac{\partial Y_1}{\partial h_1} \frac{dh_1}{dU} + \frac{\partial L_2}{\partial Y_2} \frac{\partial Y_2}{\partial h_2} \frac{dh_2}{dU} + \frac{\partial L_3}{\partial Y_3} \frac{\partial Y_3}{\partial h_3} \frac{dh_3}{dU} \dots \\ &= \frac{\partial L_1}{\partial Y_1} \frac{\partial Y_1}{\partial h_1} \frac{dh_1}{dU} + \frac{\partial L_2}{\partial Y_2} \frac{\partial Y_2}{\partial h_2} \left(\frac{\partial h_2}{\partial U} + \frac{\partial h_2}{\partial h_1} \frac{dh_1}{dU} \right) + \frac{\partial L_3}{\partial Y_3} \frac{\partial Y_3}{\partial h_3} \left(\frac{\partial h_3}{\partial U} + \frac{\partial h_3}{\partial h_2} \frac{dh_2}{dU} \right) \dots\end{aligned}\tag{14.9}$$

Generative adversarial networks

Reading list and references

[Calin, 2020, Chapter 19]
[Zhang et al., 2021, Chapter 20]

15.1 Introduction

In this lecture we will look at the use of networks to *generate* data rather than *summarise* it. In particular, we will look at the idea of the generative adversarial network (GAN): an intuitive method which has gained widespread modern use.

The essential idea of a GAN is to simultaneously train a model to produce simulated samples typical of training data, and a model which is trying to differentiate the true training data from the simulated training data. The two models (usually both neural networks) can be thought of as being in competition with each other.

15.2 Setup

Suppose we have some data samples $X_1, X_2, \dots, X_n$ which are identically distributed according to a given distribution; that is, $X_i \sim X$. Let us say that X has a density, which we will call f_{data} (everything we will do will work equally well if X has a mass function instead).

Essentially, we want to be able to produce more samples $X'_i \sim X$. We will generally say that samples $X'_i \sim f_{model}$, and which we will try and make equal to f_{data} .

In this lecture, we will slightly abuse notation for brevity: we will sometimes say that $X \sim f$, where we mean that X has the PDF f .

15.3 Density estimation

We want to try and find a distribution f_{model} which is similar to f_{data} . This is tantamount to estimating the distribution given by f_{data} . One standard way we might do this is *parametrically* by assuming a statistical model for f_{model} : that is, to assume that it comes from a family of distributions indexed by a parameter θ , so $f_{model}(x) = f_{model}(x; \theta)$.

We can then choose θ by a maximum-likelihood estimate: that is, choose $\theta = \theta^*$ where θ^* is defined as:

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^n f_{model}(X_i, \theta) = \arg \max_{\theta} \sum_{i=1}^n \ln [f_{model}(X_i, \theta)]$$

We may also estimate densities non-parametrically; for instance, by kernel density estimates.

Given a specification of a probability density function it is (moderately) straightforward to simulate samples from that distribution, at least in low dimensions.

15.4 Adversarial models

We will consider a ‘game’ between two players A and B . There is a ‘payoff’ V , which can be observed by both players, and A wants it to be high and B wants it to be low. For example

1. In a chess game between A and B , V may be the probability that A will win (according to some probability space)
2. In a market, V is the price of some object, and A (the buyer) wants it to be high and B (the seller) wants it to be low.
3. In a football game, the payoff V is team A 's score minus team B 's score.

Each player controls only some of the variables which affect V . Suppose A controls variables we will term x and B controls variables we term y .

1. In a chess game, x is the movements of A 's pieces, and y the movements of B 's
2. In a market, x is a set of things the seller controls; e.g., advertising, price; and y is the decision to buy.
3. In a football match, V is determined by the position of the ball, which is in turn determined by the positions and actions of the players. The variables x describe the positions and actions of A 's players, and the variables y describe the positions and actions of B 's players.

In the case of an adversarial network, our two players are statistical models, which in our case are neural networks.

- Player A is the *generator*, trying to generate new samples $X'_i \sim X$. It controls the variables x , which are the parameters of a *generative* neural network.
- Player B is the *discriminator*, trying to tell the samples X'_i apart from X_i . It controls the variables Y which are the parameters of a standard neural network.

In this case, the objective V is the *negative* of the error in distinguishing the values X_i from the values X'_i . Player A wants this to be as large as possible (e.g. close to 0), while player B wants it minimised (e.g., wants a large discrimination error).

We are on the side of the generator: we want to be able to generate samples which the distribution. The objective we want to maximise is now:

$$\arg \max_y \left[\min_x V(x, y) \right] \quad (15.1)$$

15.5 Generative adversarial networks

A generative adversarial network implements this idea. Formally, our generative network is a function

$$G(z, \theta^{(g)}) : \mathcal{Z} \rightarrow \mathcal{X}$$

which maps a *latent* space $\mathcal{Z}$ to the same space $\mathcal{X}$ from which the data comes. To use the network, we generate values z in $\mathcal{Z}$ randomly according to a probability density $f_{code}(z)$, which remains fixed.

As above, we denote the distribution of simulated samples as $f_{model}(x)$. We have, if Z has PDF $f_Z = f_{code}$ and:

$$X = G(Z, \theta^{(g)})$$

then X has PDF $f_X = f_{model}$. We generally have that Z has smaller dimension than X .

The discriminator network is a function

$$D(x, \theta^{(d)}) : \mathcal{X} \rightarrow [0, 1]$$

which takes a sample and returns a value between 0 and 1, where

- $D(x, \theta^{(d)}) = 1$ if the discriminator considers that x comes from the *generated* data
- $D(x, \theta^{(d)}) = 0$ if the discriminator considers that x comes from the *training* data

We illustrate a GAN as follows:

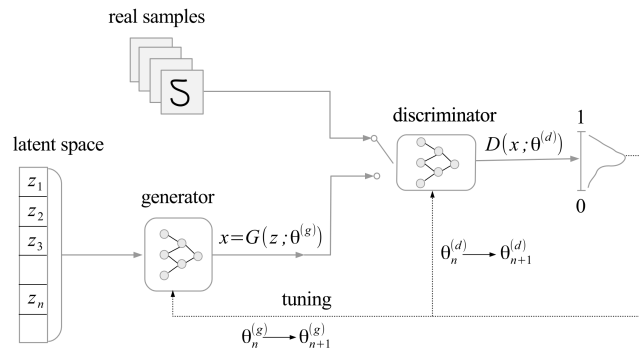


Figure 15.1: Diagram of GAN; reproduced from [Calin \[2020\]](#)

An important idea is that we will assess the discriminator by negative *log-loss*: that is, we will use the objective:

$$C(Y, \hat{Y}) = Y \ln(\hat{Y}) + (1 - Y) \ln(1 - \hat{Y}) \quad (15.2)$$

where Y is an indicator variable which is 1 if the sample is in the generated data and 0 if it is in the training data. This is *maximised* if Y is predicted well by $\hat{Y}$.

15.5.1 Loss function

Suppose we have a random variable W , which with probability p takes a values X_i with PDF f_{data} and with probability $(1 - p)$ takes a value X'_i with PDF f_{model} . We call Y the indicator variable of whether the sample comes from f_{data} (in which case $Y = 1$) or f_{model} (in which case $Y = 0$). Then we have:

$$\begin{aligned} P(Y = 1) &= p \\ f_{W|Y=1}(x) &= f_{data}(x) \\ f_{W|Y=0}(x) &= f_{model}(x) \end{aligned} \quad (15.3)$$

The quantity we want the discriminator to maximise can be written as:

$$\begin{aligned} V(G, D) &= \mathbb{E}_W \{C(Y, \hat{Y})\} \\ &= \mathbb{E}_W \{C(Y, D(W, \theta^{(d)}))\} \\ &= \mathbb{E}_Y [\mathbb{E}_{W|Y} \{C(Y, D(W, \theta^{(d)}))\}] \\ &= P(Y = 1) \mathbb{E}_{W|Y=1} \{C(Y, D(W, \theta^{(d)}))\} + P(Y = 0) \mathbb{E}_{W|Y=0} \{C(Y, D(W, \theta^{(d)}))\} \\ &= p \mathbb{E}_{X \sim f_{data}} \{C(1, D(X, \theta^{(d)}))\} \\ &\quad + (1 - p) \mathbb{E}_{X \sim f_{model}} \{C(0, D(X, \theta^{(d)}))\} \\ &= p \mathbb{E}_{X \sim f_{data}} \left\{ \ln [D(X, \theta^{(d)})] \right\} + (1 - p) \mathbb{E}_{X \sim f_{model}} \left\{ \ln [1 - D(X, \theta^{(d)})] \right\} \end{aligned} \quad (15.4)$$

and we usually use $p = 1/2$, and ignore the factor of 2, to get:

$$\begin{aligned} V(G, D) &= \mathbb{E}_{X \sim f_{data}} \left\{ \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{X \sim f_{model}} \left\{ \ln [1 - D(X, \theta^{(d)})] \right\} \\ &= \mathbb{E}_{X \sim f_{data}} \left\{ \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{Z \sim f_{code}} \left\{ \ln [1 - D(G(Z, \theta^{(g)}), \theta^{(d)})] \right\} \end{aligned}$$

15.5.2 Optimal discriminator

We now show a hopefully slightly familiar theorem (Proposition 19.4.3, [Calin \[2020\]](#)). This is essentially the Neyman-Pearson lemma (which you may remember from statistics courses) in disguise.

Theorem 34. For fixed $\theta^{(g)}$, the discriminator which maximises $V(G, D)$ is given by:

$$D(x) = \frac{f_{data}(x)}{f_{data}(x) + f_{model}(x)} \quad (15.5)$$

Proof. We will omit the arguments $\theta^{(d)}$ and $\theta^{(g)}$ since they are not needed.

Since both f_{data} and f_{model} are defined on the same space $\mathcal{X}$, we write:

$$V(G, D) = \int_{\mathcal{X}} (\ln [D(x)] f_{data}(x) + \ln [1 - D(x)] f_{model}(x)) dx$$

which can be considered a functional of $D(x)$; we write

$$V(G, D) = \int L(x, D(x)) dx \quad (15.6)$$

where the ‘Lagrangian’ $L(x, D(x))$ is given by

$$L(x, D(x)) = \ln [D(x)] f_{data}(x) + \ln [1 - D(x)] f_{model}(x)$$

For this to be maximised, we must have

$$0 = \frac{dL(D)}{dD} = \frac{f_{data}(x)}{D(x)} - \frac{f_{model}(x)}{1 - D(x)}$$

from which

$$D(x) = \frac{f_{data}(x)}{f_{data}(x) + f_{model}(x)} \quad (15.7)$$

□

Of course, we don’t know what f_{data} is - that’s the whole objective!

15.5.3 Training

To train a GAN, we firstly need to evaluate the gradient of V with respect to θ^d and $\theta^{(g)}$. We do this as follows:

$$\begin{aligned} \nabla_{\theta^{(d)}} V(G, D) &= \nabla_{\theta^{(d)}} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{X \sim f_{model}} \left\{ \ln [1 - D(X, \theta^{(d)})] \right\} \right) \\ &= \mathbb{E}_{X \sim f_{data}} \left\{ \nabla_{\theta^{(d)}} \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{X \sim f_{model}} \left\{ \nabla_{\theta^{(d)}} \ln [1 - D(X, \theta^{(d)})] \right\} \\ &= \mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} - \mathbb{E}_{X \sim f_{model}} \left\{ \frac{1}{1 - D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} \end{aligned} \quad (15.8)$$

which, given samples $X_1, X_2, \dots, X_N \sim f_{data}$ and $X'_1, X'_2, \dots, X'_N \sim f_{model}$, we can make an unbiased estimate of as:

$$\nabla_{\theta^{(d)}} V(G, D) \approx \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X_i} - \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{1 - D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X'_i}$$

since, denoting $\mathbf{X} = \{X_1, X_2, \dots, X_N\}$ and $\mathbf{X}' = \{X'_1, X'_2, \dots, X'_N\}$

$$\begin{aligned} &\mathbb{E}_{\mathbf{X}, \mathbf{X}'} \left(\frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X_i} - \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{1 - D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X'_i} \right) \\ &= \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{\mathbf{X}, \mathbf{X}'} \left(\left\{ \frac{1}{D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X_i} \right) - \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{\mathbf{X}, \mathbf{X}'} \left(\left\{ \frac{1}{1 - D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X'_i} \right) \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} - \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{X \sim f_{model}} \left\{ \frac{1}{1 - D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} \\
&= \frac{1}{N} N \mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} - \frac{1}{N} N \mathbb{E}_{X \sim f_{model}} \left\{ \frac{1}{1 - D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} \\
&= \mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} - \mathbb{E}_{X \sim f_{model}} \left\{ \frac{1}{1 - D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} \\
&= \nabla_{\theta^{(d)}} V(G, D)
\end{aligned}$$

To evaluate the gradient with respect to $\theta^{(g)}$, we proceed as follows, using the chain rule:

$$\begin{aligned}
\nabla_{\theta^{(g)}} V(G, D) &= \nabla_{\theta^{(g)}} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{X \sim f_{model}} \left\{ \ln [1 - D(X, \theta^{(d)})] \right\} \right) \\
&= \nabla_{\theta^{(g)}} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{Z \sim f_{code}} \left\{ \ln [1 - D(G(Z, \theta^{(g)}), \theta^{(d)})] \right\} \right) \\
&= \mathbb{E}_{X \sim f_{data}} \left\{ \nabla_{\theta^{(g)}} \ln [D(X, \theta^{(d)})] \right\} + \mathbb{E}_{Z \sim f_{code}} \left\{ \nabla_{\theta^{(g)}} \ln [1 - D(G(Z, \theta^{(g)}), \theta^{(d)})] \right\} \\
&= \mathbb{E}_{X \sim f_{data}} \{0\} + \mathbb{E}_{Z \sim f_{code}} \left\{ \nabla_{\theta^{(g)}} \ln [1 - D(G(Z, \theta^{(g)}), \theta^{(d)})] \right\} \\
&= \mathbb{E}_{Z \sim f_{code}} \left\{ \nabla_{\theta^{(g)}} \ln [1 - D(G(Z, \theta^{(g)}), \theta^{(d)})] \right\} \\
&= \mathbb{E}_{Z \sim f_{code}} \left[-\frac{1}{1 - D(G(Z, \theta^{(g)}), \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial x} \bigg|_{x=G(Z, \theta^{(g)})} \frac{\partial G(Z, \theta^{(g)})}{\partial \theta^{(g)}} \right]
\end{aligned}$$

which can be approximated, given (only) samples $Z'_1, Z'_2, \dots, Z'_N \sim f_{code}$ as:

$$\nabla_{\theta^{(g)}} V(G, D) \approx \frac{1}{N} \sum_{i=1}^N \left[-\frac{1}{1 - D(G(Z, \theta^{(g)}), \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial x} \bigg|_{x=G(Z, \theta^{(g)})} \frac{\partial G(Z, \theta^{(g)})}{\partial \theta^{(g)}} \right]_{Z=Z_i}$$

which, as above, is unbiased.

In practice, when training, we tend to go from a situation where the generator is useless and the discriminator is fairly good to a situation where the discriminator is useless and the generator fairly good.

15.5.4 Mode collapse

A common problem with GANs is the phenomenon of ‘mode collapse’. In this setting, the generator learns to accurately simulate only a small part of the distribution f_{data} . The canonical example is training a GAN to the MNIST dataset (which we looked at in practicals) to see if the generator can draw realistic-looking numbers, and finding that the generator learns only to draw realistic-looking 7’s. In this way, the generator has missed a ‘mode’ of the training data (in this case, all other numbers)

This can arise due to variable rates of training between the discriminator and generator. If the generator can learn quickly while the discriminator can not, then the generator may learn to accurately reproduce 7’s, while the discriminator is not yet capable of penalising this. When the generator improves, the learning rate of the discriminator has slowed, and it can no longer make large enough changes to improve.

Exercises

1. If $X = G(Z, \theta^{(g)})$, what can be said about the σ -algebras generated by X and Z ?
2. Give the unconditional PDF of W from definition 15.3.
3. Assume $G(\cdot, \theta^{(g)})$ is invertible. Express f_{model} through the functions $G(\cdot, \theta^{(g)})$ and f_{code} .
4. From the proof of theorem 34 show that the given value of $D(x)$ as a stationary point of $V(G, D)$ must be a maximum.
5. We have free choice over the parameter p in equation 15.4.
 - (a) Restate the value of $\nabla_{\theta^{(d)}} V(G, D)$ in 15.8 with p retained
 - (b) Give an unbiased estimator of your new version of $\nabla_{\theta^{(d)}} V(G, D)$ (you may assume p is known).
 - (c) Suppose that you happen to know that:

$$\text{var}_{X \sim f_{data}} \left\{ \frac{1}{D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} = V_1$$

$$\text{var}_{X \sim f_{model}} \left\{ \frac{1}{1 - D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} = V_2$$

What values might you want to choose for p ?

Solutions

1. We must have $\mathfrak{S}(X) \subseteq \mathfrak{S}(Z)$.

2. It is:

$$f_W(x) = pf_{model}(x) + (1-p)f_{data}(x) \quad (15.9)$$

3. Let $g(x) = G(x, \theta^{(g)})$, and let $g^{-1}(y)$ be its inverse (which exists by assumption). From the general formula for multivariate PDF transformation: if J_g is the Jacobian of g , then:

$$f_{model}(x) = f_{code}(g^{-1}(z))|J_{g^{-1}}| = f_{code}(g^{-1}(z))|J_g^{-1}|$$

4. We have

$$\frac{d^2 L(D)}{dD^2} = -\frac{f_{data}(x)}{D(x)^2} - \frac{f_{model}(x)}{(1-D(x))^2} < 0$$

so the critical point, if it exists, is a maximum.

5. With p included:

(a) The new expression for $\nabla_{\theta^{(d)}} V(G, D)$ is:

$$\nabla_{\theta^{(d)}} V(G, D) = p \mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\} - (1-p) \mathbb{E}_{X \sim f_{model}} \left\{ \frac{1}{1-D(X, \theta^{(d)})} \frac{\partial D(X, \theta^{(d)})}{\partial \theta^{(d)}} \right\}$$

(b) An unbiased estimator of $\nabla_{\theta^{(d)}} V(G, D)$ is derived in the same way as above and is:

$$\nabla_{\theta^{(d)}} V(G, D) \approx \frac{p}{N} \sum_{i=1}^N \left\{ \frac{1}{D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X_i} - \frac{1-p}{N} \sum_{i=1}^N \left\{ \frac{1}{1-D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X'_i}$$

(c) Let us call the above estimator $DV(\mathbf{X}, \mathbf{X}')$. The variance of the estimator is:

$$\begin{aligned} \text{var}_{\mathbf{X}, \mathbf{X}'}(DV(\mathbf{X}, \mathbf{X}')) &= \text{var} \left(\frac{p}{N} \sum_{i=1}^N \left\{ \frac{1}{D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X_i} \right. \\ &\quad \left. - \frac{1-p}{N} \sum_{i=1}^N \left\{ \frac{1}{1-D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X'_i} \right) \\ &= \left(\frac{p}{N} \right)^2 \sum_{i=1}^N \text{var} \left(\left\{ \frac{1}{D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X_i} \right) \\ &\quad + \left(\frac{1-p}{N} \right)^2 \sum_{i=1}^N \text{var} \left(\left\{ \frac{1}{1-D(x, \theta^{(d)})} \frac{\partial D(x, \theta^{(d)})}{\partial \theta^{(d)}} \right\}_{x=X'_i} \right) \\ &= \left(\frac{p}{N} \right)^2 NV_1 + \left(\frac{1-p}{N} \right)^2 (N)V_2 \\ &= \frac{p^2}{N} V_1 + \frac{(1-p)^2}{N} V_2 \end{aligned}$$

which for fixed N is minimal when $p = \frac{V_1}{V_1+V_2}$

Boltzmann machines

Reading list and references

[Calin, 2020, Chapter 20]

16.1 Introduction

In the next two lectures we will look at quite a new notion of neural networks. Up til now we have considered only feed-forward networks, with ordered neurons, and have considered neurons that behave deterministically: that is, do not introduce more noise.

We will look at neurons with *random* outputs, with distribution determined by weights and inpput. We will look at what happens if we connect these neurons together. This will turn out to have an analogy to the Ising model, which some of you may have come across in physics.

16.2 Stochastic neurons and perceptrons

So far we have looked at neurons which implement functions of the form:

$$Y = \phi \left(\sum_i w_i x_i + b \right)$$

We discussed the idea that if $\phi(x) = (1 + \exp(-x))^{-1}$, and we observe only values of X and values $Y' \in \{0, 1\}$ where $P(Y' = 1) = Y$, then we are working with a logistic regression model.

A stochastic neuron does a bit more of the work. In a general sense, Y becomes a random variable, with distribution determined by the value $(\sum_i w_i x_i + b)$. However, we will mostly be interested in binary Y , where we have:

$$(Y = 1|X) \sim \text{Bernoulli} \left(\phi \left(\sum_i w_i X_i + b \right) \right)$$

so

$$P(Y = 1|X) = \phi \left(\sum_i w_i X_i + b \right)$$
$$P(Y = 0|X) = 1 - \phi \left(\sum_i w_i X_i + b \right)$$

that is, the neuron randomly picks the value of Y with probability given by $\phi(\sum_i w_i X_i + b)$. We include the conditional on X because the input will usually also be a random variable.

16.3 Boltzmann machines

Let us call the output of a neuron its ‘state’. We will join together a set of neurons, and call and the set of outputs of all neurons in a network the state of the network.

Definition 25. A Boltzmann machine (illustrated in figure 16.1) is a neural network of n neurons in which

1. Each neuron is a stochastic neuron with binary output, with activation function given below
2. The output of each neuron is one of the inputs of every other neuron (that is, the network is fully connected)
3. Weights are *symmetric*: so if the input of neuron i which comes from neuron j has weight w_{ij} , then the input of neuron j that comes from neuron i also has weight w_{ij}
4. The weight from a neuron to itself, w_{ii} , is 0.

The state x_i of neuron i , given weights w_{ij} and bias b_i , is determined by the equation:

$$x_i | (x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \sim \text{Bernoulli} \left[\phi_{\frac{1}{T}} \left(\sum_j w_{ij} x_j + b_i \right) \right]$$

where

$$\phi_{\frac{1}{T}}(x) = (1 + e^{-\frac{x}{T}})^{-1}$$

The ‘state’ of the network is given by the set of values $(x_1, x_2, \dots, x_n)$.

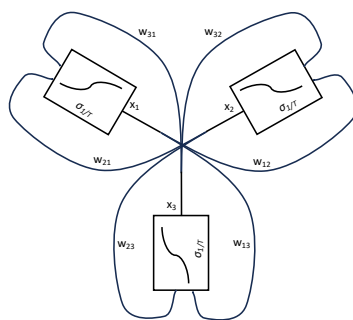


Figure 16.1: Illustration of a Boltzmann machine with $n = 3$

It is generally easiest to think of Boltzmann machines as evolving over discrete time; that is, they have a state $x^t = (x_1^t, x_2^t, \dots, x_n^t)$ at time t ; then the state $x^{t+1} = (x_1^{t+1}, x_2^{t+1}, \dots, x_n^{t+1})$ is determined by the weights and the state x^t .

Importantly, **the state updates by changing only one neuron at a time**. That is, an updating step involves:

1. Choose a neuron at random (say i)
2. Update the state of only that neuron, so

$$x_i^{t+1} = \text{Bernoulli} \left[\phi_{\frac{1}{T}} \left(\sum_j w_{ij} x_j^t + b_i \right) \right]$$

$$x_j^{t+1} = x_j^t \text{ for } j \neq i$$

We can imagine that for some states, the network will be very ‘stable’ and won’t change much, but for other states, the neuron will very quickly change into a different state. As an extreme example, suppose that in the figure 16.1 above, we have $T = 1$, all weights are equal to 1, and all biases are -10, and we happen to pick neuron 1 to update. Then we have:

$$P(x_1^{t+1} = 1 | x_2^t, x_3^t) = \phi_1 \left(\sum_j w_{1j} x_j^t + b_1 \right)$$

$$\begin{aligned}
&= \phi_1 \left(\sum_{j \neq 1} x_j - 10 \right) \\
&\leq \phi_1(-8) \\
&< \frac{1}{1000}
\end{aligned}$$

If we are in the state $x^t = (x_1^t, x_2^t, x_3^t) = (1, 1, 1)$, then with very high probability, we will immediately change to one of the states $x^t = (0, 1, 1)$, $x^t = (1, 0, 1)$, or $x^t = (1, 1, 0)$. If we are in the state $(0, 0, 0)$, by contrast, we are much stabler, and will be quite likely to stay there¹.

This resembles a physical system. If we put some sodium in water, it will react wildly, until it reaches a more stable state. If we put a ball someone on a skatepark, sometimes it will immediately roll, sometimes it will stay where it is. Roughly, the ‘stability’ of a state corresponds to its potential energy: if it has high potential energy (ball on a high point of a skatepark; lump of metallic sodium in water) it will rapidly change its state; if it has low potential energy (ball at the bottom of a bowl, sodium oxide in water) then it will not.

16.3.1 Energy of a network

Indeed, we can assign an ‘energy’ to a state, which is physically something like its potential energy. There are a few ways to do this, but one stands out as most sensible.

What should the energy of a neuron be? Basically, if its state strongly disagrees with its activation - so that next time it’s very likely to change - then it should be high energy. Roughly, we will multiply its state by something resembling the *negative* of its activation, to get the energy of a neuron x_i :

$$E(x_i) = x_i \times \left(-\frac{1}{2} \sum_j w_{ij} x_j - b_i \right)$$

The reason for the factor $-\frac{1}{2}$ will soon become clear. We define the energy of a network as the sum of energies of its neurons:

$$E(x) = \sum_i E(x_i) = \sum_i \left(x_i \times \left(-\frac{1}{2} \sum_j w_{ij} x_j - b_i \right) \right) = -\frac{1}{2} \sum_{i,j} w_{ij} x_i x_j - \sum_i b_i x_i = -\frac{1}{2} x^T W x - b^T x$$

taking the matrix W with i, j th entry w_{ij} , and the vector b with i th entry b_i .

The reason we define energy this way is that it means we can quickly describe the long-run probability that the network is in any given state. To show this, we note that (as we have defined it) the set of values x^t , $t \in \{1, 2, \dots\}$ form a Markov chain.

Theorem 35. For a Boltzmann machine with weight matrix W and bias vector b , the equilibrium distribution Z of the machine; that is, the values $p_Z(r)$ such that for any state r :

$$\lim_{t \rightarrow \infty} P(x = r) = p_Z(r) \text{ (a.s.)} \quad (16.1)$$

satisfy:

$$p_Z(r) = \frac{e^{-\frac{E(r)}{T}}}{K} \quad (16.2)$$

for a constant K

Proof. Suppose the Boltzmann machine has n neurons. Since the sequence of states x^t , $t = 1, 2, \dots$ is a Markov chain, we will show that $p(r)$ is an equilibrium distribution, so it must be unique. We will do this by showing that if $x^t \sim Z$, then $x^{t+1} \sim Z$. For a given state x , let y^k be the state that differs from x only at neuron k . Then:

$$P_{x^t \sim Z}(x^{t+1} = x) = \sum_{\text{states } y} P(x^{t+1} = x | x^t = y) P(x^t = y)$$

¹It is worth noting that stability does not necessarily mean staying in the same state; it could mean moving between some number of states reasonably freely. Roughly stability refers to whether we’re likely to move *back* to the state we came from.

$$\begin{aligned}
&= \sum_{\substack{\text{states } y \text{ one} \\ \text{different from } x}} P(x^{t+1} = x | x^t = y) P(x^t = y) \\
&= P(x^{t+1} = x | x^t = x) P(x^t = x) + \sum_{k=1}^n P(x^{t+1} = x | x^t = y^k) P(x^t = y^k) \\
&= P(x^{t+1} = x | x^t = x) p_Z(x) + \sum_{k=1}^n P(x^{t+1} = x | x^t = y^k) p_Z(y^k) \tag{16.3}
\end{aligned}$$

Let us denote, for any k :

$$u_k = \sum_i w_{ki} y_i^k + b_i = \sum_i w_{ki} x_i + b_i$$

What is $p_Z(y^k)$? Since y^k differs from x only at the value y_k^k , we have:

$$\begin{aligned}
E(y^k) &= - \sum_{i < j} w_{ij} y_i^k y_j^k - \sum_i b_i y_i^k \\
&= -y_k^k \sum_j w_{kj} y_j^k - \sum_{\substack{i < j \\ i, j \neq k}} w_{ij} y_i^k y_j^k - b_k y_k^k - \sum_{i \neq k} b_i y_i^k \\
E(x) &= -x_k \sum_j w_{kj} x_j - \sum_{\substack{i < j \\ i, j \neq k}} w_{ij} x_i x_j - b_k x_k - \sum_{i \neq k} b_i x_i
\end{aligned}$$

and noting that $x_k \neq y_k^k$, while $x_i = y_i^k$ for $k \neq i$, we have

$$E(y) = E(x) - (y_k^k - x_k) \left(\sum_j w_{kj} y_j^k + b_k \right) = E(x) - (y_k^k - x_k) u_k$$

so

$$p_Z(y^k) = \frac{e^{-\frac{E(y^k)}{T}}}{K} = \frac{e^{-\frac{E(x) - (y_k^k - x_k) u_k}{T}}}{K} = \frac{e^{-\frac{E(x)}{T}}}{K} e^{\frac{(y_k^k - x_k) u_k}{T}} = p_Z(x) \left(e^{\frac{u_k}{T}} \right)^{y_k^k - x_k}$$

And what is $P(x^{t+1} = x | x^t = y^k)$? To change from state y^k to state x we must first pick the k th neuron, and then change its state. So:

$$P(x^{t+1} = x | x^t = y^k) = \frac{1}{n} \sigma_{\frac{1}{T}}(u_k)^{x_k} \left(1 - \sigma_{\frac{1}{T}}(u_k) \right)^{1-x_k}$$

If $x_k = 1$, then $y_k^k = 0$ and we have

$$\begin{aligned}
P(x^{t+1} = x | x^t = y^k) p_Z(y^k) &= \frac{1}{n} \sigma_{\frac{1}{T}}(u_k)^{x_k} \left(1 - \sigma_{\frac{1}{T}}(u_k) \right)^{1-x_k} p_Z(x) \left(e^{\frac{u_k}{T}} \right)^{y_k^k - x_k} \\
&= \frac{1}{n} p_Z(x) \sigma_{\frac{1}{T}}(u_k) e^{-\frac{u_k}{T}} \\
&= \frac{1}{n} p_Z(x) \frac{e^{-\frac{u_k}{T}}}{1 + e^{-\frac{u_k}{T}}} \\
&= \frac{1}{n} p_Z(x) \left(1 - \frac{1}{1 + e^{-\frac{u_k}{T}}} \right) \\
&= \frac{1}{n} p_Z(x) (1 - \sigma_{\frac{1}{T}}(u_k))
\end{aligned}$$

and if $x_k = 0$, then $y_k^k = 1$ and we have

$$\begin{aligned}
P(x^{t+1} = x | x^t = y^k) p_Z(y^k) &= \frac{1}{n} \sigma_{\frac{1}{T}}(u_k)^{x_k} \left(1 - \sigma_{\frac{1}{T}}(u_k) \right)^{1-x_k} p_Z(x) \left(e^{\frac{u_k}{T}} \right)^{y_k^k - x_k} \\
&= \frac{1}{n} p_Z(x) \left(1 - \sigma_{\frac{1}{T}}(u_k) \right) e^{\frac{u_k}{T}}
\end{aligned}$$

$$\begin{aligned}
&= \frac{1}{n} p_Z(x) \left(1 - \frac{1}{1 + e^{-\frac{u_k}{T}}} \right) e^{\frac{u_k}{T}} \\
&= \frac{1}{n} p_Z(x) \frac{e^{-\frac{u_k}{T}}}{1 + e^{-\frac{u_k}{T}}} e^{\frac{u_k}{T}} \\
&= \frac{1}{n} p_Z(x) \frac{1}{1 + e^{-\frac{u_k}{T}}} \\
&= \frac{1}{n} p_Z(x) \sigma_{\frac{1}{T}}(u_k)
\end{aligned}$$

Putting this together we have:

$$\begin{aligned}
\sum_{k=1}^n P(x^{t+1} = x | x^t = y^k) p_Z(y^k) &= \sum_{k: x_k=1} \frac{1}{n} p_Z(x) (1 - \sigma_{\frac{1}{T}}(u_k)) + \sum_{k: x_k=0} \frac{1}{n} p_Z(x) \sigma_{\frac{1}{T}}(u_k) \\
&= \frac{1}{n} p_Z(x) \left(\sum_{k: x_k=1} (1 - \sigma_{\frac{1}{T}}(u_k)) + \sum_{k: x_k=0} \sigma_{\frac{1}{T}}(u_k) \right)
\end{aligned}$$

Finally, we have, using similar arguments:

$$\begin{aligned}
P(x^{t+1} = x | x^t = x) p_Z(x) &= p_Z(x) \sum_{k=1}^n P(x^{t+1} = x | x^t = x, \text{update neuron } k) P(\text{update neuron } k) \\
&= p_Z(x) \sum_{k=1}^n \sigma_{\frac{1}{T}}(u_k)^{x_k} (1 - \sigma_{\frac{1}{T}}(u_k))^{1-x_k} \frac{1}{n} \\
&= \frac{1}{n} p_Z(x) \left(\sum_{k: x_k=1} \sigma_{\frac{1}{T}}(u_k) + \sum_{k: x_k=0} (1 - \sigma_{\frac{1}{T}}(u_k)) \right)
\end{aligned} \tag{16.4}$$

and putting everything together into equation 16.3 gives

$$\begin{aligned}
P_{x^t \sim Z}(x^{t+1} = x) &= P(x^{t+1} = x | x^t = x) p_Z(x) + \sum_{k=1}^n P(x^{t+1} = x | x^t = y^k) p_Z(y^k) \\
&= \frac{1}{n} p_Z(x) \left(\sum_{k: x_k=1} \sigma_{\frac{1}{T}}(u_k) + \sum_{k: x_k=0} (1 - \sigma_{\frac{1}{T}}(u_k)) \right) \\
&\quad + \frac{1}{n} p_Z(x) \left(\sum_{k: x_k=1} (1 - \sigma_{\frac{1}{T}}(u_k)) + \sum_{k: x_k=0} \sigma_{\frac{1}{T}}(u_k) \right) \\
&= \frac{1}{n} p_Z(x) \left(\sum_{k: x_k=1} (\sigma_{\frac{1}{T}}(u_k) + (1 - \sigma_{\frac{1}{T}}(u_k))) \right. \\
&\quad \left. + \sum_{k: x_k=0} ((1 - \sigma_{\frac{1}{T}}(u_k)) + \sigma_{\frac{1}{T}}(u_k)) \right) \\
&= \frac{1}{n} p_Z(x) \left(\sum_{k: x_k=1} 1 + \sum_{k: x_k=0} 1 \right) \\
&= \frac{1}{n} p_Z(x) n \\
&= p_Z(x)
\end{aligned}$$

as required

□

This stationary distribution given by:

$$p(x) = \frac{e^{-\frac{E(x)}{T}}}{K} = \frac{e^{-\frac{1}{2} x^T W x - x^T b}}{K}$$

is called the *Boltzmann distribution*.

16.3.2 Exact learning

We first ask the question: can *any* distribution over states x be a Boltzmann distribution for some weights w ?

We can quickly answer ‘no’ in general: for an n -state Boltzmann machine, there are 2^n possible states, so a distribution is determined by $2^n - 1$ probabilities (the final probability is determined by the first $2^n - 1$). But there are only $\frac{n(n-1)}{2}$ weights, and n biases, so we only have $n + \frac{n(n-1)}{2}$ parameters, so we certainly can’t model every possible distribution.

However, if $n = 2$, it turns out we can, since we have three parameters (w_{12}, b_1, b_2) and three free probabilities. Suppose we want the equilibrium distribution/Boltzmann distribution to be (q_1, q_2, q_3, q_4) where we denote:

$$\begin{aligned} q_1 &= P(x = (0, 0)) & q_2 &= P(x = (0, 1)) \\ q_3 &= P(x = (1, 0)) & q_4 &= P(x = (1, 1)) \end{aligned}$$

Let’s write down what these are, in terms of the energy at $T = 1$. The equations determining x_1 and x_2 are:

$$\begin{aligned} P(x_1^{t+1} = 1 | x_2^t) &= \sigma_{\frac{1}{T}}(w_{12}x_2 + b_1) \\ P(x_2^{t+1} = 1 | x_1^t) &= \sigma_{\frac{1}{T}}(w_{12}x_1 + b_2) \end{aligned}$$

The energy for a configuration (x_1, x_2) is given by:

$$E(x_1, x_2) = -(w_{12}x_1x_2 + b_1x_1 + b_2x_2)$$

The Boltzmann distribution is given by $p(x) = \frac{e^{-E(x)}}{Z}$, for normalising factor Z . This is:

$$\begin{aligned} q_1 &= \frac{1}{Z} e^{\frac{-E(0,0)}{T}} = \frac{1}{Z} e^{\frac{w_{12} \cdot 0 + b_1 \cdot 0 + b_2 \cdot 0}{T}} = \frac{1}{Z} \\ q_2 &= \frac{1}{Z} e^{\frac{-E(0,1)}{T}} = \frac{1}{Z} e^{\frac{w_{12} \cdot 0 + b_1 \cdot 0 + b_2 \cdot 1}{T}} = \frac{1}{Z} e^{\frac{b_2}{T}} \\ q_3 &= \frac{1}{Z} e^{\frac{-E(1,0)}{T}} = \frac{1}{Z} e^{\frac{w_{12} \cdot 0 + b_1 \cdot 1 + b_2 \cdot 0}{T}} = \frac{1}{Z} e^{\frac{b_1}{T}} \\ q_4 &= \frac{1}{Z} e^{\frac{-E(1,1)}{T}} = \frac{1}{Z} e^{\frac{w_{12} \cdot 1 + b_1 \cdot 1 + b_2 \cdot 1}{T}} = \frac{1}{Z} e^{\frac{w_{12} + b_1 + b_2}{T}} \end{aligned}$$

From the first equation we have

$$Z = \frac{1}{q_1}$$

From the second, we thus have

$$b_2 = T \ln(Zq_2) = T \ln\left(\frac{q_2}{q_1}\right)$$

and from the third

$$b_1 = T \ln(Zq_3) = T \ln\left(\frac{q_3}{q_1}\right)$$

Thus, from the fourth, we have:

$$\begin{aligned} w_{12} &= -b_1 - b_2 + T \ln(Zq_4) \\ &= -b_1 - b_2 + T \ln\left(\frac{q_4}{q_1}\right) \\ &= -T \ln\left(\frac{q_3}{q_1}\right) - T \ln\left(\frac{q_2}{q_1}\right) + T \ln\left(\frac{q_4}{q_1}\right) \\ &= T \left(\ln\left(\frac{q_1}{q_3}\right) + \ln\left(\frac{q_1}{q_2}\right) + \ln\left(\frac{q_4}{q_1}\right) \right) \\ &= T \ln\left(\frac{q_1 q_4}{q_2 q_3}\right) \end{aligned}$$

In general, if we want a Boltzmann machine to have an equilibrium distribution given by an arbitrary distribution on a vector of length n , we can just use a Boltzmann machine with $N > n$ neurons and only observe n of them (the others are ‘hidden’).

Exercises

1. Suppose we have a Boltzmann machine and move from a state x to a state x' , where only the k th neuron changes state, from 0 to 1. Find an expression for the change in energy.
2. For the probability distribution $(q_1, q_2, q_3, q_4) = (0.3, 0.25, 0.1, 0.35)$, find weight w_{12} and biases b_1, b_2 for a two-neuron Boltzmann distribution which has (q_1, q_2, q_3, q_4) as its equilibrium distribution at $T = 2$.
3. Suppose we want to approximate an arbitrary probability distribution over the states of a binary vector of length n . We will use a Boltzmann machine with $N \gg n$ neurons, only n of which are visible. Approximately how many neurons do we need to approximate any distribution over the states of our binary vector?
4. Run a simulation on a pseudo-Boltzmann machine where instead of updating only one neuron at a time, we update all neurons, so we evaluate all values x^{t+1} rather than just one. Show by a simple example that this does not have a stationary distribution given by the Boltzmann distribution.

Solutions

1. The energy in state x is

$$\begin{aligned}
 E(x) &= - \sum_{i,j;i < j} w_{ij} x_i x_j - \sum_i b_i x_i \\
 &= - \sum_{i,j;i < j; i,j \neq k} w_{ij} x_i x_j - x_k \sum_i w_{ik} x_i - \sum_{i \neq k} b_i x_i - b_k x_k \\
 &= - \sum_{i,j;i < j; i,j \neq k} w_{ij} x_i x_j - 0 \sum_i w_{ik} x_i - \sum_{i \neq k} b_i x_i - 0 x_k \\
 &= - \sum_{i,j;i < j; i,j \neq k} w_{ij} x_i x_j - \sum_{i \neq k} b_i x_i
 \end{aligned}$$

and in state x' it is (recalling $w_{ii} = 0$):

$$\begin{aligned}
 E(x') &= - \sum_{i,j;i < j} w_{ij} x'_i x'_j - \sum_i b_i x'_i \\
 &= - \sum_{i,j;i < j; i,j \neq k} w_{ij} x_i x_j - x'_k \sum_i w_{ik} x_i - \sum_{i \neq k} b_i x_i - b_k x'_k \\
 &= - \sum_{i,j;i < j; i,j \neq k} w_{ij} x_i x_j - \sum_i w_{ik} x_i - \sum_{i \neq k} b_i x_i - b_k
 \end{aligned}$$

so the difference is:

$$E(x') - E(x) = - \sum_i w_{ik} x_i - b_k \quad (16.5)$$

2. Using the formulae from the lecture we have:

$$\begin{aligned}
 Z &= \frac{1}{q_1} = \frac{1}{0.3} \\
 b_1 &= T \ln \left(\frac{q_3}{q_1} \right) = 2 \ln \left(\frac{0.1}{0.3} \right) \\
 b_2 &= T \ln \left(\frac{q_2}{q_1} \right) = 2 \ln \left(\frac{0.25}{0.3} \right) \\
 w_{12} &= T \ln \left(\frac{q_1 q_4}{q_2 q_3} \right) = 2 \ln \left(\frac{0.3 \times 0.35}{0.25 \times 0.1} \right)
 \end{aligned}$$

3. A binary vector of length n has 2^n states, so the space of distributions over states is a simplex of dimension $2^n - 1$ (that is, the set of all real vectors of positive values of length 2^n which sum to 1). We thus need at least $2^n - 1$ parameters. A Boltzmann machine on N nodes has $\frac{N(N-1)}{2}$ weights and N biases, so we need:

$$\frac{N(N-1)}{2} + N \geq 2^n - 1 \Leftrightarrow N^2 + N - (2^{n+1} - 2) \geq 0 \Leftrightarrow N \geq \frac{1}{2}(-1 + \sqrt{2^{n+3} - 7})$$

4. Consider a two-neuron pseudo-Boltzmann machine with single weight $w_{12} = w = 1$ and biases $b_1 = b_2 = 0$, and temperature $T = 1$. Then, enumerating the states as 1 : (0,0); 2 : (0,1); 3 : (1,0); 4 : (1,1) we have the transition matrix:

$$\begin{pmatrix}
 \frac{\frac{1}{4}}{\frac{2(1+e^1)}{1}} & \frac{\frac{1}{4}}{\frac{2(1+e^1)}{1}} & \frac{\frac{1}{4}}{\frac{2(1+e^{-1})}{1}} & \frac{\frac{1}{4}}{\frac{2(1+e^{-1})}{1}} \\
 \frac{\frac{2(1+e^1)}{1}}{(1+e^1)^2} & \frac{\frac{2(1+e^{-1})}{1}}{(1+e^{-1})^2} & \frac{\frac{2(1+e^1)}{1}}{(1+e^1)(1+e^{-1})} & \frac{\frac{2(1+e^{-1})}{1}}{(1+e^{-1})(1+e^1)}
 \end{pmatrix} \quad (16.6)$$

for which the only positive left eigenvector is:

$$\begin{pmatrix}
 \frac{4}{(1+e^1)^2} \\
 \frac{2}{1+e^1} \\
 \frac{2}{1+e^1} \\
 1
 \end{pmatrix} \quad (16.7)$$

which is not expressible as a multiple of the energies of the states.

Restricted Boltzmann machines

Reading list and references

[Calin, 2020, Chapter 20]
[Hinton, 2012]

17.1 Introduction

In this section, we will look at *training* Boltzmann machines. We will then look at a ‘restricted Boltzmann machine’, which is a slightly simpler model using similar principles.

17.2 Training Boltzmann machines

We saw in the previous lecture that a Boltzmann machine on n neurons can have a range of equilibrium distributions.

Suppose that we have a multinomial distribution over the set of 0-1 strings of length n (equivalently, a discrete random variable taking values between 0 and 2^n) and we want to find weights and biases of a Boltzmann machine which has approximately this equilibrium distribution.

Let’s our distribution is called q , so $q(s)$ is the probability of sequence s . Our Boltzmann machine has an equilibrium distribution which we’ll call p (so $p(s)$ is the long-run probability that the Boltzmann machine is in state s). How do we measure the difference between p and q ?

We go back a few lectures and remember the K-L divergence:

$$D_{KL}(q||p) = \sum_s q(s) \ln \left(\frac{q(s)}{p(s)} \right)$$

This is 0 if $p = q$, and positive otherwise, so it makes sense that we should try and minimise it. We can write

$$\sum_s q(s) \ln \left(\frac{q(s)}{p(s)} \right) = \sum_s q(s) \ln (q(s)) - \sum_s q(s) \ln (p(s)) = -H(q) - \sum_s q(s) \ln (p(s))$$

and $H(q)$ stays constant, so minimising $D_{KL}(q||p)$ is equivalent to maximising $\sum_s q(s) \ln (p(s))$.

The Boltzmann distribution for a network taking the state $x = \{x_1, x_2, \dots, x_n\}$ is given by:

$$p(x) = P(X = x) = \frac{e^{-E(x)}}{K} = \frac{e^{\frac{1}{2} \sum_{i,j} w_{ij} x_i x_j + \sum_i b_i x_i}}{K} = \frac{1}{K} \exp \left(\frac{1}{2} x^t W x + x^t b \right)$$

where we have designated W as a matrix of weights. Now we have

$$\begin{aligned} \frac{\partial}{\partial w_{ij}} \ln (p(x)) &= -\frac{\partial}{\partial w_{ij}} E(x) - \frac{\partial}{\partial w_{ij}} \ln (K) \\ &= \frac{\partial}{\partial w_{ij}} \left(\frac{1}{2} x^t W x + x^t b \right) - \frac{1}{K} \frac{\partial}{\partial w_{ij}} K \\ &= x_i x_j - \frac{1}{K} \sum_y \frac{\partial}{\partial w_{ij}} e^{-E(y)} \end{aligned}$$

$$\begin{aligned}
&= x_i x_j - \sum_y \frac{e^{-E(y)}}{K} \frac{\partial}{\partial w_{ij}} (-E(y)) \\
&= x_i x_j - \left(\sum_y p(y) y_i y_j \right) \\
&= x_i x_j - \mathbb{E}\{x_i x_j\}
\end{aligned}$$

where $\mathbb{E}\{x_i x_j\}$ is taken with respect to the Boltzmann distribution of states. Similarly, we have:

$$\frac{\partial}{\partial b_j} \ln(p(x)) = x_j - \mathbb{E}\{x_j\} \quad (17.1)$$

and so

$$\begin{aligned}
-\frac{\partial}{\partial w_{ij}} D_{KL}(q||p) &= \frac{\partial}{\partial w_{ij}} \sum_x q(x) \ln(p(x)) \\
&= \sum_x q(x) \frac{\partial}{\partial w_{ij}} \ln(p(x)) \\
&= \sum_x q(x) x_i x_j - \sum_x q(x) \mathbb{E}\{x_i x_j\} \\
&= \mathbb{E}^q\{x_i x_j\} - \left(\sum_x q(x) \right) \mathbb{E}\{x_i x_j\} \\
&= \mathbb{E}^q\{x_i x_j\} - \mathbb{E}^p\{x_i x_j\}
\end{aligned}$$

where $\mathbb{E}^q$ and $\mathbb{E}^p$ refer to expectations with respect to the distributions q and p respectively. Similarly,

$$-\frac{\partial}{\partial b_j} D_{KL}(q||p) = \mathbb{E}^q\{x_j\} - \mathbb{E}^p\{x_j\} \quad (17.2)$$

The quantity $\mathbb{E}^q\{x_i x_j\}$ is known, since it is with respect to the distribution we are trying to approximate. To estimate $\mathbb{E}^p\{x_i x_j\}$, we need to run the network for a while, and count how frequently $x_i = x_j = 1$.

17.3 Restricted Boltzmann Machines (RBMs)

A Restricted Boltzmann Machine is a version of a Boltzmann machine with some weights set to 0. In particular, it is a Boltzmann machine with nodes partitioned into ‘visible’ and ‘hidden’ nodes, for which only connections between visible and hidden nodes are nonzero.

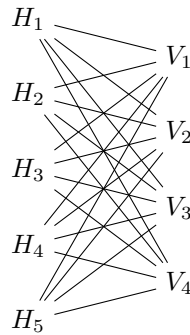


Figure 17.1: A restricted Boltzmann machine with hidden nodes $H_1 - H_5$ and visible nodes $V_1 - V_4$.

A RBM can thus be represented by a *complete bipartite graph*.

Let us start by considering a normal Boltzmann machine with n nodes, where node states are random variables $X_1, X_2, \dots, X_n$.

We now partition the nodes into m ‘visible’ nodes $V = (V_1, V_2, \dots, V_m) = (X_1, X_2, \dots, X_m)$ and $n - m$ ‘hidden’ nodes $H = (H_1, H_2, \dots, H_{n-m}) = (X_{m+1}, X_{m+2}, \dots, X_n)$. Since there are no connections between hidden nodes or between visible nodes, we will have $w_{ij} = 0$ if both i and j hidden or both i and j are visible. We denote by $b_1, b_2, \dots, b_n$ the biases of each neuron.

We can now describe a state x as a visible and hidden state: $x = (v, h) = (x_1, x_2, \dots, x_n) = (v_1, v_2, \dots, v_m, h_1, h_2, \dots, h_{n-m})$. We can make things a bit easier for ourselves by defining a few vectors and matrices:

$$\begin{aligned} v &= (v_1 \ v_2 \ \dots \ v_m)^T & h &= (h_1 \ h_2 \ \dots \ h_{n-m})^T \\ b^v &= (b_1 \ b_2 \ \dots \ b_m) & b^h &= (b_{m+1} \ b_{m+2} \ \dots \ b_n) \end{aligned}$$

and W is a matrix with i, j th entry w_{ij} . Let us also define the matrix W' such that

$$W'_{ij} = W_{i(j+m)} \quad (17.3)$$

Now:

$$W = \begin{pmatrix} \mathbf{0} & W' \\ (W')^T & \mathbf{0} \end{pmatrix}$$

The energy of a state $x = (v, h)$ is

$$\begin{aligned} E(v, h) &= E(x) \\ &= -\frac{1}{2} x^T W x - b^T x \\ &= -\frac{1}{2} (v^T \ h^T) \begin{pmatrix} \mathbf{0} & W' \\ (W')^T & \mathbf{0} \end{pmatrix} \begin{pmatrix} v \\ h \end{pmatrix} - (b^v \ b^h) \begin{pmatrix} v \\ h \end{pmatrix} \\ &= -\frac{1}{2} (v^T \mathbf{0} v + v^T W' h + h^T (W')^T v + h^T \mathbf{0} h) - b^v v - b^h h \\ &= -v^T W' h - b^v v - b^h h \end{aligned}$$

Written another way:

$$E(v, h) = - \sum_{i \text{ visible}} \sum_{j \text{ hidden}} w_{ij} v_i h_j - \sum_{i \text{ visible}} b_i v_i - \sum_{j \text{ hidden}} b_j h_j$$

noting this is the same as the energy formulation of a Boltzmann machine, but with $w_{ij} = 0$ if i and j are the same type of node.

We are mostly concerned with the probability of observing a particular configuration of the *visible* nodes. At the equilibrium Boltzmann distribution, we have

$$p(v) = P(V = v) = \sum_h P(V = v, H = h) = \frac{1}{Z} \sum_h e^{-E(v, h)}.$$

17.3.1 Training RBMs

We will not go through the laborious task of computing $\frac{\partial}{\partial w_{ij}} p(v)$, but it is fairly straightforward. Suppose that under the equilibrium distribution of the network, the random variable H represents the hidden node values, and the random variable V the visible node values. We have:

$$\begin{aligned} \frac{\partial}{\partial w_{ij}} \ln(p(v)) &= \sum_{\text{hidden states } y} p(y|v) v_i y_j + \sum_{\substack{\text{visible states } x \\ \text{hidden states } y}} p(x, y) x_i y_j \\ &= \mathbb{E}_{y \sim (H|V)} (v_i y_j) + \mathbb{E}_{(x, y) \sim (V, H)} (x_i y_j) \end{aligned} \quad (17.4)$$

In the first of these expectations, we fix the value of the visible nodes to those of our sample v , so this can be viewed as an expectation over the *data*. The second expectation is over both v and h , so can be viewed as an expectation over the *model* (see [Hinton \[2012\]](#)).

It is much easier to estimate the first term than the second. To estimate the first term, we can simply force all values of the visible nodes to v , and compute the probability that each hidden node takes the value 1 as:

$$P(H_i = 1|V = v) = \phi \left(\sum_j w_{ij} v_j + b_i \right) \quad (17.5)$$

We can thus take an unbiased sample of $H|V$ very easily.

RBMs typically have lots of nodes, so the second term is more difficult: we need to let the network run for a long time to sample $V|H$. A faster alternative, proposed by Geoffrey Hinton (of Facebook fame) is the following algorithm:

1. Set the states V of the visible units to a training vector v
2. Sample the states of the hidden units with probabilities $P(H_i = h_i|V = v)$, determined by equation 17.5. Call this sample h_v .
3. Sample the states of the visible units v_i with probabilities $P(V_i = v_i|H = h_v)$ given by

$$P(V_i = 1|H = h_v) = \phi \left(\sum_j w_{ij} (h_v)_j + b_i \right)$$

4. Treat this sample as though it were from (V, H) , and use it to approximate the gradients $\frac{\partial}{\partial w_{ij}} p(v)$.

Although appealing, and generally working in practice, this procedure does not lead to direct maximisation of $\ln(p(v))$.

Exercises

1. Show equation [17.1](#) and [17.2](#)
2. Suppose that the mean energy of a Boltzmann machine is k ; that is, $\mathbb{E}^p\{E(x)\} = k$, where p is the long-run frequency of states. Show that the entropy of the Boltzmann distribution p is given by

$$H(p) = \frac{k}{T} + \ln(Z) \quad (17.6)$$

where Z is the normalising constant for the distribution.

Solutions

1. We have:

$$\begin{aligned}\frac{\partial}{\partial b_j} \ln(p(x)) &= -\frac{\partial}{\partial b_j} E(x) - \frac{\partial}{\partial b_j} \ln(K(x)) \\ &= x_j - \sum_y p(y) x_j \\ &= x_j - \mathbb{E}\{x_j\}\end{aligned}$$

and

$$\begin{aligned}-\frac{\partial}{\partial b_j} D_{KL}(q||p) &= \frac{\partial}{\partial b_j} \sum_x q(x) \ln(p(x)) \\ &= \sum_x q(x) \frac{\partial}{\partial b_j} \ln(p(x)) \\ &= \sum_x q(x) x_j - \sum_x q(x) \mathbb{E}\{x_j\} \\ &= \mathbb{E}^q\{x_j\} - \left(\sum_x q(x)\right) \mathbb{E}\{x_j\} \\ &= \mathbb{E}^q\{x_j\} - \mathbb{E}^p\{x_j\}\end{aligned}$$

2. We have

$$\begin{aligned}H(p) &= \sum_x p(x) \ln(p(x)) \\ &= \sum_x p(x) \ln\left(\frac{e^{-E(x)}}{Z}\right) \\ &= \sum_x p(x) (-E(x) - \ln(Z)) \\ &= \frac{1}{K} \sum_x e^{-E(X)} (-E(x) - \ln(K))\end{aligned}\tag{17.7}$$

Variational autoencoders

Reading list and references

[Chollet et al., 2018, Chapter 8]
[Rocca 2019]
[Shafkat 2018]

18.1 Introduction

In this lecture, we will look a bit further at generative networks, in which we use a neural network to go from a ‘simple’ low-dimensional space or distribution to a larger high-dimensional space or distribution, usually a space of images.

We will look at an elegant architecture called a variational autoencoder, in which we explicitly try and compress our original data into a low-dimensional representation. We then use various tricks to force the network to do this in a useful way.

Variational autoencoders don’t work particularly well compared to modern methods (e.g., diffusion models, which we will look at in a later lecture) but they are instructive to learn about.

18.2 Autoencoder architecture

Recall that our general task in machine learning, supervised or unsupervised, is to find a simple representation of our data which tells us what we want to know (e.g., if we have data X and from this we want to know Y , then we want to find a ‘simpler’ X' such that $P(Y|X) \approx P(Y|X')$). As we did earlier in the course, we can frame this more explicitly in terms of the information content of random variables representing the outputs of layers of a neural network.

We have looked at the idea of a neural network which takes an image and classifies it. Essentially, this maps a complex space (images) onto a simple space (classifications):

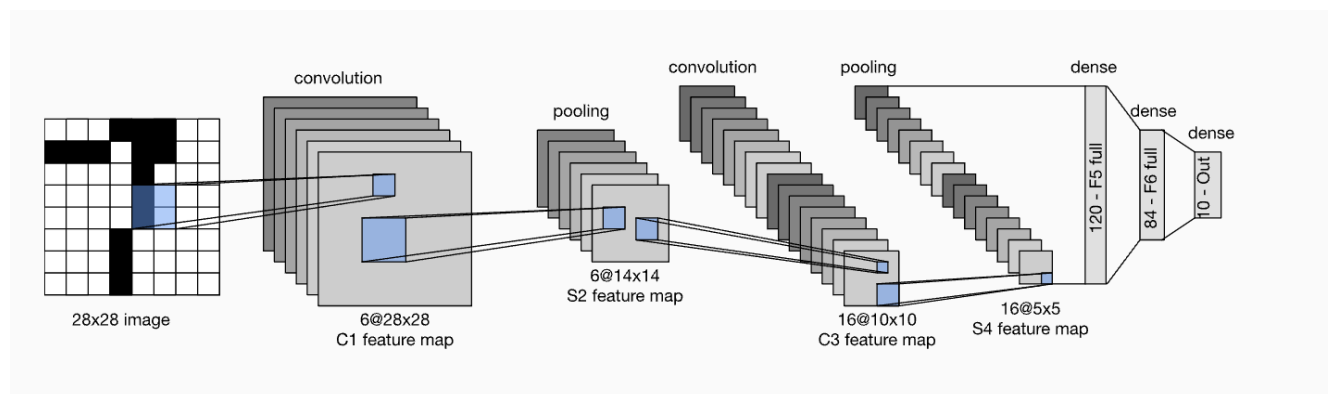


Figure 18.1: The LeNet, a typical convolutional neural network. From [Zhang et al. \[2021\]](#)

We have also looked at the idea of a generative network, which takes a sample $z \sim f_{code}$ from a simple distribution, and generates a sample $X \sim f_{model}$ from a complicated ‘target’ distribution.

An autoencoder simply chains these together: we use a ‘decoder’ network to take our original dataset and map it to a simple latent space (for instance $\mathbb{R}^2$), then use our ‘encoder’ network to map this simple space back to our original space (recall that we are calling the latent distribution f_{code} , as for GANs. The idea is to try and reconstruct the original images/samples:

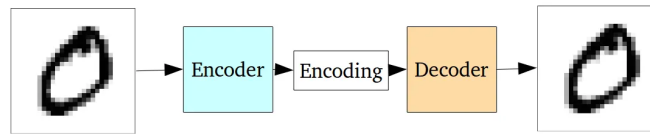


Figure 18.2: An autoencoder; from Rocca [2019]

Since this whole thing is just a big neural network, we can train the whole thing by backpropagation and gradient descent.

18.2.1 The problem with autoencoders

An immediate problem which appears with autoencoders is that we haven’t ensured that the ‘images’ of our data in the latent space preserve the topology of our data space: that is, there is no reason why images of ‘similar’ data should be close together. This is sometimes called an ‘irregular’ latent space:

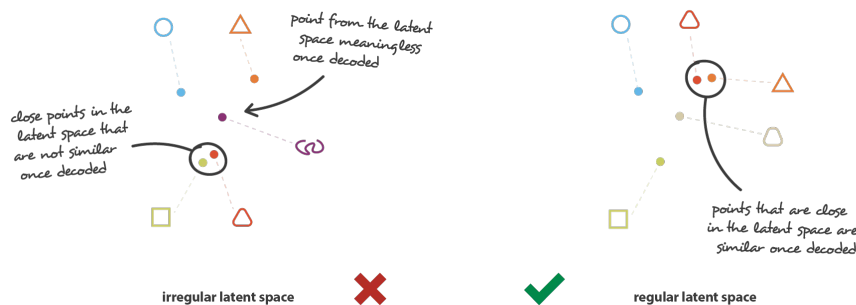


Figure 18.3: Regular latent spaces (where the topology somewhat matches our ideas of ‘similarity’ of the original images) and irregular latent spaces (where similar images are not necessarily close together). From Rocca [2019].

The latent space may be ‘smaller’ (lower-dimensional) but recall from our lectures on information theory that, if we abandon topological considerations, we can encode as much information as we like in a single real number.

18.3 Variational autoencoders

Variational autoencoders (VAEs) get around this particular problem by starting the generator off at *different* points in the latent space to those mapped by the encoder from the data.

Essentially, we map an instance of our input data to a point in a latent space, then sample a *nearby* point in the latent space, push this new point through the generator, and compare the result to our input data. This (ideally) ensures that the latent space spatially represents the data, or rather roughly preserves the topology of the data: input data elements which are ‘close’ to each other should also be ‘close’ in the latent space.

So the encoder, rather than mapping to a single point in the latent space, instead maps to a *distribution* (in practice, it maps to the parameters of a distribution) and we sample this distribution to get the input for the decoder.

18.3.1 The problem with (naive) variational autoencoders:

.... which does not really help us by itself. After all, we may simply learn to map each data point to its own far-out region of the latent space, and map each nearby point back to the original data.

The way to deal with this is to impose a penalty on the distance between the distributions given by the encoded inputs. The best way to do this turns out to be by using our old friend, the KL divergence.

Before the details, this is our basic algorithm:

1. Suppose that inputs come from a space I , and let f_{data} be the distribution of inputs over I .
2. Decide on a simple space S , usually $\mathbb{R}^m$ for small m .
3. Decide on a loss/cost function $C : I \times I \rightarrow \mathbb{R}^+$ which represents the distance between two input images. Generally this will be $C(x, x') = \frac{\|x - x'\|_2}{2c}$ for some value c (which will be explained more soon).
4. Initiate two neural networks $E : I \rightarrow S \times \mathbb{R}^{m \times m}$ (encoder) and $D : S \rightarrow I$ (decoder). The encoder maps an input data sample to a *mean* and *variance* of a distribution on S .
5. Parametrise E by $\theta^{(e)}$ and D by $\theta^{(d)}$.
6. For each training sample x , compute $(\mu_z, \sigma_z^2) = E(x, \theta^{(e)})$ as the ‘encoded’ distribution of x in the latent space.
7. Generate an $z' \sim MVN(\mu_z, \sigma_z)$, where MVN is a multivariate normal distribution with mean μ_z and variance σ_z^2 .
8. Compute $x' = E(z', \theta^{(e)})$ using the ‘encoder’.
9. Assess the performance of the autoencoder by the function:

$$\frac{\|x - x'\|_2}{2c} + D_{KL}(MVN(\mu_z, \sigma_z), MVN(0, I_m))$$

10. Update parameters $\theta^{(d)}$ and $\theta^{(e)}$ (by an algorithm we will look at next)

We note that the inclusion of the term

$$D_{KL}(MVN(\mu_z, \sigma_z), MVN(0, I_m)) \tag{18.1}$$

in the cost function ensures that:

- The variance σ_z is not too small: e.g., if σ_z is close to 0, then the K-L divergence with $MVN(0, I_m)$ will be high
- The distributions are not too spread out; if μ_z is too far from 0, the K-L divergence will again be large.
- The variance σ_z will not be too large.

18.4 Training VAEs

Training VAEs uses a couple of neat tricks. Since the function $F : x \rightarrow x'$ (given by $E(D(x, \theta^{(d)}), \theta^{(e)})$) is random-valued, it is not immediately obvious how to update parameters $(\theta^{(d)}, \theta^{(e)})$ using gradient descent.

Let us assume that our ‘latent space’ is actually a peculiarity of the data, in that our data samples X are determined by some latent values Z : in particular, we will assume that our data are generated according to the process:

1. Choose a $Z \sim f_{code}$ from the latent distribution
2. Choose $X|Z$ according to $(X|Z = z) = MVN(f(z, \theta^{(d)}), cI_n)$ where n is the dimension of the data, and $\theta^{(d)}$ is some set of parameters.

In this formulation, we want our encoder to be:

$$E(x, \theta^{(e)}) = f_{Z|X=x}(z)$$

noting that this returns a distribution, and our decoder to be:

$$D(z, \theta^{(d)}) = f(z, \theta^{(d)})$$

We will also make an assumption that:

$$Z \sim \text{MVN}(0, I_m)$$

that is, f_{code} is given by a standard normal PDF.

We will approximate $f_{Z|X=x}(z, x)$ as:

$$f_{Z|X=x}(z) \approx q_x(z, \theta^{(e)}) = \text{MVN}(g(x), h(x))$$

that is, so $E(x, \theta^{(e)}) = (g(x), h(x))$. Note that we have, for any x ,

$$\begin{aligned} \arg \min_{\theta^{(e)}} D_{KL}(q_x(z, \theta^{(e)}) || f_{Z|X=x}(z)) &= \arg \min_{\theta^{(e)}} \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln \left(\frac{q_x(z, \theta^{(e)})}{f_{Z|X=x}(z)} \right) \right\} \\ &= \arg \min_{\theta^{(e)}} \left\{ \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(q_x(z, \theta^{(e)})) \right\} - \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(f_{Z|X=x}(z)) \right\} \right\} \\ &= \arg \min_{\theta^{(e)}} \left\{ \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(q_x(z, \theta^{(e)})) \right\} - \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln \left(\frac{f_{X|Z=z}(x) f_Z(z)}{f_X(x)} \right) \right\} \right\} \\ &= \arg \min_{\theta^{(e)}} \left(\mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(q_x(z, \theta^{(e)})) \right\} - \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(f_Z(z)) \right\} \right. \\ &\quad \left. - \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(f_{X|Z=z}(x)) \right\} + \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(f_X(x)) \right\} \right) \\ &= \arg \min_{\theta^{(e)}} \left(\mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ -\ln(f_{X|Z=z}(x)) \right\} \right. \\ &\quad \left. + \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(q_x(z, \theta^{(e)})) \right\} - \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln(f_Z(z)) \right\} \right) \\ &= \arg \min_{\theta^{(e)}} \left(\mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \frac{\|x - f(z, \theta^{(d)})\|_2}{2c} \right\} + D_{KL}(q_x(z, \theta^{(e)}), f_Z(z)) \right) \\ &= \arg \min_{\theta^{(e)}} \left(\mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \frac{\|x - f(z, \theta^{(d)})\|_2}{2c} \right\} + D_{KL}(\text{MVN}(\mu_z, \sigma_z), \text{MVN}(0, I_m)) \right) \end{aligned}$$

which shows where our objective comes from (some steps omitted and left as exercise).

18.4.1 Training the decoder

Much the same goes as above when trying to optimise with respect to $\theta^{(d)}$, the parameters of the decoder. It is relatively easy to set up a gradient descent algorithm to optimise $\theta^{(d)}$. The only part of the expression to optimise which depends on $\theta^{(d)}$ is the first term, which is a straightforward function of $f(z, \theta^{(d)})$. For a given $\theta^{(e)}$, we know the distribution $q_x(z)$, so we can train $\theta^{(d)}$ by gradient descent in the usual way.

18.4.2 Training the encoder

Training $\theta^{(e)}$ is more difficult. This is essentially because we want to calculate:

$$\nabla_{\theta^{(e)}} \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln \left(\frac{q_x(z, \theta^{(e)})}{f_{Z|X=x}(z)} \right) \right\} \quad (18.2)$$

and because $q_x(z, \theta^{(e)})$ depends on $\theta^{(e)}$, we can't move the $\nabla_{\theta^{(e)}}$ inside the $\mathbb{E}_{q_x(z, \theta^{(e)})}$.

We can get around this by noting that $q_x(z, \theta^{(e)})$ is a normal random variable with mean $g(x)$ and standard deviation $h(x)$, where both $g(x)$ and $h(x)$ depend on $\theta^{(e)}$ but are constant. We note that we can say that if $Z \sim q_x(z, \theta^{(e)})$ then

$$(Z|X=x) \sim \mu_x + \sigma_x \epsilon \quad (18.3)$$

where $\epsilon \sim MVN(0, I_n)$ is a Gaussian random variable *not* depending on $\theta^{(e)}$ (this is called the *reparametrisation trick*) and μ_x, σ_x depend on $g(x), h(x)$. Using this we can write:

$$\nabla_{\theta^{(e)}} \mathbb{E}_{q_x(z, \theta^{(e)})} \left\{ \ln \left(\frac{q_x(z, \theta^{(e)})}{f_{Z|X}(z, x)} \right) \right\} = \nabla_{\theta^{(e)}} \mathbb{E}_{\epsilon} \left\{ \ln \left(\frac{q_x(\mu_x + \epsilon \sigma_x, \theta^{(e)})}{f_{Z|X}(\mu_x + \epsilon \sigma_x, x)} \right) \right\} = \mathbb{E}_{\epsilon} \left\{ \nabla_{\theta^{(e)}} \ln \left(\frac{q_x(\mu_x + \epsilon \sigma_x, \theta^{(e)})}{f_{Z|X}(\mu_x + \epsilon \sigma_x, x)} \right) \right\}$$

and now we can use backpropagation as usual.

Exercises

1. Suppose that our input space I is $\{0, 1\}^{n^2}$; the set of 0-1 sequences of length n , and f_{data} is a distribution across I . We will consider various latent spaces S , all of which will be $\{0, 1\}^r$ for some r . We will build a ‘naive’ autoencoder with encoder $E : I \rightarrow S$ and decoder $D : S \rightarrow I$.
 - (a) Suppose that $f_{data}(i) > 0$ for all $i \in I$. Show that we cannot have a perfect autoencoder (that is, for which $i = D(E(i))$ for all $i \in I$ unless $r \geq n^2$; that is, we cannot compress this input.
 - (b) Suppose that $f_{data}(i) > 0$ only for $i \in A_n$, where A_n is some set which has maximum Kolmogorov complexity $O(k(n))$ for some function k (relative to a standard computer programming language; for instance, R). Show that there exists a value $s > 0$ such that for sufficiently large n , we can choose D , E and S with $r \leq sk(n)$ such that the autoencoder is perfect.
 - (c) Suppose that we know the distribution of f_{data} . Rather than always getting a perfect answer, we want to get the right answer *most of the time* for large n . We will set $r = (1 + \epsilon)h$ for some value of h . What value of h will allow us to do this?
2. If $(X|Z = z) \sim \text{MVN}(f(z, \theta^d)^2, cI_n)$ show that

$$-\ln(f_{X|Z=z}(x)) = \frac{\|x - f(z, \theta^{(d)})\|_2^2}{2c} + q(n),$$

where $q(n)$ is a function depending only on n .

3. In the derivation of $\arg \min_{\theta^{(e)}} D_{KL}(q_x(z, \theta^{(e)}) || f_{Z|X=x}(z))$, why can the terms $q(n)$ (from the previous question) and $\mathbb{E}_{q_x(z)} \{\ln(f_Z(z))\}$ be ignored?
4. If $(Z|X = x) \sim \text{MVN}(g(x), h(x))$, where $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $h : \mathbb{R}^n \rightarrow \mathbb{R}^{m^2}$ then find functions μ_x, σ_x such that $(Z|X = x) = \mu_x + \epsilon \sigma_x$, where ϵ does not depend on $\theta^{(e)}$.

Solutions

1. (a) Since every possible input has a positive probability of occurring, in order to set S , E and D such that $i = D(E(i))$ for all $i \in I$, the encoder and decoder cannot make any mistakes. Thus we cannot have $i, j \in I$ with $i \neq j$ but $E(i) = E(j)$, because then we would have $D(E(i)) = D(E(j))$ but $i \neq j$, so we cannot have both $i = D(E(i))$ and $j = D(E(j))$.

So we need S to be able to take at least 2^n values, so we need $r \geq n$.

- (b) Given a value $i \in A_n$ with Kolmogorov complexity k , we can write a computer program of length k which outputs i . Let us set $D_1()$ as the interpreter for this computer program and $E_1(i)$ as a writer which writes the shortest computer program which outputs i .

Let $q(n) = \max \{D(i), i \in A_n\}$; that is, the longest computer program that can occur. From the question, we know that $q(n) = O(k(n))$, so there exists some $N > 0$ and $s_1 > 0$ such that whenever $n > N$ we have $q(n) < s_1 k(n)$.

Let us encode our computer program as a binary sequence. We do this by encoding every character as a binary string (as is done with, say, ASCII). Since there are a finite number of possible characters in the computer program, we can do this with a binary string of a fixed length, say w . Let E_2 be an encoder which takes a program and outputs a binary sequence, and D_2 be a decoder which takes the binary sequence and outputs a program. The binary sequence of a program of length $q(n)$ is of length at most $wq(n)$.

Now we set our encoder as $E(i) = E_2(E_1(i))$; so we write a computer program that outputs i and convert it to binary. We set our decoder as $D(sq) = D_2(D_1(sq))$ for a sequence sq . Since for $n > N$, the computer program to output anything in A_n has length at most $s_1 k(n)$, the binary sequence $D(i)$ for $i \in A_n$ is of length at most $ws_1 k_n$, so we can use $r = wk_n$, and we have designed our autoencoder as needed.

- (c) By the Source Coding Theorem (roughly) we can encode a random variable with entropy h using $(1 + \epsilon)h$ bits, and we will *usually* be able to recover it. So we want h to be the entropy of f_{data} .
2. We have:

$$\begin{aligned}
 -\ln(f_{X|Z=z}(x)) &= -\ln\left(\frac{1}{(2\pi)^{\frac{n}{2}}\sqrt{\det(cI_n)}} \exp\left(-\frac{1}{2}\left(x - f(z, \theta^{(d)})\right)^t (cI_n)^{-1} \left(x - f(z, \theta^{(d)})\right)\right)\right) \\
 &= \frac{1}{2}\left(x - f(z, \theta^{(d)})\right)^t (cI_n)^{-1} \left(x - f(z, \theta^{(d)})\right) + \ln\left((2\pi)^{\frac{n}{2}}\sqrt{\det(cI_n)}\right) \\
 &= \frac{\left(x - f(z, \theta^{(d)})\right)^t \left(x - f(z, \theta^{(d)})\right)}{2c} + q(n) \\
 &= \frac{\|x - f(z, \theta^{(d)})\|_2^2}{2c} + q(n)
 \end{aligned}$$

3. These terms do not depend on $\theta^{(e)}$, so since we are looking for the value of $\theta^{(e)}$ minimising the expression, we do not need to consider them. Although $q_x(z)$ depends on $\theta^{(e)}$, the expression inside the expectation does not, so this becomes:

$$\mathbb{E}_{q_x(z)} \{\ln(f_Z(z))\} = f_Z(z) \mathbb{E}_{q_x(z)} \{1\} = f_Z(z)$$

which does not depend on $\theta^{(e)}$.

4. We can write $\mu_x = g(x)$, straightforwardly. Suppose that σ_x is a matrix of dimension $m \times m$. Then we have:

$$\text{var}(Z|X=x) = \text{var}(\sigma_x \epsilon) = \sigma_x \text{var}(\epsilon) \sigma_x^t = \sigma_x I_n \sigma_x^t = \sigma_x \sigma_x^t$$

so we need $\sigma_x \sigma_x^t = h(x)$, from which σ_x can readily be computed through eigendecomposition of $h(x)$.

Transformers

Reading list and references

[Zhang et al., 2021, Chapter 11]
Vaswani et al. [2017]
Giacaglia [2019]
Alammar [2018]
Rush [2018]

I would recommend looking directly at some of these blogs. They use animations in a clever way to illustrate some of the ideas in this lecture.

19.1 Introduction

In this lecture we will look at the (celebrated) transformer mechanism, which is the neural network architecture used in the GPT series of models (GPT stands for General Pre-trained Transformer).

Roughly, a transformer is an improvement on an LSTM for managing sequences of data (noting that an LSTM itself is an improvement on a recurrent neural network). It works using a notion of ‘attention’ to various parts of a sequence, and we will look at what this means both formally and conceptually.

I do not expect a detailed understanding of how LSTMs or transformers work or of the attention mechanism. All that will be necessary for the exam is a rough qualitative (i.e. non-mathematical) understanding of the underlying ideas.

19.2 Long short-term memory networks (LSTMs)

A tidy way to (partly) overcome the vanishing gradient problem intrinsic to recurrent neural networks is the long short-term memory network (LSTM) Hochreiter and Schmidhuber [1997].

A recurrent network can be thought of as having *long-term* ‘memory’ encoded by weights, which are updated during training, and change slowly. The activations of the network (from calculating a forward pass) constitute *short-term* memory. In a sense, the vanishing gradient problem arises because long-term memory is too salient, and the network cannot react to new information quickly. LSTMs introduce a new type of memory encoded by a particular type of neuron.

To each h_i we attribute a *memory* C_i , sometimes called an *internal state*. We may control whether the internal state:

- should be affected by a given input,
- should affect the output
- should be changed

This is easier if we actually formalise it.

19.2.1 LSTM setup

The flow of information through an LSTM is illustrated in figure 19.1.

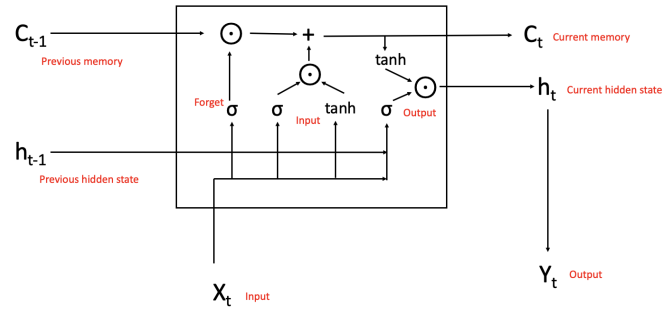


Figure 19.1: Information flow through a long-short-term-memory network. Adapted from [Zhang et al. \[2021\]](#).

The flow of information consists of a few steps. Data passes through a series of ‘gates’, called ‘forget’ and ‘input’. In detail (adapted from [Zhang et al. \[2021\]](#)):

1. We begin with what we previously had: at time t , data X_t and a hidden state h_{t-1} from the previous time, and we will output a current hidden state h_t and a Y_t . We also have a ‘memory’ from the previous time C_{t-1} , and we will output C_t .
2. We begin by computing four functions F , I , O and $\tilde{C}$ of h_t and X_t . All are neural networks, parametrised by weight matrices W_F , W_I , W_O , $W_{\tilde{C}}$ for h_t and U_F , U_I , U_O , $U_{\tilde{C}}$ for X_t respectively, and biases b_F , b_I , b_O , $b_{\tilde{C}}$. The first three use logistic activation functions $\phi(x) = \sigma(x) = (1 + \exp(-x))^{-1}$, so have values in $(0, 1)$, and $\tilde{C}$ uses $\phi(x) = \tanh(x)$ so has values in $(-1, 1)$.

$$\begin{aligned}
 F(h_{t-1}, X_t) &= \sigma(W_F h_{t-1} + U_F X_t + b_F) \\
 I(h_{t-1}, X_t) &= \sigma(W_I h_{t-1} + U_I X_t + b_I) \\
 O(h_{t-1}, X_t) &= \sigma(W_O h_{t-1} + U_O X_t + b_O) \\
 \tilde{C}(h_{t-1}, X_t) &= \sigma(W_{\tilde{C}} h_{t-1} + U_{\tilde{C}} X_t + b_{\tilde{C}})
 \end{aligned} \tag{19.1}$$

The function F is short for ‘forget’, I is ‘input’ and O is ‘output’. The ‘forget’ function tells us how much of the old C_t value we will forget, and the ‘input’ function tells us how much of the new input we will retain. The dimensions of F are the same as that of C_t ,

3. We combine F with the previous memory C_{t-1} using a Hadamard product $\odot$ (element-wise multiplication), and combine I and $\tilde{C}$ the same way. We add these together to get the new memory state C_t :

$$C_t = F(h_{t-1}, X_t) \odot C_{t-1} + I(h_{t-1}, X_t) \odot \tilde{C}(h_{t-1}, X_t)$$

4. Finally, we take the Hadamard product of $O(h_t, X_t)$ and $\tanh(C_t)$ to get the new hidden state h_t .

Why does this help? In a sense, it allows *resetting* of the hidden state, and the network learns when to do this, in particular learning to skip irrelevant observations.

19.3 Attention

We will look a little more closely the problem of dealing with a *sequence* of data. The usual example of this in the context of attention is the problem of translating text [Giacaglia \[2019\]](#). Suppose we have the following sentence in English, and we want to translate it to another language:

The Beatles are a major English band. The band was formed in Liverpool in 1960.

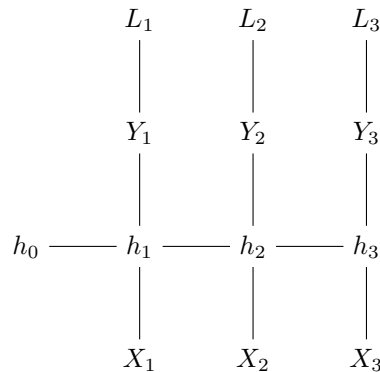
The term ‘The band’ in the second sentence refers to the term ‘The Beatles’ in the first. In order to make sense of the sentence as a whole, we need to keep the whole sentence in mind, or roughly to remember ‘The Beatles’ while we read the second part. We are maintaining ‘attention’ on the phrase ‘The Beatles’ while reading the sentence.

19.3.1 Beginnings

We have already encountered recurrent neural networks for dealing with sequences, which roughly learn a function which maps each element of the sequence to the next element (allowing for some random input). In order to keep ‘memory’ of elements of the sequence we looked at longer ago, we introduced the LSTM, which retains ‘memory’ and combines new information with this memory. However, even LSTMs struggle when we need to link information from elements a long way from each other in the sequence.

Let’s consider a rudimentary neural network which translates a sentence to another. A usual way to do this would be to chain two recurrent neural networks together, one for ‘reading’ and one for ‘writing’:

To remind ourselves: a recurrent neural network looks a bit like this:



that is, we learn to predict the next ‘hidden’ state given the current hidden state and new information X_i .

In order to get the output of a recurrent neural network, we will put each of n words in the sentence as a sequence of X_i s, and keep track of the hidden state after giving it some initial value h_0 and then giving it the X_i s in sequence. Our knowledge about the sentence after reading it is now (ideally) contained in the hidden state h_n .

In our naive setting, we would then pass our h_n to the decoder, which would then spit out a sequence of words which constituted the translated sentence.

19.3.2 Weighting states for attention

In the ‘Attention’ mechanism, we pass the decoder *the whole sequence of hidden states*, after applying a function to decide how important the state is. Generally, if the state h_i is a vector of length n , we maintain a (learnable) vector w of length n , and compute the function:

$$a_i = \text{SoftPlus}(w \cdot h_i^T) = \ln(1 + \exp(w \cdot h_i^T))$$

to get a measure of how much ‘attention’ a_i we should pay to the state h_i . We then

If, for instance, we are translating the French sentence:

L'accord sur la zone economique europeenne a ete signe en août 1992

into the English

The agreement on the European Economic Area was signed in August 1992.

using a trained translation model, the following heatmap shows how much attention is paid to each input word when decoding each output word:

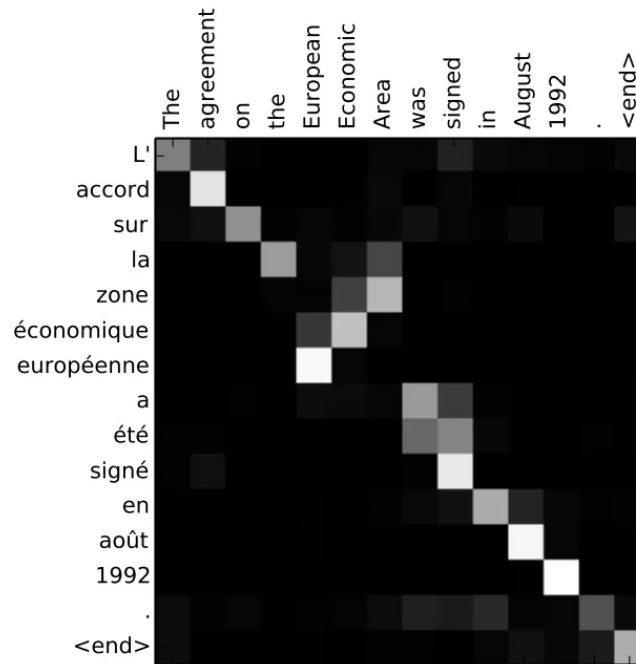


Figure 19.2: Figure from [Giacaglia \[2019\]](#), [Alammar \[2018\]](#).

In fact, what we will do is to define:

1. x_t as the sequence of words; in the above example, $x_1 = L'$, $x_2 = \text{accord}$, for $t \in 1, 2, \dots, T$.
2. h_t as the hidden states of the *encoder*, so $h_t = f(h_{t-1}, x_t)$
3. $c = h_T$; the final state of the encoder after reading the sequence
4. y_t as the output of the decoder.
5. s_t as the hidden states of the *decoder*, so $s_t = g(s_{t-1}, c, y_{t-1})$ and $y_t \sim r(s_t)$, where $r(s_t)$ is a probability distribution over possible values of y_t .

In a standard translator, we keep c fixed; in our attention-based translator, we let c change as we generate the output y ; in fact using:

$$c_t = \sum_i \alpha(s_{t-1}, h_i) h_i$$

where

$$\alpha(w, x) = \text{SoftMax}(w^T x) = \ln(1 + \exp(w^T x))$$

Note that we are using the previous hidden state s_{t-1} as our attention vector.

19.3.3 Self-attention

The very rough idea of transformers is to take a sequence x_i , then generate a new sequence z_i where the z_i mostly represents x_i , but also ‘takes account’ of all the other x_i s.

In a little more detail: we have a vector x_i of dimension d . We maintain matrices W^q , W^v of dimension $d \times r$, where usually $r \ll d$, and compute values $q_i = W^q x_i$, and $v_i = W^v x_i$, calling these values the ‘query’ and ‘value’ respectively. We also keep a matrix of ‘keys’ K of dimension $d \times m$. We then compute the values:

$$z_i = \text{SoftMax}\left(\frac{K q_i}{\sqrt{r}}\right) v_j$$

A few notes:

1. We use the *query* for item i and the *values* for all items j .
2. By multiplying the SoftMax'd score with v_j , we preserve the values of words we want to pay attention to.

19.4 Transformers

We will begin motivation of transformers by comparing convolutional and recurrent neural networks with our new ‘self-attention’ mechanism on a generalised problem. Suppose we have an input sequence of length n , and we want to output a sequence of length n . Each input element and each output element is a d -dimensional vector.

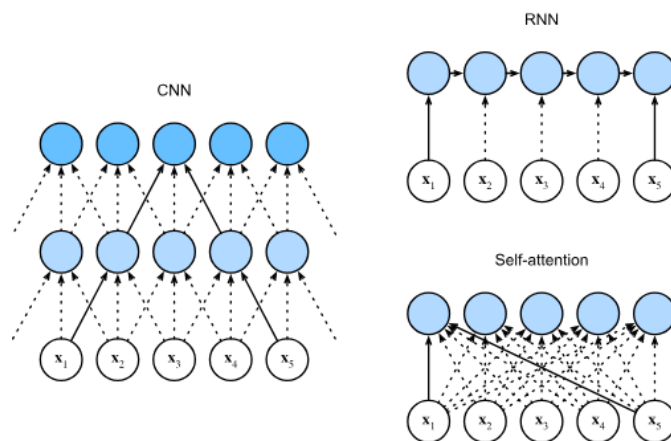


Figure 19.3: Rough diagram of a CNN, RNN, and self-attention mechanism. From [Zhang et al. \[2021\]](#), chapter 11

We will compare the three architectures on the basis of:

1. the *total* number of operations they need to do, which we will call ‘total complexity’
2. the number of operations we must perform in sequence; that is, we can only perform the i th action after performing the $(i - 1)$ th action, which we will call the ‘sequential complexity’ (this is very relevant, since it reflects the extent to which a computation is parallelisable)
3. given any two input elements, the number of operations we must do before information from both elements reaches a common neuron; we will call this ‘path length’.

We will consider a convolutional neural network using a kernel of size k . We have the following table:

	CNN	RNN	Self-attention
Total complexity	$O(knd^2)$	$O(nd^2)$	$O(n^2d)$
Sequential complexity	$O(1)$	$O(n)$	$O(1)$
Path length	$O(\log_k(n))$	$O(n)$	$O(1)$

Table 19.1: Complexity of major NN designs

We will go through the rationale for these complexities in the lecture; also see [\[Zhang et al., 2021, Chapter 11\]](#).

19.4.1 Positional encoding

The self-attention mechanism processes all input elements in parallel, so we lose the intrinsic knowledge of sequence position we would have should we have used a recurrent architecture.

To overcome this, we simply add some information to each input element to represent its position in the sequence, which is then processed along with the rest of the element.

This can be done in many ways. One popular way uses a binary-like encoding. Suppose we have n input elements, which are all vectors of length d . We combine these into an $n \times d$ matrix X . We encode positions as an $n \times d$ matrix P , where

$$P_{ij} = \begin{cases} \sin\left(\frac{i}{M^{\frac{j}{d}}}\right) & \text{for } j \text{ even} \\ \cos\left(\frac{i}{M^{\frac{j-1}{d}}}\right) & \text{for } j \text{ odd} \end{cases}$$

for some large number M (usually 10^5). From a barcode of values $(P_{i1}, P_{i2}, \dots, P_{id})$ the position can be reconstructed. This representation is advantageous because it resembles a binary enumeration of elements (Zhang et al. [2021]).

19.4.2 The transformer architecture

The overall transformer architecture is shown in the following figure from Vaswani et al. [2017]:

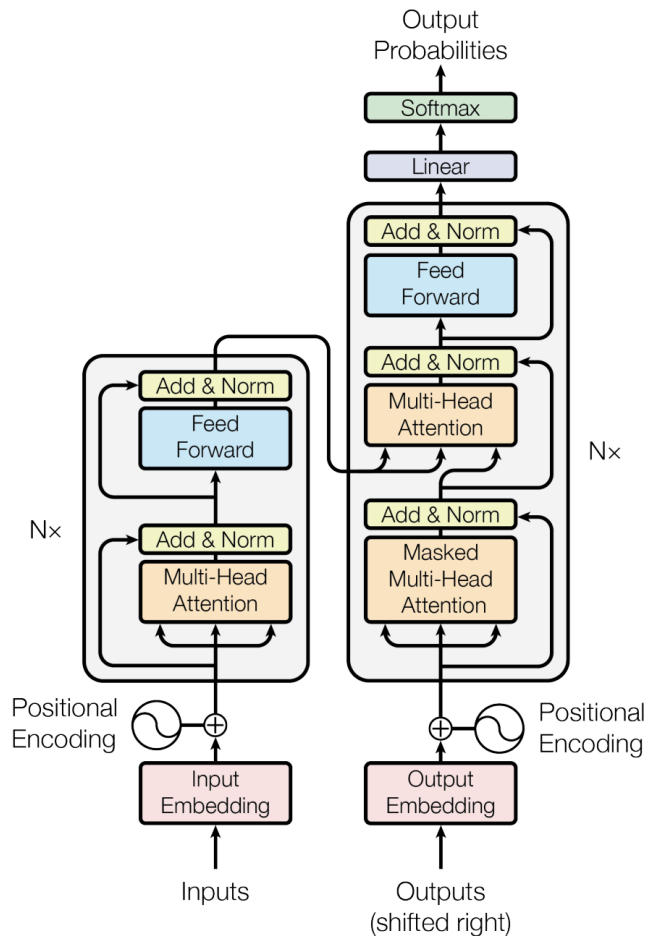


Figure 19.4: The overall transformer architecture

It is essentially a composition of a series of feed-forward and attention layers. A few notes:

1. The overall task is ‘sequence transduction’, so the network takes a sequence of inputs (‘Inputs’ on figure) representing, e.g., a sentence to be translated, *and* a sequence representing the output up to a given point (‘Outputs (shifted right)’ on figure) representing e.g. the translation so far. It then returns a probability distribution describing what the next element in the output is likely to be (‘Output probabilities’ on figure) representing e.g. the next word in the translated sentence.
2. A ‘Feed forward’ block is a standard feed-forward neural network, like those we have seen before.

3. There are layers at which data is normalised and pooled ('Add and norm')
4. There are arrows going around most blocks. These are 'residual connections', which are where given a hidden layer value $X^{(\ell)}$, we decide that for some $r > \ell$ we will designate $X^{(r)}$ (the input to layer $r + 1$) to be both the output of layer r (that is, $X^{(r-1)}$) and the values $X^{(\ell)}$; that is, we let the values skip some layers.

The transformer architecture can largely be trained by backpropagation in the usual way. See [Vaswani et al. \[2017\]](#) and [Rush \[2018\]](#) for details.

Exercises

1. What are the consequences in an LSTM if $F = 0$? What if $F = 1$?
2. What are the consequences in an LSTM if $I = 0$? What if $I = 1$?
3. The transformer architecture includes ‘residual connections’, as above. Could these be represented as a standard (fully connected) neural network with some forced values of weights/parameters (e.g. some weights forced to 1)? Why might residual networks help with function approximation?
4. Why might we divide by $\sqrt{r}$ in the self-attention expression?

Solutions

1. If $F = 0$, then the first term in equation:

$$C_t = F(h_{t-1}, X_t) \odot C_{t-1} + I(h_{t-1}, X_t) \odot \tilde{C}(h_{t-1}, X_t)$$

vanishes, and the value C_{t-1} no longer contributes to C_t . If it is always 1, then C_{t-1} always contributes (we never forget).

2. If $I = 0$, then the contribution of the second term in the equation for C_t vanishes, and the input does not contribute to C_t . If $I = 1$, then the second term always contributes.
3. In general these cannot be represented exactly as a neural network in which some weights are kept at 1: in order for an input to ‘bypass’ a layer, we need an activation function which can return any value in the input space. For instance, if we are using sigmoid activations, then the layer outputs values in $[0, 1]$, but the input might not be in this range. Of course, this can be bypassed by suitable scaling and use of weights in the next layer.

Residual networks are widely used and are particularly helpful when the neural network is trying to approximate a function $g(x, f(x))$, where g and f are relatively simple functions.

4. Roughly to preserve the variance; if X_i are IID samples from $N(0, 1)$ then

$$\text{var} \left(\frac{1}{\sqrt{r}} \sum_{i=1}^r X_r \right) = 1$$

Diffusion models

Reading list and references

[Das \[2024\]](#)

I strongly encourage reading this blog post for a deep understanding of how diffusion models work.

20.1 Introduction

In this lecture we will introduce diffusion models. We have already encountered generative models before (GANs, autoencoders); diffusion models are similar in philosophy, but are state-of-the-art.

Although GANs can work well, they are difficult to train, partly due to ‘mode collapse’. Diffusion models train a sequence of probability distributions by using a ‘score’ function, and tend to be easier to train.

I do not expect a detailed understanding of how diffusion models work: all that will be necessary for the exam is a rough qualitative (i.e. non-mathematical) understanding of the underlying ideas.

20.2 Overall idea

20.2.1 Generative networks

Diffusion models are generative, and take a similar approach to GANs and autoencoders in attempting to model a complex (target) distribution by transforming a simple one.

In spirit, a diffusion model is similar to an autoencoder, in that it takes samples from the real data, maps them gradually to a latent space, then reverses the map to get a sample similar to the real data.

Suppose that we have some complex distribution, which we will call f_{data} , from which we are trying to sample. For instance, suppose we want to generate 1000×1000 pixel images of human faces. A colour image is a set of three arrays (red, green, and blue) where each pixel takes a value between 0 and 255. Then f_{data} is defined over the (discrete) space

$$\{0, 1, 2, \dots, 255\}^{1000 \times 1000 \times 3} \quad (20.1)$$

of all possible 1000×1000 pixel colour images, and for an image I , and the value $f_{data}(I)$ will be high if I is a picture of a face, and low if I is not a picture of a face. Our goal of generating a new picture of a face is equivalent to *sampling* a new value which follows the distribution f_{data} .

The approach taken by diffusion models is similar to that taken by GANs, in that we assume that we can generate images by taking a simple *latent* distribution, f_{code} , from which we can easily sample, and transforming a sample from f_{code} to a new random variable whose distribution we hope approximates f_{data} . For example, we may have:

$$f_{code}(z) = \phi(z)$$

so we take $Z \sim f_{code} \sim N(0, 1)$, and find some function

$$F : \mathbb{R} \rightarrow \{0, 1, 2, \dots, 255\}^{1000 \times 1000 \times 3}$$

such that, approximately:

$$F(Z) \sim f_{data}$$

or rather, if I is a random variable representing images of faces, then for any image i ,

$$P(I = i) = f_{data}(i) \approx P_{Z \sim f_{code}}(F(Z) = i)$$

20.2.2 The normalising constant

The general problem of estimating an unknown probability distribution is generally to assume that the distribution comes from some class of distributions $f(\cdot; \theta)$ parametrised smoothly by parameters θ and to try and maximise something like the log-likelihood of the data with respect to θ ; that is, for data $x_1, x_2 \dots x_n$, to find:

$$\arg \max_{\theta} \mathbb{E}_{X \sim f_{data}} \{\ln(f(X; \theta))\} \approx \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \ln(f(x_i; \theta))$$

To do this, though, we need to actually be able to calculate the values $f(x_i; \theta)$. The difficulty with doing this is that we usually want a very complicated f , and while it is usually easy to specify what we want the *relative* values of $f(x; \theta)$ to be (e.g. we want $f(x_1; \theta)$ to be twice the value of $f(x_2; \theta)$) but this only enables us to specify f up to a constant; that is, we can find $\tilde{f}(x; \theta) = Z f(x; \theta)$ where

$$f(x; \theta) = \frac{\tilde{f}(x; \theta)}{Z} = \frac{\tilde{f}(x; \theta)}{\int_z \tilde{f}(z; \theta) dz} \text{ or } \frac{\tilde{f}(x; \theta)}{\sum_z \tilde{f}(z; \theta)}$$

where the sum or integral is very difficult to calculate. Unfortunately the sum or integral also depends on θ , so it is difficult to get around this.

This will be a very familiar problem to all of you who have studied Bayesian statistics (and indeed we will take a variational-Bayes style approach to addressing it). We have also seen a similar idea when looking at Boltzmann distributions:

$$P(X = x; \theta = (w_{ij}, b_i)) = \frac{e^{-E(x; \theta)}}{Z} = \frac{e^{-E(x; \theta)}}{\sum_z e^{-E(z; \theta)}} = \frac{e^{\sum_{i < j} w_{ij} x_i x_j + \sum_i b_i x_i}}{\sum_z e^{\sum_{i < j} w_{ij} z_i z_j + \sum_i b_i z_i}}$$

where it is relatively easy to calculate $e^{-E(x; \theta)}$ for any given state x , and compare it to $e^{-E(y; \theta)}$ for a state y , but for a state of length n , the sum in the denominator has 2^n terms, and is very slow to calculate.

20.2.3 The score

The way we get around this in diffusion models is by considering the ‘score’, $\nabla_x \ln(f(x; \theta))$ (note that we will need $f(x; \theta)$ to be continuous, but in practice this isn’t too big a problem). We have:

$$\nabla_x \ln(f(x; \theta)) = \nabla_x \ln\left(\frac{\tilde{f}(x; \theta)}{\int_z \tilde{f}(z; \theta) dz}\right) = \nabla_x \ln(\tilde{f}(x; \theta)) - \nabla_x \ln\left(\int_z \tilde{f}(z; \theta) dz\right) = \nabla_x \ln(\tilde{f}(x; \theta))$$

since $\int_z \tilde{f}(z; \theta) dz$ does not depend on x . This means, by working with the score, we can get around dealing with the normalising constant.

Intuitively, a derivative only involves how a function changes; it doesn’t care what value it changes from. The derivative of the log ensures that the contribution of the normalising constant is additive.

20.2.4 Diffusion model overall form

We are aiming to find a function f_{θ} parametrised by θ for which if $Z \sim f_{code}$ then $f_{\theta}(Z) \sim f_{data}$.

The type of function f_{θ} in a diffusion model is a *repeated composition* of other functions. If $s_{\theta}(x)$ is a neural network parametrised by θ , and $g(\cdot)$ is a function, and $h(x) = g(s_{\theta}(x), x)$, then roughly a diffusion model has the form:

$$f_{\theta}(x) = h(h(\dots h(h(x)) \dots))$$

The idea of the composition is that rather than trying to model f_{θ} with a neural network, we use a neural network to model its derivative (the score) and use something similar to gradient descent (e.g., repeated applications of the score).

20.3 Details

We have two basic tasks:

1. How do we estimate the score?
2. Given an estimate of the score function, how do we use it to make a sample from f_{data} ?

We will roughly address these in turn, then go on to how the diffusion model itself works.

20.3.1 How do we estimate the score?

We want to estimate the score:

$$s(x) \triangleq \nabla_x \ln(f_{data}(x))$$

We will approximate $s(x)$ with a neural network $s_\theta(x)$ parametrised by θ . To do this, we have samples x_i from f_{data} . The way we do this is to attempt to find:

$$\begin{aligned} \theta^* &= \arg \min_{\theta} \frac{1}{2} \mathbb{E}_{X \sim f_{data}} \{ \|s(X) - s_\theta(X)\|_2^2 \} \\ &= \arg \min_{\theta} \mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{2} \|s(X)\|_2^2 + \frac{1}{2} \|s_\theta(X)\|_2^2 - s(X) s_\theta(X)^T \right\} \\ &= \arg \min_{\theta} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{2} \|s_\theta(X)\|_2^2 \right\} - \int f_{data}(x) s(x) s_\theta(x)^T dx \right) \\ &= \arg \min_{\theta} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{2} \|s_\theta(X)\|_2^2 \right\} - \int f_{data}(x) \nabla_x \ln(f_{data}(x)) s_\theta(x)^T dx \right) \\ &= \arg \min_{\theta} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{2} \|s_\theta(X)\|_2^2 \right\} - \int f_{data}(x) \frac{\nabla_x f_{data}(x)}{f_{data}(x)} s_\theta(x)^T dx \right) \\ &= \arg \min_{\theta} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{2} \|s_\theta(X)\|_2^2 \right\} - \int \nabla_x f_{data}(x) s_\theta(x)^T dx \right) \\ &= \arg \min_{\theta} \left(\mathbb{E}_{X \sim f_{data}} \left\{ \frac{1}{2} \|s_\theta(X)\|_2^2 \right\} + \int f_{data}(x) \nabla_x \cdot s_\theta(x)^T dx \right) \\ &= \arg \min_{\theta} \mathbb{E}_{x \sim f_{data}} \left\{ \nabla_x \cdot s_\theta(x) + \frac{1}{2} \|s_\theta(x)\|_2^2 \right\} \end{aligned}$$

(see [Hyvärinen and Dayan \[2005\]](#) for a few more details) which can be readily fitted by gradient descent. There are additional nuances; see [Das \[2024\]](#) for details.

20.3.2 How do we use the score to sample from f_{data} ?

Suppose we have a ‘starting’ value x and know the score $s(x) = \nabla_x \ln(f_{data}(x))$. One naive-looking way to sample from $f_{data}(x)$ is to try and update x by the following:

$$x \leftarrow x + \delta s(x)$$

for some learning rate δ . Since $s(x) = 0$ at a local maximum of $f_{data}(x)$, this will push x towards modes of f_{data} (ideally) which gives something resembling a sample. For very small δ , we have

$$dx = s(x) \tag{20.2}$$

which allows us to set up a differential equation describing where x will go over time if we update by small δ .

We don’t really want to converge to the nodes. The key idea in diffusion models comes from physics. We instead introduce noise into the path of x via a [Brownian motion](#) term B_t , leading to [Langevin dynamics](#):

$$dx = s(x)dt + \sqrt{2}dB_t = \nabla_x \ln(f_{data}(x))dt + \sqrt{2dt}Z$$

where $Z \sim N(0, 1)$. That is, we decide a path for x by considering increments of size dt where

$$x_{t+1} = x_t + s(x)dt + \sqrt{2dt}Z \tag{20.3}$$

and considering the limit as $dt \rightarrow 0$ (we are skipping a lot of analysis of stochastic differential equations!)

It turns out that this leads to what we need: the distribution of x_t converges to f_{data} .

20.3.3 Putting this together

It doesn’t quite work this easily, because the score is harder to estimate than implied earlier.

The essential difficulty is that learning $s(x)$ *globally* is too hard. Realistic functions f_{data} are wildly complicated, and functions $s(x)$ are just as complicated in reality. A critical idea of diffusion models is to learn $s(x)$ *only where it is needed*. To do this, we first decide on a path for x , and then go back and learn $s(x)$ on that path.

What we end up doing is a *forward process*, which is what we did before in reverse: we start with a sample from f_{data} (e.g. a training sample) and then transform it so that its distribution moves toward f_{code} . We usually use $f_{code} = N(0, I)$, so the update is relatively easy:

$$dx = \nabla_x \ln(f_{code}(x))dt + \sqrt{2}dB_t = -xdt + \sqrt{2}dZ \quad (20.4)$$

noting that this is easier, because we already know the score of f_{code} .

Suppose we have a path x_t , where t varies continuously from 0 to ∞ . To estimate $s(x_t)$, we need samples near x_t . We don't necessarily have these. But we can simulate them.

So the idea of a diffusion model is to:

1. Start with a data sample x_i
2. Run a *forward diffusion process* (equation 20.4) which gradually maps the distribution of x_i to f_{code} (usually $N(0, I)$)
3. Use these to estimate $s(x)$ for points x_t likely to be encountered in the *backward diffusion process* (equation 20.3)
4. Run the backward diffusion process to generate a sample from f_{data}

There is a lot more detail and subtlety to these models; in particular see [Das \[2024\]](#) and associated references to get an idea of where the complexities are.

Exercises

1. The Fokker-Planck equation considers processes of the form:

$$dX_t = \mu(X_t, t)dt + \sigma(X_t, t)dB_t \quad (20.5)$$

where B_t is a standard Brownian motion. This can be roughly thought of as the continuous analogue of a sequence of random variables $X_0, X_1, X_2, \dots$, where $X_0 = 0$ and

$$X_{i+1} = X_i + \mu(X_i, i) + \sigma(X_i, i)Z \quad (20.6)$$

where $Z \sim N(0, 1)$.

- (a) In the discrete analogue, if $\mu(X_i, i) = i$ and $\sigma(X_i, i) = c$, show that $X_i \sim N\left(\frac{i(i+1)}{2}, c^2 i\right)$
- (b) If $\mu(X_t, t) = t$ and $\sigma(X_t, t) = c$ in the continuous case, conjecture the distribution of X_t .
2. Given the stochastic process in the previous question, the Fokker-Planck equation describes the evolution of the density $f_{X_t}(x)$ of X_t as a partial differential equation:

$$\frac{\partial}{\partial t} f_{X_t}(x) = -\frac{\partial}{\partial x} [\mu(x, t) f_{X_t}(x)] + \frac{\partial^2}{\partial x^2} \left[\frac{\sigma(x, t)^2}{2} f_{X_t}(x) \right]$$

- (a) With $\mu(X_t, t) = t$ and $\sigma(X_t, t) = c$, show the distribution

$$f_{X_t}(x) = N\left(\frac{t^2}{2}, c^2 t\right)$$

satisfies the Fokker-Planck equation.

- (b) If $\mu(x, t) = \nabla_x \ln(q_{data}(x))$ and $\sigma(x, t) = \sqrt{2}$, show that the PDF $f_{X_t}(x) = q_{data}(x)$ is a *stable* solution to the Fokker-Planck equation: that is, that $\frac{\partial}{\partial t} f_{X_t}(x) = 0$.

Solutions

1. (a) Since X_t is a sum of Gaussian random variables, it is itself Gaussian. The mean of X_i will be $1+2+\dots+i = \frac{i(i+1)}{2}$ and the variance of X_i will be $\underbrace{c^2 + c^2 + \dots + c^2}_{i \text{ times}} = ic^2$, from which the distribution of X_t is as above.

- (b) An obvious possible distribution for X is:

$$X_t \sim N\left(\frac{t(t+1)}{2}, c^2 t\right)$$

2. (a) We have:

$$\frac{\partial}{\partial t} f_{X_t}(x) = f_{X_t}(x) \left(\frac{1}{c^2} \left(x - \frac{t^2}{2} \right) + \frac{1}{2t^2 c^2} \left(x - \frac{t^2}{2} \right)^2 - \frac{1}{2t} \right)$$

and

$$-\frac{\partial}{\partial x} \mu(x, t) f_{X_t}(x) = -\frac{\partial}{\partial x} t f_{X_t}(x) = f_{X_t}(x) \frac{1}{c^2} \left(x - \frac{t^2}{2} \right)$$

and

$$\frac{\partial^2}{\partial x^2} \frac{\sigma(x, t)^2}{2} f_{X_t}(x) = \frac{c^2}{2} \frac{\partial^2}{\partial x^2} f_{X_t}(x) = f_{X_t}(x) \left(\frac{1}{2t^2 c^2} \left(x - \frac{t^2}{2} \right)^2 - \frac{1}{2t} \right)$$

so

$$\frac{\partial}{\partial t} f_{X_t}(x) = -\frac{\partial}{\partial x} [\mu(x, t) f_{X_t}(x)] + \frac{\partial^2}{\partial x^2} \left[\frac{\sigma(x, t)^2}{2} f_{X_t}(x) \right]$$

as needed.

- (b) We have:

$$\begin{aligned} -\frac{\partial}{\partial x} [\mu(x, t) f_{X_t}(x)] + \frac{\partial^2}{\partial x^2} \left[\frac{\sigma(x, t)^2}{2} f_{X_t}(x) \right] &= -\frac{\partial}{\partial x} [\nabla_x \ln(q_{data}(x)) q_{data}(x)] + \frac{\partial^2}{\partial x^2} \left[\frac{\sqrt{2}^2}{2} q_{data}(x) \right] \\ &= -\frac{\partial}{\partial x} \left[\frac{\nabla_x q_{data}(x)}{q_{data}(x)} q_{data}(x) \right] + \frac{\partial^2}{\partial x^2} q_{data}(x) \\ &= -\frac{\partial^2}{\partial x^2} q_{data}(x) + \frac{\partial^2}{\partial x^2} q_{data}(x) \\ &= 0 \end{aligned}$$

as needed

Revision lecture

Reading list and references

Statistical learning: [Hastie et al., 2009, Chapters 1,2,3]
Introduction to neural networks: [Calin, 2020, Chapters 1,5,6,7,8,9]
Representation: [Calin, 2020, Chapters 7,8,9], [Cybenko [1989], Leshno et al. [1993], Hornik et al. [1989]
Information theory: [Calin, 2020, Chapters 12,13], [Ash [2012]
Training: [Calin, 2020, Chapter 4,6], [Zhang et al., 2021, Chapter 5]
Standard network architectures: [Calin, 2020, Chapters 16,17,19], [Zhang et al., 2021, Chapters 7,8,9,10,20]
Energy-based networks: [Calin, 2020, Chapter 20], [Hinton [2012]
Modern networks (not examinable): [Zhang et al., 2021, Chapter 11], [Rocca [2019], Shafkat [2018], Das [2024], Vaswani et al. [2017], Giacaglia [2019], Alammar [2018], Rush [2018]

21.1 Introduction

Well done on making it through the course!

In this lecture we'll revise some key elements of what we covered. We will go through the sequence of lecture topics we covered through the year, covering basically what I expect you to be able to do for the exam. Rather than going into detail, we will highlight the main important ideas from each topic.

I emphasise that the best thing to do to revise is probably to go over the questions at the end of lecture slides and revise the problems/solutions from the formative assignments. There are also further questions in Calin [2020] and Zhang et al. [2021], which are worth looking at.

21.2 General overall principles

Inasmuch as there are general ideas on this course, these are several important takeaways. These are not necessarily useful for the exam, but are helpful heuristics for use of machine learning.

- We usually consider data to be generated according to some process which leads to an underlying data distribution.
- The general idea of machine learning can be considered as trying to find latent (low-dimensional) representations of (generally) high-dimensional data distributions. That is, for data X , we want to find some function z with $X' = z(X)$ for which:
 1. $\dim(X') \ll \dim(X)$
 2. f is well-behaved, in some sense
 3. Either z is nearly one-to-one, so we can recover X from X' , or for some outcome Y of interest we have that $Y|X'$ has approximately the same distribution as $Y|X$.

where item 3 essentially states that we 'simplify' X while retaining it's 'usefulness'.

- Usually if X comes from some high-dimensional space, we are concerned with some function $f(X)$, which we don't know, but for which we have some observations, and we want to be able to evaluate $f(X)$ at other values of X . We won't be able to evaluate f exactly, but we can approximate it with simpler functions.

- Neural networks can represent any realistic function over X , by making them wide enough. If there is latent structure in X , we can represent functions more efficiently by making the neural network deeper.
- A major advantage of neural networks over other machine learning methods is that they can be trained efficiently, using back-propagation, which depends on the network structure.

In the following sections, areas which are harder or involve more material are **bolded**. This does *not* mean they are more likely to come up in the exam, but they will generally be worth more points when tested.

21.3 Statistical learning

The idea of this lecture is to revise standard ideas from statistical learning, which you will mostly have encountered in previous courses.

You should be able to:

1. Understand the ideas of a dataset, the data distribution, expected value, decision rule, and loss/cost function.
2. Define the generalisation error
3. **Describe why we need training and testing sets, both formally (mathematically) and descriptively.**
4. Give the formula for an optimal classifier with 0-1 loss.

You do *not* need to:

1. Describe cross-validation
2. Describe anything to do with Gaussian mixture models

21.4 Introduction to neural networks

These three lectures specify the idea of a neural network. An important takeaway in general is understanding the ideas of the set of functions a neural network can represent.

You should be able to:

1. Sketch a typical abstract neuron and neural network, indicating input, activation function, bias, and output.
2. Give a formula for the output of a two-layer neural network in terms of the input, activation function(s), weights, and biases.
3. **Describe the set of functions which can be implemented by a perceptron, including using logic gates.**
4. Give a criterion (without measure theory) which a function has to satisfy to be n -discriminatory. Indicate whether common activation functions are 1-discriminatory.
5. **Work out the set of functions which can be implemented by simple two-layer neural network architectures.**
6. Give the formulas for some common activation functions.

You do *not* need to be able to

1. Describe biological neurons
2. Describe discriminatory activation functions using measure theory

21.5 Representation

These lectures are intended as a run-through of major representation results in the theory of neural networks. The most important take-away is the general heuristic ideas of why neural networks can approximate arbitrary functions; you should be able to take a simple network architecture and a simple (but general) class of functions, and design a neural network which can approximate any function in that class.

Generally, you should be able to:

1. State the standard universal approximation theorem for discriminatory activation functions, with ‘discriminatory’ as defined in lectures.
2. **Prove rigorously that neural networks with one hidden layer with ReLU or Heaviside activation functions are universal approximators for continuous functions,**
3. Understand the idea of approximating a class of functions with another, and the use of the supremum norm for this purpose.
4. Describe the set of functions which can be exactly implemented by a simple neural network.

You do *not* need to:

1. Prove Cybenko’s universal approximation theorem
2. Define or give analytic results about function spaces outside those mentioned in Cybenko’s theorem
3. Remember other universal approximation theorems, such as sum-product networks
4. Use norms on function spaces other than the supremum norm.

21.6 Information theory

These lectures are as close as we get to a link between the fundamental ideas of machine learning and the practical maths of how they work. We look at information in several ways.

21.6.1 Notions of complexity

You should:

1. Be able to give specific examples of why deep neural networks outperform shallow neural networks on certain tasks, and give a rough generalisation of why we normally use deep neural networks
2. **Show that a sawtooth function can be implemented efficiently with a deep neural network, but not with a shallow one.**
3. Recall the definition of Kolmogorov complexity and find the average Kolmogorov complexity of various sets of 0-1 sequences
4. Recall the definition of VC dimension and be able to find bounds on the VC dimension for various classes of simple functions.

21.6.2 Sigma algebras

You should:

1. Know the definition of a σ -algebra,
2. **Understand the concept of a random variable as a function from a probability space to a measurable space, and define what we mean by the σ -algebra ‘generated’ by a random variable.**
3. Given one random variable which is a function of another, say what happens to their associated σ -algebras (e.g., recognise that σ -algebras become coarser)
4. Relate this idea to standard feed-forward neural networks and describe how the σ -algebras of layer outputs change as we move through the network.

21.6.3 Entropy, mutual information

You should

1. **Be able to define and work with the ideas of entropy, differential entropy, mutual information, conditional entropy, KL divergence.**
2. Say why information should be a logarithm of probability
3. State a rough form of Shannon's Source Coding Theorem
4. Recall the basic properties of entropy, differential entropy, mutual information, and conditional entropy.
5. **Be able to prove basic inequalities regarding entropy and conditional entropy, using tools like Jensen's inequality.**

21.6.4 What you do not need to know

You do not need to:

1. Show anything about the Mandelbrot set
2. Remember or show results from measure theory other than the idea of sigma-algebras (e.g. Lebesgue integration, Radon-Nikodym theorem, filtrations)
3. Prove that the sigma-algebra generated by a function of a random variable is a subset or equal to the sigma algebra generated by that random variable
4. Use the idea of 'measurable functions'
5. Define or prove how differential entropy changes under a continuous function, or how this relates to neural networks.
6. Prove that differential entropy is maximised for a normal distribution when variance is fixed.
7. Prove how mutual information changes for functions of continuous random variables.

21.7 Training

In this series of lectures we looked at training neural networks, and general training algorithms for machine learning problems. You should be able to:

1. **Derive backpropagation formulas for a standard neural network**
2. **Describe the problems which arise if gradient descent proceeds too slowly or too fast.**
3. Describe the backtracking line search and momentum algorithms.
4. Understand the reasons for batch training
5. Roughly describe the dropout algorithm and why it is useful
6. Describe weight initialisation algorithms

You do *not* need to:

1. Give or prove analytic results regarding properties of minima, including the basin of attraction.
2. Prove rates of gradient descent convergence
3. Describe Nesterov's accelerated gradient algorithm, ADAGRAD, ADAM etc in detail

21.8 Standard network architectures

These (important) lectures cover some standard architectures common in real-world networks.

21.8.1 Convolutional neural networks

You should be able to

1. Sketch a convolutional neural network, indicating pooling layers and convolutional layers
2. Describe the 1- or 2- dimensional convolution operator and use it algebraically.
3. Describe the idea of pooling and the idea of transfer learning

21.8.2 Recurrent neural networks

You should be able to

1. Sketch and describe the operation of recurrent neural networks
2. **Derive backpropagation equations for recurrent neural networks.**
3. Understand the vanishing and exploding gradient problems and the circumstances in which they can arise.

21.8.3 Generative adversarial networks

You should be able to

1. Sketch a generative adversarial network (GAN), including the real data, generated data, discriminator and generator.
2. **Specify the loss function for a GAN, and derive the derivatives with respect to the parameters of the discriminator and generator.**
3. Describe the problem of mode collapse.

21.8.4 What you do not need to do

You do *not* need to:

1. Use continuous convolution operators
2. Specifically describe any particular convolutional network, e.g. those used in practicals
3. Describe LSTM networks
4. Use or show any properties of dynamical systems.

21.9 Energy-based networks

In this topic, we looked at a different conception of neurons which fired randomly with a given probability. As well as being of important theoretical interest, neural networks of this type can be used usefully to learn distributions.

You should be able to:

1. Sketch a stochastic neuron and describe its output as a probability distribution depending on its inputs
2. Sketch a Boltzmann machine and a restricted Boltzmann machine.
3. Describe how a Boltzmann machine evolves over time
4. **Give and use the formula for energy of a configuration of either a Boltzmann machine or a restricted Boltzmann machine, and the Boltzmann distribution of states**
5. Describe a Boltzmann machine as a Markov chain and describe formally that the long-run probability of being in a given state is given by the Boltzmann distribution
6. **Give and derive the formulas for training a Boltzmann machine.**

7. Roughly describe training procedures for restricted Boltzmann machines.

You do *not* need to:

1. Prove that the Boltzmann distribution is a stable distribution for a Boltzmann machine
2. Describe any physical analogies or derivations for the energy of a state

21.10 Modern neural networks

You should be able to:

1. Describe the operation of a variational autoencoder, including the reasons for its design
2. **Motivate and derive the usual loss function for training**
3. **Describe the reparametrisation trick for VAEs**
4. Describe (roughly) the tasks for which LSTMs, transformers and diffusion models are used.

You do *not* need to:

1. Describe the operation of an LSTM
2. Describe the operation of a transformer
3. Describe the operation of a diffusion model
4. Use stochastic differential equations or the Fokker-Planck equation

Bibliography

- Kingma DP Ba J Adam et al. A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 1412(6), 2014.
- Jay Alammar. The illustrated transformer [blog post], 2018. URL <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>.
- Martin Anthony and Peter L Bartlett. *Neural network learning: Theoretical foundations*, volume 9. Cambridge University Press, 2002.
- Robert B Ash. *Information theory*. Courier Corporation, 2012. URL <https://doc.lagout.org/Others/Information%20Theory/Information%20Theory/Information%20Theory%20-%20Robert%20Ash.pdf>.
- Peter L Bartlett and Shahar Mendelson. Rademacher and gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3(Nov):463–482, 2002.
- Stephen Bates, Trevor Hastie, and Robert Tibshirani. Cross-validation: what does it estimate and how well does it do it? *Journal of the American Statistical Association*, pages 1–12, 2023.
- Chris M Bishop. Training with noise is equivalent to tikhonov regularization. *Neural computation*, 7(1):108–116, 1995.
- Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K Warmuth. Learnability and the vapnik-chervonenkis dimension. *Journal of the ACM (JACM)*, 36(4):929–965, 1989.
- Ovidiu Calin. *Deep Learning Architectures: A Mathematical Approach*. Springer, 2020.
- Francois Chollet, Tomasz Kalinowski, and Joseph Allaire. *Deep Learning with R*. Manning, 2018.
- Erhan Cinlar. *Probability and stochastics*, volume 261. Springer, 2011.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- Ayan Das. Building diffusion model’s theory from ground up. In *The Third Blogpost Track at ICLR 2024*, 2024. URL <https://d2jud02ci9yv69.cloudfront.net/2024-05-07-diffusion-theory-from-scratch-58/blog/diffusion-theory-from-scratch/>.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Giancarlo Giacaglia. How transformers work, 2019. URL <https://towardsdatascience.com/transformers-141e32e69591>.
- Leonardo Ferreira Guilhoto. An overview of artificial neural networks for mathematicians. *Guilhoto. pdf*, 2018.
- Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer, 2009.
- Geoffrey E Hinton. A practical guide to training restricted boltzmann machines. In *Neural Networks: Tricks of the Trade: Second Edition*, pages 599–619. Springer, 2012.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- Aapo Hyvärinen and Peter Dayan. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6(4), 2005.
- Andrei N Kolmogorov. Three approaches to the quantitative definition of information. *Problems of information transmission*, 1(1):1–7, 1965.
- Moshe Leshno, Vladimir Ya Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural networks*, 6(6):861–867, 1993.
- Yurii Evgen’evich Nesterov. A method of solving a convex programming problem with convergence rate $O(k^{-2})$. In *Doklady Akademii Nauk*, volume 269, pages 543–547. Russian Academy of Sciences, 1983.
- Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *Ussr computational mathematics and mathematical physics*, 4(5):1–17, 1964.
- Desh Raj. Theory of deep learning: Role of depth, 2018. URL <https://desh2608.github.io/2018-07-28-deep-learning-theory-3/>.
- Joseph Rocca. Understanding variational autoencoders, 2019. URL <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>.
- Alexander M Rush. The annotated transformer. In *Proceedings of workshop for NLP open source software (NLP-OSS)*, pages 52–60, 2018. URL <https://nlp.seas.harvard.edu/2018/04/03/attention.html#training>.
- Irhum Shafkat. Intuitively understanding variational autoencoders, 2018. URL <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>.
- Shai Shalev-Shwartz and Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- Matus Telgarsky. Benefits of depth in neural networks. In *Conference on learning theory*, pages 1517–1539. PMLR, 2016.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Aston Zhang, Zachary C Lipton, Mu Li, and Alexander J Smola. Dive into deep learning. *arXiv preprint arXiv:2106.11342*, 2021. URL <https://d2l.ai/index.html>.